claude-windows / 75d3d121-f240-4293-afa5-b436a379ad31

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\75d3d121-f240-4293-afa5-b436a379ad31.jsonl`
- SHA-256: `7da25009fd6bc64c958d463207f5bbdc0f7e30e0252330508aae3645ea022485`
- Source modified: `2026-07-08T06:52:44+00:00`
- Imported at: `2026-07-08T15:59:19+00:00`
- project: `C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer`
- session_id: `75d3d121-f240-4293-afa5-b436a379ad31`

Transcript

1. user (2026-07-08T05:04:37.601Z)

Implement GitHub issue #521 (Khelsutra/badminton-highlight-indexer): "perf/scaling: run compile/stitch on the L4 worker; make phase?machine placement config-driven". Read issue #521 and #520 first (`gh issue view 521 -R Khelsutra/badminton-highlight-indexer`), plus the evidence in `docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_validation-latency_2026-07-07.md` §9.

Repo / environment

- Real code lives HERE: `C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer` (the sibling `C:\Users\avidu\Projects\badminton-highlight-indexer` is an empty STUB ? ignore it).
- Windows + PowerShell primary; `gcloud`?googleapis egress works but general web egress is blocked (curl to non-Google hosts returns HTTP 000). `gh` and `git` to GitHub DO work. Authed as avidullu, project `gen-lang-client-0306026826`, region `asia-southeast1`.

Why (measured, 2026-07-08 live CUJ on rev rally-control-plane-00019)

Compile/stitch runs IN-PROCESS on the CPU-only 2-vCPU control-plane and took **~12m55s** for an 11-clip / ~80s reel from a 1.83 GiB 4K source ? **~45?90s per ~7s clip**, because the per-clip branding watermark forces a full 4K re-encode with no NVENC. Validation (183s) ALSO runs on the control-plane; both are serialized by a mutex, so one run pins the control-plane for ~16 min of heavy CPU ? won't scale past ~5 concurrent users. Detection already dispatches to an L4 Cloud Run Job and is fast.

Current architecture (files to study first)

- `backend/orchestration.py`:
  - `_execute_compile(cfg, db, job, progress)` ? the compile/stitch handler (kind='compile' job): resolves segments, runs `backend.main.VideoCompiler` (ffmpeg trim+watermark+concat) under `_LOCAL_COMPUTE_LANE`, mirrors the reel to `reel_object_uri()` = `<output_root>/highlights/<video_id>/<name>`.
  - `_resolve_compile(...)` ? already documented as "run at SUBMIT AND in the WORKER (re-resolve)", so it's built to run worker-side.
  - `_maybe_dispatch_remote(...)` ? the DETECT dispatch path (control_plane ? Cloud Run Job), emits `--skip-validation`, forwards `indexing.wasb.*`, bucket-polls a `.done` marker. THIS is the pattern to mirror for compile.
  - `_cloud_available(cfg)`, `reel_object_uri`, `signed_reel_url`.
- `backend/worker/__main__.py` ? the worker entrypoint (`infer` subcommand today). Add a `compile` subcommand (or a `--phase=compile` mode) that runs `_execute_compile` worker-side and writes the reel + a done-marker.
- `backend/compute/cloud_run.py` ? Cloud Run Job dispatch + bucket-poll plumbing.
- `backend/config/models.py` ? config schema (`deployment.compute_target`, `indexing.wasb.*`, `StitchingConfig`); add the phase-placement config here.
- The cloud-serving preset (grep `cloud-serving` under `backend/config/`) ? where `compute_target=cloud_run` is set.
- `backend/main.py::VideoCompiler` (the ffmpeg wrapper; watermark path).
- Tests: `tests/test_api_cloud_dispatch.py`, `tests/test_worker.py`,

`tests/test_compute_backend.py`, `tests/test_async_jobs.py` ? mirror these for the compile-dispatch path.

Deliver (3 parts)

1. **Dispatch compile/stitch to the L4 worker.** Add a `kind='compile'` Cloud Run dispatch path mirroring `_maybe_dispatch_remote`: the control-plane submits a compile job to the `rally-worker` Job with the video_id + segment_ids + output_filename + locale, the worker runs `_execute_compile` (re-resolving via `_resolve_compile`), writes the reel to `<output_root>/highlights/<video_id>/<name>` + a `.done` marker, and the control-plane bucket-polls + returns the same `{status, filepath, download_url}` shape the SPA already expects. Prefer NVENC (`h264_nvenc`/`hevc_nvenc`) on the L4 for the trim/watermark re-encode; keep libx264 as the CPU fallback.
2. **Config-driven phase?machine placement.** Add a per-phase placement knob (e.g. `deployment.phase_placement: {validation: control_plane, segment: cloud_run_l4, compile: cloud_run_l4}` with values `control_plane | cloud_run_l4`), defaulting to today's behavior (validation+compile on control-plane, segment on cloud_run when compute_target=cloud_run) so nothing changes unless configured. Wire compile + detect to consult it; set compile?cloud_run_l4 in the cloud-serving preset. Goal: move a phase between machines by config edit, no code change.
3. **Cheaper watermark** (the dominant term): pre-rendered/overlay-filter or an NVENC filter path so the stitch isn't a full CPU re-encode. If large, land 1+2 first and file/track 3 separately.

Acceptance

- Compile can be dispatched to the L4 by config; control-plane stitch remains available as a config option (both paths tested, mocked ? mirror `tests/test_api_cloud_dispatch.py`).
- Flipping `phase_placement` moves validation / segment / compile between machines with no code change.
- New + existing tests green; `ruff check .` and `mypy` clean (match CI); `python -m tools.leak_scan` and `python -m tools.check_md_links` green; run the FULL suite (e.g. `python run_all_tests.py` / `pytest`), not just changed files; keep coverage ? the repo's `--cov-fail-under` floor.
- Update `docs/COMPUTE_DECOUPLED_SERVING/` docs for the new config + phase-placement model.

Branching safety (concurrent sessions share this checkout ? follow every time)

- The shared checkout may have active sibling WIP (e.g. `feat/validation-result-cache`) and master can move under you. Work in an ISOLATED clone/worktree; never touch the shared working tree or sibling branches.
- `git fetch origin` then branch EXPLICITLY from the remote: `git checkout -b feat/521-compile-on-l4 origin/master`. Re-verify right before commit: `git branch --show-current` + `git log --oneline origin/master..HEAD` must list ONLY your commits (re-cut from `origin/master` if a sibling commit appears).
- Stage explicit paths (`git add <files>`), never `git add -A`/`.`/`-u`.
- Open a PR against `master`, reference #521 (and note it's driven by the 2026-07-08 live-CUJ evidence in the §9 report). End the PR body with the Claude Code attribution line.

Deploy is out of scope ? implement + test + PR only. Ask before any `gcloud run deploy` / traffic change.

2. assistant (2026-07-08T05:04:56.568Z)

I'll start by following the branching-safety and repo-freshness conventions: gather context (issues, evidence, memory/handoff) and check the repo state before making any changes. Let me kick off several independent reads in parallel.

3. user (2026-07-08T05:05:18.796Z)

title: perf/scaling: run compile/stitch on the L4 worker; make phase?machine placement config-driven
state: OPEN
author: avidullu (Avi)
labels: bug, enhancement
comments: 0
assignees:

projects:
milestone:
number: 521
--

Problem

The **compile/stitch** phase runs **in-process on the CPU-only 2-vCPU control-plane** (`backend/orchestration.py::_execute_compile` ? `_LOCAL_COMPUTE_LANE` ffmpeg). **Validation also runs on the control-plane.** On the 2026-07-08 live CUJ (rev `rally-control-plane-00019`, clip "First CUJ.MP4" ? 1.83 GiB / 174.17 s / 4K-class):

Phase	Machine	Wall
Validation (#512 shared-decode)	control-plane 2 vCPU	**183 s**
Detection (WASB)	L4 worker	161.6 s
Compile / stitch (11 clips ? ~80 s reel)	**control-plane 2 vCPU**	**~12m55s**

Stitch was **~45?90 s per ~7 s clip** because the **per-clip branding watermark forces a full 4K re-encode** with no NVENC on 2 vCPU. So a single run pins the control-plane for **~16 min of heavy CPU** (validation + stitch), serialized by `_LOCAL_COMPUTE_LANE`. At even a handful of concurrent users the control-plane saturates and the API becomes unusable ? estimated **>~5 concurrent users renders it useless**.

Ask

1. **Move compile/stitch to the L4 worker** (hardware **NVENC** + 8 vCPU), the way detection already dispatches. The worker already fetches the source, and `_execute_compile` is designed to re-resolve in the worker (`_resolve_compile` is explicitly "run at SUBMIT **AND** in the WORKER"). Add a `kind='compile'` Cloud Run dispatch path.
2. **Make phase?machine placement configurable** ? a per-phase target knob (`validation`, `segment/detect`, `compile/stitch` ? `control_plane | cloud_run_l4 | <other>`) so phases can be relocated **by config, no code change**. Mirror the existing `deployment.compute_target` + `_maybe_dispatch_remote` forwarding pattern (COMPUTE_DECOUPLED_SERVING).
3. **Cheaper watermark** ? pre-rendered overlay / NVENC filter path so stitch isn't a full CPU re-encode (the dominant term above).

Acceptance

- Compile can be dispatched to the L4 by config; control-plane stitch stays available as a config option.
- Flipping config moves validation / detect / compile between machines **without a code change**.
- Measured stitch wall for this reel drops from ~13 min (2 vCPU) to sub-minute-class on L4.

Evidence

`docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_validation-latency_2026-07-07.md` §9 (2026-07-08 live CUJ). Compile timeline: `POST /api/compile` 04:37:26 ? 11× per-clip 4K re-encode+watermark (~67 s each) ? concat ~2 s ? reel mirrored 04:50:21 (`first-cuj-ks-highlight-reel.mp4`, 581.8 MB).

Filed from the 2026-07-08 live-CUJ phase breakdown.

4. user (2026-07-08T05:05:20.540Z)

title: Observability: step-level debug tracing (server + UI) to a low-latency sink (bucket/service)
state: OPEN
author: avidullu (Avi)
labels: enhancement
comments: 0
assignees:
projects:
milestone:
number: 520
--

Problem

Observing a run today means scraping Cloud Run stdout **after the fact** via `gcloud logging read`, then hand-correlating timestamps across **three** separate streams ? the control-plane service, the worker Job, and the in-process compiler ? and there is **no UI-side signal at all**. Reconstructing the phase timeline for the 2026-07-08 live CUJ (upload ? validate ? dispatch ? detect ? ingest ? compile) required manually diffing log timestamps across those streams.

Ask

A **debug/trace mode** (config- or env-gated, **off by default**) where every pipeline step ? and key UI actions ? emits a **structured event** to a **low-latency sink**, so a run can be observed live and replayed:

- **Server phases:** upload (per-part / complete), validation (per-validator wall + verdict), dispatch (job/exec id + cold-start), worker (pull, skip-validation, WASB phase), ingest, compile (per-segment trim, concat, mirror-upload).
- **UI events:** upload start/complete, Process click, poll states, Compile click, download.
- **Event shape:** `{ts, run_id, video_id, phase, step, machine, wall_ms, meta}`.
- **Sink (configurable):**
  - GCS ? one JSONL per run at `gs://<output_root>/_debug/<video_id>/trace.jsonl` (cheap, durable, replayable), and/or
  - a lightweight collector / Pub/Sub topic for **live** viewing (e.g. an `/api/debug/trace/<video_id>` SSE stream, or push to an existing observability service).
- Gate behind e.g. `observability.debug_trace: true` (+ sink target); never on by default (PII / volume).

Why now

The live CUJ (`docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_validation-latency_2026-07-07.md` §9) had to be timed **by hand** across control-plane + worker + compiler logs; a first-class per-run trace (server **and** UI) would make the whole phase breakdown a single artifact and enable low-latency live observation.

Filed from the 2026-07-08 live-CUJ observability gap.

5. user (2026-07-08T05:05:23.491Z)

```
From https://github.com/Khelsutra/badminton-highlight-indexer
 * [new branch]      docs/validation-latency-live-cuj-2026-07-08 -> origin/docs/validation-latency-live-cuj-2026-07-08
---STATUS---
On branch master
Your branch is up to date with 'origin/master'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        score_golden_gate.py
        score_local.py

nothing added to commit but untracked files present (use "git add" to track)
---BRANCH---
master
---LOG---
699d85b feat(validation): cache validation verdict by video_id to skip re-validation on reupload (#518)
71c88ef docs(runlogs): validation-latency closure report for #512 + #513 (cloud-serving) (#519)
115b83d docs(runlogs): record live smoke of the crash-before-marker fast-fail (#515) (#517)
ef41473 fix(cloud-run): fail fast when a worker execution terminates without a done-marker (#515)
f5c9dd9 chore(ui): re-vendor SPA bundle with the hosted compute-picker cloud default (#516)
```

6. assistant (2026-07-08T05:05:27.779Z)

Now let me read the evidence report §9, check memory/handoff, and locate the key architecture files.

7. user (2026-07-08T05:05:31.992Z)

```
===MEMORY DIR===
total 4
drwxr-xr-x 1 avidu 197609 0 Jul  8 10:34 .
drwxr-xr-x 1 avidu 197609 0 Jul  8 10:34 ..
===REPORT===
DEPLOY_REPORT_cloud-serving_2026-07-04.md
DEPLOY_REPORT_cloud-serving_a1-cuj_2026-07-04.md
DEPLOY_REPORT_cloud-serving_a1-e2e_2026-07-04.md
DEPLOY_REPORT_cloud-serving_alpha-all1v1_2026-07-06.md
DEPLOY_REPORT_cloud-serving_cp-rebuild_2026-07-05.md
DEPLOY_REPORT_cloud-serving_gpu-resident-nvdec_2026-07-07.md
DEPLOY_REPORT_cloud-serving_validation-latency_2026-07-07.md
README.md
RUNLOG_cloud-serving_exec-status-fastfail_2026-07-07.md
RUNLOG_truly-local_TEMPLATE.md
```

8. user (2026-07-08T05:05:33.943Z)

```
1       # DEPLOY REPORT ? cloud-serving, validation-latency closure (#512 shared-decode + #513
worker skip-revalidation), 2026-07-07
2
3       > Durable **closure record** for the two validation-latency fixes now live on the
`cloud-serving` path: **#512** (`perf(validators)`: decode `scene_cut`/`blur`/`relevance` in ONE
shared pass instead of ~125 random per-sample seeks across 3 captures) and **#513**
(`fix(serving)`: the worker skips its redundant re-validation when the control-plane already
validated + dispatched, via `--skip-validation`). Both shipped in the live image `rally-
worker@sha256:8045f54e?` (built from `origin/master` @ `f5c9dd9`; #512 `20862d7`, #513 `23c1868`
are ancestors). Measures the before/after on 3 clips spread across codecs/resolutions, all
against the real serving infra. Project `gen-lang-client-0306026826`, region `asia-southeast1`.
Honest-limits (incl. the owner-gated live CUJ) in §6.
4
5       ## 1. TL;DR
6
7       | Stage | Result |
8       |---|---|
9       | Live image carries #512+#513 | ? control-plane rev `00018-qvz` + worker Job both on
`sha256:8045f54e?` (= `f5c9dd9`, ancestors `20862d7`/`23c1868` verified) |
10      | Baseline captured (pre-fix) | ? from prod logs on rev `00017-qbd` (image `09d2edf7` =
`66756ad`, pre-#512/#513) ? no re-run needed |
11      | **#513 worker skip-revalidation** | ? real dispatch-style exec: Video 1 validation
**SKIPPED (0 s)**, runs to **SUCCESS / 13 rallies** ? the blur-reject that failed the live CUJ
is **gone** |
12      | **#512 shared-decode speedup (real L4)** | ? Video 1 re-validation **169.3 s ? 69.7 s
(2.43×)** on identical worker hardware, decision byte-identical (sharpness 11.9 < 20.0 both) |
13      | **#512 speedup (off-box A/B, breadth)** | ? 1080p H.264 **1.95×**, 4K HEVC **3.10×**,
5.3K 10-bit HEVC **3.23×**; every validator decision unchanged OLD?NEW |
14      | Live control-plane "after" number | ? owner-gated (Cloudflare Access) ? projected
**~412 s ? ~170 s (2?3 min)**; recipe in §6 |
15
16      **Bottom line:** the two fixes do exactly what they claim on live serving hardware.
**#513** turns the soft 5.3K clip from a **hard FAIL** (worker re-validated at its default
`min_blur=20`, rejected sharpness 11.9, `rc2`, no reel) into a **PASS with 13 rallies** by
trusting the control-plane's gate. **#512** cuts the frame-sampling validators ~**2?3×** (2.43×
measured on the L4; 1.95?3.23× off-box across codecs) with **zero decision drift** OLD?NEW. The
only number still owner-pending is the live control-plane validation wall on rev 00018 ? its
components are measured and it projects to ~2?3 min from the ~6m52s baseline.
17
18      ## 2. Resources
19      - **Control-plane:** `rally-control-plane` Cloud Run **service**, rev **`rally-control-
plane-00018-qvz`** (100% traffic), image `asia-southeast1-docker.pkg.dev/gen-lang-
client-0306026826/rally/rally-worker@sha256:8045f54ea9fcbf?` (built from `f5c9dd9`),
`DEPLOYMENT_PROFILE=cloud-serving`, CPU-only **2 vCPU**, edge-auth ON (`edge_auth_enabled=true`,
`cf_team_domain=khelsutra.cloudflareaccess.com`).
```

```
20      - **Worker:** `rally-worker` Cloud Run **Job**, same image `sha256:8045f54e?`, 8 vCPU /
32 GiB / 1× NVIDIA **L4**, `WASB_WEIGHTS=/gcs/models/wasb_badminton_best.pth.tar`,
`WASB_DECODE_BACKEND=nvdec_resident`, GCSFuse mount of `gs://khelsutra-rally-serving`.
21      - **Baseline (pre-fix) surface:** control-plane rev **`00017-qbd`** / worker image
`sha256:09d2edf7?` (= `66756ad` = #507+#508+#509, both **pre-#512/#513**) ? the source of the
prod baseline logs below (2026-07-07, before the 16:15 rev-00018 cutover).
22      - **Corpus:** Video 1 `gs://khelsutra-rally-
serving/videos/uploads/702636eed76145e88aa59c3000c7e415/Video 1 - Badminton.MP4` (1.41 GiB,
5312×2988 yuv420p10le HEVC, 3019 frames, MD5 `8fe6b719603dd7a7b2b6ef543139a4a3`);
`gs://khelsutra-rally-corpus/videos/golden_1080p/Boxhill_Doubles.mp4` (1080p H.264);
`gs://khelsutra-rally-corpus/videos/golden/GX010128.mp4` (2.18 GiB, 4K HEVC, 45298 frames).
23
24      ## 3. Exact commands
25      ```bash
26      # --- Provenance: prove the live image carries both fixes ---
27      gcloud run services describe rally-control-plane --region asia-southeast1 \
28        --format="value(status.latestReadyRevisionName,
spec.template.spec.containers[0].image)"   # 00018-qvz  ?@sha256:8045f54e?
29      git merge-base --is-ancestor 20862d7 f5c9dd9 && echo "#512 in live image"   # yes
30      git merge-base --is-ancestor 23c1868 f5c9dd9 && echo "#513 in live image"   # yes
31
32      # --- #513 proof: worker A/B on the LIVE image (Video 1 already staged in the serving
bucket) ---
33      V1="gs://khelsutra-rally-serving/videos/uploads/702636eed76145e88aa59c3000c7e415/Video 1
- Badminton.MP4"
34      # Run A ? WITH --skip-validation (exactly what orchestration._maybe_dispatch_remote
emits on dispatch):
35      gcloud run jobs execute rally-worker --region asia-southeast1 --async \
36        --args="^@@^infer@@--video-uri=$V1@@--out-uri=gs://khelsutra-rally-
serving/_valclose/v1-skipval@@--segmenter=wasb_hybrid@@--skip-validation@@--
set=deployment.compute_target=local_gpu@@--set=indexing.detector_impl=native@@--
set=indexing.skip_ai_handoff=true@@--set=indexing.wasb.batch_size=64@@--
set=indexing.wasb.prefetch_batches=4@@--set=indexing.wasb.timeout_sec=0"
37      # Run B ? WITHOUT --skip-validation (worker re-validates with the NEW #512 validators ?
still blur-rejects):
38      gcloud run jobs execute rally-worker --region asia-southeast1 --async \
39        --args="^@@^infer@@--video-uri=$V1@@--out-uri=gs://khelsutra-rally-serving/_valclose/v
1-reval@@--segmenter=wasb_hybrid@@--set=deployment.compute_target=local_gpu@@--
set=indexing.detector_impl=native@@--set=indexing.skip_ai_handoff=true"
40
41      # --- #512 timing: bracket "pulled ? -> in.mp4"  ?  "validation FAILED/SKIPPED"  ?
"Phase 2 (WASB) completed" ---
42      gcloud logging read 'resource.type="cloud_run_job" AND
labels."run.googleapis.com/execution_name"="rally-worker-<exec>"' \
43        --freshness 1d --format="value(timestamp, textPayload)"
44
45      # --- #512 off-box A/B (isolated clones; run_all decodes the SAME sample indices in both
trees) ---
46      git clone --local --no-hardlinks <repo> repo_old && git -C repo_old checkout 23c1868   #
pre-#512 (per-sample seeks)
47      git -C repo_old worktree add --detach repo_new f5c9dd9
# live image src (shared pass)
48      python valbench.py <repo_old|repo_new> <video> <label> 2   # times
registry.run_all(video, load_config())
49      ```
50
51      ## 4. Run results
52
53      ### 4a. Video 1 (5312×2988 10-bit HEVC, 3019 frames) ? the clip that motivated #512/#513
54
55      | Metric | BEFORE ? pre-fix (rev `00017`, img `09d2edf7`) | AFTER ? #512+#513 live (rev
`00018`, img `8045f54e`) |
56      |---|---|---|
57      | Control-plane validation | **411?412 s** (6m52s) ? passed (baked `min_blur=8.0`) | ?
live owner-gated; **projected ~170 s (2?3 min)** |
```

Worker (re-)validation	**169.3 s** ? ? **BLUR-REJECT** (sharpness 11.9 < default 20.0)	**0 s ? SKIPPED** (`--skip-validation`, #513)
? counterfactual: worker re-val if NOT skipped	*(169.3 s, OLD validators)*	**69.7 s** (NEW #512 validators, **2.43×**, same 11.9<20 decision)
End-to-end	~10m31s wall ? **FAILED** (0 rallies, no reel)	worker ? **SUCCESS** in ~2m38s exec ? **13 rallies** (reel = owner CUJ)
Rallies produced	**0** (rejected before detection)	**13** (`report.json` `status=success`, 13 segments)
Pass / fail	? **FAIL**	? **PASS**

Prod baseline evidence (rev 00017, 2026-07-07, unmodified): control-plane `13:49:18 Running validations ? 13:56:10 Video passed checks` = 411.4 s (and `14:03:56 ? 14:10:48` = 411.9 s). Worker exec `rally-worker-2lbz2` (dispatched by the 14:03 run, OLD image): `pulled 14:11:35 ? validation FAILED: blur_check ? sharpness 11.9 below 20.0 @14:14:24 ? exit(2)` ? a **successful GPU dispatch that failed after the fact**, the exact CUJ #513 fixes.

After evidence (live image `8045f54e`, 2026-07-07):

- `rally-worker-k29rk` (**Run A, --skip-validation**): `pulled 17:09:24 ? [worker] validation SKIPPED (--skip-validation) 17:09:27 ? native WASB (torch 2.6.0+cu124, cuda True) ? Phase 2 (WASB) completed in 100.9 s, 13 candidate windows (1510/3019 visible) ? emitted 13 rallies ? exit(0)`. Exec wall 157.7 s; `gpu_active`?100.9 s (WASB); `bytes_out`=`report.json`+`intervals.csv`. Output `gs://?/_valclose/v1-skipval/outputs/8fe6b719?/`, `status=success`, 13 segments.
- `rally-worker-t8llx` (**Run B, re-validation on the NEW image**): `pulled 17:09:29 ? validation FAILED: blur_check ? sharpness 11.9 below 20.0 @17:10:39 ? exit(2)`. Re-val wall **69.7 s** vs the OLD image's 169.3 s = **2.43× on identical L4 hardware**, and the **decision is byte-identical** (11.9 < 20.0) ? #512 is pure speed, zero behavior change. (This run also shows *why* #513 is needed: absent the skip, the worker's default `min_blur=20` still rejects the clip the control-plane passed at 8.0.)

4b. Off-box #512 A/B ? codec/resolution breadth (16-core local box, cv2 4.13.0, warm, `run_all`)

Isolated clones at `23c1868` (OLD, per-sample seeks) vs `f5c9dd9` (NEW, shared pass); identical sample indices ? a fair A/B. **Every validator decision is identical OLD?NEW** on every clip (no PASS/FAIL drift from the fix).

Clip	codec / res	OLD (warm)	NEW (warm)	Speedup	decisions OLD?NEW
Boxhill_Doubles	1080p H.264	10.91 s	5.60 s	**1.95×**	6/6 identical (all PASS)
GX010128	4K HEVC (45298 fr)	35.76 s	11.54 s	**3.10×**	6/6 identical (all PASS)
Video 1	5.3K 10-bit HEVC	173.20 s	53.62 s	**3.23×**	6/6 identical (all PASS?)

? On this Windows/cv2 box the 10-bit-HEVC decode yields a Laplacian variance **above** 20 so blur PASSES off-box, whereas the Linux serving stack measures **11.9** and rejects. That platform decode-stack difference is **orthogonal to #512** (which changes only *which frames are sampled*, identically in both trees); the authoritative blur verdict and the real speedup come from the L4/prod (§4a). Off-box contributes only the OLD?NEW **ratio**, which is decode-stack-independent.

The win grows with per-frame decode cost (random-seek re-decode is what #512 removes), and it **triangulates** with the real-L4 Video 1 A/B (2.43×): the off-box ratios bound the control-plane projection.

4c. Control-plane projection (2-vCPU)

Baseline **~412 s**. Applying the L4-measured **2.43×** ? **~170 s (~2m50s)**; the off-box ratios (1.95?3.23×) bracket **~2?3 min**. Matches the pre-fix design estimate (~6.5 min ? ~2 min). Exact live number is owner-gated (§6).

5. Rollout state

- **Already deployed ? this is a measurement/closure record, not a deploy.** #512+#513 went live with the `8045f54e` image cutover (control-plane rev `00018-qvz`, worker Job same

digest) on 2026-07-07. No infra change was made for this report.
89   - **Rollback (pre-existing levers):** control-plane `gcloud run services update-traffic rally-control-plane --region asia-southeast1 --to-revisions rally-control-plane-00017-qbd=100`; worker `gcloud run jobs update rally-worker --region asia-southeast1 --image ?/rally-worker@sha256:09d2edf7?`. Both revert to the pre-#512/#513 image.
90   - **Scratch outputs** written under `gs://khelsutra-rally-serving/_valclose/` (isolated prefix; safe to delete).
91
92   ## 6. Honest limits / not-verified
93   - **Live end-to-end CUJ is owner-gated.** `POST /api/process` on rev 00018 requires a Cloudflare-Access JWT (`edge_auth_enabled=true`; a direct call returns **401** `missing Cf-Access-Jwt-Assertion header`). So the two ? cells ? the **live control-plane validation wall on rev 00018** and the **stitched reel** ? need the owner to drive one upload (or hand over a CF-Access JWT / service token). Everything else here is measured on the real serving infra. **Recipe to close them:** upload "Video 1 - Badminton.MP4" via the SPA (compute=cloud), then `gcloud logging read '?service_name="rally-control-plane"?("Running validations" OR "passed checks")'` for the wall, and confirm the dispatched worker exec logs `validation SKIPPED` + emits rallies + the reel compiles. Expect CP validation ~2?3 min and a PASS.
94   - **#513's worker win is shown via a dispatch-faithful direct exec**, not a control-plane-initiated dispatch (same auth gate). The `--skip-validation` flag, its emission in `orchestration._maybe_dispatch_remote`, and its handling in `worker.cmd_infer` are unit-tested on master; Run A exercises the exact worker code path with the exact flag.
95   - **Rallies, not a reel.** Run A used `skip_ai_handoff=true` (secret-free) so the 13 "rallies" are local candidate windows (no Gemini refine, no stitched mp4) ? this validates decode+detect+windowing end-to-end and that the clip **passes**; the reel stitch is the owner-CUJ step above. The 13 segments are its inputs.
96   - **Off-box absolute times are from a Windows 16-core box (cv2/FFmpeg), not the 2-vCPU Linux control-plane** ? so the **ratios** transfer (the change is algorithmic: seeks?shared forward-hybrid pass), the absolutes don't, and (per ?) the 10-bit blur *verdict* can differ; the control-plane absolute is anchored to the prod 412 s baseline, not the local numbers. Video 1 off-box is a cross-check of the authoritative real-L4 2.43×.
97   - **#515** (crash-before-marker fast-fail) is NOT in this image (`ef41473` post-dates `f5c9dd9`); it's unrelated to validation latency and out of scope for this closure.
98
99   ## 7. Cost
100   - 2 L4 worker execs (Run A 157.7 s + Run B 109.6 s ? 268 s of L4 time) ? **~$0.5?0.7**. Off-box A/B: **$0** (local). No VM, no image build. GCS read for the 3 downloaded clips (~5.1 GiB) is intra-project.
101
102   ## 8. Closure verdict
103   ? **CLOSED ? #512 and #513 are proven on live cloud-serving hardware.** #513 converts the motivating soft-focus 5.3K clip from a post-dispatch FAIL into a 13-rally PASS by eliminating the worker's redundant re-validation (169 s + wrongful blur-reject ? 0 s); #512 speeds the frame-sampling validators **2.43× on the real L4** and **1.95?3.23× off-box across codecs**, with **zero decision drift** OLD?NEW on every clip. The only residual is the live control-plane validation wall on rev 00018 (owner-gated, projected ~2?3 min from ~6m52s) ? its components are all measured. Validation-latency work is done.
104

9. assistant (2026-07-08T05:05:41.291Z)

The report ends at §8, but §9 (the 2026-07-08 live CUJ) is referenced. A new remote branch `docs/validation-latency-live-cuj-2026-07-08` just appeared ? §9 likely lives there. Let me check.

10. user (2026-07-08T05:05:43.910Z)

===does §9 exist on the new remote branch?===

9. Update (2026-07-08) ? live SPA CUJ closes the two owner-gated cells (on rev `00019`)

The two ? cells in §4a (the **live control-plane validation wall** and the **end-to-end reel**) were owner-gated on the Cloudflare-Access `POST /api/process`. On **2026-07-08** the owner drove one real SPA upload (compute=**Cloud**) end-to-end, so both are now measured on live serving infra. Mirrors the gpu-resident report's "§5 Update" append ? the body above is unchanged; this

section records what the live run added.

Two honest deltas from the recipe (state moved after this report's 2026-07-07 image cutover):
1. **Live rev rolled `00018` ? `00019`.** Traffic is 100% on **`rally-control-plane-00019-96f`**, image `rally-worker@sha256:c6393d27b4ae?` (tag `:latest`), deployed 2026-07-07T17:06Z ? ~51 min after the `00018`/`8045f54e` cutover §2 documents; the worker Job is on the same `:latest`=`c6393d27` digest (both surfaces match). It's an undocumented forward-bump, but #512 (`20862d7`) and #513 (`23c1868`) are ancestors of `origin/master`, `f5c9dd9` (the `00018` src) is an ancestor of master, and the run **empirically** shows both fixes live (one-pass validation wall + worker `validation SKIPPED`). Measured on what is actually serving.
2. **A fresh clip, not Video 1.** The upload was **"First CUJ.MP4"** ? **1.83 GiB** (1,960,043,219 B), source duration **174.17 s** (?84 Mbps, 4K-class ? *heavier* than Video 1's 1.41 GiB), video_id `2f31b45e?`. So the live wall **corroborates** the Video-1 projection on a comparably-heavy clip; it is **not** a re-measurement of Video 1's 412 s?~170 s A/B (a different clip has a different absolute wall).

Full phase breakdown (UTC 2026-07-08, rev `00019`):

#	Phase	Window	Wall	Machine	Notes
1	Upload (client?server, 80-part multipart)	04:25:56 ? 04:28:51	**~2m55s**	client uplink	1.83 GiB @ ~84 Mbps ? owner bandwidth, not infra
2	Accept ? validation start (stage)	04:28:52 ? 04:29:22	~30 s	control-plane	stage upload to serving bucket
3	**Validation (#512 shared-decode)** ?	04:29:22 ? 04:32:25	**183 s (3m03s)**	control-plane 2 vCPU	one-pass scene_cut/blur/relevance; PASSED
4	Dispatch ? worker ready (cold start)	04:32:25 ? 04:33:20	~55 s	Cloud Run Job	image pull + GCSFuse mount + torch/cuda init
5	**Worker skip-validation (#513)** ?	04:33:20 ? 04:33:24	~4 s	L4	`validation SKIPPED`, 0 re-validation
6	Detection / segment (WASB)	04:33:24 ? 04:36:20	**161.6 s**	L4 (cuda)	15 candidate windows, 2561 visible pts
7	Ingest + finalize	04:36:20 ? 04:36:24	~4 s	control-plane	intervals?DB, `status=success`
?	(user reviews rallies, clicks Compile)	04:36:24 ? 04:37:26	~62 s idle	?	human-in-the-loop
8	**Compile / stitch** ? slow	04:37:26 ? 04:50:21	**~12m55s**	control-plane 2 vCPU	11 clips, ~67 s/clip 4K re-encode+watermark; concat ~2 s; mirror ~11 s

Server pipeline (process?success, excl. upload+idle): **7m32s**. Full CUJ (upload?reel): **~24m25s** wall.

Cell 1 ? control-plane validation AFTER (live): **183 s (3m03s)** on rev `00019`, PASSED ? the #512 shared-decode validators on the real 2-vCPU serving path, in the projected **2?3 min** band on a clip *heavier* than Video 1 (pre-#512 per-sample seeks were the ~6m52s-class baseline). Evidence: `Running validations 04:29:22.789 ? Video passed checks 04:32:25.780`.

Cell 2 ? end-to-end reel (live): `/api/compile` stitched **11 of the 15** detected rallies (0.5 s pad, per-clip branding watermark) from the 174.17 s source ? **`gs://khelsutra-rally-serving/highlights/2f31b45e?/first-cuj-ks-highlight-reel.mp4`** (**581.84 MiB**, saved 04:50:10 *"Dynamic highlight reel saved (581.8 MB)"*, mirrored 04:50:21, served via `/api/highlights` 307). **Reel ? produced end-to-end on the live control-plane** ? source fetch ? per-clip trim+watermark ? concat ? mirror-upload, all exercised.

Bonus ? #513 proof upgraded from the §6 dispatch-faithful *direct exec* to a genuine **control-plane-initiated dispatch** (`rally-worker-64bhj`): `_maybe_dispatch_remote` emitted `--skip-validation` on a live SPA run and the worker honored it (0 re-validation).

? Finding ? stitch is the scaling bottleneck (follow-ups filed). The compile ran **~13 min on the CPU-only 2-vCPU control-plane** (~45?90 s per ~7 s clip: the watermark forces a full 4K re-encode, no NVENC). With validation (183 s) *also* on the control-plane and serialized by `_LOCAL_COMPUTE_LANE`, one run pins the control-plane for ~16 min of heavy CPU ? it won't scale past a few concurrent users. Filed: **#521** (move compile/stitch to the L4 worker + make phase?machine placement config-driven) and **#520** (step-level debug tracing, server+UI, to a

low-latency sink ? this breakdown had to be hand-correlated across 3 log streams).

Cost (heuristic): one L4 detect exec (`rally-worker-64bhj`, ~222 s) ? **$0.15?0.55** (rate-dependent, dominant) + control-plane CPU for validation+stitch (~960 s @ 2 vCPU) ? **~$0.05** + reel egress (581 MB, if downloaded to internet) ? **~$0.07** + storage/ops **< $0.02** + Gemini **$0** (skip_ai_handoff). **? $0.3?0.7 total**, ~one L4 exec as projected ? but the stitch's real cost is **concurrency** (~16 min of control-plane heavy CPU), not dollars.

Net: both owner-gated cells are now live-measured on rev `00019` ? validation wall **183 s** (corroborating the 2?3 min projection) and the reel compiles end-to-end. #512 one-pass validation and #513 skip-validation-on-real-dispatch are now both directly observed in prod. The Video-1 412 s?~170 s A/B remains a projection by design (different, heavier clip).

11. assistant (2026-07-08T05:05:49.890Z)

I have the full picture from §9. Now let me study the core architecture files. Let me read them in parallel.

12. user (2026-07-08T05:05:51.195Z)

```
1       """Processing / compile orchestration, extracted from ``backend.main`` (P23; re-cut of
#390).
2
3       The full hash -> validate -> segment -> report pipeline (``_execute_processing``), the
4       Cloud Run dispatch + ingest path (``_maybe_dispatch_remote`` /
``_ingest_remote_segments``),
5       and the shared compile resolution + worker handler (``_resolve_compile`` /
``_execute_compile``).
6       Extracted verbatim from ``backend.main`` to shrink the orchestrator; behavior is
unchanged.
7
8       SEAM DISCIPLINE (same convention as ``backend.api.job_worker``): anything that must
remain
9       monkeypatchable-on-``backend.main`` or that lives in ``backend.main`` (the transitional
10      ``db_instance``/``global_config`` globals and the ``ProcessRequest`` model) is resolved
through
11      the ``backend.main`` module object at call time (``import backend.main as _main``),
never as a
12      bare local. ``backend.main`` re-exports every name below, so
``monkeypatch.setattr(backend.main,
13      ...)`` and ``import backend.main as m; m._execute_processing(...)`` keep working
unchanged.
14      This module must NOT import ``backend.main`` at module level (main imports us for the
re-export).
15      Routed seams (rebind-patched on ``backend.main`` by tests): ``global_config``,
``db_instance``,
16      ``ProcessRequest``, ``compute_video_id``, ``emit_indexer_report``, ``VideoCompiler``.
17      Object-attribute patches (``m.storage.fetch``, ``m.segmenter_registry.get``, ...) hit
the shared
18      objects and need no routing.
19      """
20
21      import csv
22      import json
23      import logging
24      import math
25      import os
26      import re
27      import tempfile
28      import threading
29      from typing import TYPE_CHECKING, Any, Dict, List, Optional, Tuple
30      from urllib.parse import quote
31
32      from backend import storage
33      from backend.config import AppConfig, StitchingConfig
```

```
    from backend.pipeline.segmenters.base import segmenter_registry
    from backend.pipeline.stitcher.watermark_i18n import localize_watermark
    from backend.pipeline.validators.base import ValidationResult, registry
    from backend.results import Failure
    from backend.utils.paths import output_base, safe_output_filename

    if (
        TYPE_CHECKING
    ):  # runtime import would be circular (main imports us for the re-export)
        from backend.main import ProcessRequest  # noqa: F401

    logger = logging.getLogger(__name__)

    # #466: serialize CPU/GPU-heavy LOCAL work ? in-process segmentation and the ffmpeg
compile stitch ?
    # so that raising the drain's worker count (to overlap I/O-bound REMOTE Cloud Run
dispatch/poll) never
    # accidentally parallelizes local work. A single self-hosted box serializes GPU work and
Gemini is
    # rate-fragile; a remote dispatch is a bucket-poll and never holds this lane, so N
remote jobs overlap
    # while local/compile jobs stay 1-at-a-time. It's a plain mutex (no nesting) ? no
deadlock; when the
    # drain is serial (max_workers=1) it's an uncontended no-op = today's behaviour.
    _LOCAL_COMPUTE_LANE = threading.BoundedSemaphore(1)


    def _finalize_reingest(db, video_id: str, segments, play_wm: int, cand_wm: int) -> None:
        """Commit a (re)segmentation with destroy-AFTER-confirm semantics.

        On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
        fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
and
        keep the prior good results intact ? a failed/empty re-run never destroys prior
data.
        """
        if segments:
            db.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
        else:
            db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)


    def _validation_failure_message(failures: List[Any]) -> str:
        details = []
        for failure in failures:
            name = getattr(failure, "validator_name", None) or "validation"
            message = (getattr(failure, "message", None) or "").strip()
            details.append(f"{name}: {message}" if message else str(name))
        if not details:
            return "Video failed validation checks."
        return "Video failed validation: " + "; ".join(details)


    def _report_config_for_input(config: Any, raw_input: str) -> Any:
        """Return report config that keeps remote-input reports out of disposable
scratch."""
        storage_config = (
            config.get("storage")
            if isinstance(config, dict)
            else getattr(config, "storage", None)
        )
        try:
            if storage.input_is_local(raw_input, storage_config):
                return config
        except ValueError:
```

```
            return config
        try:
            cfg = (
                config.model_dump(mode="json")
                if hasattr(config, "model_dump")
                else dict(config or {})
            )
        except Exception:
            cfg = {}
        report = dict(cfg.get("report") or {})
        if report.get("output_dir"):
            return config
        report["output_dir"] = os.path.join(output_base(config), "reports")
        cfg["report"] = report
        return cfg


    def _ingest_remote_segments(
        db, video_id: str, outputs_uri: str, storage_config: dict, *, max_rows: int = 20000
    ) -> int:
        """Read a cloud job's ``intervals.csv`` (start,end,shot_count,ending_reason) from
the bucket and
        persist each rally via the SAME ``db.add_play_segment`` the in-process segmenters
call ? so
        ``/api/query`` + ``/api/compile`` read the rows with zero knowledge of where they
were produced.
        Returns the count ingested. Mirrors ``cloud_run._read_json``'s fetch-to-a-real-dst
pattern (cloud
        backends REQUIRE a dst). Hardened against a hostile/corrupt worker output: a
malformed row is
        skipped, non-finite / overflowing values are rejected, the loop is bounded, and the
temp dir is
        cleaned. May raise (missing/unreadable CSV, fetch error) ? the caller treats that as
a job failure
        and prunes any partial rows (destroy-AFTER-confirm)."""
        uri = f"{outputs_uri.rstrip('/')}/intervals.csv"
        ref = storage.parse_uri(uri)
        n = 0
        with tempfile.TemporaryDirectory(prefix="cr-ingest-") as td:
            local = storage.fetch(
                uri, os.path.join(td, ref.name or "intervals.csv"), config=storage_config
            )
            with open(local, newline="", encoding="utf-8") as f:
                for row in csv.DictReader(f):
                    if n >= max_rows:
                        logger.warning(
                            "[API] cloud ingest hit the %d-row cap (%s) ? ignoring the
rest.",
                            max_rows,
                            uri,
                        )
                        break
                    try:
                        start, end = float(row["start"]), float(row["end"])
                    except (KeyError, TypeError, ValueError):
                        continue
                    if not (math.isfinite(start) and math.isfinite(end)):
                        continue  # reject inf/nan times rather than persist a junk rally
                    meta: Dict[str, Any] = {
                        "source": "cloud_run",
                        "ending_reason": (row.get("ending_reason") or "").strip(),
                    }
                    sc = (row.get("shot_count") or "").strip()
                    if sc:
                        try:
```

```
                            meta["shot_count"] = int(
                                float(sc)
                            )  # OverflowError on 'inf'/'1e400'
                        except (ValueError, OverflowError):
                            meta["shot_count"] = sc
                    db.add_play_segment(video_id, start, end, metadata=meta)
                    n += 1
        return n


    # Trajectory segmenters whose IN-PROCESS detect needs model weights present on THIS box.
A hosted
    # cloud-serving control-plane bakes NO detector weights into its image (only the worker
Cloud Run Job
    # GCSFuse-mounts them), so forcing compute="local" for one of these there can never
succeed ? the
    # guardrail in _maybe_dispatch_remote fails such a request FAST instead of the cryptic
deep
    # "WASB weights not found". (Names are the registered segmenter identities; see
backend/pipeline/segmenters.)
    _WEIGHTS_BACKED_SEGMENTERS = frozenset({"wasb_hybrid", "tracknet_hybrid",
"fusion_hybrid"})


    def _local_detector_weights_present(cfg, seg_name: str) -> bool:
        """Whether the trajectory weights the IN-PROCESS ``seg_name`` detector needs are
present + readable
        on THIS box. Mirrors the native/WSL runners' resolution (env override ?
``indexing.<det>.weights_path``)
        and the ``/api/capabilities`` probe, so the guardrail agrees with the healthcheck
that would
        otherwise fail deep in the run. Fusion follows its configured substrate (WASB
default / TrackNet).
        A non-weights segmenter returns True (nothing local to check)."""
        name = (seg_name or "").strip().lower()
        if name not in _WEIGHTS_BACKED_SEGMENTERS:
            return True
        idx = getattr(cfg, "indexing", None)
        substrate = (
            getattr(getattr(idx, "fusion", None), "substrate", "") or "wasb"
        ).strip().lower()
        if name == "tracknet_hybrid" or (name == "fusion_hybrid" and substrate ==
"tracknet"):
            path = getattr(getattr(idx, "tracknet", None), "weights_path", "") or ""
        else:  # wasb_hybrid, or fusion on the (default) WASB substrate
            path = (
                os.environ.get("WASB_WEIGHTS")
                or getattr(getattr(idx, "wasb", None), "weights_path", "")
                or ""
            )
        return bool(path) and os.path.exists(os.path.expanduser(path))


    def _cloud_available(cfg) -> bool:
        """Whether THIS server can satisfy a per-request ``compute="cloud"`` dispatch (item
3).

        True when the control plane is wired for Cloud Run: either the configured default
target is
        already ``cloud_run`` (the operator chose cloud serving), or the Cloud Run env
coordinates
        (``CLOUD_RUN_JOB`` + ``GOOGLE_CLOUD_PROJECT``) and an output bucket
(``storage.output_root``)
        are present so a forced per-request override can actually dispatch + collect a
result.
```

```
197
198         Surfaced in ``/api/capabilities`` so the SPA only offers the cloud toggle when it
would work."""
199         deployment = getattr(cfg, "deployment", None)
200         if (getattr(deployment, "compute_target", "") or "") == "cloud_run":
201             return True
202         has_job = bool(
203             os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT")
204         )
205         out_root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
206         return bool(has_job and out_root)
207
208
209     def _wait_progress_message(seg_name: str, elapsed_sec: float) -> str:
210         """The live job.progress line while a cloud GPU run is in flight (#482). Honest and
211         minute-granular: the control plane knows ELAPSED time, not frames ? frame-level %
comes
212         with the worker-side heartbeat (the #482 follow-up). Under a minute reads as
"starting"
213         (queue/cold-start), so the very first tick already shows movement."""
214         minutes = int(elapsed_sec) // 60
215         if minutes < 1:
216             return f"segmenting ({seg_name}) on cloud GPU ? starting up"
217         return f"segmenting ({seg_name}) on cloud GPU ? {minutes} min elapsed"
218
219
220     def submit_time_video_id(input_ref, cfg) -> Optional[str]:
221         """The video's md5 (the canonical ``video_id``) resolvable CHEAPLY at submit time ?
gs://
222         object metadata only, one stat, no bytes (#484). ``None`` ? unknown: local files
keep
223         deferred hashing on the drain (DEFERRED_HARDENING #16), and composite objects carry
no
224         ``md5Hash``. Never raises ? a metadata miss must not block a submit."""
225         if getattr(input_ref, "backend", "") != "gcs":
226             return None
227         sc = _storage_settings(cfg)
228         if not sc:
229             return None
230         try:
231             st = storage.get_storage("gcs", config=sc).stat(input_ref)
232             return getattr(st, "md5", None) or None
233         except Exception:  # noqa: BLE001 ? best-effort identity, the drain still hashes
234             logger.info(
235                 "[API] submit-time md5 unavailable for %s (metadata stat failed)",
236                 getattr(input_ref, "uri", input_ref),
237                 exc_info=True,
238             )
239             return None
240
241
242     def _cached_process_payload(
243         video_id: str, validation_results, segments, requested_compute
244     ) -> Dict[str, Any]:
245         """The /api/process result for a video whose detection already exists ? ONE shape
for
246         both producers (the drain's DB-cache short-circuit and the submit-time cache,
#484)."""
247         return {
248             "video_id": video_id,
249             "status": "success",
250             "message": (
251                 f"Reused {len(segments)} cached play segments. "
252                 "Pass force=true to reprocess this video."
253             ),
```

```
            "results": validation_results or [],
            "play_segments": segments,
            "cached": True,
            "compute": {"path": "cached", "requested": requested_compute},
        }


    def _read_bucket_json(uri: str, storage_config: dict) -> Optional[Dict[str, Any]]:
        """Fetch + parse one small JSON object (cloud backends require a real dst ? the
        ``_ingest_remote_segments`` pattern). Raises on fetch/parse errors; callers
decide."""
        ref = storage.parse_uri(uri)
        with tempfile.TemporaryDirectory(prefix="cr-report-") as td:
            local = storage.fetch(
                uri, os.path.join(td, ref.name or "report.json"), config=storage_config
            )
            with open(local, encoding="utf-8") as f:
                loaded = json.load(f)
        return loaded if isinstance(loaded, dict) else None


    # One re-hydrate at a time per video_id (#492 review blocker 1): two concurrent cache
hits
    # from an empty DB must not both ingest intervals.csv ? the loser waits, then finds the
    # winner's rows in the DB and reuses them. Process-wide is enough: the CP is single-
instance
    # (maxScale=1, spec D8) and SQLite serializes cross-process writers anyway.
    _REHYDRATE_LOCKS: Dict[str, threading.Lock] = {}
    _REHYDRATE_LOCKS_GUARD = threading.Lock()


    def _rehydrate_lock(video_id: str) -> threading.Lock:
        with _REHYDRATE_LOCKS_GUARD:
            return _REHYDRATE_LOCKS.setdefault(video_id, threading.Lock())


    def probe_process_cache(db, cfg, video_id: str) -> bool:
        """READ-ONLY: does completed detection for ``video_id`` already exist (DB or
bucket)?

        Safe to call before the atomic submit claim ? it writes nothing (#492 review blocker
1
        ordering). Used to decide whether a submit can skip the local-scratch fetch guard
        (blocker 2): a True probe means the request should never need CP scratch. Best-
effort:
        errors report False (? the normal dispatch path with its guards)."""
        try:
            existing = db.get_video(video_id)
            if (
                existing
                and existing.get("status") == "processed"
                and db.query_play_segments(video_id, {})
            ):
                return True
            sc = _storage_settings(cfg)
            if not sc or (sc.get("backend") or "local") == "local":
                return False
            output_root = (sc.get("output_root") or "").rstrip("/")
            if not output_root:
                return False
            report_uri = f"{output_root}/outputs/{video_id}/report.json"
            if not storage.exists(report_uri, config=sc):
                return False
            report = _read_bucket_json(report_uri, sc)
            return bool(report and report.get("status") == "success")
```

```
        except Exception:  # noqa: BLE001 ? a failed probe degrades to the guarded dispatch
path
            logger.warning(
                "[API] cache probe failed for %s ? treating as a miss", video_id,
exc_info=True
            )
            return False


    def cached_process_result(
        db, cfg, video_id: str, video_path: str, requested_compute=None
    ) -> Optional[Dict[str, Any]]:
        """A ready-to-serve /api/process result when this video's detection ALREADY exists ?
        the DB first, then the bucket (``outputs/<md5>/report.json`` with ``status=success``
        re-ingests via ``_ingest_remote_segments``): **the DB is a cache, the bucket is the
        source of truth** (#484 instant re-runs; the #485 re-hydrate mechanism). ``None`` ?
no
        completed work found (or the check failed) ? the caller dispatches normally. The
bucket
        branch mirrors the real cloud ingest exactly (watermarks + ``_finalize_reingest``),
so a
        partial earlier ingest is replaced, never double-counted.

        Concurrency (#492 review blocker 1): the whole check-then-ingest runs under a
        per-``video_id`` lock ? a concurrent caller blocks, then finds the winner's rows via
the
        in-lock DB re-check and reuses them (one segment set, ever). Callers must hold a
CLAIMED
        job before invoking (writes happen post-claim)."""
        try:
            with _rehydrate_lock(video_id):
                existing = db.get_video(video_id)
                if existing and existing.get("status") == "processed":
                    segs = db.query_play_segments(video_id, {})
                    if segs:
                        return _cached_process_payload(
                            video_id,
                            existing.get("validation_results", []),
                            segs,
                            requested_compute,
                        )
                sc = _storage_settings(cfg)
                if not sc or (sc.get("backend") or "local") == "local":
                    return None
                output_root = (sc.get("output_root") or "").rstrip("/")
                if not output_root:
                    return None
                outputs_uri = f"{output_root}/outputs/{video_id}"
                report_uri = f"{outputs_uri}/report.json"
                if not storage.exists(report_uri, config=sc):
                    return None
                report = _read_bucket_json(report_uri, sc)
                if not report or report.get("status") != "success":
                    return None
                if not video_path:
                    # Read-path re-hydrate (#485): a wiped DB has no source record. The
report
                    # carries the worker's --video-uri (P3); older reports fall back to the
                    # staged-canonical videos/<md5>/ prefix; else "" ? the segments still
                    # recover, and the next same-md5 submit re-binds the truthful path via
                    # add_video's upsert.
                    video_path = str(report.get("video_uri") or "") or _first_object_under(
                        f"{output_root}/videos/{video_id}/", sc
                    )
                play_wm, cand_wm = db.segment_watermarks(video_id)
```

```
                db.add_video(video_id, video_path, "processed", [])
                n = _ingest_remote_segments(db, video_id, outputs_uri, sc)
                if n <= 0:
                    return None
                segments = [
                    s
                    for s in db.query_play_segments(video_id, {})
                    if int(s.get("id", 0)) > play_wm
                ]
                _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
                if not segments:
                    return None
                logger.info(
                    "[API] re-hydrated %d segments for video_id=%s from %s (submit-time
cache hit)",
                    n,
                    video_id,
                    outputs_uri,
                )
                return _cached_process_payload(video_id, [], segments, requested_compute)
        except Exception:  # noqa: BLE001 ? cache misses must degrade to a normal dispatch
            logger.warning(
                "[API] submit-time cache check failed for %s ? dispatching normally",
                video_id,
                exc_info=True,
            )
            return None


    def _first_object_under(prefix_uri: str, storage_config: dict) -> str:
        """The first object URI under a bucket prefix, or "" ? best-effort source
recovery."""
        try:
            return next(iter(storage.list_uris(prefix_uri, config=storage_config)), "")
        except Exception:  # noqa: BLE001 ? recovery nicety, never a failure
            return ""


    _MD5_HEX_RE = re.compile(r"[0-9a-f]{32}")


    def read_path_rehydrate(db, cfg, video_id: str) -> bool:
        """#485, the read half of "DB = cache, bucket = truth": before a rally READ reports
        nothing for an md5-shaped id, try re-hydrating the video from ``outputs/<md5>/`` ?
so
        ``?video=<md5>`` lands on grounded state after any restart/redeploy wipe. True ? the
DB
        has the video's rows now (re-query). Non-md5 ids return False without touching the
        bucket; every failure degrades to False (the read just reports what the DB has)."""
        vid = (video_id or "").strip().lower()
        if not _MD5_HEX_RE.fullmatch(vid):
            return False
        if not probe_process_cache(db, cfg, vid):
            return False
        return cached_process_result(db, cfg, vid, "") is not None


    def _maybe_dispatch_remote(
        cfg,
        db,
        req: "ProcessRequest",
        video_id: str,
        seg_name: str,
        val_results,
        play_wm: int,
```

```
        cand_wm: int,
        notify,
    ) -> Optional[Tuple[Dict[str, Any], bool]]:
        """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path): when
``deployment.compute_target
        == "cloud_run"``, run the heavy DETECT on a Cloud Run GPU Job and INGEST its rally
intervals into
        THIS DB, so the rest of the flow (query ? compile/local-stitch) is unchanged.

        Returns ``(response_dict, ran)`` when the cloud path is taken ? ``ran`` is whether
the cloud job
        actually EXECUTED (``False`` for a pre-dispatch config rejection, so the
observability report
        doesn't claim a fake segmentation run) ? or ``None`` to fall through to the in-
process segmenter
        (**DEFAULT-OFF**: ``get_compute_backend`` returns ``None`` for every
non-``cloud_run`` target, so
        the in-process path stays byte-identical)."""
        from backend.compute import ComputeJobSpec, get_compute_backend

        def _failed(msg: str, ran: bool) -> Tuple[Dict[str, Any], bool]:
            # Shape-match the in-process segmenter-fail branch + drop any partial NEW rows
(id > play_wm);
            # prior good rows (id <= play_wm) are preserved (destroy-AFTER-confirm).
            db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
            # Provenance even on failure, and HONEST about what ran: `path` is "cloud_run"
only when the
            # job actually dispatched + executed remotely (ran=True, e.g. a non-zero worker
rc / ingest
            # error); a pre-dispatch reject (ran=False) ran NOWHERE ? "not_run". `target`
records the
            # intended backend, `requested` the picker choice, `dispatched` whether the
remote job ran.
            return (
                {
                    "video_id": video_id,
                    "status": "failed",
                    "message": msg,
                    "results": [r.model_dump() for r in val_results],
                    "play_segments": [],
                    "compute": {
                        "path": "cloud_run" if ran else "not_run",
                        "requested": req.compute,
                        "target": "cloud_run",
                        "dispatched": ran,
                    },
                },
                ran,
            )

        # SPA compute-picker (item 3): a per-request choice overrides the server's
configured default.
        # "local" forces the in-process segmenter; "cloud" forces a Cloud Run dispatch
(guarded by
        # _cloud_available so we fail clearly rather than silently running local); None /
unknown ? default.
        choice = (req.compute or "").strip().lower()
        if choice and choice not in ("local", "cloud"):
            logger.warning(
                "[API] ignoring unknown compute=%r (expected 'local'|'cloud'); using server
default.",
                req.compute,
            )
            choice = ""
        if choice == "local":
```

```
            # A hosted cloud-serving control-plane bakes NO detector weights into its image
(only the
            # worker Cloud Run Job GCSFuse-mounts them), so forcing a weights-backed
trajectory detector
            # to run IN-PROCESS here can never succeed ? it dies deep in the segmenter with
a cryptic
            # "WASB weights not found" (observed live: job f29d41d0?, 2026-07-07). Fail FAST
with an
            # actionable message instead of that silent trap. A truly-local box (not cloud-
capable), or a
            # cloud-wired box that ACTUALLY has the weights on disk, is unaffected ? it runs
in-process
            # below exactly as before (the weights-present check is the escape hatch).
            if (
                (seg_name or "").strip().lower() in _WEIGHTS_BACKED_SEGMENTERS
                and _cloud_available(cfg)
                and not _local_detector_weights_present(cfg, seg_name)
            ):
                return _failed(
                    f"this hosted server can't run detection locally ? the '{seg_name}'
model weights "
                    "live only on the cloud worker, not on the control-plane. Choose 'Run in
cloud'.",
                    False,
                )
            return None  # explicit "run on my machine" ? in-process regardless of the
server's target
        if choice == "cloud":
            if not _cloud_available(cfg):
                return _failed(
                    "cloud-serving was requested but this server isn't configured for it "
                    "(no Cloud Run target). Pick 'run on my machine', or configure cloud
serving.",
                    False,
                )
            compute = get_compute_backend(cfg, target_override="cloud_run")
        else:
            compute = get_compute_backend(
                cfg
            )  # server default (default-OFF unless cloud_run)
        if compute is None:
            return None  # default-OFF ? run the in-process segmenter below (unchanged)

        storage_cfg = getattr(cfg, "storage", None)

        # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local
to this box.
        try:
            input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
        except ValueError as e:
            return _failed(
                f"cloud-serving: unresolvable input '{req.video_path}': {e}", False
            )
        if input_ref.backend == "local":
            # A browser upload lands on THIS box's local disk; the Cloud Run job (a
different machine)
            # can't read it. Stage it UP to the input bucket first, then dispatch from there
(#461). Only
            # possible when a CLOUD input_root is configured; without one we genuinely can't
stage ? fail.
            # Classify the ROOT ITSELF (parse_uri), not storage.input_backend: the staged
URI below is
            # built by joining under input_root, and storage.put routes by
parse_uri(staged_uri).backend ?
            # so an explicit cloud root (gs://?, s3://?) stages fine with input_backend left
```

```
at its
530             # default, while a bare/local-looking root can't stage no matter what
input_backend says
531             # (resolve_input_uri only consults input_backend when input_root is EMPTY, and
an empty root
532             # gives us nothing to build the staged URI under).
533             input_root = (getattr(storage_cfg, "input_root", "") or "").strip()
534             try:
535                 root_backend = (
536                     storage.parse_uri(input_root).backend if input_root else "local"
537                 )
538             except ValueError:  # unsupported scheme in input_root ? nowhere real to stage
to
539                 root_backend = "local"
540             if root_backend == "local":
541                 return _failed(
542                     "cloud-serving needs a cloud input (e.g. gs://?), or a cloud
storage.input_root to "
543                     "stage local uploads into; a path local to this machine can't be read by
the Cloud "
544                     "Run job.",
545                     False,
546                 )
547             local_src = input_ref.key  # resolved local path of the upload
548             staged_uri =
f"{input_root.rstrip('/')}/{video_id}/{os.path.basename(local_src)}"
549             try:
550                 notify("staging upload to bucket")
551                 # Thread the active storage settings so backend-specific config survives (S3
endpoint_url/
552                 # region/addressing for R2·DO·MinIO; GCS project/client) ? not a fresh
default client.
553                 storage.put(
554                     local_src,
555                     staged_uri,
556                     config=(
557                         storage_cfg.model_dump() if storage_cfg is not None else None
558                     ),
559                 )
560             except Exception as e:  # noqa: BLE001 ? a staging failure is a clean job
failure, not a 500
561                 logger.error(
562                     "[API] cloud-serving: failed to stage %s -> %s: %s",
563                     local_src,
564                     staged_uri,
565                     e,
566                     exc_info=True,
567                 )
568                 return _failed(
569                     f"cloud-serving: failed to stage the uploaded file to {input_root}:
{e}",
570                     False,
571                 )
572             # Repoint BOTH the dispatch and the DB video record at the staged bucket object:
the L4 job
573             # DETECTs from it, and a later /api/compile re-fetches the same source (the
local upload is
574             # gone once this instance recycles). add_video is an UPSERT ? it only updates
filepath.
575             req.video_path = staged_uri
576             input_ref = storage.resolve_input_ref(staged_uri, storage_cfg)
577             db.add_video(
578                 video_id, staged_uri, "processed", [r.model_dump() for r in val_results]
579             )
580         out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
```

```
        if not out_root:
            return _failed(
                "cloud-serving needs storage.output_root set to a bucket (e.g. gs://your-
bucket) "
                "? that's where the Cloud Run job writes the result.",
                False,
            )

        notify("dispatching (cloud_run)")
        logger.info(
            "[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
            req.video_path,
            out_root,
        )
        # Forward the engine's AI-handoff stance so the cloud job runs the SAME detection:
with
        # skip_ai_handoff=True the WASB windows become rallies directly (no Gemini key
needed); without
        # forwarding the cloud job would default to the handoff and yield 0 rallies when it
has no key.
        # (player_pool / max_frames are still dropped on the cloud path ? a separate
documented follow-up.)
        overrides = []
        idx_cfg = getattr(cfg, "indexing", None)
        sk = getattr(idx_cfg, "skip_ai_handoff", None)
        if sk is not None:
            overrides.append(f"indexing.skip_ai_handoff={'true' if sk else 'false'}")
        # Forward the resolved WASB serving knobs so the PRESET governs the worker
(#506/#507). The
        # worker Job has no DEPLOYMENT_PROFILE, so it does NOT apply the cloud-serving
preset itself ?
        # without forwarding, its decode_backend + GPU-feed tuning silently fall back to the
model
        # defaults (cpu, batch 8) regardless of what the control-plane resolved.
decode_backend is
        # cuda-only (validator-enforced) and requires a worker image that understands
--decode-backend
        # (? #507); batch_size/prefetch_batches are forwarded only when set (None = the
parity default).
        wasb_cfg = getattr(idx_cfg, "wasb", None)
        if wasb_cfg is not None:
            decode_backend = getattr(wasb_cfg, "decode_backend", None)
            if decode_backend:
                overrides.append(f"indexing.wasb.decode_backend={decode_backend}")
            batch_size = getattr(wasb_cfg, "batch_size", None)
            if batch_size:
                overrides.append(f"indexing.wasb.batch_size={batch_size}")
            prefetch = getattr(wasb_cfg, "prefetch_batches", None)
            if prefetch:
                overrides.append(f"indexing.wasb.prefetch_batches={prefetch}")
        # Tell the worker to SKIP its redundant re-validation: THIS control-plane is the
authoritative
        # gate ? it already ran the validators with its resolved config just above (only a
PASS reaches
        # here; a failure returned "failed" before dispatch). A worker re-run would decode
the video a
        # second time and, because its config isn't fully forwarded, can resolve DIFFERENT
thresholds and
        # REJECT a clip this gate already passed (observed 2026-07-07: passed at
min_blur_threshold=8.0,
        # then worker-rejected at its default 20.0 after a successful GPU dispatch).
        spec = ComputeJobSpec(
            video_uri=req.video_path,
            out_uri=out_root,
            segmenter=seg_name,
```

```
630             config_overrides=tuple(overrides),
631             skip_validation=True,
632         )
633         try:
634             # Dispatch the Cloud Run Job + bucket-poll to completion. on_wait is the live
635             # heartbeat (#482): job.progress moves on every poll tick instead of freezing at
636             # "segmenting" for the whole GPU run (the 2026-07-05 silent 22-min spinner) ?
which
637             # also gives the SPA a stall signal to time out on, instead of a blind wall
clock.
638             result = compute.run_infer(
639                 spec,
640                 on_wait=lambda elapsed: notify(_wait_progress_message(seg_name, elapsed)),
641             )
642         except Exception as e:  # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
643             logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
644             return _failed(f"cloud dispatch error: {e}", True)
645         if result.returncode != 0:
646             return _failed(f"cloud job failed (returncode {result.returncode}).", True)
647         if result.video_id and result.video_id != video_id:
648             # The worker re-hashes the bucket bytes; a divergence means the input changed
between this
649             # box's hash and the worker's ? ingest under the LOCAL id but flag it so it's
diagnosable.
650             logger.warning(
651                 "[API] cloud video_id %s != local %s ? input bytes differ between the API
hash "
652                 "and the worker hash; ingesting under the local id.",
653                 result.video_id,
654                 video_id,
655             )
656
657         notify("ingesting (cloud_run)")
658         storage_dict = (
659             storage_cfg.model_dump()  # type: ignore[union-attr]  # hasattr-guarded; mypy
can't narrow
660             if hasattr(storage_cfg, "model_dump")
661             else {}
662         )
663         try:
664             _ingest_remote_segments(db, video_id, result.outputs_uri, storage_dict)
665         except Exception as e:  # noqa: BLE001 ? a missing/partial/corrupt result must not
500 or leak rows
666             logger.error("[API] cloud ingest failed: %s", e, exc_info=True)
667             return _failed(
668                 f"cloud-serving: could not read the result from {result.outputs_uri}: {e}",
669                 True,
670             )
671
672         # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then
drops the old ones.
673         segments = [
674             s for s in db.query_play_segments(video_id, {}) if int(s.get("id", 0)) > play_wm
675         ]
676         _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
677         # Compute-path PROVENANCE: the GCP-verifiable proof the cloud path ran. `execution`
is the real
678         # Cloud Run execution name (cross-check: gcloud run jobs executions describe
<execution>).
679         provenance = {
680             "path": "cloud_run",
681             "requested": req.compute,
682             "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
683             "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
```

```
            "execution": result.execution,
            "outputs_uri": result.outputs_uri,
        }
        logger.info(
            "[COMPUTE] cloud_run dispatched: execution=%s job=%s region=%s out=%s
video_id=%s",
            result.execution,
            provenance["job"],
            provenance["region"],
            result.outputs_uri,
            video_id,
        )
        return (
            {
                "video_id": video_id,
                "status": "success",
                "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
'{seg_name}').",
                "results": [r.model_dump() for r in val_results],
                "play_segments": segments,
                "compute": provenance,
            },
            True,
        )


    def _execute_processing(
        config=None, db=None, req=None, progress=None
    ) -> Dict[str, Any]:
        """The full hash ? validate ? segment ? report pipeline for one video.

        Accepts explicit ``config``/``db`` (the new app-factory pattern) OR reads the
        module-level ``global_config``/``db_instance`` (backward-compatible with existing
        tests that pass only ``req``). Route handlers always pass explicit params; older
        test code that calls ``_execute_processing(req)`` still works.

        ``progress`` is an optional callable(stage: str) used to surface coarse job
progress.
        Raises on unexpected internal errors ? the caller records those as a job failure
        """
        import backend.main as _main  # call-time seam: main's globals/model stay patchable

        # Backward-compatible shim: if called as _execute_processing(req) (old pattern),
        # shift positional args ? req arrives in position 0 (config), progress in position 1
(db).
        if config is not None and not isinstance(config, AppConfig):
            # Old signature: _execute_processing(req, progress=None)
            if req is None and db is not None and isinstance(db, _main.ProcessRequest):
                req = db
                db = None
            elif req is None and isinstance(config, _main.ProcessRequest):
                req = config
                config = None
        # Resolve config/db from module globals when not passed explicitly
        if config is None:
            config = _main.global_config
        if db is None:
            db = _main.db_instance

        notify = progress or (lambda stage: None)

        notify("hashing")
        # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
(identity ?
        # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
```

```
original
744         # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
745         # consumers (hash / validators / segmenter) use the resolved local path.
746         local_video = ""
747         try:
748             local_video = storage.acquire_local_input(
749                 req.video_path, getattr(config, "storage", None)
750             )
751             video_id = _main.compute_video_id(local_video)
752         except Exception:
753             storage.cleanup_acquired_local_input(
754                 req.video_path, local_video, getattr(config, "storage", None)
755             )
756             raise
757         try:
758             existing_video = db.get_video(video_id)
759         except Exception:
760             storage.cleanup_acquired_local_input(
761                 req.video_path, local_video, getattr(config, "storage", None)
762             )
763             raise
764         was_reingest = existing_video is not None
765
766         # typed AppConfig: attribute access for the default segmenter (with safe fallback)
767         default_seg = getattr(
768             getattr(config, "indexing", None), "default_segmenter", "motion"
769         )
770         seg_name = req.segmenter or default_seg
771         segmenter_actual = None
772         segmentation_ran = False
773         val_results: List[Any] = []
774         val_context: Dict[str, Any] = {}
775         # Compute-path PROVENANCE (the validation method): which backend actually ran the
DETECT ?
776         # "in_process" once the local segmenter is reached, set on the response as
"cloud_run" by
777         # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
fail).
778         compute_actual: Optional[str] = None
779         response: Optional[Dict[str, Any]] = None
780         try:
781             if (
782                 existing_video
783                 and existing_video.get("status") == "processed"
784                 and not req.force
785             ):
786                 cached_segments = db.query_play_segments(video_id, {})
787                 if cached_segments:
788                     logger.info(
789                         "[API] Reusing cached segmentation for video_id=%s; force=true
bypasses cache.",
790                         video_id,
791                     )
792                     response = _cached_process_payload(
793                         video_id,
794                         existing_video.get("validation_results", []),
795                         cached_segments,
796                         getattr(req, "compute", None),
797                     )
798                     return response
799
800             # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for
the report) ?
801             #    UNLESS this exact content already has a cached verdict under the current
```

```
validator config.
802             notify("validating")
803             # VALIDATION-RESULT CACHE: video_id is the MD5 of the file bytes, so an
identical-content
804             # reupload keys the same row. When that content was ALREADY validated under the
SAME validator
805             # configuration, reuse the stored verdict instead of re-decoding the clip
through the whole
806             # validator suite ? minutes on a heavy 4K clip on the CPU-only control-plane,
which the
807             # 2026-07-07 CUJ re-paid on every retry after a segmentation failure (validation
passed but
808             # detection failed ? no cached segments, so the whole-pipeline cache above
misses and we land
809             # here). The fingerprint folds in the validation.* config + sport + validator
set + a schema
810             # version, so ANY config change misses and re-validates (never serve a stale
PASS/FAIL).
811             # ``force=true`` bypasses the reuse and re-writes a fresh verdict below (mirrors
how it bypasses
812             # the whole-pipeline cache). Composes with #513 (the worker skips its own re-
validation on a
813             # cloud dispatch), so a post-failure reupload reaches segmentation in ~0
validation time.
814             val_fingerprint = registry.config_fingerprint(config)
815             # The whole read ? fingerprint-match ? replay is guarded: ANY cache fault
(unreadable row,
816             # corrupt or schema-incompatible stored results) degrades to a real validation
rather than
817             # failing the job. The cache is a pure speedup ? never a new failure mode.
818             reused_validation = False
819             if not req.force:
820                 try:
821                     cached = db.get_validation_cache(video_id)
822                     if cached is not None and cached.get("fingerprint") == val_fingerprint:
823                         # Replay the stored ValidationResult list; val_context stays {} (no
fresh
824                         # ffprobe_meta ? the report's media block degrades gracefully to
size-only, the
825                         # deliberate cost of skipping the decode). Segmentation reads
neither of these.
826                         val_results = [
827                             ValidationResult(**r) for r in cached.get("results", [])
828                         ]
829                         reused_validation = True
830                         logger.info(
831                             "[API] Reusing cached validation verdict (passed=%s) for
video_id=%s; "
832                             "validator config unchanged ? skipping the validator suite. "
833                             "force=true re-validates.",
834                             cached.get("passed"),
835                             video_id,
836                         )
837                 except Exception:  # noqa: BLE001 ? any cache read/replay fault degrades to
a real validation
838                     reused_validation = False
839                     logger.warning(
840                         "[API] validation-cache reuse failed for video_id=%s ? re-
validating.",
841                         video_id,
842                         exc_info=True,
843                     )
844             if not reused_validation:
845                 logger.info(f"Running validations for {req.video_path}...")
846                 val_results, val_context = registry.run_all_with_context(local_video,
```

```
config)
            failures = [r for r in val_results if not r.passed]
            if not reused_validation:
                # Persist the verdict keyed by (content, config) so the NEXT identical
reupload skips the
                # decode. Best-effort (the writer swallows + logs its own faults); a miss
just re-validates.
                db.set_validation_cache(
                    video_id,
                    val_fingerprint,
                    not failures,
                    [r.model_dump() for r in val_results],
                )

            if failures:
                # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
                # status; it no longer destroys the video's prior good segments.
                db.add_video(
                    video_id,
                    req.video_path,
                    "failed",
                    [r.model_dump() for r in val_results],
                )
                response = {
                    "video_id": video_id,
                    "status": "failed",
                    "message": _validation_failure_message(failures),
                    "results": [r.model_dump() for r in val_results],
                }
                return response

            # 2. Run segmenter & indexer
            logger.info("[API] Video passed checks. Segmenting and indexing...")
            db.add_video(
                video_id, req.video_path, "processed", [r.model_dump() for r in val_results]
            )

            segmenter_cls = segmenter_registry.get(seg_name)
            if not segmenter_cls:
                # Submit-time validation already 400s unknown names; this is the defensive
                # in-worker path (e.g. config default pointing at an unregistered
segmenter).
                available = list(segmenter_registry.list_available().keys())
                response = {
                    "video_id": video_id,
                    "status": "failed",
                    "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
                    "results": [r.model_dump() for r in val_results],
                    "play_segments": [],
                }
                return response

            logger.info(f"[API] Using segmenter: {seg_name}")
            notify(f"segmenting ({seg_name})")
            segmenter_actual = seg_name
            # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
            # if the run yields segments; otherwise drop the new partial rows and keep the
old.
            play_wm, cand_wm = db.segment_watermarks(video_id)
            # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run,
run the heavy
            # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
```

```
            # DEFAULT-OFF: returns None for every non-cloud target ? the in-process
segmenter below runs
            # byte-identically. (player_pool/max_frames aren't threaded to the worker
`infer` CLI yet, so
            # they're dropped on the cloud path in this slice ? a documented follow-up.)
            remote = _maybe_dispatch_remote(
                config, db, req, video_id, seg_name, val_results, play_wm, cand_wm, notify
            )
            if remote is not None:
                response, segmentation_ran = (
                    remote  # `ran` = did the cloud job actually execute (report)
                )
                return response  # the cloud branch already stamped response["compute"]
(path=cloud_run)
            # In-process from here on ? record the actual path for the provenance stamp in
`finally`.
            compute_actual = "in_process"
            segmenter = segmenter_cls(db, config)
            try:
                # Serialize the CPU/GPU-heavy in-process detection (see
_LOCAL_COMPUTE_LANE). A remote
                # dispatch never reaches here (it returned above), so remote polls still
overlap freely.
                with _LOCAL_COMPUTE_LANE:
                    result = segmenter.process_video(
                        video_id, local_video, req.player_pool, req.max_frames
                    )
            except Exception:
                # A raising segmenter must not leave half-written rows: restore the pre-run
                # state (same destroy-after-confirm contract as the Failure branch), then
let
                # the job worker record the failure.
                db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
                raise
            segmentation_ran = True

            if isinstance(result, Failure):
                logger.error(f"[API] Segmenter failed: {result.message}")
                db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
                response = {
                    "video_id": video_id,
                    "status": "failed",
                    "message": f"Segmenter '{seg_name}' failed: {result.message}",
                    "results": [r.model_dump() for r in val_results],
                    "play_segments": [],
                }
                return response

            segments = result.value if hasattr(result, "value") else result
            notify("finalizing")
            _finalize_reingest(db, video_id, segments, play_wm, cand_wm)

            response = {
                "video_id": video_id,
                "status": "success",
                "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",  # type: ignore[arg-type]  # hasattr-unwrap above; Success arm has
.value
                "results": [r.model_dump() for r in val_results],
                "play_segments": segments,
            }
            return response
        finally:
            # Compute-path PROVENANCE stamp (the validation method): the cloud branch
already set
```

```
            # response["compute"] (path=cloud_run + the Cloud Run execution); for every
other path stamp
            # the actual path here so the job result PROVES which backend ran (in_process /
not_run) and
            # what the picker REQUESTED ? a silent fallback (requested cloud, ran local) is
then visible.
            if isinstance(response, dict):
                response.setdefault(
                    "compute",
                    {
                        "path": compute_actual or "not_run",
                        "requested": getattr(req, "compute", None),
                    },
                )
            prov = response.get("compute", {}) if isinstance(response, dict) else {}
            logger.info(
                "[COMPUTE] video_id=%s requested=%s actual=%s dispatched=%s status=%s",
                video_id,
                getattr(req, "compute", None),
                prov.get("path"),
                prov.get("dispatched"),
                (response or {}).get("status") if isinstance(response, dict) else None,
            )
            # Always emit the per-video observability report (next to the video, or to
            # report.output_dir). Never raises. Surface the paths in the response. (The
compute-path
            # proof lives on the job result + the [COMPUTE] log, not the report ?
obsreport's record_run
            # doesn't surface arbitrary run signals.)
            paths = _main.emit_indexer_report(
                db=db,
                video_id=video_id,
                # Resolved LOCAL path, not the raw URI: obsreport writes the report next to
the
                # video via os.path.abspath(video_path), and a remote URI (gs://, gcs://,
local://)
                # collapses to a literal scheme dir at the repo root (see #397). The source
URI is
                # still preserved in DB provenance via db.add_video.
                video_path=local_video,
                config=_report_config_for_input(config, req.video_path),
                val_results=val_results,
                val_context=val_context,
                segmenter_requested=req.segmenter,
                segmenter_actual=segmenter_actual,
                was_reingest=was_reingest,
                segmentation_ran=segmentation_ran,
            )
            if paths and isinstance(response, dict):
                response["reports"] = paths
            storage.cleanup_acquired_local_input(
                req.video_path, local_video, getattr(config, "storage", None)
            )


    def _remote_input_size_bytes(input_ref, storage_config) -> Optional[int]:
        """Best-effort size (bytes) of a remote storage source, or ``None`` if unknowable.

        Used only to pre-reserve scratch for a fetch-to-local (P2). Never raises: a stat
that needs a
        missing cloud SDK, the network, or a credential returns ``None`` so the free-space
guard degrades
        to a no-op rather than blocking a job it cannot measure."""
        try:
            sc = (
```

```
                storage_config.model_dump()
                if hasattr(storage_config, "model_dump")
                else (storage_config or {})
            )
            backend = storage.get_storage(input_ref.backend, config=sc)
            return int(backend.stat(input_ref).size)
        except Exception as e:  # noqa: BLE001 - measuring is best-effort; inability must
not block
            logger.debug(
                "free-space: could not stat remote source %s: %s", input_ref.uri, e
            )
            return None


    class _CompileError(Exception):
        """A submit-time-validatable compile problem ? carries the HTTP status + detail so
the API path
        maps it to an HTTPException (fast 4xx) and the worker path maps it to a failed-job
result."""

        def __init__(self, status_code: int, detail: str):
            super().__init__(detail)
            self.status_code = status_code
            self.detail = detail


    def _resolve_compile(
        db, cfg, video_id: str, segment_ids: List[int], output_filename: str
    ):
        """Shared compile resolution ? run at SUBMIT (fast-fail 4xx) AND in the WORKER (re-
resolve, since a
        row could change between submit and run). Verifies the video exists, the requested
segments match
        and survive the ``stitching.min_shot_count`` floor (segments WITHOUT a known
shot_count are exempt
        so manual/legacy picks can't silently vanish), and the output name is traversal-
safe. Raises
        ``_CompileError(404/400)``. Returns ``(video_row, kept_segments,
safe_out_name)``."""
        video = db.get_video(video_id)
        if not video:
            raise _CompileError(404, "Indexed video record not found.")
        # Reject path traversal / absolute names (os.path.join would otherwise write
anywhere on disk).
        try:
            out_name = safe_output_filename(output_filename)
        except ValueError as e:
            raise _CompileError(400, str(e)) from e

        all_segments = db.query_play_segments(video_id, {})
        matched = [s for s in all_segments if s["id"] in segment_ids]
        if not matched:
            raise _CompileError(400, "No matching play segments found to stitch.")

        stitching = getattr(cfg, "stitching", None) or StitchingConfig()
        kept = []
        for s in matched:
            meta = s.get("metadata") or {}
            sc = meta.get("shot_count") if isinstance(meta, dict) else None
            try:
                if sc is not None and int(sc) < stitching.min_shot_count:
                    continue
            except (TypeError, ValueError):
                pass  # unparseable count -> treat as unknown, keep
            kept.append(s)
```

```
        if len(kept) < len(matched):
            logger.info(
                "[compile] stitching.min_shot_count=%s: dropped %d/%d segments below the
floor.",
                stitching.min_shot_count,
                len(matched) - len(kept),
                len(matched),
            )
        if not kept:
            raise _CompileError(
                400,
                f"All {len(matched)} matching segments fall below "
                f"stitching.min_shot_count={stitching.min_shot_count}.",
            )
        return video, kept, out_name


    def reel_object_uri(cfg, video_id: str, name: str) -> str:
        """The bucket URI a compiled reel is mirrored to for a direct signed-URL download
(#463), or "" when
        the output isn't a cloud bucket (truly-local ? serve the local file). Shared by the
compile
        mirror-upload and the /api/highlights redirect:
``<output_root>/highlights/<video_id>/<name>``."""
        root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
        if not root or "://" not in root:
            return ""
        return f"{root.rstrip('/')}/highlights/{video_id}/{name}"


    def _storage_settings(cfg) -> Optional[Dict[str, Any]]:
        """The active storage settings dict (per-backend config for the storage layer ? S3
endpoint/region,
        GCS project/client/signed_url_ttl), or None. Threaded into storage calls so a non-
default backend
        isn't silently constructed as a fresh default."""
        sc = getattr(cfg, "storage", None)
        return sc.model_dump() if sc is not None else None


    def signed_reel_url(cfg, video_id: str, name: str) -> Optional[str]:
        """A short-lived signed URL for a reel mirrored to the output bucket, so
/api/highlights can
        307-redirect off the app's response path (bypassing Cloud Run's ~32 MiB cap + the
ephemeral local
        /tmp; #463). None ? no bucket copy ? serve the local file. Uses the active storage
config for BOTH
        the existence check and the signing (extracted here to keep backend/main.py lean ?
P23/P24)."""
        reel_uri = reel_object_uri(cfg, video_id, name)
        if not reel_uri:
            return None
        settings = _storage_settings(cfg)
        if storage.exists(reel_uri, config=settings):
            return storage.signed_url(reel_uri, config=settings, method="GET")
        return None


    def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:
        """Worker handler for a ``kind='compile'`` job (#329): re-resolve the segments,
stitch the reel,
        and return the {status, filepath, download_url} result the SPA polls for. Returns a
        ``status='failed'`` result for an EXPECTED problem (missing video / no segments / a
stitch error)
        so the SPA always gets a clean verdict instead of a worker-crash error."""
```

```
1122        import backend.main as _main  # call-time seam: VideoCompiler stays patchable on
backend.main
1123
1124        notify = progress or (lambda stage: None)
1125        params = json.loads(job.get("params") or "{}")
1126        video_id = job.get("video_id") or ""
1127        segment_ids = params.get("segment_ids") or []
1128        locale = params.get("locale")
1129
1130        notify("resolving")
1131        try:
1132            video, kept, out_name = _resolve_compile(
1133                db, cfg, video_id, segment_ids, params.get("output_filename", "")
1134            )
1135        except _CompileError as e:
1136            return {"status": "failed", "message": e.detail, "video_id": video_id}
1137
1138        stitching = getattr(cfg, "stitching", None) or StitchingConfig()
1139        output_dir = (
1140            getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
1141        )
1142        # Localize the branding watermark to the UI locale the SPA rendered in (recorded on
the job);
1143        # unknown/absent locale keeps the configured default unchanged.
1144        localized_watermark = localize_watermark(stitching.watermark, locale)
1145        compiler = _main.VideoCompiler(
1146            output_dir,
1147            begin_padding=stitching.begin_padding,
1148            end_padding=stitching.end_padding,
1149            watermark=localized_watermark,
1150        )
1151        time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
1152
1153        notify("stitching")
1154        local_src = ""
1155        try:
1156            # The stored `filepath` is the SOURCE ref (gs://, s3://, gdrive://, or a local
path). Resolve to
1157            # a LOCAL file the stitcher can read ? identity for a local path, fetch-to-
scratch for a URI (the
1158            # same seam the process path uses); without it, compiling a bucket-ingested
video can't open gs://.
1159            local_src = storage.acquire_local_input(
1160                video["filepath"], getattr(cfg, "storage", None)
1161            )
1162            # The ffmpeg stitch is CPU-heavy ? serialize it so a raised drain worker count
(for remote
1163            # dispatch) doesn't run multiple compiles at once on the box (see
_LOCAL_COMPUTE_LANE).
1164            with _LOCAL_COMPUTE_LANE:
1165                out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
1166        except Exception as e:  # noqa: BLE001 ? a stitch/ffmpeg failure is a clean job
FAILURE, not a crash
1167            logger.error(
1168                "[compile] stitching failed for video_id=%s: %s", video_id, e, exc_info=True
1169            )
1170            return {
1171                "status": "failed",
1172                "message": f"Highlight stitching failed: {e}",
1173                "video_id": video_id,
1174            }
1175        finally:
1176            if local_src:
1177                storage.cleanup_acquired_local_input(
1178                    video["filepath"], local_src, getattr(cfg, "storage", None)
```

```
1179                )
1180
1181        # #463: mirror the reel to the output bucket so /api/highlights can 307 to a signed
URL ? Cloud Run
1182        # caps a buffered response at ~32 MiB (a real reel is bigger) and the local /tmp
copy dies when the
1183        # instance recycles. A mirror-upload failure must NOT fail an otherwise-good stitch
(the endpoint
1184        # still falls back to the local file on the same instance).
1185        reel_uri = reel_object_uri(cfg, video_id, out_name)
1186        if reel_uri:
1187            try:
1188                notify("uploading reel")
1189                # Thread the active storage settings so the configured backend (S3
endpoint/region, GCS
1190                # project/client) is used, not a fresh default.
1191                storage.put(out_path, reel_uri, config=_storage_settings(cfg))
1192            except Exception as e:  # noqa: BLE001 ? best-effort mirror; local stitch
already succeeded
1193                logger.warning("[compile] could not mirror reel to %s: %s", reel_uri, e)
1194
1195        # `filepath` is server-local (not browser-reachable); also return the download URL
so the SPA needn't
1196        # hand-build it (P6 / PER_RALLY_SELECTION.md §4.3).
1197        download_url = f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
1198        notify("finished")
1199        return {
1200            "status": "success",
1201            "filepath": out_path,
1202            "download_url": download_url,
1203            "video_id": video_id,
1204        }
1205
```

13. user (2026-07-08T05:05:51.816Z)

```
1       """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3           python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4               [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5           python -m backend.worker train ...        # Gen-0 train-from-bucket (follow-up)
6
7       The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
``local://``
8       is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing
local runs
9       are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
10      ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
key) ?
11      the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
12
13      This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
CLI uses
14      (validator registry ? segmenter ? report), never a forked pipeline.
15      """
16
17      from __future__ import annotations
18
19      import argparse
20      import csv
21      import json
22      import logging
23      import os
24      import re
25      import sys
```

```
    import tempfile
    from typing import Any, List

    from backend import storage
    from backend.compute import ComputeJobSpec, get_compute_backend
    from backend.config import (
        StartupCheckError,
        enforce_startup_checks,
        load_config,
        probe_cuda_available,
    )
    from backend.infrastructure.database import Database
    from backend.pipeline.segmenters.base import segmenter_registry
    from backend.pipeline.validators.base import registry as validator_registry
    from backend.reporting import emit_indexer_report
    from backend.results import Failure
    from backend.utils.ffmpeg import ffprobe_bin  # P2a: single ffmpeg/ffprobe resolver
    from backend.utils.hashing import compute_video_id

    logger = logging.getLogger("backend.worker")


    def _coerce(v: str) -> Any:
        """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
str."""
        low = v.lower()
        if low in ("true", "false"):
            return low == "true"
        for cast in (int, float):
            try:
                return cast(v)
            except ValueError:
                pass
        return v


    def _apply_overrides(config: Any, sets: List[str]) -> None:
        """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
        for kv in sets or []:
            key, sep, val = kv.partition("=")
            if not sep:
                raise ValueError(f"--set expects k=v, got {kv!r}")
            parts = key.split(".")
            obj = config
            for p in parts[:-1]:
                obj = getattr(obj, p)
            setattr(obj, parts[-1], _coerce(val))


    def _write_intervals_csv(segments: List[dict], path: str) -> None:
        """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
friendly
        artifact (same schema as the rally-annotator labels)."""
        with open(path, "w", newline="") as f:
            w = csv.writer(f)
            w.writerow(["start", "end", "shot_count", "ending_reason"])
            for s in segments:
                w.writerow(
                    [
                        f"{float(s.get('start_time', 0.0)):.3f}",
                        f"{float(s.get('end_time', 0.0)):.3f}",
                        s.get("shot_count", ""),
                        s.get("ending_reason", s.get("shot_count_confidence", "")),
                    ]
```

```
                )


    def _write_report_json(
        video_id: str, segments: List[dict], status: str, path: str, video_uri: str = ""
    ) -> None:
        with open(path, "w") as f:
            json.dump(
                {
                    "video_id": video_id,
                    "status": status,
                    "segment_count": len(segments),
                    # The SOURCE the detection ran on (CP-rebuild P3, #485): a restart-wiped
                    # control plane re-hydrating from outputs/<md5>/ restores the video row
                    # with its truthful path ? without this, only the segments recover.
                    "video_uri": video_uri,
                    "segments": [
                        {
                            "start": float(s.get("start_time", 0.0)),
                            "end": float(s.get("end_time", 0.0)),
                        }
                        for s in segments
                    ],
                },
                f,
                indent=2,
            )


    def _serialize_and_push_outputs(
        scratch: str,
        out_uri: str,
        video_id: str,
        segments: List[dict],
        status: str,
        storage_cfg: dict,
        video_uri: str = "",
    ) -> List[str]:
        """Write intervals.csv + report.json under ``<scratch>/outputs/<video_id>/`` and
push both to
        ``<out_uri>/outputs/<video_id>/``; return the pushed URIs.

        Shared by the SUCCESS path and the failure path (``_fail``) so a detector
timeout/abort leaves a
        ``status="failed"`` report.json in the bucket ? the control plane's source of truth
        (``orchestration.cached_process_result`` / ``probe_process_cache`` treat any
non-``success``
        report as "no usable result" ? retry, and the restart-recovery poll waits for
report.json to
        EXIST) ? instead of a missing artifact or a false 0-segment ``"success"``."""
        out_local = os.path.join(scratch, "outputs", video_id)
        os.makedirs(out_local, exist_ok=True)
        _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
        _write_report_json(
            video_id,
            segments,
            status,
            os.path.join(out_local, "report.json"),
            video_uri=video_uri,
        )
        out_prefix = out_uri.rstrip("/")
        pushed: List[str] = []
        for fn in sorted(os.listdir(out_local)):
            uri = f"{out_prefix}/outputs/{video_id}/{fn}"
            storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
```

```
149             pushed.append(uri)
150         return pushed
151
152
153     def _run_startup_checks(config: Any) -> bool:
154         """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
155         best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
156         cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true
no-op
157         (returns True, ZERO work) when no deployment.profile is selected.
158
159         The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily
imports
160         torch, and pre-M5 the worker's infer/train path was torch-free (detection is
delegated to a
161         WSL/native subprocess). Probing only when a profile is set keeps the default-OFF
path a true
162         no-op of work, not just of output."""
163         profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
164         cuda = (
165             probe_cuda_available() if profile else None
166         )  # torch-free on the default-OFF path
167         try:
168             enforce_startup_checks(config, cuda_available=cuda, logger=logger)
169             return True
170         except StartupCheckError:
171             return False  # already logged at ERROR by enforce_startup_checks
172
173
174     def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
175         """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc
once the
176         outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
177         container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never
re-dispatch.
178
179         A remote job that never writes a completion marker exits via one of two CLEAN codes
(distinct
180         from the local 2/3), never a raw traceback:
181           * rc 4 ? the bucket poll exhausted its budget (``PolicyTimeout``): the execution
may still be
182             running (the shim best-effort cancels it).
183           * rc 5 ? the execution has TERMINATED without a marker
(``RemoteExecutionFailed``): a worker
184             crash/OOM/pre-marker abort caught FAST by the Executions-API probe, so instead
of the
185             ~65-min opaque wait the operator gets an immediate, log-pointing failure."""
186         from backend.compute import RemoteExecutionFailed
187         from backend.infrastructure.execution_policy import PolicyTimeout
188
189         spec = ComputeJobSpec(
190             video_uri=args.video_uri,
191             out_uri=args.out_uri,
192             segmenter=args.segmenter,
193             config_overrides=tuple(args.set or ()),
194             skip_validation=bool(getattr(args, "skip_validation", False)),
195         )
196         try:
197             result = compute.run_infer(spec)
198         except RemoteExecutionFailed as e:
199             logger.error(
200                 "[worker] remote execution terminated without a completion marker: %s", e
```

```
            )
            return 5
        except PolicyTimeout as e:
            logger.warning(
                "[worker] remote compute did not complete (no marker within budget): %s", e
            )
            return 4
        logger.info(
            "[worker] remote compute done: video_id=%s rc=%d outputs=%s",
            result.video_id,
            result.returncode,
            result.outputs_uri,
        )
        return result.returncode


    def _write_done_marker(
        done_uri: str,
        video_id: str,
        returncode: int,
        segment_count: int,
        status: str,
        storage_cfg: dict,
        scratch: str,
    ) -> None:
        """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer)
for a remote
        dispatcher's bucket-poll. Best-effort ? never raises into the worker's success
return."""
        try:
            marker = {
                "video_id": video_id,
                "returncode": returncode,
                "segment_count": segment_count,
                "status": status,
            }
            local = os.path.join(scratch, "_done.json")
            with open(local, "w", encoding="utf-8") as f:
                json.dump(marker, f)
            storage.put(local, done_uri, config=storage_cfg)
            logger.info("[worker] wrote completion marker -> %s", done_uri)
        except Exception as e:  # never fail a good run because the marker push hiccuped
            logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)


    def cmd_infer(args: argparse.Namespace) -> int:
        config = load_config(args.config) if args.config else load_config()
        _apply_overrides(config, args.set)
        if not _run_startup_checks(config):
            return 2

        # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
process.
        # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
through to the
        # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
(the
        # dispatcher forces compute_target=local_gpu), so it never recursively re-
dispatches.
        compute = get_compute_backend(config)
        if compute is not None:
            return _dispatch_remote(compute, args)

        storage_cfg = config.storage.model_dump()

```

```
        scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
        os.makedirs(scratch, exist_ok=True)
        config.output.output_dir = scratch

        # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
        local_video = storage.fetch(
            args.video_uri, os.path.join(scratch, "in.mp4"), config=storage_cfg
        )
        print(f"[worker] pulled {args.video_uri} -> {local_video}")

        # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
        video_id = compute_video_id(local_video)

        # 3. VALIDATE (same registry as the main CLI) ? UNLESS the dispatcher already did.
        # On the cloud-serving path the CONTROL-PLANE is the authoritative validation gate:
it runs the
        # SAME validators with ITS resolved config and only dispatches on PASS. Re-
validating here is
        # redundant, decodes the whole video a SECOND time, and is outright WRONG when the
worker's config
        # resolves different thresholds than the control-plane's ? observed 2026-07-07: a
5312x2988 clip
        # (sharpness 11.9) PASSED the control-plane at validation.min_blur_threshold=8.0,
then the worker
        # REJECTED it at its own default 20.0, so a SUCCESSFUL GPU dispatch failed after the
fact.
        # --skip-validation (set by the dispatcher in orchestration._maybe_dispatch_remote)
skips it; the
        # direct ``worker infer`` CLI has no control-plane, so it still validates by
default.
        if getattr(args, "skip_validation", False):
            print(
                "[worker] validation SKIPPED (--skip-validation): the dispatcher is the
authoritative "
                "gate and already validated this input."
            )
            val_results, val_context = [], {}
        else:
            val_results, val_context = validator_registry.run_all_with_context(
                local_video, config
            )
        failures = [r for r in val_results if not r.passed]
        db = Database(os.path.join(scratch, config.output.db_name))
        db.add_video(
            video_id,
            local_video,
            "failed" if failures else "processed",
            [r.model_dump() for r in val_results],
        )

        def _fail(rc: int) -> int:
            # A worker-side FAILURE (validation, unknown segmenter, or a detector
timeout/abort the
            # segmenter surfaced as a Failure) must NOT masquerade as a 0-rally "success".
Write + push
            # a status="failed" report.json/intervals.csv (0 segments) so the control
plane's bucket
            # truth (submit-cache probe, restart-recovery) and the alpha-user result surface
see an
            # explicit FAILURE ? the wf294 silent-failure fix. Best-effort: a push hiccup
must never
            # mask the original failure. THEN the M6 done-marker (real rc + status) so a
remote
```

```
            # dispatcher's bucket-poll terminates FAST instead of hanging until its budget
expires.
            try:
                _serialize_and_push_outputs(
                    scratch,
                    args.out_uri,
                    video_id,
                    [],
                    "failed",
                    storage_cfg,
                    str(getattr(args, "video_uri", "") or ""),
                )
            except Exception as e:  # never mask the real failure on an artifact-push hiccup
                logger.warning("[worker] could not push failure artifacts: %s", e)
            # DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).
            if getattr(args, "done_uri", None):
                _write_done_marker(
                    args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
                )
            return rc

        segments: List[dict] = []
        segmenter_actual = None
        segmentation_ran = False
        status = "failed"
        try:
            if failures:
                print(
                    "[worker] validation FAILED: "
                    + "; ".join(f"{r.validator_name}: {r.message}" for r in failures)
                )
                return _fail(2)
            seg_name = args.segmenter or config.indexing.default_segmenter
            segmenter_cls = segmenter_registry.get(seg_name)
            if not segmenter_cls:
                print(
                    f"[worker] unknown segmenter: {seg_name!r} "
                    f"(have {list(segmenter_registry.list_available())})"
                )
                return _fail(2)
            segmenter_actual = seg_name
            result = segmenter_cls(db, config).process_video(
                video_id, local_video, max_frames=args.max_frames
            )
            segmentation_ran = True
            if isinstance(result, Failure):
                print(f"[worker] segmenter FAILED: {result.message}")
                return _fail(3)
            segments = result.value if hasattr(result, "value") else result
            status = "success"
            # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
            # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
            # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
            if not segments:
                detector_impl = (
                    os.environ.get("RALLY_DETECTOR_IMPL")
                    or getattr(config.indexing, "detector_impl", None)
                    or "auto"
                )
                logger.warning(
                    "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
```

```
                    "The detector substrate yielded no usable trajectories/windows ? check
the detector "
                    "logs above; common causes: a missing ${AI_PROVIDER}_API_KEY with "
                    "skip_ai_handoff=False (the segmenter bails BEFORE detection ? set the
key or "
                    "indexing.skip_ai_handoff=true; cloud-serving now fails this at
startup), native "
                    "runner UNAVAILABLE (weights/repo/WASB_PYTHON env), the WASB env
produced no "
                    "detections, or the input resolution differs from the tuned proxy
resolution. "
                    "Re-run with -v for the native command + frame counts.",
                    video_id,
                    segmenter_actual,
                    detector_impl,
                )
        finally:
            # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
            emit_indexer_report(
                db=db,
                video_id=video_id,
                video_path=local_video,
                config=config,
                val_results=val_results,
                val_context=val_context,
                segmenter_requested=args.segmenter,
                segmenter_actual=segmenter_actual,
                was_reingest=False,
                segmentation_ran=segmentation_ran,
            )

        # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
        pushed = _serialize_and_push_outputs(
            scratch,
            args.out_uri,
            video_id,
            segments,
            status,
            storage_cfg,
            str(getattr(args, "video_uri", "") or ""),
        )

        print(
            f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):"
        )
        for u in pushed:
            print("   ", u)

        # M6: a remote dispatcher passes --done-uri; write the SUCCESS completion marker so
its bucket-poll
        # terminates (carries video_id ? the dispatcher's output path + rc=0). Worker
FAILURES (rc 2/3)
        # write their own marker via _fail() above, so the dispatcher fails FAST either way
(no 30-min hang
        # on a bad video). DEFAULT-OFF: no --done-uri ? no marker written (the unchanged
local run).
        if getattr(args, "done_uri", None):
            _write_done_marker(
                args.done_uri, video_id, 0, len(segments), status, storage_cfg, scratch
            )
        return 0


```

```
422     #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a
423     #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
collision).
424     _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")
425
426     #: Leading ISO-date prefix on a canonical label name (``labels/<YYYY-MM-
DD>_<name>.rallies.csv``,
427     #: docs/DATA_IN_GCS.md §7.1). It disambiguates recycled GoPro names by authoring date
but is NOT
428     #: part of the clip's join key, so it's stripped when deriving the videos/ stem.
429     _LABEL_DATE_PREFIX = re.compile(r"^\d{4}-\d{2}-\d{2}_")
430
431
432     def _label_clip_name(fname: str) -> str:
433         """Derive the clip name (the ``videos/<name>.<ext>`` join key) from a label
filename.
434
435         Strips both the ``.rallies.csv`` / ``.csv`` suffix and an optional leading ISO-date
prefix, so a
436         canonical dated label and a legacy bare label resolve to the SAME bare clip stem:
437
438         * ``2026-06-22_GX020094.rallies.csv`` -> ``GX020094``       (canonical, date-
stamped)
439         * ``GX020094.rallies.csv``            -> ``GX020094``       (legacy bare ? same
stem)
440         * ``GX020094_proxy.rallies.csv``      -> ``GX020094_proxy`` (``_proxy`` is part of
the name, kept)
441         * ``2026-06-21_mahadevpura_1.rallies.csv`` -> ``mahadevpura_1`` (underscores in
<name> survive)
442
443         Without the date strip, ``--trajectory-source detector`` can't match a dated label
to its bare
444         video stem, so every clip is skipped and train aborts with ``<2 videos``
(DATA_IN_GCS §7b #1).
445         """
446         name = fname.replace(".rallies.csv", "").replace(".csv", "")
447         return _LABEL_DATE_PREFIX.sub("", name)
448
449
450     def _probe_video_fps_width(path: str):
451         """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.
452
453         The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a
WRONG
454         fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles
every time).
455         cv2's ``_probe_fps_width`` silently defaults to (30, 1280) on a read miss ? fine for
the
456         inference report (it flags the fallback) but corrupting for REAL training data. So
here we
457         probe with ffprobe (same tool as ``get_video_duration``) and return None so the
caller SKIPS
458         rather than guesses.
459         """
460         import json
461         import subprocess
462
463         try:
464             out = subprocess.check_output(
465                 [
466                     ffprobe_bin(),
467                     "-v",
468                     "error",
469                     "-select_streams",
```

```
                    "v:0",
                    "-show_entries",
                    "stream=r_frame_rate,width",
                    "-of",
                    "json",
                    path,
                ],
                stderr=subprocess.DEVNULL,
            ).decode()
            st = (json.loads(out).get("streams") or [{}])[0]
            num, _, den = str(st.get("r_frame_rate", "0/1")).partition("/")
            fps = float(num) / float(den or "1")
            width = float(st.get("width") or 0)
            if fps > 0 and width > 0:
                return fps, width
        except (
            subprocess.SubprocessError,
            ValueError,
            KeyError,
            OSError,
            json.JSONDecodeError,
        ):
            pass
        return None


    def _resolve_train_detector(args: argparse.Namespace, config: Any):
        """Build the trajectory DetectorRunner for `--trajectory-source detector`, honouring
        detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU
box,
        stub=CPU mock). Returns the runner, or prints an error + returns None.

        Reuses the segmenter's `_resolve_runner` seam so the worker and the live CLI build
the
        detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
has
        no shuttle detector and is rejected.
        """
        from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter

        seg_cls = segmenter_registry.get(args.segmenter)
        if not seg_cls:
            print(
                f"[worker] unknown segmenter: {args.segmenter!r} "
                f"(have {list(segmenter_registry.list_available())})"
            )
            return None
        seg = seg_cls(None, config)
        if not isinstance(seg, TrajectoryHybridSegmenter):
            print(
                f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
                f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid / fusion_hybrid)."
            )
            return None
        try:
            runner, _ = seg._resolve_runner(config.indexing)
        except NotImplementedError as e:
            # e.g. detector_impl='native' for a family without a native runner (tracknet, or
            # fusion with a tracknet substrate). Fail cleanly (rc!=0), not with a raw
traceback.
            print(f"[worker] detector not available: {e}")
            return None
        ok, msg = runner.healthcheck()
        if not ok:
            print(f"[worker] detector environment not ready: {msg}")
```

```
531             return None
532         print(f"[worker] detector ready ({args.segmenter}): {msg}")
533         return runner
534 
535 
536     def cmd_train(args: argparse.Namespace) -> int:
537         """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
shell out
538         to training.gen0.harness (backend must NOT import training) -> push the artifact
bundle.
539 
540         Two trajectory sources (the train pipeline + harness are identical either way):
541           * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
synthetic
542             shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
CI.
543           * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
544             DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
This
545             is the M3.5 "first real training" path; only the trajectory source flips.
546         """
547         import subprocess
548 
549         config = load_config(args.config) if args.config else load_config()
550         _apply_overrides(config, args.set)
551         if not _run_startup_checks(config):
552             return 2
553         storage_cfg = config.storage.model_dump()
554 
555         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
556         labels_dir = os.path.join(scratch, "labels")
557         traj_dir = os.path.join(scratch, "trajectories")
558         videos_dir = os.path.join(scratch, "videos")
559         os.makedirs(labels_dir, exist_ok=True)
560         os.makedirs(traj_dir, exist_ok=True)
561 
562         corpus = args.corpus_uri.rstrip("/")
563         label_uris = sorted(
564             u
565             for u in storage.list_uris(f"{corpus}/labels/", config=storage_cfg)
566             if u.lower().endswith(".csv")
567         )
568         if len(label_uris) < 2:
569             print(
570                 f"[worker] need >=2 labeled videos for leave-one-video-out; found
{len(label_uris)} "
571                 f"under {corpus}/labels/"
572             )
573             return 2
574 
575         # Real-detector source: resolve the runner once + index the bucket's videos/ by
stem.
576         runner = None
577         video_by_stem: dict = {}
578         if args.trajectory_source == "detector":
579             os.makedirs(videos_dir, exist_ok=True)
580             runner = _resolve_train_detector(args, config)
581             if runner is None:
582                 return 2
583             for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
584                 vname = storage.parse_uri(vuri).name
585                 if not vname.lower().endswith(_VIDEO_EXTS):
586                     continue  # skip sidecars (.srt/.json/thumbnails) so they can't shadow
the video
587                 video_by_stem[os.path.splitext(vname)[0]] = vuri
```

```
588
589         print(f"[worker] trajectory source: {args.trajectory_source}")
590         specs: List[dict] = []
591         for uri in label_uris:
592             fname = storage.parse_uri(uri).name  # e.g. 2026-06-22_GX020094.rallies.csv
593             name = _label_clip_name(fname)  # -> GX020094 (date prefix stripped)
594             local_label = storage.fetch(
595                 uri, os.path.join(labels_dir, fname), config=storage_cfg
596             )
597             if args.trajectory_source == "detector":
598                 vuri = video_by_stem.get(name)
599                 if not vuri:
600                     print(
601                         f"[worker] no video for {name!r} under {corpus}/videos/ ? skipping."
602                     )
603                     continue
604                 local_video = storage.fetch(
605                     vuri,
606                     os.path.join(videos_dir, storage.parse_uri(vuri).name),
607                     config=storage_cfg,
608                 )
609                 video_id = compute_video_id(local_video)
610                 # Real fps/width drive the trajectory frame->time mapping; PROBE (don't
guess) ? a
611                 # wrong fps silently mis-aligns every label, so skip the video if ffprobe
can't read it.
612                 meta = _probe_video_fps_width(local_video)
613                 if meta is None:
614                     print(
615                         f"[worker] could not probe fps/width for {name!r} (ffprobe) ?
skipping (won't guess)."
616                     )
617                     continue
618                 fps, frame_width = meta
619                 traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
620                 if not traj:
621                     print(
622                         f"[worker] detector produced no trajectory for {name!r} ? skipping."
623                     )
624                     continue
625                 specs.append(
626                     {
627                         "name": name,
628                         "trajectory": traj,
629                         "labels": local_label,
630                         "fps": fps,
631                         "frame_width": frame_width,
632                     }
633                 )
634             else:
635                 from backend.pipeline.detectors.stub_runner import (
636                     synth_trajectory_from_labels,
637                 )
638
639                 traj = os.path.join(traj_dir, f"{name}_traj.csv")
640                 synth_trajectory_from_labels(
641                     local_label, traj, fps=args.fps, frame_width=args.frame_width
642                 )
643                 specs.append(
644                     {
645                         "name": name,
646                         "trajectory": traj,
647                         "labels": local_label,
648                         "fps": args.fps,
649                         "frame_width": args.frame_width,
```

```
                    }
                )

        if len(specs) < 2:
            print(
                f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}."
            )
            return 2
        manifest_path = os.path.join(scratch, "manifest.json")
        with open(manifest_path, "w", encoding="utf-8") as f:
            json.dump(specs, f, indent=2)
        src_label = (
            "real detector" if args.trajectory_source == "detector" else "CPU-mock stub"
        )
        print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")

        out_dir = os.path.join(scratch, "artifacts")
        cmd = [
            sys.executable,
            "-m",
            "training.gen0.harness",
            "--manifest",
            manifest_path,
            "--out",
            out_dir,
            "--version",
            args.version,
        ]
        if args.floor:
            floor_local = (
                storage.fetch(
                    args.floor, os.path.join(scratch, "floor.json"), config=storage_cfg
                )
                if "://" in args.floor
                else args.floor
            )
            cmd += ["--floor", floor_local]
        print("[worker] training:", " ".join(cmd))
        rc = subprocess.run(cmd).returncode
        if rc != 0:
            print(f"[worker] gen0 harness failed (rc={rc})")
            return rc

        # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
        bundle = os.path.join(out_dir, args.version)
        out_prefix = args.out_uri.rstrip("/")
        pushed = []
        for fn in sorted(os.listdir(bundle)):
            uri = f"{out_prefix}/weights/{args.version}/{fn}"
            storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
            pushed.append(uri)
        print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
        for u in pushed:
            print("   ", u)
        return 0


    def build_parser() -> argparse.ArgumentParser:
        p = argparse.ArgumentParser(
            prog="python -m backend.worker",
            description="Storage-aware worker: pull ? run pipeline ? push.",
        )
        p.add_argument(
            "-v",
```

```
714             "--verbose",
715             action="store_true",
716             help="DEBUG logging (the native detector command, frame counts, per-stage
detail)",
717         )
718         sub = p.add_subparsers(dest="cmd", required=True)
719
720         pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
721         pi.add_argument(
722             "--video-uri", required=True, help="local path / local:// / s3:// / gcs://"
723         )
724         pi.add_argument(
725             "--out-uri",
726             required=True,
727             help="output prefix (outputs/<video_id>/? is appended)",
728         )
729         pi.add_argument(
730             "--segmenter", default=None, help="default: indexing.default_segmenter"
731         )
732         pi.add_argument(
733             "--config", default=None, help="config.json path (default: ./config.json)"
734         )
735         pi.add_argument(
736             "--scratch", default=None, help="local scratch dir (default: a temp dir)"
737         )
738         pi.add_argument("--max-frames", type=int, default=None)
739         pi.add_argument(
740             "--done-uri",
741             default=None,
742             help="M6: storage URI for a completion MARKER (video_id + rc) written on a "
743             "successful run, so a remote (Cloud Run) dispatcher's bucket-poll terminates. "
744             "Default-OFF: omit it for the normal local run.",
745         )
746         pi.add_argument(
747             "--set",
748             action="append",
749             default=[],
750             metavar="k=v",
751             help="config override, e.g. --set indexing.skip_ai_handoff=true",
752         )
753         pi.add_argument(
754             "--skip-validation",
755             action="store_true",
756             help="skip the worker's re-validation: the dispatcher (control-plane) already
ran the "
757             "validators with its resolved config and only dispatches on PASS, so re-
validating here is "
758             "redundant, decodes the video again, and can contradict the gate. Default-OFF:
the direct "
759             "`worker infer` CLI still validates.",
760         )
761         pi.set_defaults(func=cmd_infer)
762
763         pt = sub.add_parser(
764             "train",
765             help="Gen-0 train-from-bucket (labels [+videos] -> trajectories -> harness)",
766         )
767         pt.add_argument(
768             "--corpus-uri",
769             required=True,
770             help="prefix containing labels/ (+ videos/ for "
771             "--trajectory-source detector), e.g. s3://bucket or gcs://bucket",
772         )
773         pt.add_argument(
774             "--out-uri",
```

```
            required=True,
            help="output prefix (weights/<version>/ is appended)",
        )
        pt.add_argument("--version", required=True, help="semver for the artifact bundle")
        pt.add_argument(
            "--trajectory-source",
            choices=["synth", "detector"],
            default="synth",
            help="synth = CPU-mock label-consistent trajectories (default, no GPU); "
            "detector = REAL shuttle trajectories via the resolved DetectorRunner "
            "(needs videos/ in the corpus; honours detector_impl).",
        )
        pt.add_argument(
            "--segmenter",
            default="wasb_hybrid",
            help="trajectory hybrid whose detector is used for --trajectory-source
detector",
        )
        pt.add_argument(
            "--floor", default=None, help="floor baseline JSON (local path or storage URI)"
        )
        pt.add_argument("--config", default=None)
        pt.add_argument("--scratch", default=None)
        pt.add_argument(
            "--fps",
            type=float,
            default=30.0,
            help="synth trajectory fps (detector probes the video)",
        )
        pt.add_argument(
            "--frame-width",
            type=float,
            default=1280.0,
            help="synth trajectory frame width (detector probes the video)",
        )
        pt.add_argument("--set", action="append", default=[], metavar="k=v")
        pt.set_defaults(func=cmd_train)
        return p


    def _setup_logging(verbose: bool) -> None:
        """Configure logging for the worker CLI so the pipeline's existing INFO/ERROR
diagnostics are
        VISIBLE ? the worker previously ran with no logging config, so every
segmenter/detector log
        (detector choice, 'Running native WASB inference', availability failures) was
silently dropped.
        That is exactly why a 0-rally cold-start gave nothing to analyse. Configured here
(the CLI
        entry), not at import, so importing the module never reconfigures a caller's
logging."""
        # force=True: basicConfig is a no-op if the root already has handlers (e.g. a
library
        # configured logging at import), which would leave the level at WARNING and re-
suppress the
        # diagnostics. Forcing reconfiguration means the worker reliably controls its own
verbosity.
        logging.basicConfig(
            level=logging.DEBUG if verbose else logging.INFO,
            format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
            datefmt="%H:%M:%S",
            force=True,
        )


```

```
831     def main(argv=None) -> int:
832         args = build_parser().parse_args(argv)
833         _setup_logging(getattr(args, "verbose", False))
834         return args.func(args)
835
836
837     if __name__ == "__main__":
838         sys.exit(main())
839
```

14. assistant (2026-07-08T05:05:58.441Z)

This is a large task. Let me set up a todo list and continue studying the key files: the compute backend, config models, VideoCompiler, and the job dispatch layer.

15. user (2026-07-08T05:05:59.457Z)

```
1       """``CloudRunCompute`` ? dispatch one inference job to a Cloud Run GPU **Job** and
bucket-poll
2       for its completion (COMPUTE_DECOUPLED_SERVING M6, decisions D16 + D7).
3
4       Flow (the cloud-serving compute path):
5         1. **Dispatch** a Cloud Run Job execution whose container runs ``python -m
backend.worker infer
6            --video-uri <gs> --out-uri <gs> --done-uri <gs> ?`` (the SAME worker, on the L4
GPU).
7         2. **Bucket-poll** ``done_uri`` via ``execution_policy.poll_until`` + the storage
layer (D16 ? the
8            completion signal is a tiny marker object the worker writes, NOT the Cloud Run API
status, so
9            it is storage-backed and reuses the existing park/resume primitives). Scale-to-zero
(D7): the
10           cold-start is absorbed by the async poll. On a poll TIMEOUT (``max_wait_sec`` can
undercut the
11           job's own ``--task-timeout`` budget) the shim does ONE final marker check ? a slow-
but-healthy
12           worker that finished just past the deadline is rescued, not wasted ? then best-
effort CANCELS
13           the still-running execution so the L4 doesn't keep billing until ``--task-
timeout``.
14           *Fast negative path:* while the marker stays the SOLE success signal, each poll
also probes the
15           Cloud Run Executions API ? if the execution has TERMINATED without ever writing a
marker (a
16           worker crash/OOM/argparse-abort before the marker push), the shim fails fast +
diagnostically
17           (``RemoteExecutionFailed``) instead of spinning the full ~65-min budget on a futile
wait.
18        3. **Read** the marker (it carries ``video_id`` + the worker's returncode) ? fetch
19           ``outputs/<video_id>/report.json`` ? ``ComputeResult``.
20
21      The dispatched container is forced to run **INLINE** (``compute_target=local_gpu`` +
native detector)
22      so it never recursively re-dispatches ? the dispatcher decides cloud, the container just
computes.
23
24      The Cloud Run trigger is an **injected client** (``CloudRunJobClient``) so the whole
shim is unit-tested
25      with a fake (no GCP). The real ``GoogleCloudRunJobClient`` lazy-imports ``google-cloud-
run`` and is
26      owner-verified on the M7 real run.
27      """
28
29      from __future__ import annotations
30
```

```
    import json
    import logging
    import os
    import tempfile
    import uuid
    from abc import ABC, abstractmethod
    from typing import Any, Callable, List, Optional

    from backend.compute.base import (
        ComputeBackend,
        ComputeJobSpec,
        ComputeResult,
        RemoteExecutionFailed,
    )
    from backend.infrastructure.execution_policy import (
        DEFAULT_POLICY,
        ExecutionPolicy,
        PolicyTimeout,
        poll_until,
    )
    from backend.storage import fetch, get_storage, parse_uri

    LOG = logging.getLogger("compute.cloud_run")

    # The container worker must run INLINE on the GPU ? never re-enter the cloud-dispatch
branch.
    # (compute_target=local_gpu disarms the cmd_infer dispatch guard; native = the L4 WASB
runner.)
    _DEFAULT_CONTAINER_OVERRIDES: tuple[str, ...] = (
        "deployment.compute_target=local_gpu",
        "indexing.detector_impl=native",
    )


    class CloudRunJobClient(ABC):
        """Triggers a Cloud Run **Job** execution with per-execution container arg
overrides.

        Abstracted so ``CloudRunCompute`` is testable with a fake (no GCP). A real
implementation
        overrides the job's container ``args`` for this execution and starts it."""

        @abstractmethod
        def run_job(
            self,
            job_name: str,
            argv: List[str],
            *,
            region: str,
            project: Optional[str] = None,
        ) -> str:
            """Start a Cloud Run Job execution running ``python -m backend.worker <argv>``;
return an
            execution id/name (for logs). Does NOT block on completion (the caller bucket-
polls)."""

        @abstractmethod
        def cancel_execution(self, execution: str) -> None:
            """Cancel a running execution by the name ``run_job`` returned. Called when the
bucket-poll
            times out so the abandoned execution doesn't keep billing the GPU until
``--task-timeout``.
            May raise ? the caller treats every failure as best-effort (the original poll
timeout is
            NEVER masked by a cancel error)."""
```

```
87
88          def execution_terminal_state(self, execution: str) -> Optional[str]:
89              """Return the execution's TERMINAL state as a short label (``"Failed"`` /
``"Cancelled"`` /
90              ``"Succeeded"``) if it has finished, else ``None`` (still pending/running, or
the state
91              can't be determined).
92
93              ADVISORY, not authoritative: it feeds ``run_infer``'s fast negative path so a
worker that
94              terminated WITHOUT writing the done-marker (crash/OOM/pre-marker abort) fails
fast instead
95              of bucket-polling the full budget. The done-marker stays the ONLY success
signal.
96
97              NON-abstract with a default of ``None`` (additive / default-OFF): a client that
can't ? or
98              chooses not to ? poll the Executions API degrades transparently to today's
marker-only
99              polling. Implementations MAY raise on a transient API error ? ``run_infer``
treats a raise
100             as ``None`` (unknown ? keep waiting), so a flaky status API never fails a
healthy run."""
101             return None
102
103
104     class GoogleCloudRunJobClient(CloudRunJobClient):
105         """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real
run; CI uses
106         a fake, so this code path is never exercised in tests (the SDK isn't a test
dependency)."""
107
108         def run_job(
109             self,
110             job_name: str,
111             argv: List[str],
112             *,
113             region: str,
114             project: Optional[str] = None,
115         ) -> str:
116             # A None/empty project would build a bogus "projects/None/locations/..." path
(job_path
117             # requires a real str). Fail loudly BEFORE the SDK import so the misconfig is
exercisable
118             # in CI (where google-cloud-run is absent) and so mypy narrows project to str
for job_path.
119             if not project:
120                 raise RuntimeError(
121                     "GOOGLE_CLOUD_PROJECT is unset ? the real Cloud Run dispatch needs a GCP
project id "
122                     "to resolve the job path; set it on the control plane (see
get_compute_backend)."
123                 )
124             try:
125                 from google.cloud import run_v2  # type: ignore[attr-defined]
126             except Exception as exc:  # pragma: no cover - real SDK not present in CI
127                 raise RuntimeError(
128                     "google-cloud-run is required for CloudRunCompute's real client; install
it on the "
129                     "control plane (it is intentionally NOT a CI/test dependency)."
130                 ) from exc
131             client = (
132                 run_v2.JobsClient()
133             )  # pragma: no cover - exercised only on the real M7 run
134             name = client.job_path(project, region, job_name)
```

```
            overrides = run_v2.RunJobRequest.Overrides(
                container_overrides=[
                    run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)
                ]
            )
            op = client.run_job(
                request=run_v2.RunJobRequest(name=name, overrides=overrides)
            )
            return (
                getattr(op, "metadata", None)
                and getattr(op.metadata, "name", "")
                or "execution"
            )

        def cancel_execution(self, execution: str) -> None:
            # run_job falls back to the literal string "execution" when the operation
metadata carries
            # no name ? that is NOT a resolvable resource name. Fail loudly BEFORE the SDK
import
            # (mirrors run_job's project guard, #342) so the misuse is exercisable in CI
(where
            # google-cloud-run is absent).
            if "/executions/" not in (execution or ""):
                raise RuntimeError(
                    f"not a fully-qualified Cloud Run execution name: {execution!r} ?
expected "
                    "projects/<p>/locations/<l>/jobs/<j>/executions/<e> (run_job could not
resolve "
                    "one from the operation metadata, so there is nothing to cancel by
name)."
                )
            try:
                from google.cloud import run_v2  # type: ignore[attr-defined]
            except Exception as exc:  # pragma: no cover - real SDK not present in CI
                raise RuntimeError(
                    "google-cloud-run is required to cancel a Cloud Run execution; install
it on the "
                    "control plane (it is intentionally NOT a CI/test dependency)."
                ) from exc
            client = (
                run_v2.ExecutionsClient()
            )  # pragma: no cover - exercised only on a real run
            client.cancel_execution(request=run_v2.CancelExecutionRequest(name=execution))

        def execution_terminal_state(self, execution: str) -> Optional[str]:
            # Advisory status probe (see the ABC docstring): fetch the execution and
classify it via the
            # pure, unit-tested ``_classify_execution_state``. A non-resolvable name /
absent SDK returns
            # None (can't probe ? keep marker-polling) ? unlike run_job/cancel this never
raises on bad
            # input, because the probe is best-effort and must not fail a healthy run.
            if "/executions/" not in (execution or ""):
                return None
            try:
                from google.cloud import run_v2  # type: ignore[attr-defined]
            except Exception:  # pragma: no cover - real SDK not present in CI
                return None
            client = (
                run_v2.ExecutionsClient()
            )  # pragma: no cover - exercised only on a real run
            exe = client.get_execution(  # pragma: no cover - real run only
                request=run_v2.GetExecutionRequest(name=execution)
            )
            return _classify_execution_state(exe)  # pragma: no cover - real run only
```

```
190
191
192     def _classify_execution_state(exe: Any) -> Optional[str]:
193         """Map a Cloud Run ``run_v2.Execution`` to a terminal state label (``"Failed"`` /
``"Cancelled"``
194         / ``"Succeeded"``), or ``None`` if it is still pending/running. PURE (no
SDK/network) so it is
195         unit-tested against both real proto-plus ``Execution`` objects and lightweight fakes
? the actual
196         ``get_execution`` fetch is the only part that can't run in CI.
197
198         ``google-cloud-run`` is a **proto-plus** library, so a ``google.protobuf.Timestamp``
field is
199         marshalled to a native Python ``datetime`` when set and to ``None`` when unset ? NOT
to a proto
200         ``Timestamp`` with a ``.seconds`` field (a ``datetime`` has ``.second``, singular).
Terminality is
201         therefore "``completion_time`` is not None"; the per-task counts (plain ints) then
classify it,
202         with a cancel taking precedence (a cancelled execution reports ``cancelled_count >
0``)."""
203         if getattr(exe, "completion_time", None) is None:
204             return None  # no terminal completion timestamp yet ? still pending/running
205         if getattr(exe, "cancelled_count", 0):
206             return "Cancelled"
207         if getattr(exe, "failed_count", 0):
208             return "Failed"
209         if getattr(exe, "succeeded_count", 0):
210             return "Succeeded"
211         return "Failed"  # terminal but no positive success ? treat as failed (conservative)
212
213
214     class CloudRunCompute(ComputeBackend):
215         """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""
216
217         def __init__(
218             self,
219             *,
220             run_client: CloudRunJobClient,
221             job_name: str,
222             region: str,
223             project: Optional[str] = None,
224             storage_config: Optional[dict] = None,
225             policy: ExecutionPolicy = DEFAULT_POLICY,
226             token: Optional[str] = None,
227             container_overrides: Optional[tuple[str, ...]] = None,
228         ) -> None:
229             self._client = run_client
230             self._job_name = job_name
231             self._region = region
232             self._project = project
233             self._storage_config = storage_config or {}
234             self._policy = policy
235             # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
collide.
236             # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic
(tests).
237             self._token = token
238             self._container_overrides = (
239                 _DEFAULT_CONTAINER_OVERRIDES
240                 if container_overrides is None
241                 else tuple(container_overrides)
242             )
243
244         def run_infer(
```

```
            self,
            spec: ComputeJobSpec,
            *,
            on_wait: "Callable[[float], None] | None" = None,
        ) -> ComputeResult:
            out_root = spec.out_uri.rstrip("/")
            token = self._token or uuid.uuid4().hex
            done_uri = f"{out_root}/_dispatch/{token}.done"
            argv: List[str] = [
                "infer",
                "--video-uri",
                spec.video_uri,
                "--out-uri",
                spec.out_uri,
                "--done-uri",
                done_uri,
            ]
            if spec.segmenter:
                argv += ["--segmenter", spec.segmenter]
            if spec.skip_validation:
                # The control-plane already validated + only dispatches on PASS ? tell the
worker not to
                # re-decode and re-validate (redundant, and its config may resolve different
thresholds).
                argv.append("--skip-validation")
            for kv in (*spec.config_overrides, *self._container_overrides):
                argv += ["--set", kv]

            execution = self._client.run_job(
                self._job_name, argv, region=self._region, project=self._project
            )
            LOG.info(
                "cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
                self._job_name,
                execution,
                done_uri,
            )

            try:
                marker = poll_until(
                    lambda: self._poll_check(done_uri, str(execution or "")),
                    self._policy,
                    label="cloud-run-infer",
                    logger=LOG,
                    # Live heartbeat (#482): each still-waiting tick reaches the caller so
                    # job.progress moves during the whole GPU run instead of freezing.
                    on_tick=on_wait,
                )
            except PolicyTimeout as timeout_exc:
                # RemoteExecutionFailed (the fast negative path in _poll_check) is NOT a
PolicyTimeout,
                # so it propagates past here uncaught ? the dispatch handler maps it to a
distinct,
                # clearly-labelled failure instead of the ~65-min opaque timeout this branch
handles.
                marker = self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
            video_id = str(marker.get("video_id", ""))
            # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
            # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
            rc = int(marker.get("returncode", 1))
            if not video_id:
                LOG.warning(
                    "cloud-run: job=%s completion marker present but empty/unreadable (no "
```

```
                    "video_id) ? treating as failure",
                    self._job_name,
                )
                rc = rc or 1
            outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
            report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
            LOG.info(
                "cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id, rc
            )
            return ComputeResult(
                returncode=rc,
                outputs_uri=outputs_uri,
                video_id=video_id,
                report=report,
                execution=str(execution or ""),
            )

        def _poll_check(self, done_uri: str, execution: str) -> Optional[dict]:
            """One bucket-poll iteration, hardened with an Executions-API fast negative
path.

            The done-marker is the SOLE success signal and is checked FIRST ? a present
marker always
            wins (even if the execution's status later flips), so a worker that wrote a
SUCCESS *or a
            FAILURE* marker resolves through the normal path with its real returncode. Only
when the
            marker is absent do we consult the execution status: a worker that TERMINATED
without ever
            writing a marker (crash / OOM / an old image argparse-aborting on an unknown
flag) would
            otherwise make this poll spin the full ``remote_poll_max_wait_sec`` (~65 min)
before failing
            opaquely. Detecting that terminal state collapses the whole class to a prompt,
actionable
            failure.

            Returns the marker dict (resolve the poll), ``None`` (keep waiting), or raises
            ``RemoteExecutionFailed`` (abort the poll immediately ? the execution is over,
no marker
            will ever appear)."""
            marker = self._read_marker(done_uri)
            if marker is not None:
                return marker  # success signal ? authoritative even if status later flips
            state = self._terminal_execution_state(execution)
            if state is None:
                return None  # still pending/running, or status unknown ? keep bucket-
polling
            # The execution is terminal but no marker is present. Re-check the marker ONCE
to close the
            # race where the worker wrote it between our two reads (mirrors the post-timeout
rescue in
            # _handle_poll_timeout). If it is genuinely absent, the process is gone and no
marker will
            # EVER appear ? fail fast + diagnostically rather than wait out the whole poll
budget.
            marker = self._read_marker(done_uri)
            if marker is not None:
                return marker
            raise RemoteExecutionFailed(
                f"cloud-run: job={self._job_name} execution={execution or '<unknown>'}
reached "
                f"terminal state {state!r} with NO completion marker ? the worker exited
before "
                "writing one (crash / OOM / pre-marker abort, or a failed marker upload on
```

```
an "
352                 "otherwise-Succeeded run). The execution is over, so the marker will never
appear; "
353                 "inspect the Cloud Run Job execution log to diagnose: "
354                 f"`gcloud run jobs executions describe {execution or '<execution>'} "
355                 f"--region {self._region}`."
356             )
357
358         def _terminal_execution_state(self, execution: str) -> Optional[str]:
359             """Best-effort wrapper over the client's ``execution_terminal_state`` probe. A
raising or
360             flaky status API must NEVER kill a healthy wait (the done-marker stays the sole
authority),
361             so any exception ? and an empty/unresolvable execution name ? degrades to
``None`` (unknown
362             ? keep polling) and is DEBUG-logged, exactly today's marker-only behaviour."""
363             if not execution:
364                 return None
365             try:
366                 return self._client.execution_terminal_state(execution)
367             except Exception:  # noqa: BLE001 ? advisory probe: never let it kill the
bucket-poll
368                 LOG.debug(
369                     "cloud-run: execution status probe failed for %s ? treating as unknown",
370                     execution,
371                     exc_info=True,
372                 )
373                 return None
374
375         def _handle_poll_timeout(
376             self, done_uri: str, execution: str, timeout_exc: PolicyTimeout
377         ) -> dict:
378             """Poll deadline hit ? rescue a just-finished result, else cancel + re-raise.
379
380             The poll budget (``max_wait_sec``, default 1800s) can undercut the deployed
job's own
381             ``--task-timeout`` (3600s), so a slow-but-healthy worker may finish just past
the deadline:
382             ONE final marker check rescues that completed GPU run instead of wasting it. If
the marker
383             is genuinely absent, the execution is still running ? best-effort cancel it
(otherwise the
384             L4 keeps billing until ``--task-timeout``) and re-raise a ``PolicyTimeout``
carrying the
385             execution name so the operator can verify the state in the GCP console. A cancel
failure
386             NEVER masks the original timeout."""
387             try:
388                 late = self._read_marker(done_uri)
389             except Exception:  # noqa: BLE001 ? the final check is best-effort too: absence
? failure
390                 LOG.warning(
391                     "cloud-run: final post-timeout marker check failed ? treating as
absent",
392                     exc_info=True,
393                 )
394                 late = None
395             if late is not None:
396                 LOG.warning(
397                     "cloud-run: job=%s exec=%s done-marker found on the final post-timeout
check "
398                     "? rescuing the completed result",
399                     self._job_name,
400                     execution,
401                 )
```

```
402                 return late
403             try:
404                 self._client.cancel_execution(execution)
405             except Exception as cancel_exc:  # noqa: BLE001 ? best-effort: never mask the
timeout
406                 cancel_note = f"best-effort cancel FAILED ({cancel_exc})"
407                 LOG.warning(
408                     "cloud-run: best-effort cancel of execution=%s failed: %s",
409                     execution,
410                     cancel_exc,
411                     exc_info=True,
412                 )
413             else:
414                 cancel_note = "cancel requested"
415                 LOG.info("cloud-run: requested cancel of execution=%s", execution)
416             raise PolicyTimeout(
417                 f"{timeout_exc} [cloud-run job={self._job_name} "
418                 f"execution={execution or '<unknown>'}; {cancel_note} ? verify the execution
"
419                 "state in the GCP console]"
420             ) from timeout_exc
421
422         # --- storage-backed completion signal (bucket-poll) ---
423         def _read_marker(self, done_uri: str) -> Optional[dict]:
424             """The done-marker dict if the worker has written it, else None ? poll_until
keeps waiting."""
425             ref = parse_uri(done_uri)
426             if not get_storage(ref.backend, config=self._storage_config).exists(ref):
427                 return None
428             return (
429                 self._read_json(done_uri) or {}
430             )  # empty dict still terminates the poll (present == done)
431
432         def _read_json(self, uri: str) -> dict:
433             # Download to a real local dst FIRST: the cloud backends (gcs/s3) REQUIRE a dst
? a None dst
434             # raises ValueError ? unlike local's identity passthrough. Without an explicit
dst this read
435             # would silently return {} for every gs://|s3:// object (the only backends this
shim targets).
436             try:
437                 ref = parse_uri(uri)
438                 with tempfile.TemporaryDirectory(prefix="cr-read-") as td:
439                     dst = os.path.join(td, ref.name or "obj.json")
440                     local = fetch(uri, dst, config=self._storage_config)
441                     with open(local, "r", encoding="utf-8") as f:
442                         data: Any = json.load(f)
443                 return data if isinstance(data, dict) else {}
444             except Exception:
445                 LOG.warning("cloud-run: could not read %s", uri, exc_info=True)
446                 return {}
447
```

16. user (2026-07-08T05:06:00.017Z)

```
1       """Pluggable compute layer: in-process (default) + a Cloud Run GPU Job dispatch shim.
2
3       COMPUTE_DECOUPLED_SERVING M6. ``get_compute_backend(config)`` is the seam: it returns a
4       ``ComputeBackend`` ONLY when a non-local ``deployment.compute_target`` is selected, and
``None``
5       otherwise ? ``None`` means "run the worker's existing in-process path" (today's
behaviour, byte
6       identical). So the seam is **default-OFF**: with no profile / ``local_gpu`` /
``cpu_mock`` nothing
7       changes. Mirrors ``backend/storage``'s lazy factory idiom (cloud SDKs imported only when
```

```
needed).
8       """
9
10      from __future__ import annotations
11
12      import dataclasses
13      import os
14      from typing import Any, Optional
15
16      from backend.compute.base import (
17          ComputeBackend,
18          ComputeJobSpec,
19          ComputeResult,
20          RemoteExecutionFailed,
21      )
22      from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy
23
24      __all__ = [
25          "ComputeBackend",
26          "ComputeJobSpec",
27          "ComputeResult",
28          "RemoteExecutionFailed",
29          "get_compute_backend",
30      ]
31
32
33      def _storage_dict(config: Any) -> dict:
34          storage = getattr(config, "storage", None)
35          if storage is None:
36              return {}
37          if hasattr(storage, "model_dump"):
38              return storage.model_dump()  # type: ignore[no-any-return]
39          return dict(storage) if isinstance(storage, dict) else {}
40
41
42      def get_compute_backend(
43          config: Any,
44          *,
45          run_client: Optional[Any] = None,
46          policy: ExecutionPolicy = DEFAULT_POLICY,
47          token: Optional[str] = None,
48          target_override: Optional[str] = None,
49      ) -> Optional[ComputeBackend]:
50          """Return the compute backend for ``config.deployment.compute_target``, or ``None``
to run
51          in-process (the default-OFF path).
52
53          Only ``compute_target == "cloud_run"`` returns a non-None backend (a
``CloudRunCompute``). Its
54          Cloud Run coordinates come from the control-plane env (``CLOUD_RUN_JOB`` /
``CLOUD_RUN_REGION``
55          / ``GOOGLE_CLOUD_PROJECT``) so no secrets live in config. ``run_client`` is
injectable (tests
56          pass a fake; production defaults to the real ``google-cloud-run`` client, owner-
verified on M7).
57
58          ``target_override`` lets a single request force a target (the SPA compute-picker
maps a
59          per-request ``compute="cloud"`` to ``target_override="cloud_run"``) without mutating
the
60          server's configured default. ``None`` falls back to
``config.deployment.compute_target``.
61          """
62          target = (
63              target_override
```

```
64              if target_override is not None
65              else getattr(getattr(config, "deployment", None), "compute_target", "") or ""
66          )
67          if target != "cloud_run":
68              return None  # local_gpu / cpu_mock / "" ? run inline (today's path, unchanged)
69
70          # Lazy import so the cloud SDK shim is only pulled when a cloud target is actually
selected.
71          from backend.compute.cloud_run import CloudRunCompute, GoogleCloudRunJobClient
72
73          # Derive the poll budget from config UNLESS the caller injected an explicit policy ?
74          # deployment.remote_poll_max_wait_sec must outlast the worker Job's --task-timeout
so a
75          # slow-but-healthy paid GPU run is never abandoned mid-flight (CP-rebuild §4.3; the
old
76          # 1800s default undercut real 1080p60 jobs). Identity check on DEFAULT_POLICY keeps
an
77          # explicitly-passed policy (tests, callers with bespoke budgets) untouched.
78          if policy is DEFAULT_POLICY:
79              max_wait = getattr(
80                  getattr(config, "deployment", None), "remote_poll_max_wait_sec", None
81              )
82              if max_wait:
83                  policy = dataclasses.replace(DEFAULT_POLICY, max_wait_sec=float(max_wait))
84
85          job_name = os.environ.get("CLOUD_RUN_JOB", "rally-worker")
86          region = os.environ.get("CLOUD_RUN_REGION", "asia-south1")
87          project = os.environ.get("GOOGLE_CLOUD_PROJECT")
88          client = run_client if run_client is not None else GoogleCloudRunJobClient()
89          return CloudRunCompute(
90              run_client=client,
91              job_name=job_name,
92              region=region,
93              project=project,
94              storage_config=_storage_dict(config),
95              policy=policy,
96              token=token,
97          )
98
```

17. assistant (2026-07-08T05:06:05.692Z)

Let me read the compute base, config models, and find the compile job dispatch/API sites.

18. user (2026-07-08T05:06:07.211Z)

```
1       """The ``ComputeBackend`` ABC ? one interface for *where* the rally core runs.
2
3       COMPUTE_DECOUPLED_SERVING M6. Mirrors the ``StorageBackend`` ABC+factory idiom
4       (``backend/storage/``): a ``ComputeBackend`` decides **where** the pure
DETECT?WINDOW?STITCH
5       core executes ? in-process on the local box (truly-local) or on a **Cloud Run GPU Job**
6       (cloud-serving) ? without the core ever branching on the profile (D1).
7
8       The seam is **default-OFF**: only ``deployment.compute_target == "cloud_run"`` selects a
9       non-local backend. Every other target (``""`` / ``local_gpu`` / ``cpu_mock``) keeps the
worker's
10      existing in-process path **byte-identical** ? the factory returns ``None`` (= "run
inline").
11      """
12
13      from __future__ import annotations
14
15      from abc import ABC, abstractmethod
16      from dataclasses import dataclass, field
```

```
17      from typing import Callable, Optional
18
19
20      class RemoteExecutionFailed(RuntimeError):
21          """A dispatched remote job's execution reached a TERMINAL state without ever writing
its
22          completion marker ? the worker crashed / OOM'd / aborted (e.g. an old image
argparse-rejecting
23          an unknown flag) before the marker push, or the marker upload itself failed on an
otherwise
24          Succeeded run.
25
26          Distinct from ``PolicyTimeout`` (poll budget exhausted while the execution may STILL
be
27          running): this says the execution is DEFINITIVELY over and no marker will ever
appear, so the
28          dispatcher must stop bucket-polling immediately instead of burning the full
29          ``remote_poll_max_wait_sec`` (~65 min) on a guaranteed-futile wait. The message
carries the
30          job + execution name + terminal state + the ``gcloud`` command to read the execution
log, so
31          the whole "worker crashed before the done-marker" class fails fast AND
diagnostically."""
32
33
34      @dataclass(frozen=True)
35      class ComputeJobSpec:
36          """One inference job, profile-agnostic. ``video_id`` is the MD5 the worker will
(re)compute
37          from the SAME input bytes, so the output always lands at
``<out_uri>/outputs/<video_id>/`` ?
38          deterministic regardless of where the job ran."""
39
40          video_uri: str  # input (gs://? for cloud; a path/local:// for local)
41          out_uri: str  # output ROOT (gs://? bucket for cloud)
42          segmenter: Optional[str] = None
43          # Raw worker ``--set k=v`` overrides. A tuple (not list/dict) so the spec stays
frozen/hashable.
44          config_overrides: tuple[str, ...] = ()
45          # Skip the worker's redundant re-validation. The DISPATCHER (the cloud-serving
control-plane) is
46          # the authoritative validation gate: it runs the SAME validators with ITS resolved
config and only
47          # dispatches on PASS, so a worker re-run is (a) redundant, (b) a second full-video
decode, and
48          # (c) outright WRONG when the worker's config resolves different thresholds ? it can
REJECT a clip
49          # the gate already passed. Default False keeps the direct ``worker infer`` CLI (no
control-plane)
50          # validating exactly as before.
51          skip_validation: bool = False
52
53
54      @dataclass(frozen=True)
55      class ComputeResult:
56          """The terminal state of a dispatched job. ``returncode`` mirrors the worker's exit
code
57          (0 ok / 2 validation-or-config / 3 segmenter-failure), plus **4 = remote dispatch
timeout or an
58          empty/unreadable completion marker** (the cloud path only). ``video_id`` is known
once the job's
59          done-marker is read; ``outputs_uri`` is where ``intervals.csv`` / ``report.json``
landed."""
60
61          returncode: int
```

```
62          outputs_uri: str = ""
63          video_id: str = ""
64          report: dict = field(default_factory=dict)
65          # The backend's execution identity for PROVENANCE (e.g. the Cloud Run execution
name) ? surfaced
66          # on the job result so "the cloud path actually ran" is independently verifiable
(gcloud run jobs
67          # executions describe <execution>). Empty for the in-process path / backends without
one.
68          execution: str = ""
69
70
71      class ComputeBackend(ABC):
72          """Backend-agnostic compute. ``run_infer`` dispatches ONE inference job and returns
only once
73          its outputs are durably available (bucket-poll for cloud), so a caller treats local
and cloud
74          execution identically."""
75
76          @abstractmethod
77          def run_infer(
78              self,
79              spec: ComputeJobSpec,
80              *,
81              on_wait: "Callable[[float], None] | None" = None,
82          ) -> ComputeResult:
83              """Run ``spec`` to completion and return its ``ComputeResult``.
84
85              ``on_wait(elapsed_sec)`` (optional) is invoked periodically while the backend is
86              still WAITING on the remote work ? the caller's live-progress heartbeat (#482),
so a
87              long GPU run doesn't look frozen. Implementations without a wait loop may ignore
it."""
88
```

19. user (2026-07-08T05:06:08.012Z)

```
1       """Pydantic models for the application configuration.
2
3       This module defines the canonical, validated, typed configuration for the
4       Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6       Unknown / typo'd config keys are rejected at load time via extra="forbid".
7       """
8
9       from __future__ import annotations
10
11      from typing import Any, List, Literal, Optional
12
13      from pydantic import BaseModel, Field, field_validator, model_validator
14
15      Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18      class ValidationRelevanceConfig(BaseModel):
19          court_green_lower: list[int] = Field(default=[35, 40, 40])
20          court_green_upper: list[int] = Field(default=[85, 255, 255])
21          court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24      class ValidationConfig(BaseModel):
25          min_resolution_width: int = 1280
26          min_resolution_height: int = 720
27          min_fps: float = 29.9
28          min_blur_threshold: float = 20.0
```

```
        max_scene_cuts_per_minute: float = 0.5
        pts_max_gap_seconds: float = Field(
            default=10.0,
            ge=0.0,
            description=(
                "Maximum allowed keyframe gap (seconds) before flagging as spliced. "
                "GoPro H.264/H.265 recordings typically have GOP intervals of 1?4s; "
                "a gap >10s suggests edited/missing content. Mobile phones, screen "
                "recordings, and other encoders may use different GOP sizes ? tune "
                "this threshold for your recording device."
            ),
        )
        relevance: ValidationRelevanceConfig = Field(
            default_factory=ValidationRelevanceConfig
        )


    class TrackNetConfig(BaseModel):
        wsl_distro: str = "Ubuntu"
        repo_dir: str = "~/models/TrackNetV4"
        conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
        conda_env: str = "TrackNetV4"
        weights_path: str = ""
        queue_length: int = 5
        stage_in_wsl: bool = True
        wsl_stage_dir: str = "~/clips"
        timeout_sec: int = 1800  # 30 min; kills a hung WSL/GPU call (0 = no timeout)
        velocity_threshold: float = 0.02
        # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
        # decoded frame cache) are deleted automatically ? they are pure intermediates,
        # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
        # true only for debugging. On FAILURE they are always kept so a re-run can resume.
        keep_frames: bool = False
        # Absolute FLOOR (GB) on WSL free space (at wsl_stage_dir) before a run, on top of
the
        # ~15×-source headroom the frame-stage guard always enforces (D11 ? it now BLOCKS,
not warns).
        # 0 = no extra floor (the headroom guard still applies).
        min_free_gb: float = 0.0


    class WasbIndexConfig(BaseModel):
        """Typed config for the live WASB shuttle substrate (indexing.wasb).

        Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
        these knobs actually take effect ? previously this whole block was silently dropped
        because nested config sections defaulted to extra='ignore'.
        """

        wsl_distro: str = "Ubuntu"
        repo_dir: str = "~/models/WASB-SBDT"
        conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
        conda_env: str = "wasb"
        weights_path: str = (
            "~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
        )
        sport: str = "badminton"
        wsl_stage_dir: str = "~/clips"
        timeout_sec: int = 1800  # 30 min; kills a hung WSL/GPU call (0 = no timeout)
        velocity_threshold: float = 0.02
        # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner.
---
        # Env vars override these (WASB_DEVICE / WASB_PYTHON) for a config-free GPU box.
        # torch device for the native runner. "auto" = cuda-if-available-else-cpu (the M4
CPU fallback
```

```
90          # for a no-GPU box); "cuda" stays strict (hard-fails without a GPU ? no silent slow-
CPU downgrade).
91          device: str = "cuda"  # auto | cuda | mps | cpu
92          python_bin: str = (
93              "python"  # the `wasb` conda env python (invoked by path, no `conda activate`)
94          )
95          # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
96          # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
97          # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
98          keep_frames: bool = False
99          # Absolute FLOOR (GB) on WSL free space (at wsl_stage_dir) before a run, on top of
the
100         # ~15×-source headroom the frame-stage guard always enforces (D11 ? it now BLOCKS,
not warns).
101         # 0 = no extra floor (the headroom guard still applies).
102         min_free_gb: float = 0.0
103         # FAST PATH ? decode the video on the fly and feed frames straight to the detector,
104         # skipping the slow per-frame PNG extraction that dominates a long clip (~46k PNGs
for a
105         # 12-min 1080p60 video, which overran the 30-min timeout). Output is bit-identical
to the
106         # PNG path (PNG is lossless) ? VERIFIED on a real GPU 2026-06-20 (native stream ==
WSL disk,
107         # 1194/1195 frames byte-identical; native run-to-run byte-identical /
deterministic).
108         # Tri-state:
109         #   None       = per-runner default ? the WSL runner keeps DISK extraction
(unchanged Windows
110         #                behaviour); the Linux-native runner (GPU box) defaults to STREAMING
(the perf win).
111         #   True/False = force on/off for BOTH runners (e.g. set False for a container cv2
can't seek).
112         stream_video: Optional[bool] = None
113         # GPU-feed tuning for the native runner (measured on the first real L4 serving run,
114         # 2026-07-03: the default batch 8 with single-process loading left the GPU <1%
utilized ?
115         # the pipeline was CPU-feed-bound). None = wasb_infer.py's parity defaults (batch 8,
no
116         # prefetch ? byte-identical to the historical path). The cloud-serving preset opts
into
117         # the measured fast values; local/WSL behaviour is unchanged unless set explicitly.
118         batch_size: Optional[int] = None
119         # Depth of the bounded producer-thread queue that overlaps CPU-side batch assembly
120         # (decode + PIL + ToTensor) with GPU inference in streaming mode. None/0 = off.
121         prefetch_batches: Optional[int] = None
122         # GPU-RESIDENT DECODE (#506) ? DEFAULT OFF. "nvdec_resident" moves
decode+resize+normalize
123         # onto the GPU with no host round-trip: PyNvVideoCodec NVDEC decode ? affine
`grid_sample`
124         # resize replicating WASB's cv2.warpAffine EXACTLY (the #506 lesson: a naive
F.interpolate
125         # silently over-detects ? F1 0.58?0.07 ? and only shows on hard clips) ? GPU
ImageNet
126         # normalize ? HRNet. Validated on a real L4 (GX010128, SERVED preset, 2026-07-06):
~1.7×
127         # fps (35.8?60.2), GPU util 47?79% / CPU 62?35%, F1 0.574 ? baseline 0.582. Native
(Linux)
128         # runner only ? needs the torch-2.x + PyNvVideoCodec WASB env
(deploy/cloudrun/Dockerfile)
129         # with libnvcuvid/libnvidia-encode present, and device=cuda; the WSL runner ignores
it.
130         # Flip gate: the full 15-clip golden set at the SERVED preset (GX010128 must pass).
131         decode_backend: Literal["cpu", "nvdec_resident"] = "cpu"
```

```
132
133         @model_validator(mode="after")
134         def _nvdec_resident_requires_cuda(self) -> "WasbIndexConfig":
135             # NVDEC decode is CUDA-only; reject the contradictory combo at config load
instead of
136             # failing deep inside a paid GPU run. "auto" is allowed here ? the native
runner's
137             # healthcheck hard-fails if auto resolves to cpu with this backend on.
138             if self.decode_backend == "nvdec_resident" and self.device not in ("cuda",
"auto"):
139                 raise ValueError(
140                     f"indexing.wasb.decode_backend='nvdec_resident' requires device 'cuda'
or "
141                     f"'auto' (got {self.device!r}) ? NVDEC decode runs on the GPU."
142                 )
143             return self
144
145         @field_validator("batch_size")
146         @classmethod
147         def _batch_size_positive(cls, v: Optional[int]) -> Optional[int]:
148             # A DataLoader batch must be >=1; reject at config time so a typo can't reach
149             # `wasb_infer.py --batch-size` as an invalid value (the runner only passes the
flag
150             # when truthy, so 0 stays "off/parity-default").
151             if v is not None and v < 1:
152                 raise ValueError(f"indexing.wasb.batch_size must be >= 1 when set (got
{v})")
153             return v
154
155         @field_validator("prefetch_batches")
156         @classmethod
157         def _prefetch_batches_non_negative(cls, v: Optional[int]) -> Optional[int]:
158             # 0 = off (the default). Reject negatives: the runner passes any truthy value as
159             # `--prefetch-batches`, and `_PrefetchIter` clamps depth with max(1,
int(depth)), so a
160             # negative would SILENTLY enable prefetch at depth 1 instead of failing loudly.
161             if v is not None and v < 0:
162                 raise ValueError(
163                     f"indexing.wasb.prefetch_batches must be >= 0 when set (got {v})"
164                 )
165             return v
166
167
168     class FusionConfig(BaseModel):
169         """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).
170
171         Every default reproduces control behaviour: the fusion segmenter persists the
172         shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
is
173         true ? the person-cue features, but it does NOT gate the served handoff unless a
174         threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court
mask
175         is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-
only);
176         supplied ? off-court persons (spectators / adjacent courts) are excluded.
177         """
178
179         # Which trajectory substrate seeds the windows (shuttle stays primary).
180         substrate: Literal["wasb", "tracknet"] = "wasb"
181         # Windowing velocity threshold for the fusion family (the engine reads
182         # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
183         # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
184         velocity_threshold: float = 0.02
185         # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
```

```
        # enabled ? so the default path never imports torch / touches the GPU.
        cue_flags: dict[str, bool] = Field(default_factory=dict)
        # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
        # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
        min_crossings: int = Field(default=0, ge=0)
        # Served Gate B: when the person cue is on, drop windows with median in-court
players
        # < this. 0 = OFF.
        min_players: int = Field(default=0, ge=0)
        # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
        court_polygon: Optional[list[list[float]]] = None
        # Net line for the person two-sided-motion split (person features only ? the shuttle
        # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
        net_axis: Literal["x", "y"] = "y"
        net_position: float = Field(default=0.5, ge=0.0, le=1.0)


    class IndexingConfig(BaseModel):
        default_segmenter: str = "yolo_hybrid"
        use_gpu: bool = True
        # Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
        # WSL runner (default, production); "stub" = a deterministic CPU mock (no GPU/WSL)
        # so the whole pipeline is regression-testable on a CPU box / CI; "native" = the
        # Linux-native WASB runner (no WSL/conda-activate; a GPU box ? M3.5). The
        # RALLY_DETECTOR_IMPL env var overrides this. See
pipeline/detectors/{stub,native_wasb}_runner.py.
        detector_impl: str = "auto"
        motion_threshold: float = 0.08
        min_segment_duration: float = 3.0
        dead_time_merge_gap: float = 2.0
        chunk_duration: float = 90.0
        chunk_overlap: float = 10.0
        detector_model: str = "fasterrcnn_resnet50_fpn"
        detector_score_threshold: float = 0.5
        yolo_velocity_threshold: float = 0.15
        yolo_frame_skip: int = 3
        yolo_tracker: str = "bytetrack.yaml"
        yolo_prefetch_workers: int = 2
        min_action_window_duration: float = 2.0
        # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF; > 0
= a window
        # longer than this many seconds is recursively split at its largest internal
inactivity gap (?
        # ``window_min_split_gap`` s ? the most likely rally boundary), so one window can't
swallow
        # several rallies + the dead time between them. Never merges, only cuts (recall
can't drop).
        # *** R1 MILESTONE DEFAULT-ON (cap=6): the smallest safe local-windowing flip.
Measured n=13
        # golden, R2 tolF1: baseline?milestone (cap=6 + R4 below) ?+0.222, sign 12/0,
p=0.0005; cap=6
        # beats cap=12 ?+0.110, sign 12/0; net over-seg guard PASS (mean merge_rate
0.651?0.161). ***
        max_window_duration: float = 6.0
        window_min_split_gap: float = (
            0.5  # min internal gap (s) that qualifies as a split point
        )
        # HARD-CAP fallback for W1: when True, an over-long window with NO internal
inactivity gap
        # (a continuously-active trajectory ? e.g. the real-footage mahadevpura-1 174s blob)
is
        # chopped at its midpoint until ? max_window_duration, making the cap a true upper
bound.
```

```
237         # Only cuts (recall can't drop). OFF by default until measured/promoted.
238         window_hard_cap: bool = False
239         # R4 rally-END inactivity trim (s, 0 = OFF). After a rally the shuttle keeps moving
slowly
240         # (picked up / walked) above velocity_thresh, so the window over-extends its END
(the measured
241         # |?end| gap vs golden). Trim the post-rally tail back to the last frame whose
trailing
242         # window stays dense with FAST motion (velocity > inactivity_rally_thresh). Only
cuts.
243         # *** R1 MILESTONE DEFAULT-ON (1.5/0.20/0.30): on top of cap=6, R4 adds a small but
significant
244         # end-precision gain ? ?tolF1 +0.040, sign 9/1/3, p=0.022, |?end| 4.96?3.31s (n=13).
The W1 cap
245         # is the dominant lever; R4 is the marginal increment. See QUALITY_ITERATIONS.md
"R1". ***
246         window_inactivity_end: float = 1.5
247         inactivity_rally_thresh: float = 0.20
248         inactivity_min_density: float = 0.30
249         # R4 density DENOMINATOR fix (code-review finding B). The R4 trim's fast-fraction
denominator is
250         # the COUNT of active samples in the trailing window, which sparse tracking inflates
(one fast
251         # sample after a gap ? 100% dense ? tail held open). When True, the denominator is
the window's
252         # TEMPORAL length instead. No-op under dense tracking (sample-count == temporal-
span), so it only
253         # corrects the sparse-track case. *** P2b PROMOTED DEFAULT-ON (2026-06-29): golden-
set A/B at the
254         # SERVED operating point (n=15, 595 rallies) ? tolF1 0.353?0.364 (?tolF1 +0.010,
sign 10W/4L, NO
255         # regression), |?end| 2.97?2.72s, seg_count_ratio ?0.103 (p=0.006), split_rate
?0.010 (p=0.016),
256         # R1 over-seg guard PASS; served_regresses(eps=0.0)=False ?
promote_if_served_safe(structural)=True.
257         # See docs/RALLY_DETECTION_FIXES_PLAN.md §7/§8. ***
258         inactivity_temporal_density: bool = True
259         # R5 blob-fallback (default OFF). LAST-RESORT splitter for the continuously-active
blob: after
260         # the W1 gap-split AND the R4 inactivity trim have each had their chance, a group
STILL longer
261         # than window_blob_factor × max_window_duration (no internal gap, no rally-end
signal ? e.g.
262         # mahadevpura-1's ~65s residual ? 11× a 6s cap) is midpoint-chopped to ? the cap as
a FORCED cut
263         # (logged + flagged on the window) so the degenerate single-window outcome is
avoided and the
264         # clip is surfaced as needing rally-STATE cues, not a bigger cap. Differs from
window_hard_cap
265         # (which chops BEFORE R4 and over-segments normal clips): blob-fallback runs AFTER
R4 and only
266         # force-cuts the genuine blob (the factor gate). Only cuts (recall can't drop).
267         window_blob_fallback: bool = False
268         # Magnitude gate that keeps R5 surgical: only a group longer than this ×
max_window_duration is a
269         # blob (a normal long rally ~1-2× the cap is left alone; the mahadevpura-1 residual
is ~11×).
270         # Default 4.0 is do-no-harm on the golden corpus (rescues only the continuously-
active
271         # clips, every other clip a no-op within noise) ? see docs/QUALITY_ITERATIONS.md
"R5".
272         window_blob_factor: float = 4.0
273         log_yolo_candidates: bool = True
274         store_yolo_candidates: bool = True
275         ai_handoff_padding: float = 2.0
```

```
276         ai_max_workers: int = 5
277         # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
278         # chunks of a high-bitrate (4K) source blow past the provider upload cap
279         # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
280         # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
281         # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
282         # Matches gemini_refine's --clip-scale-width default.
283         ai_chunk_scale_width: int = 1280
284         # FULLY-LOCAL mode (default OFF): when true, the trajectory-hybrid segmenters
285         # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the Phase-3 AI handoff
entirely and
286         # emit their local candidate windows AS the rallies ? no provider, no API key,
nothing
287         # uploaded. Lets the local detector run standalone (e.g. to benchmark it against the
Gemini
288         # path, or to operate with zero cloud dependency). The pure `gemini` segmenter
ignores this.
289         skip_ai_handoff: bool = False
290         # Optional debug knob: trim every video to the first N seconds before processing.
291         debug_duration: Optional[float] = None
292
293         tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
294         wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
295         fusion: FusionConfig = Field(default_factory=FusionConfig)
296
297         @field_validator("default_segmenter")
298         @classmethod
299         def segmenter_must_be_registered(cls, v: str) -> str:
300             # Lazy import: the segmenter registry imports modules that may import
301             # config, so we resolve it at validation time rather than at module load.
302             from backend.pipeline.segmenters.base import segmenter_registry
303
304             if segmenter_registry.get(v) is None:
305                 available = sorted(segmenter_registry.list_available().keys())
306                 raise ValueError(
307                     f"Segmenter '{v}' is not registered. Available: {', '.join(available)}"
308                 )
309             return v
310
311         @model_validator(mode="after")
312         def _chunk_step_must_be_positive(self) -> "IndexingConfig":
313             # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
314             # step makes `while t < duration: t += step` loop forever, growing memory before
any
315             # GPU work. Reject the bad config at load time instead.
316             if self.chunk_overlap >= self.chunk_duration:
317                 raise ValueError(
318                     f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
"
319                     f"({self.chunk_duration}); otherwise chunking never advances."
320                 )
321             return self
322
323
324     class OutputConfig(BaseModel):
325         default_highlights_name: str = "highlights.mp4"
326         db_name: str = "sports_indexer.db"
327         # Directory where the DB, highlight reels, and processed clips are written.
328         # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
329         output_dir: str = "output"
330
331
332     class WatermarkConfig(BaseModel):
333         """Branding overlay burned into each highlight clip during the per-clip re-encode
```

```
        (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
        (under `stitching.watermark` in config.json) to turn it off. The text is fully
        configurable. The concat stage stays stream-copy (-c copy)."""

        enabled: bool = True
        text: str = "Generated by Khelsutra.guru"
        logo_path: str = ""  # optional transparent PNG overlay; "" = no logo
        font_path: str = ""  # optional .ttf; "" = use the fontconfig `font` family below
        font: str = "Sans"  # fontconfig family used when font_path is empty
        font_size_ratio: float = Field(
            default=0.035, gt=0.0, le=0.5
        )  # text height as a fraction of frame height
        opacity: float = Field(default=0.85, ge=0.0, le=1.0)
        box: bool = True  # faint backing box behind the text for legibility
        margin: int = Field(default=24, ge=0)  # px inset from the chosen corner
        position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = (
            "bottom-right"
        )


    class StitchingConfig(BaseModel):
        begin_padding: float = 0.5
        end_padding: float = 0.5
        use_cache: bool = False
        min_shot_count: int = 1
        # Long-rally reel filter (seconds; 0 = OFF). Drop detected rally pairs whose
duration
        # (end ? start) is below this from the highlight reel, BEFORE compiling. A cleaner
        # reel-selection knob than `min_shot_count`: shot_count is unlabeled / "Unknown" on
the
        # fully-local no-AI path, whereas duration is derived from the rally's own
start/end. The
        # local detector over-fires and its false positives are mostly SHORT (motion blips,
        # background-court fragments), so this keeps the reel to the eye-catching long
rallies.
        # OFF by default ? the detector output is unchanged, only which rallies reach the
reel.
        min_rally_duration: float = Field(default=0.0, ge=0.0)
        watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)


    class LoggingConfig(BaseModel):
        """Logging knobs for the run entrypoints (see backend/utils/logging_setup.py)."""

        # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
        # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
        # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
        # them to WARNING; set true to restore full chatter for request-flow debugging.
        verbose_http: bool = False


    class SecurityConfig(BaseModel):
        # When set, /api/process video_path must resolve under this directory (hosted/tenant
        # mode). Default None preserves the single-user local 'any local file' workflow.
        allowed_video_root: Optional[str] = None

        # --- Chunked upload (B2, PR-2) ---
        # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
        # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
        # contain upload_dir or /api/process will reject the uploaded paths.
        upload_dir: str = "output/uploads"
        # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
        # free-tier edge proxies cap request bodies there (HOSTING_PLAN §2).
        max_upload_mb: int = 4096
        max_part_mb: int = 95
```

```
394         allowed_upload_extensions: List[str] = ["mp4", "mov"]
395
396         # --- M9 edge auth (Cloudflare Access, D9) ---
397         # DEFAULT-OFF: edge_auth_enabled=False ? no JWT is required/verified (the local
single-user
398         # path is unchanged). When True, /api/process requires + verifies the `Cf-Access-
Jwt-Assertion`
399         # header against the team JWKS (so a direct hit to the Cloud Run URL can't spoof the
identity
400         # header) and tags the job with the verified user_id (owner_id). The team domain
(e.g.
401         # ``https://<team>.cloudflareaccess.com``) supplies the JWKS + issuer; the AUD is
the CF Access
402         # application's audience tag. NO secrets here ? these are public identifiers.
403         edge_auth_enabled: bool = False
404         cf_team_domain: str = (
405             ""  # e.g. https://<team>.cloudflareaccess.com (issuer + /certs JWKS)
406         )
407         cf_access_aud: str = ""  # the CF Access application AUD tag the token must carry
408         # (CORS allow-origins is set via the RALLY_CORS_ALLOW_ORIGINS env var ? read at
import in
409         # backend/main.py so importing the app does no filesystem I/O; default ["*"].
Credentials stay
410         # OFF ? Cf-Access is a header, not a cookie.)
411
412
413     class StorageConfig(BaseModel):
414         """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
415         behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + DO
Spaces
416         + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = a
locally MOUNTED
417         Google Drive (``gdrive.mount_root``). Per-backend settings are loose dicts passed to
the constructor ?
418         **secrets are never stored here**, only endpoints / regions / file paths / source-
selectors
419         (boto3/google auth chains supply credentials). See ``backend/storage/``."""
420
421         backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
422         # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
423         output_root: str = ""
424         # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
`backend`/`output_root`).
425         # Default ``local`` / "" keeps today's behaviour: a bare path / ``local://`` URI is
read IN PLACE
426         # (identity, no copy). Set these to read inputs from a bucket / mounted Drive ? a
bare/relative
427         # input name is resolved against ``input_root`` (e.g.
``input_root="gs://bucket/videos"`` + "x.mp4"
428         # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with
``input_backend``
429         # (``"gcs"`` + "bucket/x.mp4" => ``gcs://bucket/x.mp4``). An explicit scheme on the
input
430         # (``gs://?``/``s3://?``) or an absolute local path ALWAYS wins. Non-local inputs
are fetched to a
431         # scratch dir before processing (see ``backend.storage.acquire_local_input``).
432         input_backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
433         input_root: str = ""
434         # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
435         # One folder per video keyed by the MD5 video_id:
<root>/[<workspace_prefix>/]<video_id>/...
436         # so a local deploy and a cloud (bucket) deploy share ONE relative layout (only the
root
437         # differs). DEFAULT-OFF: convergence is phased + flipped via a Launch Report (D5).
```

```
When OFF
438         # the legacy flat layout is used unchanged.
439         single_folder_per_video: bool = False
440         # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
441         workspace_root: str = ""
442         # Cloud namespace prefix under the root so per-video folders sit tidily beside
443         # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>/). "" = root
directly.
444         workspace_prefix: str = "workspaces"
445         local: dict[str, Any] = Field(default_factory=dict)
446         s3: dict[str, Any] = Field(default_factory=dict)
447         gcs: dict[str, Any] = Field(default_factory=dict)
448         gdrive: dict[str, Any] = Field(default_factory=dict)
449         # gdrive-api settings (e.g. {"root_folder_id": "<id>"}); read via the
hyphen?underscore lookup in
450         # backend.storage.get_storage. NO secrets ? creds come from ADC over the Drive
scope.
451         gdrive_api: dict[str, Any] = Field(default_factory=dict)
452
453
454     class DeploymentConfig(BaseModel):
455         """Deployment profile ? one switch that selects a coherent bundle of compute +
storage
456         knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md``
§4.3).
457
458         **Pure / additive:** the empty default profile (``""``) applies NO preset, so the
pipeline
459         behaves exactly as before this section existed. A profile is opt-in (``config.json``
460         ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); auto-detect only ever
461         *suggests* one (D15 ? cloud spend is never entered silently). Nothing in the core
reads
462         these fields yet (M1): selecting a profile works purely by pre-setting the EXISTING
knobs
463         (``indexing.detector_impl`` / ``skip_ai_handoff``, ``storage.backend``) in the
loader.
464
465         Valid values mirror ``backend/config/presets.py`` (parity-pinned by a unit test)."""
466
467         # "" = no profile (today's manual knobs). Others apply presets.PROFILE_PRESETS.
468         profile: Literal["", "truly-local", "cloud-serving", "cpu-mock"] = ""
469         # Where the core runs. Set by the profile preset; overridable in config.json. Inert
in M1
470         # (recorded for observability; consumed by the ComputeBackend seam from M2+).
471         compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""
472         # Cloud-serving throughput cap (#466): how many DETECT jobs the control-plane drain
may dispatch +
473         # bucket-poll CONCURRENTLY when compute_target=="cloud_run". Each is an I/O-bound
Cloud Run Job
474         # execution (the GPU work is on ephemeral L4s, WASB-only has no Gemini), so ? unlike
a truly-local
475         # single box ? they need not serialize. Bounds concurrent PAID L4 executions. Non-
cloud_run targets
476         # ignore this and stay serial (max_workers=1), preserving the truly-local GPU/Gemini
serialization.
477         max_concurrent_remote_jobs: int = Field(default=4, ge=1, le=64)
478         # Poll budget for one dispatched remote job (CONTROL_PLANE_DURABLE_REBUILD §4.3).
Must sit
479         # ABOVE the worker Job's own --task-timeout (3600s) so the worker's timeout stays
the
480         # authority and the dispatcher's one-shot post-timeout marker rescue still applies ?
the old
481         # 1800s ExecutionPolicy default undercut real 1080p60 jobs (~22 min detect +
queue/cold-start,
482         # observed on the 2026-07-05 owner CUJ) and could abandon a slow-but-healthy paid
```

```
GPU run.
483         remote_poll_max_wait_sec: float = Field(default=3900.0, gt=0)
484
485
486     class BillingConfig(BaseModel):
487         """Per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8).
488
489         **DEFAULT-OFF:** ``enabled=False`` ? nothing records cost (the v6 cost columns stay
NULL =
490         today's behaviour). When enabled, a job's cost is computed + stored in **USD** via
the versioned
491         ``pricebook`` DB table and **displayed in INR** at ``fx_inr_per_usd``. The default
pricebook
492         version is the seeded ``placeholder-v0`` (every rate ``0.0`` ? cost ``0``) until the
owner
493         inserts a real, calibrated version and points ``pricebook_version`` at it.
494
495         ? Even with ``enabled=True`` + a real pricebook, recorded cost stays **0** until a
follow-up
496         wires the worker to emit per-job ``usage`` (gpu/wall/egress/gemini) ? M8 lands the
ledger +
497         pricebook + the recording seam; usage emission (the metering hooks) is a separate
step."""
498
499         enabled: bool = False
500         pricebook_version: str = (
501             "placeholder-v0"  # the seeded $0 version; owner flips to a real one
502         )
503         fx_inr_per_usd: float = 83.0  # USD?INR display rate (Bangalore market)
504         margin: float = 0.0  # resale markup (?0); 0 = bill at cost
505
506
507     class FlywheelConfig(BaseModel):
508         """Data-flywheel harness (COMPUTE_DECOUPLED_SERVING M11, D12) ? the
consent?capture?eval?goldenize
509         loop: on a CONSENTED adult job, durably **capture** the source + rally output into
the per-video
510         workspace ? **shadow-eval** owned-vs-Gemini ? **propose promotion** to the golden
TRAINING set, so
511         the owned model climbs toward the D11 ship floor.
512
513         **DEFAULT-OFF + INERT:** ``enabled=False`` ? nothing is captured/evaluated (byte-
identical to
514         today). ? **Consent is FAKED-DENY until counsel + the consent schema land:** even
``enabled=True``
515         captures **nothing**, because ``flywheel.consent_attested`` currently returns
``False`` for every
516         job (the privacy guardrail ? D10: adults-only, explicit opt-in, derived-data + auto-
delete-raw).
517         This is the PLUMBING only; the real consent gate (counsel-reviewed ToS/DPA ? consent
columns ?
518         wiring) + the owned-model + golden-label data needed to *activate* shadow-
eval/promotion are
519         owner/counsel/GPU scope. Pairs with M9-consent."""
520
521         enabled: bool = False
522         workspace_prefix: str = (
523             "flywheel"  # per-video workspace prefix for captured artifacts
524         )
525         shadow_eval: bool = True  # run the owned-vs-Gemini shadow comparison on capture
526         min_promote_n: int = 5  # promote_if_served_safe `min_n` (the served-safe gate)
527
528
529     class AppConfig(BaseModel):
530         """Root configuration object.
```

```
531
532         All runtime configuration should be accessed via an instance of this class
533         (or the loader) rather than raw dict lookups.
534         """
535
536         sport: Sport = "badminton"
537         ai_provider: str = "gemini"
538         # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
flagship;
539         # the alias never deprecates underneath us. Eval reports record the run date
alongside.
540         ai_model: str = "gemini-flash-latest"
541
542         validation: ValidationConfig = Field(default_factory=ValidationConfig)
543         indexing: IndexingConfig = Field(default_factory=IndexingConfig)
544         output: OutputConfig = Field(default_factory=OutputConfig)
545         stitching: StitchingConfig = Field(default_factory=StitchingConfig)
546         security: SecurityConfig = Field(default_factory=SecurityConfig)
547         logging: LoggingConfig = Field(default_factory=LoggingConfig)
548         storage: StorageConfig = Field(default_factory=StorageConfig)
549         deployment: DeploymentConfig = Field(default_factory=DeploymentConfig)
550         billing: BillingConfig = Field(default_factory=BillingConfig)
551         flywheel: FlywheelConfig = Field(default_factory=FlywheelConfig)
552
553         # Reject unknown keys ? typed config migration is complete (D3).
554         model_config = {
555             "extra": "forbid",
556             "validate_assignment": True,
557         }
558
559
560     # Convenience type alias for callers that want to annotate
561     Config = AppConfig
562
```

20. user (2026-07-08T05:06:09.125Z)

```
36:# work: CPU/GPU-heavy in-process detection + the compile stitch serialize via
38:# and compile/local-override jobs stay serial. The count is fixed at first _get_executor()
creation.
75:    OVERLAPS. This does NOT parallelize CPU/GPU-heavy LOCAL work ? in-process detection and
the compile
77:    single-box GPU + rate-fragile Gemini invariant holds, and compile/local-override jobs
stay serial.
157:    # Dispatch by kind: 'compile' (#329, the async reel stitch) vs 'process' (NULL/default ?
the
159:    # jobs feed the M8 cost ledger + M11 flywheel (a compile has no detection
usage/segments).
160:    is_compile = job.get("kind") == "compile"
162:        if is_compile:
163:            result = _main._execute_compile(
180:        if not is_compile:
```

21. assistant (2026-07-08T05:06:15.520Z)

Now let me read the job worker dispatch, the compile API endpoint, and the VideoCompiler/ffmpeg stitcher.

22. user (2026-07-08T05:06:16.922Z)

```
1       """Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3).
2
3       Extracted verbatim from backend.main to shrink the orchestrator; behavior is unchanged.
```

```
4       The worker DRAINS the jobs table via `claim_due_job` and honours WAIT_AND_RESUME: a job
5       that raises `JobWaiting` is parked (status 'waiting' + `next_poll_at` + progress) and
6       re-claimed when due ? with no central scheduler (the DB is the clock).
7
8       The monkeypatch contract these functions rely on (and that tests depend on):
9       every cross-function seam ? `db_instance`, `global_config`, `_execute_processing`,
10      `_arm_wake_timer`, `_get_executor`, `_MAX_DRAIN_WAKE_SEC` ? is resolved THROUGH the
11      `backend.main` module object at call-time (`import backend.main as _main`), never as a
12      bare local. `backend.main` re-exports these names, so `monkeypatch.setattr(backend.main,
13      ...)` still intercepts the inter-function calls (e.g. a parked job arms the *patched*
14      `_arm_wake_timer`, so no real daemon timer leaks in tests). Importing `backend.main`
lazily
15      inside the functions also avoids the circular import at module load (main.py imports
this
16      module; this module must not import main at import time).
17      """
18
19      import json
20      import logging
21      import os
22      import threading
23      import traceback
24      from concurrent.futures import ThreadPoolExecutor
25      from typing import Any, Dict, Optional
26
27      logger = logging.getLogger(__name__)
28
29      # Async job executor (DEFERRED_HARDENING_PLAN #16). max_workers=1 by default: a single
30      # self-hosted box serializes GPU work, and the Gemini provider is rate-fragile under
31      # parallel uploads (2026-06-10 ops learning: serial uploads + backoff). The segmenters
32      # already parallelize their AI handoff internally via their own pools.
33      # EXCEPTION (#466): when the server CAN dispatch to Cloud Run (see
_cloud_dispatch_possible), a REMOTE
34      # job is only a bucket-poll (I/O-bound; GPU is on ephemeral L4s, WASB-only has no
Gemini), so the drain
35      # scales to deployment.max_concurrent_remote_jobs to overlap those. This does NOT
parallelize local
36      # work: CPU/GPU-heavy in-process detection + the compile stitch serialize via
37      # orchestration._LOCAL_COMPUTE_LANE regardless of the worker count, so the single-box
invariant holds
38      # and compile/local-override jobs stay serial. The count is fixed at first
_get_executor() creation.
39      #
40      # ? O2 RISK (CODE_CLEANUP_AUDIT §3c): a single worker means ONE stuck or hung job
41      # (e.g. hung Gemini generate, hung WSL GPU call, hung Cloud Run poll) blocks ALL
42      # subsequent jobs ? they stay 'queued' forever. Mitigations already in place:
43      #   - ExecutionPolicy op_timeout_sec kills hung generate/processing calls
44      #   - detector timeout_sec kills hung WSL/GPU calls
45      #   - Cloud Run poll has a max_wait_sec bound
46      #   - ffmpeg/ffprobe subprocesses use bounded media timeouts
47      # A future slice should add a per-job wall-clock timeout (the executor itself
48      # cannot enforce timeouts on the Thread).
49      _job_executor: Optional[ThreadPoolExecutor] = None
50      # Drain worker count, fixed at first _get_executor() creation. 1 (serial) by default;
configure_drain()
51      # raises it for cloud_run (#466). A module global (not a _get_executor arg) so the many
call sites +
52      # the tests that monkeypatch `_get_executor` as a no-arg lambda are unchanged.
53      _drain_max_workers: int = 1
54
55
56      def _cloud_dispatch_possible(config: Any = None) -> bool:
57          """True if this server CAN dispatch a job to Cloud Run ? either the profile sets
58          ``deployment.compute_target=cloud_run``, OR the per-request ``compute='cloud'``
override path is
```

```
        wired (``CLOUD_RUN_JOB`` + ``GOOGLE_CLOUD_PROJECT`` env + an output bucket). Mirrors
        orchestration._cloud_available so drain concurrency tracks the ACTUAL remote-
dispatch capability,
        not just the server default (so compute='cloud' overrides on a local-default server
parallelize too)."""
        dep = getattr(config, "deployment", None)
        if getattr(dep, "compute_target", "") == "cloud_run":
            return True
        if os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT"):
            stor = getattr(config, "storage", None)
            if (getattr(stor, "output_root", "") or "").strip():
                return True
        return False


    def _resolve_drain_workers(config: Any = None) -> int:
        """Drain concurrency (#466): a server that CAN dispatch to Cloud Run (see
_cloud_dispatch_possible)
        may run ``deployment.max_concurrent_remote_jobs`` drain workers so I/O-bound remote
dispatch/poll
        OVERLAPS. This does NOT parallelize CPU/GPU-heavy LOCAL work ? in-process detection
and the compile
        stitch serialize via orchestration._LOCAL_COMPUTE_LANE regardless of the worker
count ? so the
        single-box GPU + rate-fragile Gemini invariant holds, and compile/local-override
jobs stay serial.
        A server with no remote capability stays serial (1). Defaults to 1 when config is
absent
        (tests / non-server) ? behaviour unchanged."""
        if _cloud_dispatch_possible(config):
            dep = getattr(config, "deployment", None)
            return max(1, int(getattr(dep, "max_concurrent_remote_jobs", 1) or 1))
        return 1


    def configure_drain(config: Any = None) -> None:
        """Set the drain worker count from ``config`` BEFORE the executor is created (#466).
Call once at
        server startup. No-op effect once the executor already exists (the count is fixed at
creation)."""
        global _drain_max_workers
        _drain_max_workers = _resolve_drain_workers(config)


    def _get_executor() -> ThreadPoolExecutor:
        """Lazy module-global executor; tests monkeypatch this for inline execution. Worker
count is
        ``_drain_max_workers`` (1 by default; raised for cloud_run via ``configure_drain``,
#466)."""
        global _job_executor
        if _job_executor is None:
            _job_executor = ThreadPoolExecutor(
                max_workers=_drain_max_workers, thread_name_prefix="jobworker"
            )
        return _job_executor


    class JobWaiting(Exception):
        """Raised inside the pipeline to SELF-SCHEDULE a job (EXECUTION_POLICY.md P3,
        WAIT_AND_RESUME). It means "I can't finish now ? park me and resume in `delay_sec`",
        NOT a failure. The worker persists `next_poll_at` + `progress` (via
`set_job_waiting`),
        releases the thread, and re-claims the job when due (`claim_due_job`). No work to
date
        is the segmenter's `_execute_processing`, run UNDER the destroy-after-confirm
```

```
contract.
110
111         The wiring is additive: the production segmenter path never raises this today (zero
112         behaviour change), so a job runs exactly as before. The hook is what a later slice
113         (resumable AI handoff) raises once a long op exhausts its in-process wait budget.
114         """
115
116         def __init__(self, delay_sec: float, progress: Optional[str] = None) -> None:
117             super().__init__(f"job parked for {delay_sec:.0f}s")
118             self.delay_sec = max(0.0, float(delay_sec))
119             self.progress = progress
120
121
122     def _job_to_request(job: Dict[str, Any]):
123         """Reconstruct the submit-time ProcessRequest from a claimed job row, so a job
124         re-claimed by `claim_due_job` (queued OR resumed-from-waiting) runs identically to
125         the original submit. The row stores exactly the ProcessRequest fields."""
126         import backend.main as _main
127
128         raw_params = job.get("params")
129         params = (
130             raw_params if isinstance(raw_params, dict) else json.loads(raw_params or "{}")
131         )
132         return _main.ProcessRequest(
133             video_path=job["video_path"],
134             player_pool=job.get("player_pool"),
135             max_frames=job.get("max_frames"),
136             segmenter=job.get("segmenter"),
137             # v8 compute-picker: the worker runs _execute_processing ?
_maybe_dispatch_remote, which
138             # honours this, so it MUST survive submit?reconstruct (unlike locale, which is
only read off
139             # the job row for analytics). NULL ? server default = pre-picker behaviour.
140             compute=job.get("compute"),
141             force=bool(params.get("force")),
142         )
143
144
145     def _run_claimed_job(job: Dict[str, Any], config: Any, db: Any) -> None:
146         """Run ONE job already claimed (`status='running'`) by `claim_due_job`, recording
its
147         terminal state ? OR parking it for resume on `JobWaiting` (WAIT_AND_RESUME).
148
149         Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done'
means
150         the worker finished ? the pipeline's own verdict (validation failure etc.) lives in
151         result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means
the
152         job self-scheduled and will be re-claimed when `next_poll_at` is due.
153         """
154         import backend.main as _main
155
156         job_id = job["id"]
157         # Dispatch by kind: 'compile' (#329, the async reel stitch) vs 'process'
(NULL/default ? the
158         # original /api/process pipeline). Both record their terminal state the same way;
only process
159         # jobs feed the M8 cost ledger + M11 flywheel (a compile has no detection
usage/segments).
160         is_compile = job.get("kind") == "compile"
161         try:
162             if is_compile:
163                 result = _main._execute_compile(
164                     config,
165                     db,
```

```
                    job,
                    progress=lambda stage: db.set_job_progress(job_id, stage),
                )
            else:
                req = _main._job_to_request(job)
                result = _main._execute_processing(
                    config,
                    db,
                    req,
                    progress=lambda stage: db.set_job_progress(job_id, stage),
                )
            if result.get("video_id"):
                db.set_job_video_id(job_id, result["video_id"])
            db.mark_job_done(job_id, result)
            if not is_compile:
                _maybe_record_job_cost(
                    job, result, config, db
                )  # M8 cost ledger (default-OFF; process jobs only)
                _maybe_run_flywheel(
                    job, result, config
                )  # M11 data-flywheel (process jobs only)
        except _main.JobWaiting as w:
            # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
            logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
            db.set_job_waiting(job_id, w.delay_sec, progress=w.progress)
        except Exception as e:
            logger.error(f"Job {job_id} crashed: {e}\n{traceback.format_exc()}")
            db.mark_job_failed(job_id, f"{type(e).__name__}: {e}")


    def _maybe_record_job_cost(
        job: Dict[str, Any], result: Dict[str, Any], config: Any, db: Any
    ) -> None:
        """M8 cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8). DEFAULT-OFF: a no-op unless
        ``config.billing.enabled``. When on, compute the job's cost from ``result['usage']``
        (= {gpu_seconds, wall_seconds, egress_gb, gemini_usd, gpu_type}) × the versioned
pricebook, and
        persist it.

        ? **Honest gap (M8 scope):** the pipeline does **not** emit ``result['usage']`` yet,
so even with
        ``billing.enabled=True`` AND a real calibrated pricebook, every recorded cost is
**0** until a
        FOLLOW-UP wires the worker to emit usage (gpu/wall/egress/gemini). M8 lands the
ledger + pricebook
        + the recording seam; the usage-emission step is separate (the metering hooks on the
real run).
        With the placeholder pricebook the cost is 0 regardless.

        Best-effort ? never fails a completed job because cost bookkeeping hiccuped. ``job``
is the
        claimed row (carries ``owner_id``); ``result`` is the pipeline output."""

        billing_cfg = getattr(config, "billing", None)
        if not billing_cfg or not getattr(billing_cfg, "enabled", False):
            return
        try:
            usage = (result or {}).get("usage") or {}
            db.record_job_cost(
                job["id"],
                usage=usage,
                pricebook_version=billing_cfg.pricebook_version,
                gpu_type=usage.get("gpu_type"),
                compute_backend=(result or {}).get("compute_backend"),
                owner_id=job.get("owner_id"),
```

```
                margin=billing_cfg.margin,
            )
            logger.info(
                "Recorded cost for job %s (pricebook=%s).",
                job["id"],
                billing_cfg.pricebook_version,
            )
        except Exception as e:  # never wedge a completed job on bookkeeping
            logger.warning(
                "Cost recording for job %s failed (non-fatal): %s", job.get("id"), e
            )


    def _maybe_run_flywheel(
        job: Dict[str, Any], result: Dict[str, Any], config: Any
    ) -> None:
        """M11 data-flywheel (COMPUTE_DECOUPLED_SERVING M11, D12). DEFAULT-OFF + INERT: a
no-op unless
        ``config.flywheel.enabled`` AND attested consent ? and consent is a **faked hard-
deny** today
        (``flywheel.consent_attested`` returns False for every job until counsel + the
consent schema
        land), so this captures **nothing** in production. When activated it captures the
consented job's
        artifacts into the per-video workspace (? later labeling ? the golden training set).

        Best-effort ? never fails a completed job because the flywheel hiccuped. ``job`` is
the claimed
        row (carries ``owner_id``); ``result`` is the pipeline output."""

        flywheel_cfg = getattr(config, "flywheel", None)
        if not flywheel_cfg or not getattr(flywheel_cfg, "enabled", False):
            return
        try:
            from backend import flywheel

            flywheel.run_for_job(job, result, config)
        except Exception as e:  # never wedge a completed job on the flywheel
            logger.warning("Flywheel for job %s failed (non-fatal): %s", job.get("id"), e)


    def _drain_due_jobs(config: Any, db: Any) -> int:
        """One worker tick: claim and run every currently-runnable job (queued, or waiting
whose
        `next_poll_at` is due), earliest-due first, until the table has nothing runnable.

        Each parked job (`JobWaiting` ? status 'waiting') drops out of this drain and is
picked
        up by a FUTURE drain once due; we schedule that follow-up drain via
`_schedule_drain` so
        a single-worker box keeps making progress with no central timer (the DB is the
clock).
        Returns the number of jobs that reached a terminal/parked state this tick.
        """
        import backend.main as _main

        ran = 0
        while True:
            job = db.claim_due_job()
            if job is None:
                break
            ran += 1
            _main._run_claimed_job(job, config, db)
        # If anything is parked-but-not-yet-due, arrange a wake-up so it resumes without a
new
```

```
280         # submit. On the single-box executor this is a delayed re-drain; multi-worker just
relies
281         # on the next submit/drain. Best-effort ? never raises into the worker.
282         _main._schedule_next_drain_if_waiting(config, db)
283         return ran
284
285
286     def _schedule_next_drain_if_waiting(config: Any, db: Any) -> None:
287         """If any job is parked in 'waiting', arm a re-drain for when the soonest one is
due, so a
288         self-scheduled job resumes with no further client submit. The drain itself re-checks
289         due-ness, so an early wake is harmless and a clamped wake for a far-future job is
fine."""
290         import backend.main as _main
291
292         try:
293             delay = db.seconds_until_next_due()
294         except Exception as e:  # pragma: no cover - defensive; never wedge the worker
295             logger.error(f"Could not compute next-due delay: {e}")
296             return
297         if delay is None:
298             return  # nothing waiting
299         # Clamp so a far-future job doesn't pin a thread forever; the drain re-checks due-
ness.
300         _main._arm_wake_timer(max(0.0, min(delay, _main._MAX_DRAIN_WAKE_SEC)), config, db)
301
302
303     def _arm_wake_timer(delay_sec: float, config: Any, db: Any) -> None:
304         """Arm a single daemon timer that re-submits a drain after ``delay_sec`` (no central
305         scheduler ? the DB is the clock). Isolated as the sole timer-creating seam so tests
can
306         monkeypatch it to a no-op and assert the *scheduling decision* without real
waiting."""
307         import backend.main as _main
308
309         timer = threading.Timer(
310             delay_sec,
311             lambda: _main._get_executor().submit(_main._drain_due_jobs, config, db),
312         )
313         timer.daemon = True
314         timer.start()
315
316
317     #: A parked job further out than this is re-checked by a capped wake rather than one
long
318     #: sleeping timer ? keeps the wake cadence bounded without a central scheduler.
319     _MAX_DRAIN_WAKE_SEC = 60.0
320
321
322     #
----------------------------------------------------------------------------------------
323     # Restart-safe remote jobs (CONTROL_PLANE_DURABLE_REBUILD P3, #471) ? the 2026-07-05
loss shape:
324     # a running cloud detection's done-marker poll lives only in this process's memory, so a
restart
325     # orphaned the job while the L4 worker kept computing; its 29 rallies landed in the
bucket with
326     # no listener. With the md5 persisted at submit (P4), outputs/<md5>/report.json IS a
durable
327     # done-signal the restarted control plane can re-arm on ? completing the job WITHOUT
ever
328     # re-dispatching (no double GPU spend).
329
330     #: Resume-poll cadence. Module-level so tests drive the loop with a zero-wait cadence.
331     _RESUME_POLL_EVERY_SEC = 15.0
```

```
332
333
334     def resume_interrupted_remote_jobs(config: Any, db: Any) -> int:
335         """Re-arm the durable done-signal poll for md5-keyed RUNNING process jobs a restart
336         orphaned (the rows ``recover_stale_jobs`` deliberately left alone). Call once at
startup,
337         after ``recover_stale_jobs`` and before any drain. Each job resumes on its own
daemon
338         thread (bounded by the handful of rows a single-instance CP can have in flight);
rows are
339         NEVER re-dispatched from here. Returns the number of resumes started.
340
341         On a box that cannot dispatch to Cloud Run at all (the appliance), no remote worker
can
342         exist ? the rows get today's honest terminal failure instead."""
343         rows = db.interrupted_remote_process_jobs()
344         if not rows:
345             return 0
346         if not _cloud_dispatch_possible(config):
347             for job in rows:
348                 db.mark_job_failed(str(job["id"]), "interrupted by server restart")
349             logger.warning(
350                 "[JOBS] %d interrupted md5-keyed process job(s) failed: this server cannot "
351                 "dispatch to Cloud Run, so no remote worker can be in flight.",
352                 len(rows),
353             )
354             return 0
355         # Prove each row was actually CLOUD-dispatched (#494 review round 2): an explicit
356         # compute='cloud' always was; NULL means "server default", which is remote ONLY when
the
357         # default target itself is cloud_run (on a default-local server with cloud-override
358         # capability, a NULL row ran in-process ? no remote worker exists to resume).
359         default_is_cloud = (
360             getattr(getattr(config, "deployment", None), "compute_target", "") ==
"cloud_run"
361         )
362         started = 0
363         for job in rows:
364             # Same normalization the execution path applies (_maybe_dispatch_remote
lowercases
365             # req.compute) ? "LOCAL"/" cloud " must mean at recovery exactly what they meant
at run.
366             requested = (str(job.get("compute") or "")).strip().lower()
367             if requested != "cloud" and not default_is_cloud:
368                 db.mark_job_failed(str(job["id"]), "interrupted by server restart")
369                 continue
370             t = threading.Thread(
371                 target=_resume_one_remote_job,
372                 args=(config, db, dict(job)),
373                 daemon=True,
374                 name=f"resume-{str(job['id'])[:8]}",
375             )
376             t.start()
377             started += 1
378         if started:
379             logger.warning(
380                 "[JOBS] re-armed the durable done-signal poll for %d interrupted remote
job(s).",
381                 started,
382             )
383         return started
384
385
386     def _resume_one_remote_job(config: Any, db: Any, job: Dict[str, Any]) -> None:
387         """One orphaned job's resume: poll ``outputs/<md5>/report.json`` (bounded by the
```

```
same
388         ``deployment.remote_poll_max_wait_sec`` budget a live dispatch gets), then complete
or
389         fail the job HONESTLY from what the bucket says. Never dispatches."""
390         from backend import orchestration, storage
391         from backend.infrastructure.execution_policy import (
392             ExecutionPolicy,
393             PolicyTimeout,
394             poll_until,
395         )
396
397         job_id = str(job["id"])
398         video_id = str(job["video_id"])
399         try:
400             raw_params = job.get("params")
401             params = (
402                 raw_params
403                 if isinstance(raw_params, dict)
404                 else json.loads(raw_params or "{}")
405             )
406             if params.get("force"):
407                 # Defense-in-depth (#494 review): the SQL carve-out already excludes forced
rows,
408                 # but a forced job must NEVER complete from the bucket cache ? force
deliberately
409                 # bypasses it, and nothing proves the report belongs to the forced run
rather
410                 # than an older completed one.
411                 db.mark_job_failed(
412                     job_id,
413                     "control plane restarted during a forced reprocess; the cached result "
414                     "cannot be trusted for force=true ? re-submit with force",
415                 )
416                 return
417             if (str(job.get("compute") or "")).strip().lower() == "local":
418                 # Defense-in-depth (#494 review round 2): an explicit local run has no
remote
419                 # worker; a same-md5 bucket report would be an OLDER run's output, not this
job's.
420                 db.mark_job_failed(job_id, "interrupted by server restart")
421                 return
422             sc = orchestration._storage_settings(config)
423             output_root = ((sc or {}).get("output_root") or "").rstrip("/")
424             if not sc or not output_root:
425                 db.mark_job_failed(
426                     job_id,
427                     "interrupted by server restart (no output bucket configured to recover
from)",
428                 )
429                 return
430             report_uri = f"{output_root}/outputs/{video_id}/report.json"
431             max_wait = float(
432                 getattr(getattr(config, "deployment", None), "remote_poll_max_wait_sec",
3900.0)
433                 or 3900.0
434             )
435             policy = ExecutionPolicy(
436                 poll_every_n_sec=_RESUME_POLL_EVERY_SEC,
437                 op_timeout_sec=120.0,
438                 max_wait_sec=max_wait,
439             )
440             db.set_job_progress(job_id, "recovering: waiting for the in-flight cloud run")
441             try:
442                 poll_until(
443                     lambda: True if storage.exists(report_uri, config=sc) else None,
```

```
444                     policy,
445                     label=f"resume-{job_id[:8]}",
446                     logger=logger,
447                     # The #493 live heartbeat: job.progress moves once per still-waiting
tick.
448                     on_tick=lambda elapsed: db.set_job_progress(
449                         job_id,
450                         "recovering: waiting for the in-flight cloud run ? "
451                         f"{int(elapsed) // 60} min",
452                     ),
453                 )
454             except PolicyTimeout:
455                 db.mark_job_failed(
456                     job_id,
457                     "control plane restarted mid-run and the cloud worker's result never "
458                     "appeared within the poll budget; re-submit the video (a finished result
"
459                     "re-hydrates instantly)",
460                 )
461                 return
462             payload = orchestration.cached_process_result(
463                 db, config, video_id, str(job.get("video_path") or "")
464             )
465             if payload is not None:
466                 db.mark_job_done(job_id, payload)
467                 logger.warning(
468                     "[JOBS] resumed job %s: ingested the in-flight worker's result for %s.",
469                     job_id,
470                     video_id,
471                 )
472                 return
473             # The report exists but did not realize into a usable result ? surface its
verdict.
474             status = "unreadable"
475             try:
476                 report = orchestration._read_bucket_json(report_uri, sc) or {}
477                 status = str(report.get("status") or "unreadable")
478             except Exception:  # noqa: BLE001 ? the failure message is best-effort
479                 pass
480             db.mark_job_failed(
481                 job_id,
482                 f"cloud worker finished without a usable result after a control-plane
restart "
483                 f"(report status: {status}); re-submit the video",
484             )
485         except Exception as e:  # noqa: BLE001 ? a resume crash must land as a job failure,
not vanish
486             logger.error("[JOBS] resume of job %s crashed: %s", job_id, e, exc_info=True)
487             try:
488                 db.mark_job_failed(job_id, f"restart recovery failed: {e}")
489             except Exception:  # noqa: BLE001
490                 pass
491
```

23. user (2026-07-08T05:06:17.321Z)

```
1       import logging
2       import os
3       import subprocess
4       import tempfile
5       from pathlib import Path
6       from typing import Any, Dict, List, Optional, Tuple, Union
7
8       from backend.utils.ffmpeg import (  # P2a: single ffmpeg/ffprobe resolver
9           ffmpeg_bin,
```

```
        ffmpeg_timeout_sec,
        ffprobe_bin,
        ffprobe_timeout_sec,
        timeout_label,
    )

    logger = logging.getLogger(__name__)

    # Bundled, commercially-clean default watermark font (Apache-2.0). Shipped so the
overlay
    # renders out-of-the-box without a system fontconfig setup (a common gap on Windows,
where
    # `font='Sans'` fails). Overridable via stitching.watermark.font_path. This module lives
at
    # backend/pipeline/stitcher/, so parents[2] == backend/.
    _BUNDLED_FONT = (
        Path(__file__).resolve().parents[2] / "assets" / "fonts" / "RobotoSlab-Regular.ttf"
    )


    class VideoCompiler:
        def __init__(
            self,
            output_dir: str,
            end_padding: float = 1.0,
            begin_padding: float = 0.5,
            watermark: Optional[Any] = None,
        ):
            self.output_dir = output_dir
            self.end_padding = end_padding
            self.begin_padding = begin_padding
            # Optional branding overlay. Accepts a WatermarkConfig model or a plain dict;
            # normalized to a dict (or None) so this module stays config-package-agnostic.
            self.watermark = self._normalize_watermark(watermark)
            os.makedirs(output_dir, exist_ok=True)

        @staticmethod
        def _normalize_watermark(watermark: Optional[Any]) -> Optional[Dict[str, Any]]:
            if watermark is None:
                return None
            if hasattr(watermark, "model_dump"):
                return watermark.model_dump()
            if isinstance(watermark, dict):
                return dict(watermark)
            return None

        @staticmethod
        def _ff_quote(path: str) -> str:
            """Make a filesystem path safe to embed inside single quotes in an ffmpeg
            filtergraph operand (drawtext fontfile/textfile, movie= source): forward
            slashes (Windows-friendly), escaped single quotes, AND escaped colons.

            The colon MUST be escaped even inside single quotes: ffmpeg's filtergraph
            option parser treats ':' as the option separator, so a Windows drive-letter
            path like ``C:/dir/font.ttf`` otherwise truncates the operand at ``C`` and
            the encode fails with "Invalid argument". This silently disabled the
            on-by-default bundled-font watermark (and absolute logo paths) on Windows."""
            return path.replace("\\", "/").replace("'", r"\'").replace(":", r"\:")

        @staticmethod
        def _parse_time(val: Union[int, float, str]) -> float:
            """Normalizes a timestamp value to float seconds.
            Handles: float/int (pass-through), plain numeric strings ("12.4"),
            and colon-separated strings ("MM:SS", "HH:MM:SS", "H:MM:SS.ms").
            This mirrors the parse_time logic used in gemini.py to ensure
```

```
            the stitcher can always consume whatever format the indexer produced."""
            if isinstance(val, (int, float)):
                return float(val)
            if isinstance(val, str):
                val = val.strip()
                if ":" in val:
                    # Handle MM:SS, HH:MM:SS, etc.
                    parts = val.split(":")
                    parts.reverse()
                    total = 0.0
                    for i, p in enumerate(parts):
                        try:
                            total += float(p) * (60**i)
                        except ValueError:
                            pass
                    return total
                try:
                    return float(val)
                except ValueError:
                    logger.warning(
                        f"Could not parse timestamp value '{val}' as a number. Defaulting to
0.0."
                    )
                    return 0.0
            logger.warning(
                f"Unexpected timestamp type {type(val).__name__} for value '{val}'.
Defaulting to 0.0."
            )
            return 0.0

        @staticmethod
        def _merge_overlapping(
            segments: List[Tuple[float, float]],
        ) -> List[Tuple[float, float]]:
            """Normalize, sort, and merge overlapping/duplicate (start, end) ranges.

            A highlight reel must never replay the same footage. The same rally can land in
            the segment list more than once ? e.g. two detector runs accumulated for one
            video (add_video is an UPSERT, so re-running without a prune leaves both sets),
or
            chunk-overlap duplicates near boundaries. Stitching those verbatim plays the
rally
            twice. Merging any range that overlaps or abuts the previous one collapses true
            duplicates/overlaps while preserving genuinely distinct (gapped) rallies.
            Degenerate rows (end <= start) are dropped. Timestamps may be float/int/str.
            """
            parsed = []
            for raw_s, raw_e in segments:
                s = VideoCompiler._parse_time(raw_s)
                e = VideoCompiler._parse_time(raw_e)
                if e > s:
                    parsed.append((s, e))
            parsed.sort()
            merged: List[Tuple[float, float]] = []
            for s, e in parsed:
                if merged and s <= merged[-1][1]:  # overlaps/abuts previous -> extend it
                    merged[-1] = (merged[-1][0], max(merged[-1][1], e))
                else:
                    merged.append((s, e))
            return merged

        def _get_video_duration(self, video_path: str) -> float:
            """Probes the video file to get its total duration in seconds.
            Returns 0.0 if it cannot be determined."""
            try:
```

```
                timeout = ffprobe_timeout_sec()
                result = subprocess.run(
                    [
                        ffprobe_bin(),
                        "-v",
                        "error",
                        "-show_entries",
                        "format=duration",
                        "-of",
                        "default=noprint_wrappers=1:nokey=1",
                        video_path,
                    ],
                    capture_output=True,
                    text=True,
                    check=True,
                    timeout=timeout,
                )
                return float(result.stdout.strip())
            except subprocess.TimeoutExpired:
                logger.warning(
                    "Could not determine video duration via ffprobe: timed out after %s",
                    timeout_label(ffprobe_timeout_sec()),
                )
                return 0.0
            except Exception as e:
                logger.warning(f"Could not determine video duration via ffprobe: {e}")
                return 0.0

        def _watermark_filter(self, tmp_dir: str) -> Optional[str]:
            """Build the ffmpeg -vf filtergraph that burns the configured watermark (text
            and/or logo PNG) into a clip during the per-clip re-encode. Returns None when
            watermarking is disabled or has nothing to draw. The concat stage stays -c copy
            because the mark is already baked into every clip identically."""
            wm = self.watermark
            if not wm or not wm.get("enabled"):
                return None

            text = (wm.get("text") or "").strip()
            logo_path = (wm.get("logo_path") or "").strip()
            if logo_path and not os.path.exists(logo_path):
                logger.warning(
                    f"[watermark] logo not found: {logo_path} ? skipping logo overlay."
                )
                logo_path = ""
            if not text and not logo_path:
                return None

            margin = int(wm.get("margin", 24))
            opacity = float(wm.get("opacity", 0.85))
            position = str(wm.get("position", "bottom-right"))
            # drawtext uses lowercase w/h (main) + tw/th (text); overlay uses W/H (main) +
w/h (logo).
            dt_xy = {
                "bottom-right": f"x=w-tw-{margin}:y=h-th-{margin}",
                "bottom-left": f"x={margin}:y=h-th-{margin}",
                "top-right": f"x=w-tw-{margin}:y={margin}",
                "top-left": f"x={margin}:y={margin}",
            }.get(position, f"x=w-tw-{margin}:y=h-th-{margin}")
            ov_xy = {
                "bottom-right": f"W-w-{margin}:H-h-{margin}",
                "bottom-left": f"{margin}:H-h-{margin}",
                "top-right": f"W-w-{margin}:{margin}",
                "top-left": f"{margin}:{margin}",
            }.get(position, f"W-w-{margin}:H-h-{margin}")

```

```
            drawtext = ""
            if text:
                ratio = float(wm.get("font_size_ratio") or 0.035)
                divisor = max(1, round(1.0 / ratio))
                # textfile= sidesteps the notorious drawtext text= escaping for arbitrary
strings.
                tf_path = os.path.join(tmp_dir, "watermark.txt")
                with open(tf_path, "w", encoding="utf-8") as fh:
                    fh.write(text)
                # Font precedence: explicit font_path ? bundled Apache-2.0 default (renders
                # without fontconfig) ? fontconfig family (last-resort, environment-
dependent).
                font_path = (wm.get("font_path") or "").strip()
                if not font_path and _BUNDLED_FONT.exists():
                    font_path = str(_BUNDLED_FONT)
                if font_path:
                    font_opt = f"fontfile='{self._ff_quote(font_path)}'"
                else:
                    font_opt = f"font='{wm.get('font', 'Sans')}'"
                box = ""
                if wm.get("box", True):
                    box = f":box=1:boxcolor=black@{min(opacity, 0.5):.2f}:boxborderw=10"
                drawtext = (
                    f"drawtext=textfile='{self._ff_quote(tf_path)}':{font_opt}"
                    f":fontcolor=white@{opacity:.2f}:fontsize=h/{divisor}:{dt_xy}{box}"
                )

            if logo_path and drawtext:
                return
f"movie='{self._ff_quote(logo_path)}'[wm];[in][wm]overlay={ov_xy},{drawtext}"
            if logo_path:
                return f"movie='{self._ff_quote(logo_path)}'[wm];[in][wm]overlay={ov_xy}"
            return drawtext

        def _trim_one_segment(
            self,
            idx: int,
            raw_start: Union[int, float, str],
            raw_end: Union[int, float, str],
            segments: List[Tuple[float, float]],
            video_duration: float,
            tmp_dir: str,
            raw_video_path: str,
            watermark_filter: Optional[str],
        ) -> Tuple[Optional[str], Optional[Tuple[int, float, float, str]]]:
            """Trim a single segment into its own clip file (extracted verbatim from the
            per-segment body of compile_highlights). Returns (clip_path, skip_record): on
            success the clip path and None; on a skip (invalid/out-of-range/ffmpeg failure)
            None and the (idx, start, end, reason) record to record in skipped_segments."""
            # Normalize timestamps to float seconds ? handles float, int,
            # plain numeric strings, and colon-separated formats like "MM:SS"
            start = self._parse_time(raw_start)
            end = self._parse_time(raw_end)
            segment_label = (
                f"Segment #{idx + 1}/{len(segments)} [{start:.2f}s - {end:.2f}s]"
            )

            # Validate the segment times
            if start < 0:
                logger.warning(
                    f"{segment_label}: start_time is negative ({start:.2f}s). Clamping to
0."
                )
                start = 0.0

```

```
            if start >= end:
                logger.error(
                    f"{segment_label}: start_time ({start:.2f}s) >= end_time ({end:.2f}s).
Skipping invalid segment."
                )
                return None, (idx, start, end, "start >= end")

            # Check if segment extends beyond video duration
            if video_duration > 0:
                if start >= video_duration:
                    logger.error(
                        f"{segment_label}: start_time ({start:.2f}s) is beyond the video
duration ({video_duration:.2f}s). "
                        f"This segment cannot be extracted ? the video ran out. Skipping."
                    )
                    return None, (idx, start, end, "start beyond video end")

                if end > video_duration:
                    original_end = end
                    end = video_duration
                    logger.warning(
                        f"{segment_label}: end_time ({original_end:.2f}s) exceeds video
duration ({video_duration:.2f}s). "
                        f"Clamping end_time to {video_duration:.2f}s. The segment may be
truncated."
                    )

            clip_path = os.path.join(tmp_dir, f"clip_{idx}.mp4")

            # Precise sub-second trim using fast re-encoding to preserve stream headers
            padded_start = max(0.0, start - self.begin_padding)
            padded_end = end + self.end_padding

            # Clamp padded_end to video duration if known
            if video_duration > 0 and padded_end > video_duration:
                padded_end = video_duration
                logger.info(
                    f"{segment_label}: Padded end ({end + self.end_padding:.2f}s) exceeds
video duration. "
                    f"Clamping to {video_duration:.2f}s (end padding partially applied)."
                )

            trim_duration = padded_end - padded_start
            logger.info(
                f"Trimming {segment_label} -> [{padded_start:.3f}s - {padded_end:.3f}s]
(duration: {trim_duration:.2f}s)"
            )

            base_cmd = [
                ffmpeg_bin(),
                "-y",
                "-ss",
                str(round(padded_start, 3)),
                "-to",
                str(round(padded_end, 3)),
                "-i",
                raw_video_path,
                "-c:v",
                "libx264",
                "-preset",
                "superfast",
                "-crf",
                "22",
                "-c:a",
                "aac",
```

```
                "-b:a",
                "128k",
            ]
            vf_args = ["-vf", watermark_filter] if watermark_filter else []

            # Try with the watermark first; if that encode fails (e.g. a bad font/logo
            # path), retry once WITHOUT it so a branding misconfig can never nuke the
            # whole reel. Clips stay codec-identical either way, so concat -c copy holds.
            attempts = [vf_args, []] if vf_args else [[]]
            produced = False
            last_exc: Optional[subprocess.CalledProcessError] = None
            cmd = base_cmd + [clip_path]
            timeout = ffmpeg_timeout_sec()
            for attempt_vf in attempts:
                cmd = base_cmd + attempt_vf + [clip_path]
                try:
                    subprocess.run(
                        cmd, capture_output=True, check=True, timeout=timeout
                    )
                except subprocess.TimeoutExpired:
                    reason = f"ffmpeg timeout after {timeout_label(timeout)}"
                    logger.error(f"{segment_label}: FFmpeg trim timed out.")
                    print(f"  Command: {' '.join(cmd)}")
                    print(f"  {reason}")
                    return None, (idx, start, end, reason)
                except subprocess.CalledProcessError as e:
                    last_exc = e
                    if attempt_vf and vf_args:
                        _err = (
                            e.stderr.decode(errors="replace").strip()[-300:]
                            if getattr(e, "stderr", None)
                            else ""
                        )
                        logger.warning(
                            f"{segment_label}: watermark encode failed; retrying without it.
"
                            f"ffmpeg: {_err or '(no stderr captured)'}"
                        )
                        continue
                    break
                if os.path.exists(clip_path) and os.path.getsize(clip_path) > 0:
                    produced = True
                    if not attempt_vf and vf_args:
                        logger.warning(
                            f"{segment_label}: encoded WITHOUT the watermark (the watermark
filter failed)."
                        )
                    break
                last_exc = None  # ffmpeg succeeded but produced nothing
                break

            if produced:
                return clip_path, None
            elif last_exc is not None:
                stderr_msg = last_exc.stderr.decode(errors="replace").strip()
                logger.error(f"{segment_label}: FFmpeg trim failed.")
                print(f"  Command: {' '.join(cmd)}")
                print(
                    f"  FFmpeg stderr: {stderr_msg[-500:]}"
                )  # Last 500 chars to avoid flooding
                return None, (idx, start, end, f"ffmpeg error: {stderr_msg[-200:]}")
            else:
                logger.error(
                    f"{segment_label}: FFmpeg exited successfully but output clip is missing
or empty. Skipping."
```

```
                )
                return None, (idx, start, end, "empty output clip")

        def compile_highlights(
            self,
            raw_video_path: str,
            segments: List[Tuple[float, float]],
            output_filename: str,
        ) -> str:
            """
            Extracts selected segments from a raw video and stitches them together
            into a seamless highlights file, applying end_padding to prevent abrupt endings.

            .. warning::
               This method uses a two-stage codec contract: per-clip re-encodes must use
               IDENTICAL encoder settings (codec, preset, CRF, pixel format) so the final
               concat can stream-copy (``-c copy``) without re-encoding. Changing the
               clip encode settings without updating the concat assumptions will produce
               broken output.
            """
            if not segments:
                raise ValueError("No highlight segments provided to compile.")

            # Never stitch the same footage twice: collapse overlapping/duplicate ranges
            # (e.g. accumulated re-run segments or chunk-overlap dupes) before trimming.
            original_count = len(segments)
            segments = self._merge_overlapping(segments)
            if len(segments) < original_count:
                logger.info(
                    "Merged %d overlapping/duplicate segment(s) before stitching: %d ->
%d.",
                    original_count - len(segments),
                    original_count,
                    len(segments),
                )
            if not segments:
                raise ValueError("No valid (start < end) highlight segments after merge.")

            if not os.path.exists(raw_video_path):
                raise FileNotFoundError(
                    f"[COMPILER ERROR] Source video file not found: {raw_video_path}"
                )

            # Defense-in-depth: never let the output name escape output_dir (the API also
            # validates this at the request boundary).
            output_path = os.path.join(self.output_dir, os.path.basename(output_filename))

            # Probe video duration so we can detect out-of-range segments
            video_duration = self._get_video_duration(raw_video_path)
            if video_duration > 0:
                logger.info(f"Source video duration: {video_duration:.2f}s")
            else:
                logger.warning(
                    "Video duration unknown. Cannot validate segment boundaries against
video length."
                )

            # We will create temporary clips and stitch them using FFmpeg concat demuxer
            temp_clips = []
            skipped_segments = []

            # Create a temp directory within output_dir
            with tempfile.TemporaryDirectory(dir=self.output_dir) as tmp_dir:
                logger.info(
                    f"Trimming {len(segments)} segments
```

```
(begin_padding={self.begin_padding}s, end_padding={self.end_padding}s)..."
                )
                # Build the (optional) watermark filtergraph once; it is applied during
every
                # per-clip re-encode so the mark is baked into each clip identically.
                watermark_filter = self._watermark_filter(tmp_dir)
                if watermark_filter:
                    logger.info("[watermark] applying branding overlay to each clip.")
                for idx, (raw_start, raw_end) in enumerate(segments):
                    clip_path, skip_record = self._trim_one_segment(
                        idx,
                        raw_start,
                        raw_end,
                        segments,
                        video_duration,
                        tmp_dir,
                        raw_video_path,
                        watermark_filter,
                    )
                    if clip_path is not None:
                        temp_clips.append(clip_path)
                    if skip_record is not None:
                        skipped_segments.append(skip_record)

                # Report summary of skipped segments
                if skipped_segments:
                    logger.info(
                        f"\n[SUMMARY] {len(skipped_segments)} out of {len(segments)}
segments were SKIPPED:"
                    )
                    for seg_idx, seg_start, seg_end, reason in skipped_segments:
                        logger.info(
                            f"  - Segment #{seg_idx + 1} [{seg_start:.2f}s -
{seg_end:.2f}s]: {reason}"
                        )
                    logger.info("")

                if not temp_clips:
                    raise RuntimeError(
                        f"[COMPILER ERROR] Failed to generate ANY valid highlight clips. "
                        f"All {len(segments)} segments were skipped. "
                        f"Check the errors above for details. "
                        f"Common causes: video file is corrupted, segments are out of range,
or ffmpeg is not installed."
                    )

                logger.info(
                    f"Successfully trimmed {len(temp_clips)}/{len(segments)} clips.
Preparing to stitch..."
                )

                # Create a list file for FFmpeg concat demuxer
                list_file_path = os.path.join(tmp_dir, "clips.txt")
                # encoding pinned: a non-ASCII clip path would crash on Windows (cp1252
default);
                # ffmpeg's concat demuxer reads the list as UTF-8 on every OS.
                with open(list_file_path, "w", encoding="utf-8") as f:
                    for clip in temp_clips:
                        # The concat demuxer resolves relative entries against the list
file's
                        # directory. Use absolute paths so a temp-dir clip does not become
                        # output/tmp.../output/tmp.../clip_0.mp4 on Windows.
                        safe_path = Path(clip).resolve().as_posix().replace("'", "'\\''")
                        f.write(f"file '{safe_path}'\n")

```

```
499                 logger.info("Stitching clips together...")
500                 # Stitch them together without re-encoding (since they were encoded with
identical settings)
501                 concat_cmd = [
502                     ffmpeg_bin(),
503                     "-y",
504                     "-f",
505                     "concat",
506                     "-safe",
507                     "0",
508                     "-i",
509                     list_file_path,
510                     "-c",
511                     "copy",
512                     output_path,
513                 ]
514
515                 try:
516                     timeout = ffmpeg_timeout_sec()
517                     subprocess.run(
518                         concat_cmd, capture_output=True, check=True, timeout=timeout
519                     )
520
521                     # Final verification
522                     if not os.path.exists(output_path) or os.path.getsize(output_path) == 0:
523                         raise RuntimeError(
524                             f"[COMPILER ERROR] FFmpeg concatenation appeared to succeed, "
525                             f"but output file is missing or empty: {output_path}"
526                         )
527
528                     output_size_mb = os.path.getsize(output_path) / (1024 * 1024)
529                     logger.info(
530                         f"Dynamic highlight reel saved: {output_path} ({output_size_mb:.1f}
MB)"
531                     )
532                     return output_path
533                 except subprocess.CalledProcessError as e:
534                     stderr_msg = e.stderr.decode(errors="replace").strip()
535                     raise RuntimeError(
536                         f"[COMPILER ERROR] FFmpeg concatenation failed.\n"
537                         f"  Command: {' '.join(concat_cmd)}\n"
538                         f"  FFmpeg stderr: {stderr_msg[-500:]}"
539                     ) from e
540                 except subprocess.TimeoutExpired as e:
541                     raise RuntimeError(
542                         f"[COMPILER ERROR] FFmpeg concatenation timed out after
{timeout_label(timeout)}.\n"
543                         f"  Command: {' '.join(concat_cmd)}"
544                     ) from e
545
```

24. assistant (2026-07-08T05:06:21.405Z)

Now let me find the compile API endpoint, the job submission, and the cloud-serving preset.

25. user (2026-07-08T05:06:24.016Z)

```
67:from backend.pipeline.stitcher.compiler import VideoCompiler
228:# The processing/compile ORCHESTRATION (hash?validate?segment?report pipeline, Cloud Run
229:# dispatch+ingest, compile resolution + worker handler) now lives in backend.orchestration
232:# call-time `_main._execute_compile(...)` seam, and monkeypatch.setattr(backend.main, ...)
237:    _execute_compile,
244:    _resolve_compile,
479:    """List compiled highlight reels present in the output dir, newest first, with a
download URL.
```

```
481:    NB: reels are stored FLAT in output_dir under the bare name passed to /api/compile ? the
engine
483:    this is a global list. (Per-video reel tracking would need a DB record + compile change
? a
512:    """Stream a compiled reel by its bare filename (no video_id ? reels are flat in
output_dir).
525:        raise HTTPException(status_code=404, detail=f"No compiled reel named '{name}'.")
1043:    """Stream a compiled highlight reel ? `filename` is the bare name given to /api/compile
1076:            detail=f"No compiled file named '{name}'. Compile first via /api/compile.",
1098:@app.post("/api/compile", status_code=202)
1099:def compile_highlights(req: CompileRequest, request: Request):
1114:        video, _kept, out_name = _resolve_compile(
1127:    if not request.app.state.db.create_compile_job(
1137:            status_code=500, detail="Failed to persist compile job record."
1381:        compiler = VideoCompiler(
1388:            res_path = compiler.compile_highlights(
1393:            print(f"[ERROR] Failed to compile trimmed segment: {e}")
1429:    # /api/compile ? reads the same value instead of a write-only runtime hack.
1480:            f"[JOBS] Recovered stale jobs from a previous run ? {requeued} compile re-
queued, "
```

26. user (2026-07-08T05:06:24.395Z)

```
1       """Deployment-profile presets + environment auto-detection (COMPUTE_DECOUPLED_SERVING
M1).
2
3       A *deployment profile* is a single switch that applies a coherent BUNDLE of existing
4       config knobs, so the truly-local / cloud-serving / cpu-mock modes don't drift apart one
5       knob at a time (the design in ``docs/COMPUTE_DECOUPLED_SERVING/README.md`` §4.3).
6
7       Precedence (lowest ? highest), realised by the loader::
8
9           built-in model defaults  <  profile preset  <  config.json  <  env  <  worker --set
10
11      This module is **pure data + pure functions** ? no IO, and (critically) **no torch
import at
12      module load**. The loader applies a preset ONLY when a profile is EXPLICITLY selected
13      (``config.json`` ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); with no
14      profile selected the loader is byte-identical to the pre-M1 behaviour (``profile=""`` ==
15      today). Auto-detect only ever **suggests** a profile ? it never auto-applies one (D15:
cloud
16      spend is never entered silently).
17      """
18
19      from __future__ import annotations
20
21      import copy
22      import os
23      from typing import Any, Mapping, Optional
24
25      # Canonical enum values, defined ONCE here. ``models.DeploymentConfig``'s ``Literal``
mirrors
26      # these and ``tests/test_config_deployment.py`` pins parity so the two can never drift.
27      PROFILES: tuple[str, ...] = ("truly-local", "cloud-serving", "cpu-mock")
28      COMPUTE_TARGETS: tuple[str, ...] = ("local_gpu", "cloud_run", "cpu_mock")
29
30      # Each profile maps 1:1 to its canonical compute_target. Used to apply the preset and to
warn
31      # when config.json explicitly overrides compute_target into a state inconsistent with
the profile.
32      COMPUTE_TARGET_FOR_PROFILE: dict[str, str] = {
33          "truly-local": "local_gpu",
34          "cloud-serving": "cloud_run",
35          "cpu-mock": "cpu_mock",
36      }
```

```
37
38      # A preset is a partial, nested override dict over the model defaults. It touches ONLY
knobs
39      # that EXIST today ? M1 is pure/additive: the new
``deployment.{profile,compute_target}`` plus
40      # ``indexing.{detector_impl,skip_ai_handoff}`` and ``storage.backend``. Later milestones
extend
41      # these bundles ? ``input_backend`` (M2) and the configurable output/workspace base (M3,
42      # Launch-Report-gated) ? deliberately NOT set here so M1 never pre-empts a gated flip.
43      PROFILE_PRESETS: dict[str, dict[str, Any]] = {
44          "truly-local": {
45              "deployment": {"profile": "truly-local", "compute_target": "local_gpu"},
46              # detector_impl stays "auto" (the runner picks WSL vs native by env on the GPU
box).
47              # Zero-cloud / privacy-first ? run fully local with no Gemini handoff.
48              "indexing": {"skip_ai_handoff": True},
49              "storage": {"backend": "local"},
50          },
51          "cloud-serving": {
52              "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
53              # Linux L4 (cuda); Gemini handoff ON until the owned model clears the #172 floor
(D11).
54              # wasb GPU-feed tuning: the parity defaults (batch 8, no prefetch) left the paid
L4
55              # <1% utilized on the first real serving run (2026-07-03) ? the pipeline was
56              # CPU-feed-bound. Serving opts into the measured fast values; local/WSL
unchanged.
57              "indexing": {
58                  "detector_impl": "native",
59                  "skip_ai_handoff": False,
60                  # GPU-resident NVDEC decode (#506/#507): validated on a real L4 (golden gate
PASS,
61                  # F1 0.574 ? baseline 0.582 @ SERVED) AND on the live Cloud Run L4 (the
served
62                  # trajectory reproduced the VM run ? 26479 ? 26481 visible on GX010128),
then rolled
63                  # out + canaried (DEPLOY_REPORT_cloud-serving_gpu-resident-
nvdec_2026-07-07). This makes
64                  # nvdec the code-level serving default so it survives an image rebuild; the
worker's
65                  # WASB_DECODE_BACKEND env was the reversible rollout lever. cuda-only
(validator-enforced).
66                  "wasb": {
67                      "batch_size": 64,
68                      "prefetch_batches": 4,
69                      "decode_backend": "nvdec_resident",
70                  },
71              },
72              "storage": {"backend": "gcs"},
73          },
74          "cpu-mock": {
75              "deployment": {"profile": "cpu-mock", "compute_target": "cpu_mock"},
76              # Deterministic CPU mock ? no GPU/WSL, no provider/API. The CI / regression
profile.
77              "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
78              "storage": {"backend": "local"},
79          },
80      }
81
82
83      def is_known_profile(profile: str) -> bool:
84          """True for a non-empty, recognised profile name."""
85          return bool(profile) and profile in PROFILE_PRESETS
86
87
```

```
88      def preset_for(profile: str) -> dict[str, Any]:
89          """Return a deep COPY of the preset bundle for ``profile`` (so callers can't mutate
the
90          module-level table), or ``{}`` for ``""`` / an unknown profile."""
91          return copy.deepcopy(PROFILE_PRESETS.get(profile, {}))
92
93
94      def _truthy(val: Optional[str]) -> bool:
95          """Common env-var truthiness (``1/true/yes/on``, case/space-insensitive)."""
96          if val is None:
97              return False
98          return val.strip().lower() in {"1", "true", "yes", "on"}
99
100
101     def _cuda_available() -> bool:
102         """Best-effort CUDA probe. torch is heavy + optional, so it is imported lazily here
and
103         ONLY when a caller opts into ``probe_cuda``; any import/runtime failure means 'no
GPU'."""
104         try:
105             # Intentionally lazy + local: keeps the config-load path torch-free (torch is
heavy
106             # and optional). Only reached when a caller opts into probe_cuda.
107             import torch
108
109             return bool(torch.cuda.is_available())
110         except Exception:
111             return False
112
113
114     def probe_cuda_available() -> bool:
115         """Public best-effort CUDA probe (lazy torch import; ``False`` on any failure).
116
117         Exposed for the per-profile startup checks (``backend/config/startup.py``) so a
caller
118         that *wants* the GPU heads-up can pass the result in without the startup module
importing
119         torch itself. Best-effort by design: if torch lives only in a detector subprocess
env
120         (native WASB), this returns ``False`` in the main process ? which is why the startup
checks
121         treat a GPU mismatch as a WARNING (the runner healthcheck is the authority), never a
hard
122         error that could false-fail."""
123         return _cuda_available()
124
125
126     def suggest_profile(
127         env: Optional[Mapping[str, str]] = None, *, probe_cuda: bool = False
128     ) -> str:
129         """SUGGEST (never auto-apply ? D15) a deployment profile from the environment.
130
131         Signals, in priority order:
132
133           - ``K_SERVICE`` set (Cloud Run)        ? ``"cloud-serving"``
134           - ``CI`` truthy                        ? ``"cpu-mock"``
135           - ``probe_cuda`` and a CUDA GPU present ? ``"truly-local"``
136           - otherwise                            ? ``""`` (no strong suggestion)
137
138         ``probe_cuda`` defaults to ``False`` so the common config-load path stays cheap and
never
139         imports torch; a CLI ``--detect`` or cloud-serving startup that wants the GPU probe
passes
140         ``probe_cuda=True``. The result is advisory only: the caller decides whether to act
on it.
```

```
141         """
142         e = os.environ if env is None else env
143         if e.get("K_SERVICE"):
144             return "cloud-serving"
145         if _truthy(e.get("CI")):
146             return "cpu-mock"
147         if probe_cuda and _cuda_available():
148             return "truly-local"
149         return ""
150
```

27. assistant (2026-07-08T05:06:27.370Z)

Let me read the compile API endpoint and how compile jobs store their params.

28. user (2026-07-08T05:06:29.382Z)

```
1080
1081
1082    @app.post("/api/query")
1083    def query_play_segments(req: QueryRequest, request: Request):
1084        filters = {
1085            "server": req.server,
1086            "receiver": req.receiver,
1087            "duration_min": req.duration_min,
1088            "event_type": req.event_type,
1089        }
1090        segments = request.app.state.db.query_play_segments(req.video_id, filters)
1091        if not segments and read_path_rehydrate(
1092            request.app.state.db, request.app.state.config, req.video_id
1093        ):  # #485: DB = cache, bucket = truth ? re-hydrate before reporting nothing
1094            segments = request.app.state.db.query_play_segments(req.video_id, filters)
1095        return {"play_segments": segments}
1096
1097
1098    @app.post("/api/compile", status_code=202)
1099    def compile_highlights(req: CompileRequest, request: Request):
1100        """Submit a reel-COMPILE job (202 + job_id); poll GET /api/jobs/{job_id} for the
reel (#329).
1101
1102        Async, mirroring /api/process: the ffmpeg stitch (trim + re-encode each clip +
concat + watermark)
1103        can run minutes on CPU ? past the ~100 s edge/proxy timeout ? so doing it
synchronously returned a
1104        Cloudflare 524 while the engine kept going (the reel built, but the client saw
"failed" and never
1105        got the URL). The cheap checks (video exists, segments match + survive the floor,
safe filename)
1106        fast-fail HERE with a 4xx; the stitch runs on the worker. On `done`, the job's
`result` carries
1107        {filepath, download_url} ? the same shape the sync endpoint returned, now under
/api/jobs/{id}."""
1108        # M9 edge-auth tenant (DEFAULT-OFF ? None); a missing/spoofed identity fails 401
before queuing.
1109        try:
1110            owner_id = edge_auth.resolve_tenant(request.headers, request.app.state.config)
1111        except edge_auth.EdgeAuthError as e:
1112            raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
1113        try:
1114            video, _kept, out_name = _resolve_compile(
1115                request.app.state.db,
1116                request.app.state.config,
1117                req.video_id,
1118                req.segment_ids,
1119                req.output_filename,
```

```
            )
        except _CompileError as e:
            raise HTTPException(status_code=e.status_code, detail=e.detail) from e

        job_id = uuid.uuid4().hex
        # Persist the video's SOURCE ref (the NOT NULL video_path); the worker re-fetches
the row by id so
        # the freshest filepath wins. owner_id/locale ride along for M9 multi-tenant + the
localized reel.
        if not request.app.state.db.create_compile_job(
            job_id,
            req.video_id,
            video["filepath"],
            req.segment_ids,
            out_name,
            locale=req.locale,
            owner_id=owner_id,
        ):
            raise HTTPException(
                status_code=500, detail="Failed to persist compile job record."
            )
        _get_executor().submit(
            _drain_due_jobs, request.app.state.config, request.app.state.db
        )
        return JSONResponse(
            status_code=202,
            content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
        )


    # --- CLI Debug Utility & Orchestration ---
    def run_cli_diagnose_clip(video_path: str, timestamp: float):
        """CLI debugging utility to examine segment properties around a timestamp."""
        print("=" * 60)
        print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
        print("=" * 60)

        cfg = load_config()  # legacy wrapper still works, returns dict for now

        # 1. Validation check diagnostics
        print("[DIAGNOSTIC] Running validation framework...")
        results = registry.run_all(video_path, cfg)
        for r in results:
            status_symbol = "[PASS]" if r.passed else "[FAIL]"
            print(f"  {status_symbol} {r.validator_name}: {r.message}")
            if r.details:
                print(f"         Details: {r.details}")

        # 2. Local segment analysis around the timestamp (sampling +/- 5 seconds)
        print("-" * 60)
        print("[DIAGNOSTIC] Analyzing local motion around the timestamp...")
        # Lazy import: cv2 is NOT installed in the LIGHT tier (torch-free default)
        # and this --debug-clip path is CLI-only, not on the served API path.
        import cv2

        cap = cv2.VideoCapture(video_path)
        if not cap.isOpened():
            print("[FAIL] OpenCV could not open video.")
            return

        fps = cap.get(cv2.CAP_PROP_FPS)
        total_frames = cap.get(cv2.CAP_PROP_FRAME_COUNT)

        start_frame = max(0, int((timestamp - 3.0) * fps))
        end_frame = min(int(total_frames), int((timestamp + 3.0) * fps))
```

```
        # Measure frame-by-frame changes in sample region
        cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
        prev_gray = None
        motions = []

        for _ in range(start_frame, end_frame):
            ret, frame = cap.read()
            if not ret:
                break
            gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
            gray = cv2.GaussianBlur(gray, (21, 21), 0)

            if prev_gray is not None:
                diff = cv2.absdiff(prev_gray, gray)
                _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
                motion_ratio = cv2.countNonZero(thresh) / (
                    thresh.shape[0] * thresh.shape[1]
                )
                motions.append(motion_ratio)
            prev_gray = gray

        cap.release()

        if motions:
            avg_motion = sum(motions) / len(motions)
            # Support both legacy dict (from wrapper) and future direct model
            if hasattr(cfg, "indexing"):
                threshold = getattr(cfg.indexing, "motion_threshold", 0.08)
            else:
                threshold = cfg.get("indexing", {}).get("motion_threshold", 0.08)  # type:
ignore[attr-defined]
            print(f"  Sample Frame Count: {len(motions)}")
            print(
                f"  Average motion index around timestamp: {round(avg_motion, 4)}
(threshold: {threshold})"
            )
            if avg_motion > threshold:
                print(
                    "  Result: [ACTIVE PLAY] High motion detected. Segment classifies as
active play."
                )
            else:
                print(
                    "  Result: [DEAD TIME] Idle movement. Segment classified as dead time."
                )
        else:
            print("  [ERROR] No visual frames were extracted in the sample range.")

        print("=" * 60)


    def _enforce_startup_or_exit(config) -> None:
        """M5: run the startup checks for the server path and ``sys.exit(1)`` on a FATAL
finding BEFORE
        any work ? a fatal incoherent profile (e.g. cloud-serving pointed at local-only
storage) OR, for
        the exposed server (``include_exposure=True``, M9 hardening), a half-configured
exposure (edge
        auth on without the ``allowed_video_root`` jail / CF team+AUD). A strict no-op when
no
        ``deployment.profile`` is selected AND edge auth is off (today's behaviour). No GPU
probe here ?
        the server stays torch-light; the detector healthcheck is the GPU authority.
Extracted so the
```

```
1239        fail-fast wiring + the default-OFF no-op are unit-testable."""
```

29. user (2026-07-08T05:06:30.709Z)

```
462:        # v9: the jobs table now holds TWO kinds of work ? `kind`='process' (the original,
NULL ?
463:        # process for pre-v9 rows) and `kind`='compile' (the async reel stitch, #329;
/api/compile
465:        # is the kind-specific JSON payload a compile job carries
(segment_ids/output_filename/locale)
467:        self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN kind TEXT")
```

30. assistant (2026-07-08T05:06:36.312Z)

Let me read the compile-job DB methods and existing tests to mirror the patterns.

31. user (2026-07-08T05:06:36.588Z)

No matches found

32. user (2026-07-08T05:06:39.298Z)

```
=== compile-related tests ===
test_api_cloud_dispatch.py
test_async_jobs.py
test_compiler.py
test_compute_backend.py
test_job_worker_loop.py
test_job_worker_module_seam.py
test_worker.py
=== cov floor ===
.github/workflows/ci.yml:        run: python run_all_tests.py --cov=backend --cov=training
--cov-report=term --cov-fail-under=70
.github/workflows/nightly.yml:        run: python run_all_tests.py --cov=backend --cov=training
--cov-report=term --cov-fail-under=70
```

33. user (2026-07-08T05:06:43.927Z)

```
295:        # re-claims it via `claim_due_job` when due ? with NO master node: the table IS
462:        # v9: the jobs table now holds TWO kinds of work ? `kind`='process' (the original,
NULL ?
463:        # process for pre-v9 rows) and `kind`='compile' (the async reel stitch, #329;
/api/compile
465:        # is the kind-specific JSON payload a compile job carries
(segment_ids/output_filename/locale)
467:        self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN kind TEXT")
```

34. user (2026-07-08T05:06:47.170Z)

```
Found 2 files
backend\infrastructure\database.py
backend\infrastructure\database_jobs.py
```

35. user (2026-07-08T05:06:49.810Z)

```
1       import json
2       import logging
3       from typing import Any, Dict, List, Optional, Tuple
4
5       from backend import billing  # pure cost math (no backend imports -> no cycle)
6
7       logger = logging.getLogger(__name__)
8
9
```

```
    class DatabaseJobsMixin:
        def _connect(self):  # pragma: no cover - implemented by Database facade
            raise NotImplementedError

        def _execute_write(  # pragma: no cover - implemented by Database facade
            self, sql: str, params: Tuple[Any, ...], error_msg: str
        ) -> bool:
            raise NotImplementedError

        # --- Async job store (DEFERRED_HARDENING_PLAN #16) ------------------------------
        # Writers follow the repo convention (swallow + logger.error, return bool); the
        # worker logs loudly on a failed transition so a job can't silently wedge.

        def create_job(
            self,
            job_id: str,
            video_path: str,
            segmenter: Optional[str] = None,
            player_pool: Optional[List[str]] = None,
            max_frames: Optional[int] = None,
            policy: Optional[Dict[str, Any]] = None,
            owner_id: Optional[str] = None,
            locale: Optional[str] = None,
            compute: Optional[str] = None,
            force: bool = False,
        ) -> bool:
            """Insert a new job in state 'queued'. ``policy`` is an optional JSON
ExecutionPolicy
            (EXECUTION_POLICY.md) the worker honours; None ? the worker's default policy.
``owner_id``
            (M9, D9) is the verified tenant from edge auth ? NULL for the local single-user
(default).
            ``locale`` (v7) is the UI language the submission came from ? NULL when
unrecorded.
            ``compute`` (v8) is the SPA compute-picker choice ("local"/"cloud") the worker
reconstructs +
            honours; NULL ? the server default (byte-identical to pre-picker behaviour).
``force`` opts
            into deliberate reprocessing; false leaves params NULL for old-row
compatibility."""
            params = json.dumps({"force": True}) if force else None
            return self._execute_write(
                "INSERT INTO jobs (id, video_path, segmenter, player_pool, max_frames,
status, policy, "
                "owner_id, locale, compute, params) VALUES (?, ?, ?, ?, ?, 'queued', ?, ?,
?, ?, ?)",
                (
                    job_id,
                    video_path,
                    segmenter,
                    json.dumps(player_pool) if player_pool is not None else None,
                    max_frames,
                    json.dumps(policy) if policy is not None else None,
                    owner_id,
                    locale,
                    compute,
                    params,
                ),
                f"Failed to create job {job_id}",
            )

        def create_compile_job(
            self,
            job_id: str,
            video_id: str,
```

```
            video_path: str,
            segment_ids: List[int],
            output_filename: str,
            locale: Optional[str] = None,
            owner_id: Optional[str] = None,
        ) -> bool:
            """Insert an async REEL-COMPILE job (#329) in state 'queued', `kind`='compile'.
The stitch
            params (segment_ids / output_filename / locale) ride the JSON `params` column
since they don't
            map onto the process columns; `video_path` is the target video's source ref (the
table's
            NOT NULL column) and `video_id` is set up-front (so /api/jobs surfaces it
immediately). The
            worker dispatches on `kind` and runs the ffmpeg stitch off the request
thread."""
            params = json.dumps(
                {
                    "segment_ids": list(segment_ids),
                    "output_filename": output_filename,
                    "locale": locale,
                }
            )
            return self._execute_write(
                "INSERT INTO jobs (id, video_path, video_id, status, owner_id, locale, kind,
params) "
                "VALUES (?, ?, ?, 'queued', ?, ?, 'compile', ?)",
                (job_id, video_path, video_id, owner_id, locale, params),
                f"Failed to create compile job {job_id}",
            )

        def count_jobs_by_locale(self) -> Dict[str, int]:
            """PMF analytics: how many jobs were submitted per UI locale. Unrecorded locales
(NULL ?
            CLI / pre-v7 / older clients) bucket under '(unset)'. Returns {locale: count},
busiest first."""
            with self._connect() as conn:
                rows = (
                    conn.cursor()
                    .execute(
                        "SELECT COALESCE(locale, '(unset)') AS loc, COUNT(*) AS n "
                        "FROM jobs GROUP BY loc ORDER BY n DESC, loc"
                    )
                    .fetchall()
                )
            return {str(r[0]): int(r[1]) for r in rows}

        def mark_job_running(self, job_id: str) -> bool:
            return self._execute_write(
                "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP, "
                "progress='starting' WHERE id = ?",
                (job_id,),
                f"Failed to mark job {job_id} running",
            )

        def set_job_progress(self, job_id: str, progress: str) -> bool:
            return self._execute_write(
                "UPDATE jobs SET progress=? WHERE id = ?",
                (progress, job_id),
                f"Failed to set progress for job {job_id}",
            )

        def set_job_video_id(self, job_id: str, video_id: str) -> bool:
            return self._execute_write(
                "UPDATE jobs SET video_id=? WHERE id = ?",
```

```
                (video_id, job_id),
                f"Failed to set video_id for job {job_id}",
            )

        def mark_job_done(self, job_id: str, result: Dict[str, Any]) -> bool:
            """Job EXECUTED to completion. The pipeline's own verdict (success/failed
            validation) is inside `result` ? job status 'done' means 'the worker
finished'."""
            return self._execute_write(
                "UPDATE jobs SET status='done', progress='finished', result=?, "
                "finished_at=CURRENT_TIMESTAMP WHERE id = ?",
                (json.dumps(result), job_id),
                f"Failed to mark job {job_id} done",
            )

        def mark_job_failed(self, job_id: str, error: str) -> bool:
            """Worker crashed / raised ? infrastructure failure, not a pipeline verdict."""
            return self._execute_write(
                "UPDATE jobs SET status='failed', error=?, "
                "finished_at=CURRENT_TIMESTAMP WHERE id = ?",
                (error, job_id),
                f"Failed to mark job {job_id} failed",
            )

        def get_job(self, job_id: str) -> Optional[Dict[str, Any]]:
            """Fetch one job; `result`/`player_pool` JSON deserialized."""
            with self._connect() as conn:
                row = (
                    conn.cursor()
                    .execute("SELECT * FROM jobs WHERE id = ?", (job_id,))
                    .fetchone()
                )
                if not row:
                    return None
                d = dict(row)
                d["result"] = json.loads(d["result"]) if d["result"] else None
                d["player_pool"] = (
                    json.loads(d["player_pool"]) if d["player_pool"] else None
                )
                d["policy"] = json.loads(d["policy"]) if d.get("policy") else None
                return d

        def list_jobs(
            self,
            owner_id: Optional[str] = None,
            statuses: Optional[List[str]] = None,
            limit: int = 50,
        ) -> List[Dict[str, Any]]:
            """List a tenant's jobs, newest first ? the /api/jobs processing-queue view so a
client can
            discover in-flight work it didn't submit (a new tab/device, cleared storage).
``owner_id``
            NULL ? the local single-user's NULL-owner rows; a value ? ONLY that tenant's
rows (M9, D9
            isolation ? never leak another tenant's jobs). ``statuses`` filters execution
state (e.g.
            ['queued','running','waiting'] for the active queue); None ? any state. Light
projection: the
            heavy ``result`` JSON is reduced to ``result_status`` (the pipeline verdict) so
the queue stays
            cheap to poll."""
            owner_clause = "owner_id IS NULL" if owner_id is None else "owner_id = ?"
            params: List[Any] = [] if owner_id is None else [owner_id]
            clauses = [owner_clause]
            if statuses:
```

```
                clauses.append("status IN (%s)" % ",".join("?" for _ in statuses))
                params.extend(statuses)
            sql = (
                "SELECT id, kind, status, progress, video_id, video_path, compute, error,
result, "
                "created_at, started_at, finished_at FROM jobs WHERE "
                + " AND ".join(clauses)
                + " ORDER BY created_at DESC, id DESC LIMIT ?"
            )
            params.append(int(limit))
            with self._connect() as conn:
                rows = conn.cursor().execute(sql, tuple(params)).fetchall()
            out: List[Dict[str, Any]] = []
            for row in rows:
                d = dict(row)
                res = json.loads(d["result"]) if d.get("result") else None
                d["result_status"] = (res or {}).get("status") if res else None
                d.pop("result", None)
                out.append(d)
            return out

        def find_active_process_job(
            self, owner_id: Optional[str], video_path: str
        ) -> Optional[Dict[str, Any]]:
            """Dedup key for /api/process idempotency: the {id, status} of an already-ACTIVE
            (queued|running|waiting) PROCESS job for the same (owner_id, video_path), else
None ? so a
            re-submit of the same video (a tester who can't see their running job) returns
that job
            instead of dispatching a second, PAID GPU run. Compile jobs (kind='compile') are
excluded;
            the owner scoping mirrors ``list_jobs`` (NULL ? local single-user)."""
            owner_clause = "owner_id IS NULL" if owner_id is None else "owner_id = ?"
            params: Tuple[Any, ...] = (
                (video_path,) if owner_id is None else (owner_id, video_path)
            )
            with self._connect() as conn:
                row = (
                    conn.cursor()
                    .execute(
                        "SELECT id, status FROM jobs WHERE "
                        + owner_clause
                        + " AND video_path = ? AND status IN ('queued', 'running',
'waiting') "
                        "AND (kind IS NULL OR kind != 'compile') "
                        "ORDER BY created_at DESC LIMIT 1",
                        params,
                    )
                    .fetchone()
                )
            return {"id": str(row["id"]), "status": str(row["status"])} if row else None

        def claim_or_create_process_job(
            self,
            job_id: str,
            video_path: str,
            segmenter: Optional[str] = None,
            player_pool: Optional[List[str]] = None,
            max_frames: Optional[int] = None,
            policy: Optional[Dict[str, Any]] = None,
            owner_id: Optional[str] = None,
            locale: Optional[str] = None,
            compute: Optional[str] = None,
            force: bool = False,
            video_id: str = "",
```

```
        ) -> Optional[Dict[str, Any]]:
            """ATOMIC idempotent submit for a PROCESS job (fixes the read-before-insert race
? PR #481
            review): in ONE write-locked statement, insert ``job_id`` as 'queued' ONLY IF no
active
            (queued|running|waiting) process job already exists for (owner_id, video_path).
Two concurrent
            identical submits (a double-click / retry) therefore can't both insert ? SQLite
serializes
            writers, and the second's ``INSERT ? WHERE NOT EXISTS`` sees the first's row
(same
            compare-and-set idiom as ``claim_due_job``, no read-then-write gap). ``force``
skips the guard
            (deliberate reprocess ? always inserts). Returns
``{"job_id","status","created"}`` ? ``created``
            False means an active job already existed and the caller must NOT dispatch a
second run ? or
            ``None`` on failure.

            ``video_id`` is the submit-time MD5 when the caller could resolve one cheaply
(#484 ?
            gs:// object metadata); "" ? unknown-yet (the drain fills it after hashing, as
before).
            Persisting it at CREATE is what lets a restarted control plane re-associate a
running
            remote job with its bucket outputs (CONTROL_PLANE_DURABLE_REBUILD §4.2)."""
            params = json.dumps({"force": True}) if force else None
            vals = (
                job_id,
                video_path,
                video_id,
                segmenter,
                json.dumps(player_pool) if player_pool is not None else None,
                max_frames,
                json.dumps(policy) if policy is not None else None,
                owner_id,
                locale,
                compute,
                params,
            )
            owner_clause = "owner_id IS NULL" if owner_id is None else "owner_id = ?"
            guard = () if owner_id is None else (owner_id,)
            cols = (
                "id, video_path, video_id, segmenter, player_pool, max_frames, status,
policy, "
                "owner_id, locale, compute, params"
            )
            try:
                with self._connect() as conn:
                    cur = conn.cursor()
                    if force:
                        cur.execute(
                            f"INSERT INTO jobs ({cols}) VALUES (?, ?, ?, ?, ?, ?, 'queued',
?, ?, ?, ?, ?)",
                            vals,
                        )
                        conn.commit()
                        return {"job_id": job_id, "status": "queued", "created": True}
                    # Conditional insert ? one atomic statement: only if no active process
job exists for
                    # this (owner, input). SQLite serializes writers, so a racing second
submit sees the row.
                    cur.execute(
                        f"INSERT INTO jobs ({cols}) "
                        "SELECT ?, ?, ?, ?, ?, ?, 'queued', ?, ?, ?, ?, ? WHERE NOT EXISTS
```

```
("
                        "SELECT 1 FROM jobs WHERE " + owner_clause + " AND video_path = ? "
                        "AND status IN ('queued', 'running', 'waiting') "
                        "AND (kind IS NULL OR kind != 'compile'))",
                        vals + guard + (video_path,),
                    )
                    conn.commit()
                    if cur.rowcount == 1:
                        return {"job_id": job_id, "status": "queued", "created": True}
                    # Lost the race (or a pre-existing active job): return the winner, no
second dispatch.
                    row = (
                        cur.execute(
                            "SELECT id, status FROM jobs WHERE " + owner_clause + " AND
video_path = ? "
                            "AND status IN ('queued', 'running', 'waiting') "
                            "AND (kind IS NULL OR kind != 'compile') ORDER BY created_at
DESC LIMIT 1",
                            guard + (video_path,),
                        ).fetchone()
                    )
                    if row:
                        return {"job_id": str(row["id"]), "status": str(row["status"]),
"created": False}
                    return None
            except Exception as e:
                logger.error(f"Failed to claim-or-create process job {job_id}: {e}")
                return None

        # --- Per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8)
----------------------------
        def get_pricebook_rates(
            self, version: str, gpu_type: Optional[str] = None
        ) -> Dict[str, float]:
            """Resolve a flat ``{resource: unit_price_usd}`` for a pricebook ``version``.
The per-second
            GPU rate is selected by ``gpu_type``; non-GPU resources (wall/egress) carry
``gpu_type=''``.
            An unknown version ? ``{}`` (so every cost computes to 0.0)."""
            gt = gpu_type or ""
            rates: Dict[str, float] = {}
            with self._connect() as conn:
                rows = (
                    conn.cursor()
                    .execute(
                        "SELECT resource, gpu_type, unit_price_usd FROM pricebook WHERE
version = ?",
                        (version,),
                    )
                    .fetchall()
                )
            for r in rows:
                res, row_gt, price = (
                    r["resource"],
                    (r["gpu_type"] or ""),
                    float(r["unit_price_usd"]),
                )
                if res == billing.GPU_SECONDS:
                    if row_gt == gt:  # the GPU rate for THIS gpu_type
                        rates[res] = price
                else:
                    rates[res] = price  # wall/egress: gpu_type-independent
            return rates

        def record_job_cost(
```

```
            self,
            job_id: str,
            *,
            usage: Dict[str, float],
            pricebook_version: str,
            gpu_type: Optional[str] = None,
            compute_backend: Optional[str] = None,
            owner_id: Optional[str] = None,
            margin: float = 0.0,
        ) -> Optional[Dict[str, Any]]:
            """Compute the job's cost from ``usage`` × the pricebook + persist every cost
column.
            Returns the stored cost dict, or ``None`` on write failure. The caller gates
this on
            ``billing.enabled`` (default-OFF) ? nothing records until billing is turned
on."""
            rates = self.get_pricebook_rates(pricebook_version, gpu_type)
            # SILENT-UNDER-BILL GUARD: a GPU-seconds AMOUNT with no RATE for this gpu_type
would bill at
            # $0 indistinguishably from a free run (a new SKU / typo / gpu_type=None).
Surface it loudly +
            # in the breakdown so the gap is auditable, not invisible. (A missing amount =
$0 IS safe.)
            gpu_amt = float(usage.get(billing.GPU_SECONDS, 0.0) or 0.0)
            unpriced_gpu = gpu_amt > 0 and billing.GPU_SECONDS not in rates
            if unpriced_gpu:
                logger.warning(
                    "[BILLING] no pricebook rate for gpu_type=%r in version=%r ? %.1f GPU-
seconds billed "
                    "at $0 (UNPRICED; recorded in cost_breakdown_json). Add a rate or fix
gpu_type.",
                    gpu_type,
                    pricebook_version,
                    gpu_amt,
                )
            cost_usd, breakdown = billing.compute_cost_usd(usage, rates)
            if unpriced_gpu:
                breakdown["unpriced_gpu_seconds"] = gpu_amt
            if margin:
                marked = billing.apply_margin(cost_usd, margin)
                breakdown["margin"] = round(
                    marked - cost_usd, 6
                )  # keep ?(breakdown) == cost_usd
                cost_usd = marked
            bytes_out = int(float(usage.get(billing.EGRESS_GB, 0.0) or 0.0) * 1e9)
            ok = self._execute_write(
                "UPDATE jobs SET owner_id=?, compute_backend=?, gpu_type=?,
gpu_active_seconds=?, "
                "wall_seconds=?, bytes_out=?, gemini_cost_usd=?, pricebook_version=?,
cost_usd=?, "
                "cost_breakdown_json=? WHERE id = ?",
                (
                    owner_id,
                    compute_backend,
                    gpu_type,
                    usage.get(billing.GPU_SECONDS),
                    usage.get(billing.WALL_SECONDS),
                    bytes_out,
                    usage.get(billing.GEMINI_USD),
                    pricebook_version,
                    cost_usd,
                    json.dumps(breakdown),
                    job_id,
                ),
                f"Failed to record cost for job {job_id}",
```

```
            )
            if not ok:
                return None
            return {
                "cost_usd": cost_usd,
                "cost_breakdown": breakdown,
                "pricebook_version": pricebook_version,
            }

        def get_job_cost(self, job_id: str) -> Optional[Dict[str, Any]]:
            """The job's cost row (``None`` if the job doesn't exist; ``cost_usd`` is
``None`` until
            recorded). ``cost_breakdown`` is the deserialized JSON breakdown."""
            with self._connect() as conn:
                row = (
                    conn.cursor()
                    .execute(
                        "SELECT id, video_id, owner_id, compute_backend, gpu_type,
gpu_active_seconds, "
                        "wall_seconds, bytes_out, gemini_cost_usd, pricebook_version,
cost_usd, "
                        "cost_breakdown_json FROM jobs WHERE id = ?",
                        (job_id,),
                    )
                    .fetchone()
                )
            if not row:
                return None
            d = dict(row)
            raw = d.pop("cost_breakdown_json", None)
            d["cost_breakdown"] = json.loads(raw) if raw else {}
            return d

        def get_video_cost(self, video_id: str) -> Dict[str, Any]:
            """Aggregate cost across a video's jobs: total ``cost_usd`` + the per-job rows.
A video is
            1:1 with an infer job, but a re-ingest can produce several ? sum them."""
            with self._connect() as conn:
                rows = (
                    conn.cursor()
                    .execute(
                        "SELECT id, cost_usd, pricebook_version, cost_breakdown_json FROM
jobs "
                        "WHERE video_id = ?",
                        (video_id,),
                    )
                    .fetchall()
                )
            jobs: List[Dict[str, Any]] = []
            total = 0.0
            for r in rows:
                cost = r["cost_usd"]
                if cost is not None:
                    total += float(cost)
                jobs.append(
                    {
                        "job_id": r["id"],
                        "cost_usd": cost,
                        "pricebook_version": r["pricebook_version"],
                        "cost_breakdown": json.loads(r["cost_breakdown_json"])
                        if r["cost_breakdown_json"]
                        else {},
                    }
                )
            return {"video_id": video_id, "total_cost_usd": round(total, 6), "jobs": jobs}
```

```
462
463         # --- Self-scheduling execution policy (EXECUTION_POLICY.md P2)
------------------------
464         def set_job_policy(self, job_id: str, policy: Optional[Dict[str, Any]]) -> bool:
465             """Attach/replace a job's JSON ExecutionPolicy."""
466             return self._execute_write(
467                 "UPDATE jobs SET policy=? WHERE id = ?",
468                 (json.dumps(policy) if policy is not None else None, job_id),
469                 f"Failed to set policy for job {job_id}",
470             )
471
472         def set_job_waiting(
473             self, job_id: str, delay_sec: float, progress: Optional[str] = None
474         ) -> bool:
475             """Self-schedule: park a job in 'waiting' until ``delay_sec`` from now,
releasing the
476             worker. `claim_due_job` re-picks it when due ? cooperative, no master.
``next_poll_at`` is
477             computed in SQLite's own UTC format so it compares directly to
CURRENT_TIMESTAMP."""
478             return self._execute_write(
479                 "UPDATE jobs SET status='waiting', "
480                 "next_poll_at=datetime('now', ?), "
481                 "progress=COALESCE(?, progress) WHERE id = ?",
482                 (f"+{max(0, int(delay_sec))} seconds", progress, job_id),
483                 f"Failed to set job {job_id} waiting",
484             )
485
486         def seconds_until_next_due(self) -> Optional[float]:
487             """Seconds until the soonest parked ('waiting') job becomes due, so the worker
can
488             schedule a wake-up (EXECUTION_POLICY.md P3 self-scheduling). Returns 0.0 if one
is
489             already due, or None if nothing is waiting. A NULL ``next_poll_at`` ? due now
(0.0)."""
490             try:
491                 with self._connect() as conn:
492                     row = (
493                         conn.cursor()
494                         .execute(
495                             "SELECT MIN(CASE WHEN next_poll_at IS NULL THEN 0 ELSE "
496                             "  (julianday(next_poll_at) - julianday('now')) * 86400.0 END)
AS secs "
497                             "FROM jobs WHERE status='waiting'"
498                         )
499                         .fetchone()
500                     )
501                 if row is None or row["secs"] is None:
502                     return None
503                 return max(0.0, float(row["secs"]))
504             except Exception as e:
505                 logger.error(f"Failed to compute next-due delay: {e}")
506                 return None
507
508         def claim_due_job(self) -> Optional[Dict[str, Any]]:
509             """Atomically claim ONE runnable job ? 'queued', or 'waiting' whose
``next_poll_at`` is due ?
510             flipping it to 'running'. Compare-and-set on (id, status) so concurrent workers
never
511             double-claim (the DB row-claim is the only coordination needed; no master).
Earliest-due
512             first. Returns the claimed job dict, or None ONLY when a fresh SELECT finds
nothing
513             runnable ? never for a lost race.
514
```

```
            A lost CAS RETRIES instead of returning: under the parallel drain (#466) several
workers
            can SELECT the same head-of-queue row, and if the losers reported None the
caller
            (``_drain_due_jobs``) would read it as "queue empty" and stop with runnable jobs
still
            queued ? stranding dispatch capacity until some later submit/wake. The retry
loop is
            deliberately UNBOUNDED: every lost race means another worker successfully
claimed that
            row, so the runnable set shrank and global progress is guaranteed (lock-free
progress);
            a retry cap would just reintroduce the false-empty. One connection serves all
attempts ?
            the SELECT is an autocommit read and each CAS commits before the next iteration,
so no
            write lock is ever held across a retry (busy_timeout covers momentary
contention)."""
            try:
                with self._connect() as conn:
                    cur = conn.cursor()
                    while True:
                        row = cur.execute(
                            "SELECT id, status FROM jobs WHERE status='queued' "
                            "OR (status='waiting' AND (next_poll_at IS NULL OR next_poll_at
<= CURRENT_TIMESTAMP)) "
                            "ORDER BY COALESCE(next_poll_at, created_at) ASC LIMIT 1"
                        ).fetchone()
                        if not row:
                            return None  # nothing runnable ? a TRUE empty, not a lost race
                        cur.execute(
                            "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP
"
                            "WHERE id=? AND status=?",
                            (row["id"], row["status"]),
                        )
                        conn.commit()
                        if cur.rowcount == 1:
                            break  # claimed it
                        # Lost the race: the winner flipped this row; re-SELECT for the next
one.
                return self.get_job(row["id"])
            except Exception as e:
                logger.error(f"Failed to claim a due job: {e}")
                return None

        def fail_stale_jobs(self, reason: str = "interrupted by server restart") -> int:
            """Crash recovery: any job still queued/running from a previous process is dead
            (the executor is in-process). Mark failed so clients see a terminal state, not a
            hang. Returns the number of jobs swept.

            NOTE: ``recover_stale_jobs`` is the preferred startup call ? it RE-QUEUES stale
            ``compile`` jobs (idempotent CPU stitch, safe to retry) instead of failing them.
This
            blanket-fail method stays for callers/tests that want the old behaviour."""
            try:
                with self._connect() as conn:
                    cur = conn.cursor()
                    cur.execute(
                        "UPDATE jobs SET status='failed', error=?,
finished_at=CURRENT_TIMESTAMP "
                        "WHERE status IN ('queued', 'running')",
                        (reason,),
                    )
                    conn.commit()
```

```
                    return cur.rowcount
            except Exception as e:
                logger.error(f"Failed to sweep stale jobs: {e}")
                return 0

        def recover_stale_jobs(
            self, reason: str = "interrupted by server restart"
        ) -> Tuple[int, int]:
            """Crash recovery, kind-aware (#329 serving reliability). Called once at
startup.

            A ``compile`` job is a **CPU-only, idempotent** ffmpeg re-stitch (re-resolving
the same
            segments yields the same reel), so a compile left queued/running by a killed
instance is
            **re-queued** ? the startup drain re-runs it and the client keeps polling until
it gets
            the reel, instead of seeing a spurious "failed". A ``process`` job is an
expensive,
            non-idempotent GPU detection pass; auto-retrying could double-spend, so it is
**failed**
            (terminal, as before) and the client re-submits deliberately ? EXCEPT a RUNNING
process
            job whose ``video_id`` is already known (CP-rebuild P3): its remote worker may
still be
            computing, and ``outputs/<md5>/report.json`` is a durable done-signal the
restarted
            control plane can re-associate by md5. Those rows are LEFT RUNNING for
            ``job_worker.resume_interrupted_remote_jobs`` to complete or fail honestly ?
never
            re-dispatched (no double GPU spend). This is exactly the 2026-07-05 loss shape:
the
            worker finished 45 min after the restart and its 29 rallies had no listener.

            ``force=true`` rows are NOT resumable (#494 review): force deliberately bypasses
the
            completed-work cache, and without a persisted per-dispatch identity the bucket
report
            cannot be proven to belong to the forced run rather than an older completed one
? so a
            resumed forced job could be marked done from exactly the stale result the user
asked
            to bypass. Forced rows keep today's honest terminal failure on restart.

            ``compute='local'`` rows are NOT resumable either (#494 review round 2): an
explicit
            local override forces the in-process segmenter ? no remote worker exists, and a
bucket
            report under the same md5 would be an OLDER run's output, not this job's.
``compute``
            NULL (server default) stays in the carve-out at the SQL level; the resume pass ?
which
            has the config ? fails it unless the server default target is actually
cloud_run.

            Returns ``(requeued_compile, failed_process)``."""
            try:
                with self._connect() as conn:
                    cur = conn.cursor()
                    # Re-queue interrupted COMPILE jobs (idempotent ? safe to retry
automatically).
                    cur.execute(
                        "UPDATE jobs SET status='queued', started_at=NULL,
next_poll_at=NULL, "
                        "error=NULL WHERE status IN ('queued', 'running') AND
```

```
kind='compile'"
609                     )
610                     requeued = cur.rowcount
611                     # Fail interrupted PROCESS jobs (GPU, not safe to auto-retry) ? except
the
612                     # md5-keyed RUNNING ones, which the resume pass owns (see docstring).
613                     cur.execute(
614                         "UPDATE jobs SET status='failed', error=?,
finished_at=CURRENT_TIMESTAMP "
615                         "WHERE status IN ('queued', 'running') AND (kind IS NULL OR kind !=
'compile') "
616                         "AND NOT (status = 'running' AND video_id IS NOT NULL AND video_id
!= '' "
617                         "AND COALESCE(json_extract(params, '$.force'), 0) != 1 "
618                         "AND (compute IS NULL OR LOWER(TRIM(compute)) != 'local'))",
619                         (reason,),
620                     )
621                     failed = cur.rowcount
622                     conn.commit()
623                     return requeued, failed
624             except Exception as e:
625                 logger.error(f"Failed to recover stale jobs: {e}")
626                 return 0, 0
627
628         def interrupted_remote_process_jobs(self) -> List[Dict[str, Any]]:
629             """The RUNNING md5-keyed process jobs a restart left behind ? the rows
630             ``recover_stale_jobs`` deliberately did NOT fail (CP-rebuild P3). Called once at
631             startup, BEFORE the drain runs, so every 'running' process row here is orphaned
by
632             definition (the executor is in-process; no live worker thread can exist yet)."""
633             try:
634                 with self._connect() as conn:
635                     rows = conn.execute(
636                         "SELECT id, video_id, video_path, owner_id, compute FROM jobs "
637                         "WHERE status = 'running' AND (kind IS NULL OR kind != 'compile') "
638                         "AND video_id IS NOT NULL AND video_id != '' "
639                         "AND COALESCE(json_extract(params, '$.force'), 0) != 1 "
640                         "AND (compute IS NULL OR LOWER(TRIM(compute)) != 'local')"
641                     ).fetchall()
642                     return [dict(r) for r in rows]
643             except Exception as e:
644                 logger.error(f"Failed to list interrupted remote process jobs: {e}")
645                 return []
646
647
```

36. assistant (2026-07-08T05:11:57.940Z)

I now understand the architecture well. Let me read the existing tests to mirror their patterns before implementing.

37. user (2026-07-08T05:11:58.949Z)

```
1       """API cloud-serving dispatch (COMPUTE_DECOUPLED_SERVING M6 ? the API path).
2
3       Proves the /api/process job path can offload DETECT to a Cloud Run GPU Job and ingest
the rally
4       intervals back into the API DB, so /api/query + /api/compile (local stitch) are
unchanged ? all
5       GPU-free + cloud-free via a FAKE CloudRunJobClient that writes report.json +
intervals.csv + the
6       done-marker into a local "bucket". Cardinal invariant: DEFAULT-OFF ? only
compute_target=cloud_run
7       dispatches; every other target runs the in-process segmenter byte-identically.
8       """
```

```

    import csv
    import json
    import os

    import pytest

    pytest.importorskip("httpx")
    from fastapi.testclient import TestClient

    import backend.compute as compute_pkg
    import backend.main as m
    from backend.compute.cloud_run import CloudRunJobClient
    from backend.config.models import AppConfig
    from backend.infrastructure.database import Database
    from backend.infrastructure.execution_policy import ExecutionPolicy
    from backend.pipeline.validators.base import ValidationResult

    _FAST = ExecutionPolicy(poll_every_n_sec=0.0, op_timeout_sec=5.0, max_wait_sec=5.0)


    def _pass():
        return [ValidationResult(validator_name="x", passed=True, message="ok", details={})]


    class _InlineExecutor:
        def submit(self, fn, *a, **k):
            fn(*a, **k)


    class _FakeCloudClient(CloudRunJobClient):
        """Simulates the Cloud Run worker: on dispatch, write report.json + intervals.csv +
the done-marker
        into the local bucket exactly where the real worker would, so the bucket-poll +
ingest resolve."""

        def __init__(self, *, video_id="vidCloud", returncode=0, rows=None):
            self.calls: list[list[str]] = []
            self.video_id, self.returncode = video_id, returncode
            self.rows = (
                rows
                if rows is not None
                else [(2.74, 6.91, 5, "winner"), (8.74, 11.38, 3, "unforced_error")]
            )

        def run_job(self, job_name, argv, *, region, project=None):
            self.calls.append(list(argv))
            out_uri = argv[argv.index("--out-uri") + 1].rstrip("/")
            done_uri = argv[argv.index("--done-uri") + 1]
            if self.returncode == 0:
                outputs = os.path.join(out_uri, "outputs", self.video_id)
                os.makedirs(outputs, exist_ok=True)
                with open(os.path.join(outputs, "report.json"), "w", encoding="utf-8") as f:
                    json.dump(
                        {
                            "video_id": self.video_id,
                            "segment_count": len(self.rows),
                            "status": "success",
                        },
                        f,
                    )
                with open(
                    os.path.join(outputs, "intervals.csv"),
                    "w",
                    newline="",
```

```
                    encoding="utf-8",
                ) as f:
                    w = csv.writer(f)
                    w.writerow(["start", "end", "shot_count", "ending_reason"])
                    for s, e, sc, er in self.rows:
                        w.writerow([f"{s:.3f}", f"{e:.3f}", sc, er])
            os.makedirs(os.path.dirname(done_uri), exist_ok=True)
            with open(done_uri, "w", encoding="utf-8") as f:
                json.dump(
                    {
                        "video_id": self.video_id,
                        "returncode": self.returncode,
                        "segment_count": len(self.rows),
                        "status": "success",
                    },
                    f,
                )
            return "exec-1"

        def cancel_execution(self, execution):
            raise AssertionError(
                "cancel must never fire on the happy path"
            )  # pragma: no cover


    @pytest.fixture
    def cloud_env(tmp_path, monkeypatch):
        """API wired for cloud_run with a LOCAL bucket (the local storage backend makes the
bucket read an
        identity file read ? no GCS). A gs:// input is 'fetched' to a local fake file for
hash+validate."""
        db = Database(str(tmp_path / "t.db"))  # noqa: F841
        monkeypatch.setattr(m, "db_instance", db)
        monkeypatch.setattr(m, "db_instance", db)
        bucket = tmp_path / "bucket"
        bucket.mkdir()
        cfg = AppConfig.model_validate(
            {  # noqa: F841
                "deployment": {"compute_target": "cloud_run"},
                "storage": {"backend": "local", "output_root": str(bucket)},
            }
        )
        monkeypatch.setattr(m, "global_config", cfg)
        monkeypatch.setattr(m, "global_config", cfg)
        monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
        # A real gs:// input would be fetched to hash+validate; return a local fake so that
path is offline.
        fake_local = tmp_path / "fetched.mp4"
        fake_local.write_bytes(b"FAKEVIDEOBYTES")
        monkeypatch.setattr(
            m.storage, "acquire_local_input", lambda path, scfg: str(fake_local)
        )
        monkeypatch.setattr(
            m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {})
        )
        return db, bucket, monkeypatch


    def _inject_fake(monkeypatch, fake):
        """Make backend.compute.get_compute_backend (re-imported inside
_maybe_dispatch_remote) build a
        real CloudRunCompute around the FAKE client + the fast no-wait policy + a fixed
token. ``**kw``
        forwards the per-request ``target_override`` (item 3) so the cloud-picker path
resolves too."""
```

```
        real = compute_pkg.get_compute_backend
        monkeypatch.setattr(
            compute_pkg,
            "get_compute_backend",
            lambda config, **kw: real(
                config, run_client=fake, policy=_FAST, token="tok", **kw
            ),
        )


    def _meta(row):
        md = row.get("metadata")
        return json.loads(md) if isinstance(md, str) else (md or {})


    def test_cloud_dispatch_ingests_segments(cloud_env):
        db, _bucket, monkeypatch = cloud_env
        fake = _FakeCloudClient()
        _inject_fake(monkeypatch, fake)

        out = m._execute_processing(
            m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
        )

        assert out["status"] == "success"
        assert len(out["play_segments"]) == 2
        rows = db.query_play_segments(out["video_id"], {})
        assert len(rows) == 2
        metas = [_meta(r) for r in rows]
        assert all(md["source"] == "cloud_run" for md in metas)
        assert {md.get("shot_count") for md in metas} == {5, 3}
        assert {md.get("ending_reason") for md in metas} == {"winner", "unforced_error"}


    def test_cloud_dispatch_builds_correct_spec(cloud_env):
        _db, bucket, monkeypatch = cloud_env
        fake = _FakeCloudClient()
        _inject_fake(monkeypatch, fake)

        m._execute_processing(
            m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
        )

        assert len(fake.calls) == 1  # exactly one Cloud Run Job execution
        argv = fake.calls[0]
        assert argv[argv.index("--video-uri") + 1] == "gs://b/clip.mp4"
        assert argv[argv.index("--out-uri") + 1] == str(bucket)
        assert argv[argv.index("--segmenter") + 1] == "wasb_hybrid"
        joined = " ".join(argv)
        assert (
            "deployment.compute_target=local_gpu" in joined
        )  # container runs INLINE (never re-dispatch)
        assert "indexing.detector_impl=native" in joined
        assert (
            "indexing.skip_ai_handoff=false" in joined
        )  # the engine's AI-handoff stance is forwarded


    def test_cloud_dispatch_forwards_skip_ai_handoff_true(cloud_env):
        """When the engine runs no-AI (skip_ai_handoff=True), the cloud job must too ? else
it tries the
        Gemini handoff (which the cloud job has no key for) and yields 0 rallies. The flag
is forwarded."""
        _db, _bucket, monkeypatch = cloud_env
        m.global_config.indexing.skip_ai_handoff = True
```

```
        fake = _FakeCloudClient()
        _inject_fake(monkeypatch, fake)
        m._execute_processing(
            m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
        )
        assert "indexing.skip_ai_handoff=true" in " ".join(fake.calls[0])


    def test_cloud_dispatch_forwards_wasb_serving_knobs(cloud_env):
        """The worker Job has no DEPLOYMENT_PROFILE, so it never applies the cloud-serving
preset ?
        the control-plane MUST forward its resolved WASB serving knobs (decode_backend +
GPU-feed
        tuning) or the worker silently reverts to the model defaults (cpu decode, batch 8),
making the
        preset flip (#506/#507) a no-op on the real dispatch path. Regression guard for that
wiring."""
        _db, _bucket, monkeypatch = cloud_env
        m.global_config.indexing.wasb.decode_backend = "nvdec_resident"
        m.global_config.indexing.wasb.batch_size = 64
        m.global_config.indexing.wasb.prefetch_batches = 4
        fake = _FakeCloudClient()
        _inject_fake(monkeypatch, fake)
        m._execute_processing(
            m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
        )
        joined = " ".join(fake.calls[0])
        assert "indexing.wasb.decode_backend=nvdec_resident" in joined
        assert "indexing.wasb.batch_size=64" in joined
        assert "indexing.wasb.prefetch_batches=4" in joined


    def test_cloud_dispatch_omits_unset_wasb_tuning(cloud_env):
        """decode_backend defaults to cpu (always forwarded ? harmless, keeps worker ==
control-plane),
        but batch_size/prefetch_batches default to None and must NOT be forwarded (None =
the worker's
        parity default; forwarding an empty value would reach argparse as a bad token)."""
        _db, _bucket, monkeypatch = cloud_env
        # defaults: decode_backend="cpu", batch_size=None, prefetch_batches=None
        fake = _FakeCloudClient()
        _inject_fake(monkeypatch, fake)
        m._execute_processing(
            m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
        )
        joined = " ".join(fake.calls[0])
        assert "indexing.wasb.decode_backend=cpu" in joined
        assert "indexing.wasb.batch_size" not in joined
        assert "indexing.wasb.prefetch_batches" not in joined


    def test_cloud_dispatch_skips_worker_revalidation(cloud_env):
        """The control-plane is the AUTHORITATIVE validation gate: it validated + only
dispatches on PASS
        (registry.run_all_with_context is stubbed to pass in cloud_env). Re-validating on
the worker is
        redundant, decodes the video a second time, and ? because the control-plane's
validation config
        isn't forwarded ? can REJECT a clip the gate already passed (observed 2026-07-07:
passed at
        validation.min_blur_threshold=8.0, then worker-rejected at its default 20.0). So the
dispatch MUST
        carry --skip-validation, telling the worker to trust the gate. Regression guard for
that wiring."""
        _db, _bucket, monkeypatch = cloud_env
```

```
        fake = _FakeCloudClient()
        _inject_fake(monkeypatch, fake)
        m._execute_processing(
            m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
        )
        assert "--skip-validation" in fake.calls[0]


    def test_cloud_failure_marks_failed_no_rows(cloud_env):
        db, _bucket, monkeypatch = cloud_env
        # pre-seed a prior good row ? must be preserved on a cloud failure (destroy-AFTER-
confirm).
        db.add_video("vidLocalPrior", "gs://b/clip.mp4", "processed", [])
        fake = _FakeCloudClient(returncode=1)
        _inject_fake(monkeypatch, fake)

        out = m._execute_processing(
            m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
        )
        assert out["status"] == "failed"
        assert "returncode 1" in out["message"]
        assert out["play_segments"] == []
        assert db.query_play_segments(out["video_id"], {}) == []
        # It DID dispatch + run remotely (then failed rc=1), so the path is honestly
cloud_run + dispatched.
        assert (
            out["compute"]["path"] == "cloud_run" and out["compute"]["dispatched"] is True
        )


    def test_cloud_requires_a_bucket_input(cloud_env, tmp_path):
        _db, _bucket, monkeypatch = cloud_env
        fake = _FakeCloudClient()
        _inject_fake(monkeypatch, fake)
        local_path = str(tmp_path / "local_only.mp4")

        out = m._execute_processing(
            m.ProcessRequest(video_path=local_path, segmenter="wasb_hybrid")
        )
        assert out["status"] == "failed"
        assert "cloud input" in out["message"]
        assert fake.calls == []  # never dispatched a local-only path


    def test_cloud_stages_local_upload_to_bucket(cloud_env, tmp_path, monkeypatch):
        """A browser upload lands on THIS box's local disk; under cloud-serving with a CLOUD
input_root it
        is STAGED to the bucket and the L4 job dispatches from the staged gs:// URI (not
hard-failed) ? #461.
        The DB video record is repointed at the staged source so a later /api/compile can
re-fetch it."""
        db, _bucket, _mp = cloud_env
        m.global_config.storage.input_backend = "gcs"
        m.global_config.storage.input_root = "gs://in-bucket/videos"
        puts: list = []

        def _fake_put(src, uri, **kw):
            puts.append((str(src), uri, kw.get("config")))
            return uri

        monkeypatch.setattr(m.storage, "put", _fake_put)
        fake = _FakeCloudClient()
        _inject_fake(monkeypatch, fake)
        local_path = str(tmp_path / "upload.mp4")

```

```
        out = m._execute_processing(
            m.ProcessRequest(video_path=local_path, segmenter="wasb_hybrid")
        )

        assert out["status"] == "success"
        # 1) the local upload was staged UP to the input bucket, namespaced by video_id
        assert len(puts) == 1
        src, dest, put_cfg = puts[0]
        assert src == local_path
        assert dest.startswith("gs://in-bucket/videos/") and dest.endswith("/upload.mp4")
        # the active storage settings are threaded into put() (so a non-default S3 endpoint
/ GCS project /
        # etc. survives) ? not a fresh default client. The model_dump carries
input_root/input_backend.
        assert put_cfg is not None and put_cfg.get("input_root") == "gs://in-bucket/videos"
        assert put_cfg.get("input_backend") == "gcs"
        # 2) the L4 job dispatched from the STAGED gs:// URI, not the unreadable local path
        assert len(fake.calls) == 1
        argv = fake.calls[0]
        assert argv[argv.index("--video-uri") + 1] == dest
        # 3) the DB video record now points at the staged source (so compile re-fetches it)
        row = db.get_video(out["video_id"])
        assert row is not None and row["filepath"] == dest


    def test_cloud_stages_when_input_root_is_cloud_and_input_backend_default(
        cloud_env, tmp_path, monkeypatch
    ):
        """input_root='gs://?' with input_backend left at its DEFAULT ('local') is a valid
cloud config ?
        the storage layer classifies an explicit root by its scheme (resolve_input_uri joins
under
        input_root before ever consulting input_backend), so the staging gate must too (PR
#467 review).
        Regression: the gate previously required input_backend != 'local' and hard-failed
this shape."""
        _db, _bucket, _mp = cloud_env
        m.global_config.storage.input_root = "gs://in-bucket/videos"
        # input_backend deliberately NOT set ? stays at the model default 'local'
        assert m.global_config.storage.input_backend == "local"
        puts: list = []

        def _fake_put(src, uri, **kw):
            puts.append((str(src), uri))
            return uri

        monkeypatch.setattr(m.storage, "put", _fake_put)
        fake = _FakeCloudClient()
        _inject_fake(monkeypatch, fake)
        local_path = str(tmp_path / "upload.mp4")

        out = m._execute_processing(
            m.ProcessRequest(video_path=local_path, segmenter="wasb_hybrid")
        )

        assert out["status"] == "success"  # NOT the 'needs a cloud input' pre-dispatch
failure
        assert len(puts) == 1
        src, dest = puts[0]
        assert src == local_path
        assert dest.startswith("gs://in-bucket/videos/") and dest.endswith("/upload.mp4")
        # dispatch continued from the STAGED gs:// URI
        assert len(fake.calls) == 1
        argv = fake.calls[0]
        assert argv[argv.index("--video-uri") + 1] == dest
```

```
365
366
367     def test_cloud_local_upload_without_input_root_still_fails(cloud_env, tmp_path,
monkeypatch):
368         """No cloud input_root configured ? we genuinely can't stage a local upload ? hard-
fail (no
369         dispatch), rather than dispatching a path the Cloud Run job can't read."""
370         _db, _bucket, _mp = cloud_env  # fixture leaves input_root unset
(input_backend=local)
371         fake = _FakeCloudClient()
372         _inject_fake(monkeypatch, fake)
373         called = []
374         monkeypatch.setattr(m.storage, "put", lambda *a, **k: called.append(a))
375
376         out = m._execute_processing(
377             m.ProcessRequest(video_path=str(tmp_path / "up.mp4"), segmenter="wasb_hybrid")
378         )
379         assert out["status"] == "failed"
380         assert "cloud input" in out["message"]
381         assert fake.calls == [] and called == []  # neither staged nor dispatched
382
383
384     def test_default_off_runs_in_process(tmp_path, monkeypatch):
385         """compute_target='' ? get_compute_backend returns None ? the in-process segmenter
runs
386         byte-identically (the gate is inert). No Cloud Run dispatch."""
387         db = Database(str(tmp_path / "t.db"))  # noqa: F841
388         monkeypatch.setattr(m, "db_instance", db)
389         monkeypatch.setattr(m, "db_instance", db)
390         cfg = AppConfig()  # compute_target defaults to ""  # noqa: F841
391         monkeypatch.setattr(m, "global_config", cfg)
392         monkeypatch.setattr(m, "global_config", cfg)
393         monkeypatch.setattr(m, "global_config", cfg)
394         monkeypatch.setattr(
395             m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {})
396         )
397         video = tmp_path / "v.mp4"
398         video.write_bytes(b"hashable-bytes")
399
400         class _InProcSeg:
401             def __init__(self, db_, cfg_):
402                 self.db = db_
403
404             def process_video(self, vid, path, *a, **k):
405                 sid = self.db.add_play_segment(vid, 0.0, 5.0)
406                 return [
407                     {
408                         "id": sid,
409                         "start_time": 0.0,
410                         "end_time": 5.0,
411                         "duration": 5.0,
412                         "metadata": {},
413                     }
414                 ]
415
416         monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
417         out = m._execute_processing(
418             m.ProcessRequest(video_path=str(video), segmenter="motion")
419         )
420         assert out["status"] == "success"
421         assert len(out["play_segments"]) == 1
422         assert "cloud_run" not in out["message"]
423
424
425     def test_cloud_ingest_failure_marks_failed_no_leak(cloud_env):
```

```
        """A rc=0 marker but a MISSING intervals.csv (a partial push) must fail cleanly +
leak no rows
        (destroy-AFTER-confirm), not 500 or hang."""
        db, _bucket, monkeypatch = cloud_env

        class _NoCsvClient(_FakeCloudClient):
            def run_job(self, job_name, argv, *, region, project=None):
                self.calls.append(list(argv))
                out_uri = argv[argv.index("--out-uri") + 1].rstrip("/")
                done_uri = argv[argv.index("--done-uri") + 1]
                outputs = os.path.join(out_uri, "outputs", self.video_id)
                os.makedirs(outputs, exist_ok=True)
                with open(os.path.join(outputs, "report.json"), "w", encoding="utf-8") as f:
                    json.dump({"video_id": self.video_id, "status": "success"}, f)
                # deliberately DO NOT write intervals.csv
                os.makedirs(os.path.dirname(done_uri), exist_ok=True)
                with open(done_uri, "w", encoding="utf-8") as f:
                    json.dump({"video_id": self.video_id, "returncode": 0}, f)
                return "exec"

        _inject_fake(monkeypatch, _NoCsvClient())
        out = m._execute_processing(
            m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
        )
        assert out["status"] == "failed"
        assert "could not read the result" in out["message"]
        assert db.query_play_segments(out["video_id"], {}) == []


    def test_cloud_video_id_divergence_is_tolerated_with_warning(cloud_env, caplog):
        """If the worker re-hashes different bytes (result.video_id != local id), ingest
still proceeds
        under the LOCAL id but logs a WARNING so the orphaned-report condition is
diagnosable."""
        db, _bucket, monkeypatch = cloud_env
        _inject_fake(monkeypatch, _FakeCloudClient(video_id="DIVERGED"))
        with caplog.at_level("WARNING", logger="backend.main"):
            out = m._execute_processing(
                m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
            )
        assert out["status"] == "success"
        assert len(db.query_play_segments(out["video_id"], {})) == 2
        assert any(
            "DIVERGED" in r.getMessage() and "!= local" in r.getMessage()
            for r in caplog.records
        )


    class _InProcSeg:
        """Minimal in-process segmenter (writes one play row) ? proves the local path
actually ran."""

        def __init__(self, db_, cfg_):
            self.db = db_

        def process_video(self, vid, path, *a, **k):
            sid = self.db.add_play_segment(vid, 0.0, 5.0)
            return [
                {
                    "id": sid,
                    "start_time": 0.0,
                    "end_time": 5.0,
                    "duration": 5.0,
                    "metadata": {},
                }
```

```
            ]


    # --- item 3: SPA compute-picker (per-request compute override + cloud_available
capability) ---


    def test_per_request_local_forces_in_process(cloud_env):
        """Server configured for cloud (compute_target=cloud_run), but a per-request
compute='local'
        forces the in-process segmenter and NEVER dispatches to Cloud Run."""
        db, _bucket, monkeypatch = cloud_env
        fake = _FakeCloudClient()
        _inject_fake(monkeypatch, fake)
        monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)

        out = m._execute_processing(
            m.global_config,
            m.db_instance,
            m.ProcessRequest(
                video_path="gs://b/clip.mp4", segmenter="motion", compute="local"
            ),
        )
        assert out["status"] == "success"
        assert len(out["play_segments"]) == 1
        assert "cloud_run" not in out["message"]
        assert (
            fake.calls == []
        )  # explicit "run on my machine" ? no dispatch despite a cloud server


    def test_per_request_cloud_forces_dispatch_on_local_server(tmp_path, monkeypatch):
        """Server default is LOCAL (compute_target=''), but a per-request compute='cloud'
forces a Cloud
        Run dispatch when the control plane is wired (CLOUD_RUN_JOB + GOOGLE_CLOUD_PROJECT +
a bucket)."""
        db = Database(str(tmp_path / "t.db"))  # noqa: F841
        monkeypatch.setattr(m, "db_instance", db)
        bucket = tmp_path / "bucket"
        bucket.mkdir()
        cfg = AppConfig.model_validate(
            {  # noqa: F841
                "deployment": {"compute_target": ""},  # server default = in-process
                "storage": {"backend": "local", "output_root": str(bucket)},
            }
        )
        monkeypatch.setattr(m, "global_config", cfg)
        monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
        fake_local = tmp_path / "fetched.mp4"
        fake_local.write_bytes(b"FAKEVIDEOBYTES")
        monkeypatch.setattr(
            m.storage, "acquire_local_input", lambda path, scfg: str(fake_local)
        )
        monkeypatch.setattr(
            m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {})
        )
        monkeypatch.setenv("CLOUD_RUN_JOB", "rally-worker")
        monkeypatch.setenv("GOOGLE_CLOUD_PROJECT", "proj")
        fake = _FakeCloudClient()
        _inject_fake(monkeypatch, fake)

        out = m._execute_processing(
            m.global_config,
            m.db_instance,
            m.ProcessRequest(
```

```
                video_path="gs://b/clip.mp4", segmenter="wasb_hybrid", compute="cloud"
            ),
        )
        assert out["status"] == "success"
        assert len(out["play_segments"]) == 2
        assert (
            len(fake.calls) == 1
        )  # per-request override dispatched despite the LOCAL server default


    def test_per_request_cloud_unconfigured_fails_clearly(tmp_path, monkeypatch):
        """compute='cloud' on a server NOT wired for it fails clearly (no silent local
fallback, no
        dispatch) so the picker can surface 'cloud isn't available here'."""
        db = Database(str(tmp_path / "t.db"))  # noqa: F841
        monkeypatch.setattr(m, "db_instance", db)
        cfg = AppConfig.model_validate(
            {"deployment": {"compute_target": ""}}
        )  # no bucket, no env  # noqa: F841
        monkeypatch.setattr(m, "global_config", cfg)
        monkeypatch.setattr(
            m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {})
        )
        monkeypatch.delenv("CLOUD_RUN_JOB", raising=False)
        monkeypatch.delenv("GOOGLE_CLOUD_PROJECT", raising=False)
        video = tmp_path / "v.mp4"
        video.write_bytes(b"hashable-bytes")

        out = m._execute_processing(
            m.global_config,
            m.db_instance,
            m.ProcessRequest(
                video_path=str(video), segmenter="wasb_hybrid", compute="cloud"
            ),
        )
        assert out["status"] == "failed"
        assert "isn't configured" in out["message"]


    def test_unknown_compute_value_falls_back_to_server_default(cloud_env, caplog):
        """An unrecognised compute value is ignored (logged) and the server default applies
? here a
        cloud_run server still dispatches to the cloud."""
        db, _bucket, monkeypatch = cloud_env
        fake = _FakeCloudClient()
        _inject_fake(monkeypatch, fake)
        with caplog.at_level("WARNING", logger="backend.main"):
            out = m._execute_processing(
                m.global_config,
                m.db_instance,
                m.ProcessRequest(
                    video_path="gs://b/clip.mp4", segmenter="wasb_hybrid", compute="banana"
                ),
            )
        assert out["status"] == "success"
        assert len(fake.calls) == 1  # fell back to the configured cloud_run default
        assert any("unknown compute" in r.getMessage() for r in caplog.records)


    def test_capabilities_reports_cloud_available(cloud_env):
        """A cloud_run-configured server advertises deployment.cloud_available=True (the SPA
shows the
        cloud toggle only then)."""
        _db, _bucket, _monkeypatch = cloud_env
        app = m.create_app(m.global_config, m.db_instance)
```

```
        client = TestClient(app)
        resp = client.get("/api/capabilities")
        caps = resp.json()
        assert caps["deployment"]["cloud_available"] is True
        assert caps["deployment"]["compute_target"] == "cloud_run"


    def test_capabilities_cloud_unavailable_by_default(monkeypatch, tmp_path):
        """A default (local) server with no Cloud Run env advertises
cloud_available=False."""
        cfg = AppConfig()
        db = Database(str(tmp_path / "t.db"))
        app = m.create_app(cfg, db)
        monkeypatch.delenv("CLOUD_RUN_JOB", raising=False)
        monkeypatch.delenv("GOOGLE_CLOUD_PROJECT", raising=False)
        client = TestClient(app)
        resp = client.get("/api/capabilities")
        caps = resp.json()
        assert caps["deployment"]["cloud_available"] is False


    def test_full_api_process_to_query_via_cloud(cloud_env):
        """End-to-end through the public API: POST /api/process (gs:// input) ? the inline
worker dispatches
        to (fake) Cloud Run ? job done ? POST /api/query returns the ingested segments."""
        db, _bucket, monkeypatch = cloud_env
        fake = _FakeCloudClient(video_id="vidCloud2")
        _inject_fake(monkeypatch, fake)
        app = m.create_app(m.global_config, db)
        client = TestClient(app)

        r = client.post(
            "/api/process",
            json={"video_path": "gs://b/clip.mp4", "segmenter": "wasb_hybrid"},
        )
        assert r.status_code == 202
        job_id = r.json()["job_id"]

        job = client.get(f"/api/jobs/{job_id}").json()
        assert job["status"] == "done"
        video_id = job["video_id"]
        assert video_id

        q = client.post("/api/query", json={"video_id": video_id})
        assert q.status_code == 200
        segs = q.json()["play_segments"]
        assert len(segs) == 2
        assert all(_meta(s)["source"] == "cloud_run" for s in segs)


    def test_local_override_survives_api_boundary(cloud_env):
        """REGRESSION: the picker choice must survive the async submit?worker?reconstruct
boundary. On a
        cloud_run server, POST /api/process with compute='local' must persist (jobs.compute)
+ reconstruct
        (_job_to_request) the choice and run IN-PROCESS ? not dispatch to Cloud Run. (An
earlier cut
        dropped compute in create_job/_job_to_request, silently nullifying the picker
through /api/process.)"""
        db, _bucket, monkeypatch = cloud_env
        fake = _FakeCloudClient()
        _inject_fake(monkeypatch, fake)
        monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
        app = m.create_app(m.global_config, db)
        client = TestClient(app)
```

```

        r = client.post(
            "/api/process",
            json={
                "video_path": "gs://b/clip.mp4",
                "segmenter": "motion",
                "compute": "local",
            },
        )
        assert r.status_code == 202
        job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
        assert job["status"] == "done"
        assert (
            fake.calls == []
        )  # 'local' honoured across the boundary ? never dispatched to Cloud Run
        segs = db.query_play_segments(job["video_id"], {})
        assert len(segs) == 1
        assert all(
            _meta(s).get("source") != "cloud_run" for s in segs
        )  # in-process rows, not cloud


    def test_cloud_override_survives_api_boundary_on_local_server(tmp_path, monkeypatch):
        """REGRESSION (mirror): on a LOCAL-default server wired for cloud (env + bucket),
POST /api/process
        with compute='cloud' must survive the boundary and dispatch to Cloud Run."""
        db = Database(str(tmp_path / "t.db"))  # noqa: F841
        monkeypatch.setattr(m, "db_instance", db)
        bucket = tmp_path / "bucket"
        bucket.mkdir()
        cfg = AppConfig.model_validate(
            {  # noqa: F841
                "deployment": {"compute_target": ""},  # server default = in-process
                "storage": {"backend": "local", "output_root": str(bucket)},
            }
        )
        monkeypatch.setattr(m, "global_config", cfg)
        monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
        fake_local = tmp_path / "fetched.mp4"
        fake_local.write_bytes(b"FAKEVIDEOBYTES")
        monkeypatch.setattr(
            m.storage, "acquire_local_input", lambda path, scfg: str(fake_local)
        )
        monkeypatch.setattr(
            m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {})
        )
        monkeypatch.setenv("CLOUD_RUN_JOB", "rally-worker")
        monkeypatch.setenv("GOOGLE_CLOUD_PROJECT", "proj")
        fake = _FakeCloudClient(video_id="vidBoundary")
        _inject_fake(monkeypatch, fake)
        app = m.create_app(m.global_config, db)
        client = TestClient(app)

        r = client.post(
            "/api/process",
            json={
                "video_path": "gs://b/clip.mp4",
                "segmenter": "wasb_hybrid",
                "compute": "cloud",
            },
        )
        assert r.status_code == 202
        job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
        assert job["status"] == "done"
        assert (
```

```
733             len(fake.calls) == 1
734         )  # 'cloud' honoured across the boundary ? dispatched despite local default
735         segs = db.query_play_segments(job["video_id"], {})
736         assert len(segs) == 2
737         assert all(_meta(s)["source"] == "cloud_run" for s in segs)
738
739
740     # --- hosted control-plane guardrail: compute='local' for a weights-backed detector
------------
741     # A hosted cloud-serving control-plane bakes NO detector weights (only the worker Cloud
Run Job mounts
742     # them), so a forced compute='local' for wasb/tracknet/fusion can't run in-process
there. It must fail
743     # FAST with an actionable "run in the cloud" message ? not fall through to the in-
process segmenter
744     # that dies deep with a cryptic "WASB weights not found" (observed live: job f29d41d0?,
2026-07-07).
745
746
747     def test_local_on_cloud_server_without_weights_fails_fast(cloud_env, monkeypatch):
748         """compute='local' + a weights-backed detector on a cloud_run server with NO local
weights ?
749         a clear pre-dispatch failure (never dispatched, never the cryptic deep 'weights not
found')."""
750         _db, _bucket, _mp = cloud_env
751         monkeypatch.delenv("WASB_WEIGHTS", raising=False)
752         m.global_config.indexing.wasb.weights_path =
"/nonexistent/wasb_badminton_best.pth.tar"
753         fake = _FakeCloudClient()
754         _inject_fake(monkeypatch, fake)
755
756         out = m._execute_processing(
757             m.ProcessRequest(
758                 video_path="gs://b/clip.mp4", segmenter="wasb_hybrid", compute="local"
759             )
760         )
761         assert out["status"] == "failed"
762         assert "can't run detection locally" in out["message"]
763         assert "Run in cloud" in out["message"]
764         assert "weights not found" not in out["message"]  # NOT the cryptic deep failure
765         assert fake.calls == []  # never dispatched
766         # Honest provenance: requested local, ran NOWHERE (pre-dispatch reject).
767         assert out["compute"]["requested"] == "local"
768         assert out["compute"]["dispatched"] is False
769         assert out["compute"]["path"] == "not_run"
770
771
772     def test_local_on_cloud_server_with_weights_present_runs_in_process(
773         cloud_env, tmp_path, monkeypatch
774     ):
775         """Escape hatch: the guardrail must NOT over-block ? on a cloud-wired box that
ACTUALLY has the
776         detector weights on disk, compute='local' still runs the in-process segmenter (no
dispatch)."""
777         _db, _bucket, _mp = cloud_env
778         weights = tmp_path / "wasb_badminton_best.pth.tar"
779         weights.write_bytes(b"WEIGHTS")
780         monkeypatch.delenv("WASB_WEIGHTS", raising=False)
781         m.global_config.indexing.wasb.weights_path = str(weights)
782         monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
783         fake = _FakeCloudClient()
784         _inject_fake(monkeypatch, fake)
785
786         out = m._execute_processing(
787             m.ProcessRequest(
```

```
                video_path="gs://b/clip.mp4", segmenter="wasb_hybrid", compute="local"
            )
        )
        assert out["status"] == "success"
        assert fake.calls == []  # in-process, not dispatched
        assert out["compute"]["path"] == "in_process"


    def test_local_non_weights_segmenter_on_cloud_server_still_runs_in_process(
        cloud_env, monkeypatch
    ):
        """The guardrail is scoped to weights-backed detectors: a weight-free segmenter
(motion) with
        compute='local' on a cloud server is unaffected ? it runs in-process, never fails
fast."""
        _db, _bucket, _mp = cloud_env
        m.global_config.indexing.wasb.weights_path =
"/nonexistent/should_not_matter.pth.tar"
        monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
        fake = _FakeCloudClient()
        _inject_fake(monkeypatch, fake)

        out = m._execute_processing(
            m.ProcessRequest(
                video_path="gs://b/clip.mp4", segmenter="motion", compute="local"
            )
        )
        assert out["status"] == "success"
        assert fake.calls == []
        assert out["compute"]["path"] == "in_process"


    def test_local_weights_backed_on_pure_local_server_is_not_guardrailed(
        tmp_path, monkeypatch
    ):
        """The guardrail only fires on a cloud-capable server. On a pure-local server (no
cloud), a
        weights-backed compute='local' request falls through to the in-process segmenter as
before ? the
        'run in the cloud' message would be nonsense there (no cloud to fall back to)."""
        db = Database(str(tmp_path / "t.db"))  # noqa: F841
        monkeypatch.setattr(m, "db_instance", db)
        cfg = AppConfig.model_validate({"deployment": {"compute_target": ""}})  # noqa: F841
        monkeypatch.setattr(m, "global_config", cfg)
        monkeypatch.setattr(
            m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {})
        )
        monkeypatch.delenv("CLOUD_RUN_JOB", raising=False)
        monkeypatch.delenv("GOOGLE_CLOUD_PROJECT", raising=False)
        monkeypatch.delenv("WASB_WEIGHTS", raising=False)
        cfg.indexing.wasb.weights_path = "/nonexistent/wasb.pth.tar"
        monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
        video = tmp_path / "v.mp4"
        video.write_bytes(b"hashable-bytes")

        out = m._execute_processing(
            m.ProcessRequest(
                video_path=str(video), segmenter="wasb_hybrid", compute="local"
            )
        )
        # Not the hosted fast-fail ? it ran in-process (the local box owns its own weights
story).
        assert out["status"] == "success"
        assert out["compute"]["path"] == "in_process"

```

```
847
848     def test_local_tracknet_on_cloud_server_without_weights_fails_fast(cloud_env,
monkeypatch):
849         """The guardrail also covers tracknet_hybrid (not just wasb): with no TrackNet
weights configured
850         (the default), compute='local' on a cloud_run server fails fast, never
dispatching."""
851         _db, _bucket, _mp = cloud_env
852         monkeypatch.delenv("WASB_WEIGHTS", raising=False)
853         m.global_config.indexing.tracknet.weights_path = ""  # default: no local TrackNet
weights
854         fake = _FakeCloudClient()
855         _inject_fake(monkeypatch, fake)
856
857         out = m._execute_processing(
858             m.ProcessRequest(
859                 video_path="gs://b/clip.mp4", segmenter="tracknet_hybrid", compute="local"
860             )
861         )
862         assert out["status"] == "failed"
863         assert "can't run detection locally" in out["message"]
864         assert fake.calls == []
865         assert out["compute"]["path"] == "not_run"
866
867
868     def test_local_fusion_tracknet_substrate_checks_tracknet_not_wasb(
869         cloud_env, tmp_path, monkeypatch
870     ):
871         """Pins the substrate switch: fusion_hybrid on the TrackNet substrate must gate on
the TrackNet
872         weights, not WASB. WASB weights PRESENT but TrackNet weights ABSENT must still fail
fast ? if the
873         check wrongly read indexing.wasb.weights_path it would take the escape hatch and run
in-process."""
874         _db, _bucket, _mp = cloud_env
875         wasb = tmp_path / "wasb_badminton_best.pth.tar"
876         wasb.write_bytes(b"WEIGHTS")
877         monkeypatch.delenv("WASB_WEIGHTS", raising=False)
878         m.global_config.indexing.wasb.weights_path = str(wasb)  # WASB present ?
879         m.global_config.indexing.fusion.substrate = "tracknet"
880         m.global_config.indexing.tracknet.weights_path = "/nonexistent/tracknet.pth"  # ?
TrackNet absent
881         fake = _FakeCloudClient()
882         _inject_fake(monkeypatch, fake)
883
884         out = m._execute_processing(
885             m.ProcessRequest(
886                 video_path="gs://b/clip.mp4", segmenter="fusion_hybrid", compute="local"
887             )
888         )
889         assert out["status"] == "failed"  # gated on TrackNet weights, which are absent
890         assert "can't run detection locally" in out["message"]
891         assert fake.calls == []
892
893
894     def test_local_fusion_default_substrate_with_wasb_weights_runs_in_process(
895         cloud_env, tmp_path, monkeypatch
896     ):
897         """Escape hatch for fusion on its DEFAULT (wasb) substrate: with WASB weights
present, compute=
898         'local' runs in-process (proves fusion_hybrid reads wasb weights on the default
substrate)."""
899         _db, _bucket, _mp = cloud_env
900         wasb = tmp_path / "wasb_badminton_best.pth.tar"
901         wasb.write_bytes(b"WEIGHTS")
```

```
        monkeypatch.delenv("WASB_WEIGHTS", raising=False)
        m.global_config.indexing.wasb.weights_path = str(wasb)
        # fusion.substrate defaults to "wasb"
        monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
        fake = _FakeCloudClient()
        _inject_fake(monkeypatch, fake)

        out = m._execute_processing(
            m.ProcessRequest(
                video_path="gs://b/clip.mp4", segmenter="fusion_hybrid", compute="local"
            )
        )
        assert out["status"] == "success"
        assert fake.calls == []
        assert out["compute"]["path"] == "in_process"


    # --- compute-path PROVENANCE (the validation method: prove which backend ran)
------------------


    def test_cloud_dispatch_stamps_compute_provenance(cloud_env):
        """A cloud run stamps result['compute'] with path=cloud_run + the REAL Cloud Run
execution name
        (the GCP-verifiable proof) + the requested choice + job/region/outputs ? so the
intended path is
        provable from the job result alone (no guessing from segment metadata)."""
        _db, _bucket, monkeypatch = cloud_env
        _inject_fake(monkeypatch, _FakeCloudClient())

        out = m._execute_processing(
            m.ProcessRequest(
                video_path="gs://b/clip.mp4", segmenter="wasb_hybrid", compute="cloud"
            )
        )
        prov = out["compute"]
        assert prov["path"] == "cloud_run"
        assert prov["requested"] == "cloud"
        assert (
            prov["execution"] == "exec-1"
        )  # the fake client's execution name (real one in prod)
        assert prov["outputs_uri"].endswith("/outputs/vidCloud/")
        assert prov["job"] and prov["region"]  # the dispatch coordinates are recorded


    def test_local_override_stamps_in_process_provenance(cloud_env):
        """On a cloud server, compute='local' runs in-process AND the result proves it:
path=in_process,
        requested=local ? so a 'ran local despite a cloud server' is auditable (no silent
ambiguity)."""
        _db, _bucket, monkeypatch = cloud_env
        _inject_fake(monkeypatch, _FakeCloudClient())
        monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)

        out = m._execute_processing(
            m.ProcessRequest(
                video_path="gs://b/clip.mp4", segmenter="motion", compute="local"
            )
        )
        assert out["status"] == "success"
        assert out["compute"] == {"path": "in_process", "requested": "local"}


    def test_default_in_process_stamps_provenance(tmp_path, monkeypatch):
        """A default-OFF (no cloud) run stamps path=in_process, requested=None ? the byte-
```

```
identical local
962         path is now self-describing."""
963         db = Database(str(tmp_path / "t.db"))  # noqa: F841
964         monkeypatch.setattr(m, "db_instance", db)
965         monkeypatch.setattr(m, "db_instance", db)
966         monkeypatch.setattr(m, "global_config", AppConfig())
967         m.app.state.config = AppConfig()
968         monkeypatch.setattr(
969             m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {})
970         )
971         monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
972         video = tmp_path / "v.mp4"
973         video.write_bytes(b"hashable-bytes")
974
975         out = m._execute_processing(
976             m.ProcessRequest(video_path=str(video), segmenter="motion")
977         )
978         assert out["compute"] == {"path": "in_process", "requested": None}
979
980
981     def test_cloud_requested_unconfigured_stamps_not_dispatched(tmp_path, monkeypatch):
982         """compute='cloud' on a server that can't dispatch fails clearly AND records it:
target cloud_run,
983         requested=cloud, dispatched=False ? proving the cloud job never ran (no silent local
fallback)."""
984         db = Database(str(tmp_path / "t.db"))  # noqa: F841
985         monkeypatch.setattr(m, "db_instance", db)
986         monkeypatch.setattr(m, "db_instance", db)
987         monkeypatch.setattr(
988             m,
989             "global_config",
990             AppConfig.model_validate({"deployment": {"compute_target": ""}}),
991         )
992         m.app.state.config = AppConfig.model_validate(
993             {"deployment": {"compute_target": ""}}
994         )
995         monkeypatch.setattr(
996             m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {})
997         )
998         monkeypatch.delenv("CLOUD_RUN_JOB", raising=False)
999         monkeypatch.delenv("GOOGLE_CLOUD_PROJECT", raising=False)
1000        video = tmp_path / "v.mp4"
1001        video.write_bytes(b"hashable-bytes")
1002
1003        out = m._execute_processing(
1004            m.ProcessRequest(
1005                video_path=str(video), segmenter="wasb_hybrid", compute="cloud"
1006            )
1007        )
1008        assert out["status"] == "failed"
1009        # Honest: it ran NOWHERE (pre-dispatch reject) ? path=not_run, but the intent is
recorded.
1010        assert out["compute"] == {
1011            "path": "not_run",
1012            "requested": "cloud",
1013            "target": "cloud_run",
1014            "dispatched": False,
1015        }
1016
1017
1018    def test_validation_failure_stamps_not_run_provenance(tmp_path, monkeypatch):
1019        """A validation failure never reaches a segmenter ? path=not_run (nothing
executed)."""
1020        db = Database(str(tmp_path / "t.db"))  # noqa: F841
1021        monkeypatch.setattr(m, "db_instance", db)
```

```
1022        monkeypatch.setattr(m, "global_config", AppConfig())
1023        m.app.state.config = AppConfig()
1024        fail = [
1025            ValidationResult(validator_name="x", passed=False, message="bad", details={})
1026        ]
1027        monkeypatch.setattr(m.registry, "run_all_with_context", lambda *a, **k: (fail, {}))
1028        video = tmp_path / "v.mp4"
1029        video.write_bytes(b"hashable-bytes")
1030
1031        out = m._execute_processing(m.ProcessRequest(video_path=str(video)))
1032        assert out["status"] == "failed"
1033        assert out["compute"]["path"] == "not_run"
1034
1035
1036    def test_full_api_process_job_result_carries_cloud_provenance(cloud_env):
1037        """End-to-end: POST /api/process (compute=cloud) ? the job result the SPA polls
carries the
1038        cloud_run provenance + execution, so the UI/operator can PROVE the cloud path was
taken."""
1039        _db, _bucket, monkeypatch = cloud_env
1040        _inject_fake(monkeypatch, _FakeCloudClient(video_id="vidProv"))
1041        app = m.create_app(m.global_config, _db)
1042        client = TestClient(app)
1043
1044        r = client.post(
1045            "/api/process",
1046            json={
1047                "video_path": "gs://b/clip.mp4",
1048                "segmenter": "wasb_hybrid",
1049                "compute": "cloud",
1050            },
1051        )
1052        assert r.status_code == 202
1053        job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
1054        assert job["status"] == "done"
1055        assert job["result"]["compute"]["path"] == "cloud_run"
1056        assert job["result"]["compute"]["execution"] == "exec-1"
1057
1058
1059    # --- #468: local compute lane ? CPU/GPU-heavy in-process work serializes, remote polls
don't --------
1060
1061
1062    def test_in_process_detection_runs_under_the_local_lane(cloud_env):
1063        """The in-process segmenter holds orchestration._LOCAL_COMPUTE_LANE while it runs,
so raising the
1064        drain worker count (to overlap I/O-bound remote dispatch) never runs two local
detections at once.
1065        A remote dispatch returns before this path, so remote polls never hold the lane
(they overlap)."""
1066        import backend.orchestration as orch
1067
1068        _db, _bucket, monkeypatch = cloud_env
1069        held = {}
1070
1071        class _LaneCheckSeg:
1072            def __init__(self, db_, cfg_):
1073                self.db = db_
1074
1075            def process_video(self, vid, path, *a, **k):
1076                # the wrapping `with _LOCAL_COMPUTE_LANE` holds the size-1 lane ? a non-
blocking re-acquire
1077                # must FAIL while detection runs (proving concurrent local detections are
excluded).
1078                got = orch._LOCAL_COMPUTE_LANE.acquire(blocking=False)
```

```
1079                held["busy"] = not got
1080                if got:
1081                    orch._LOCAL_COMPUTE_LANE.release()
1082                sid = self.db.add_play_segment(vid, 0.0, 5.0)
1083                return [
1084                    {"id": sid, "start_time": 0.0, "end_time": 5.0, "duration": 5.0,
"metadata": {}}
1085                ]
1086
1087        monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _LaneCheckSeg)
1088        out = m._execute_processing(
1089            m.global_config,
1090            m.db_instance,
1091            m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="motion",
compute="local"),
1092        )
1093        assert out["status"] == "success"
1094        assert held.get("busy") is True  # detection ran while the local lane was held
1095
1096
1097    def test_wait_progress_message_is_honest_and_minute_granular():
1098        """#482: the CP knows ELAPSED time, not frames ? the heartbeat says so plainly,
shows
1099        movement from the first tick ("starting up" during queue/cold-start), then counts
minutes."""
1100        from backend.orchestration import _wait_progress_message
1101
1102        early = _wait_progress_message("wasb_hybrid", 12.0)
1103        assert "wasb_hybrid" in early and "starting up" in early
1104        later = _wait_progress_message("wasb_hybrid", 125.0)
1105        assert later.endswith("2 min elapsed")
1106
```

38. assistant (2026-07-08T05:12:04.778Z)

I have a thorough understanding. Let me set up the isolated worktree (per branching-safety rules) and check the test environment, while reading the remaining test patterns.

39. user (2026-07-08T05:12:09.087Z)

```
=== venv? ===
=== python ===
Python 3.13.13
=== ruff/mypy ===
ruff 0.15.16
mypy 2.1.0 (compiled: yes)
=== existing worktrees ===
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer
699d85b [master]
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/cfg-wt        20b2d5a [claude/wasb-
config-validation]
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/compile-wt    b09a1d4 [claude/compile-
reliability]
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/pr458-pre-wt  355dad6 (detached HEAD)
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/pr458-wt      5de1ef2 (detached HEAD)
```

40. user (2026-07-08T05:12:09.105Z)

```
1       """Worker dispatcher: pull (storage) ? run pipeline ? push (storage).
2
```

```
    `cmd_infer` is exercised end-to-end over the LOCAL storage backend with the validator +
    segmenter mocked, so it needs no real video/GPU/network. The real S3 round-trip is validated
    on the droplet against MinIO.
    """

    import json

    import pytest

    from backend.config import load_config
    from backend.results import Failure
    from backend.worker import __main__ as wmain

    # ---------------------------------------------------------------- pure helpers


    @pytest.mark.parametrize(
        "raw,expected",
        [
            ("true", True),
            ("False", False),
            ("3", 3),
            ("2.5", 2.5),
            ("wasb_hybrid", "wasb_hybrid"),
        ],
    )
    def test_coerce(raw, expected):
        assert wmain._coerce(raw) == expected and type(wmain._coerce(raw)) is type(expected)


    def test_apply_overrides_on_typed_config():
        cfg = _fresh_config()
        wmain._apply_overrides(
            cfg,
            [
                "indexing.skip_ai_handoff=true",
                "indexing.min_segment_duration=1.5",
                "sport=tennis",
            ],
        )
        assert cfg.indexing.skip_ai_handoff is True
        assert cfg.indexing.min_segment_duration == 1.5
        assert cfg.sport == "tennis"


    def test_apply_overrides_rejects_bad_format():
        cfg = _fresh_config()
        with pytest.raises(ValueError):
            wmain._apply_overrides(cfg, ["no_equals_sign"])


    def test_write_intervals_csv(tmp_path):
        segs = [
            {
                "start_time": 2.0,
                "end_time": 5.0,
                "shot_count": 4,
                "shot_count_confidence": "LocalNoAI",
            },
            {"start_time": 9.0, "end_time": 12.5},
        ]
        out = tmp_path / "intervals.csv"
        wmain._write_intervals_csv(segs, str(out))
        lines = out.read_text().splitlines()
```

```
67          assert lines[0] == "start,end,shot_count,ending_reason"
68          assert lines[1].startswith("2.000,5.000,4,")
69          assert lines[2].startswith("9.000,12.500,,")
70
71
72      def test_write_report_json(tmp_path):
73          out = tmp_path / "report.json"
74          wmain._write_report_json(
75              "vid1", [{"start_time": 1.0, "end_time": 2.0}], "success", str(out)
76          )
77          data = json.loads(out.read_text())
78          assert data["video_id"] == "vid1" and data["segment_count"] == 1
79          assert data["segments"][0] == {"start": 1.0, "end": 2.0}
80
81
82      def test_parser_infer_accepts_set_and_uris():
83          args = wmain.build_parser().parse_args(
84              [
85                  "infer",
86                  "--video-uri",
87                  "s3://b/v.mp4",
88                  "--out-uri",
89                  "s3://b",
90                  "--set",
91                  "indexing.skip_ai_handoff=true",
92                  "--set",
93                  "sport=tennis",
94              ]
95          )
96          assert args.cmd == "infer" and args.video_uri == "s3://b/v.mp4"
97          assert args.set == ["indexing.skip_ai_handoff=true", "sport=tennis"]
98
99
100     def test_parser_infer_skip_validation_flag():
101         base = ["infer", "--video-uri", "s3://b/v.mp4", "--out-uri", "s3://b"]
102         # Default-OFF: the direct `worker infer` CLI still validates.
103         assert wmain.build_parser().parse_args(base).skip_validation is False
104         # The dispatcher passes it to disable the redundant worker re-validation.
105         assert wmain.build_parser().parse_args(base + ["--skip-validation"]).skip_validation
is True
106
107
108     # ------------------------------------------------------------- cmd_infer (local,
mocked)
109
110
111     class _OKResult:
112         passed = True
113         validator_name = "fake"
114         message = "ok"
115
116         def model_dump(self):
117             return {"validator_name": "fake", "passed": True, "message": "ok"}
118
119
120     class _FailResult(_OKResult):
```

41. assistant (2026-07-08T05:12:22.907Z)

No repo venv ? deps are in global Python 3.13, so a worktree anywhere can run the suite. There are sibling-session worktrees on other branches (the WIP the task warned about) ? I'll stay isolated. Let me create my worktree from `origin/master`.

42. user (2026-07-08T05:12:26.505Z)

```
Exit code 1
Preparing worktree (new branch 'feat/521-compile-on-l4')
branch 'feat/521-compile-on-l4' set up to track 'origin/master'.
error: unable to create file docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-
serving_gpu-resident-nvdec_2026-07-07.md: Filename too long
error: unable to create file docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-
serving_validation-latency_2026-07-07.md: Filename too long
fatal: Could not reset index file to revision 'HEAD'.
---VERIFY---
/usr/bin/bash: line 1: cd: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-
khelsutra-guru-badminton-highlight-
indexer/75d3d121-f240-4293-afa5-b436a379ad31/scratchpad/wt-521: No such file or directory
```

43. assistant (2026-07-08T05:12:33.414Z)

Windows MAX_PATH hit on the long scratchpad path. Let me clean up and use a short worktree path.

44. user (2026-07-08T05:12:41.620Z)

```
error: failed to delete '.git/worktrees/a1-master-worktree': Permission denied
error: failed to delete '.git/worktrees/bhi-wt-worker-reval': Permission denied
error: failed to delete '.git/worktrees/fix-worktree': Permission denied
error: failed to delete '.git/worktrees/master-compare': Permission denied
error: failed to delete '.git/worktrees/pr465': Permission denied
error: failed to delete '.git/worktrees/pr465-current-merge': Permission denied
error: failed to delete '.git/worktrees/pr465-merge': Permission denied
error: failed to delete '.git/worktrees/pr467': Permission denied
error: failed to delete '.git/worktrees/pr469': Permission denied
error: failed to delete '.git/worktrees/wt-465fix': Permission denied
error: failed to delete '.git/worktrees/wt-a1-runlog': Permission denied
error: failed to delete '.git/worktrees/wt-claudemd-r25': Permission denied
error: failed to delete '.git/worktrees/wt-cloudrun-cancel': Permission denied
error: failed to delete '.git/worktrees/wt-cloudrun-dispatch': Permission denied
error: failed to delete '.git/worktrees/wt-cp-rebuild-spec': Permission denied
error: failed to delete '.git/worktrees/wt-dockerfile-auth': Permission denied
error: failed to delete '.git/worktrees/wt-docs': Permission denied
error: failed to delete '.git/worktrees/wt-docs-reconcile': Permission denied
error: failed to delete '.git/worktrees/wt-drain-concurrency': Permission denied
error: failed to delete '.git/worktrees/wt-ffmpeg-timeouts': Permission denied
error: failed to delete '.git/worktrees/wt-gdrive-api-safety': Permission denied
error: failed to delete '.git/worktrees/wt-idempotent-process': Permission denied
error: failed to delete '.git/worktrees/wt-input-jail': Permission denied
error: failed to delete '.git/worktrees/wt-output-dir-config': Permission denied
error: failed to delete '.git/worktrees/wt-p1-genyaml': Permission denied
error: failed to delete '.git/worktrees/wt-p2-upload-stream': Permission denied
error: failed to delete '.git/worktrees/wt-p4-md5cache': Permission denied
error: failed to delete '.git/worktrees/wt-p5a-pollknob': Permission denied
error: failed to delete '.git/worktrees/wt-p6-heartbeat': Permission denied
error: failed to delete '.git/worktrees/wt-pr481-review': Permission denied
error: failed to delete '.git/worktrees/wt-retention-sweep': Permission denied
error: failed to delete '.git/worktrees/wt-review-492-mergecheck': Permission denied
error: failed to delete '.git/worktrees/wt-review-493-mergecheck': Permission denied
error: failed to delete '.git/worktrees/wt-scratch-reclaim': Permission denied
error: failed to delete '.git/worktrees/wt-signed-download': Permission denied
error: failed to delete '.git/worktrees/wt-source-cache': Permission denied
error: failed to delete '.git/worktrees/wt-upload-dir': Permission denied
error: failed to delete '.git/worktrees/wt-upload-staging': Permission denied
Deleted branch feat/521-compile-on-l4 (was 699d85b).
--- retry with short path ---
Preparing worktree (new branch 'feat/521-compile-on-l4')
branch 'feat/521-compile-on-l4' set up to track 'origin/master'.
HEAD is now at 699d85b feat(validation): cache validation verdict by video_id to skip re-
validation on reupload (#518)
---VERIFY---
feat/521-compile-on-l4
```

```
699d85b feat(validation): cache validation verdict by video_id to skip re-validation on reupload
(#518)
---ONLY MY COMMITS (should be empty)---
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

45. assistant (2026-07-08T05:12:56.753Z)

Worktree is clean at `C:/kg-wt521` (off origin/master 699d85b, no sibling commits). Let me establish a green baseline before changing anything, and set up progress tracking.

46. user (2026-07-08T05:13:51.566Z)

```
..................................................................... [ 41%]
..................................................................... [ 83%]
............................                                         [100%]
============================== warnings summary ===============================
tests/test_api_cloud_dispatch.py::test_full_api_process_to_query_via_cloud
tests/test_api_cloud_dispatch.py::test_full_api_process_to_query_via_cloud
tests/test_api_cloud_dispatch.py::test_local_override_survives_api_boundary
tests/test_api_cloud_dispatch.py::test_local_override_survives_api_boundary
tests/test_api_cloud_dispatch.py::test_cloud_override_survives_api_boundary_on_local_server
tests/test_api_cloud_dispatch.py::test_cloud_override_survives_api_boundary_on_local_server
tests/test_api_cloud_dispatch.py::test_full_api_process_job_result_carries_cloud_provenance
tests/test_api_cloud_dispatch.py::test_full_api_process_job_result_carries_cloud_provenance
  C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
    warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
172 passed, 8 warnings in 50.78s
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

47. assistant (2026-07-08T05:13:57.689Z)

Baseline is green (172 passed). Note: shell cwd resets to main repo each call, so I'll always `cd` to the worktree. Let me read the compute-backend and async-jobs test patterns, and the rest of test_worker.py.

48. user (2026-07-08T05:13:58.323Z)

```
1       """ComputeBackend seam ? Cloud Run dispatch shim (COMPUTE_DECOUPLED_SERVING M6).
2
3       All GPU-free + cloud-free: the Cloud Run trigger is a FAKE ``CloudRunJobClient`` that
simulates the
4       worker (writes the done-marker + report to a local "bucket" = a tmp dir), so the
dispatch?bucket-poll
5       ?read flow is proven for $0. The cardinal invariant: **default-OFF** ? only
``compute_target=cloud_run``
6       selects a backend; everything else returns ``None`` (= run the worker's existing in-
process path).
7       """
8
9       import json
10      import os
11      import tempfile
12
13      import pytest
14
15      from backend.compute import (
16          ComputeJobSpec,
17          RemoteExecutionFailed,
```

```
18          get_compute_backend,
19      )
20      from backend.compute.cloud_run import CloudRunCompute, CloudRunJobClient
21      from backend.config import AppConfig
22      from backend.infrastructure.execution_policy import ExecutionPolicy, PolicyTimeout
23
24      # A fast, no-wait policy (the fake client writes the marker synchronously, so the first
poll hits).
25      _FAST = ExecutionPolicy(poll_every_n_sec=0.0, op_timeout_sec=5.0, max_wait_sec=5.0)
26
27
28      class _FakeRunClient(CloudRunJobClient):
29          """Simulates a Cloud Run Job: on dispatch, write the report + done-marker into the
local bucket
30          exactly where the real worker would, so CloudRunCompute's bucket-poll resolves
immediately."""
31
32          def __init__(self, *, video_id="vidCloud", segment_count=2, returncode=0):
33              self.calls: list[dict] = []
34              self.video_id, self.segment_count, self.returncode = (
35                  video_id,
36                  segment_count,
37                  returncode,
38              )
39
40          def run_job(self, job_name, argv, *, region, project=None):
41              self.calls.append(
42                  {"job": job_name, "argv": list(argv), "region": region, "project": project}
43              )
44              out_uri = argv[argv.index("--out-uri") + 1].rstrip("/")
45              done_uri = argv[argv.index("--done-uri") + 1]
46              outputs = os.path.join(out_uri, "outputs", self.video_id)
47              os.makedirs(outputs, exist_ok=True)
48              with open(os.path.join(outputs, "report.json"), "w", encoding="utf-8") as f:
49                  json.dump(
50                      {
51                          "video_id": self.video_id,
52                          "segment_count": self.segment_count,
53                          "status": "success",
54                      },
55                      f,
56                  )
57              os.makedirs(os.path.dirname(done_uri), exist_ok=True)
58              with open(done_uri, "w", encoding="utf-8") as f:
59                  json.dump(
60                      {
61                          "video_id": self.video_id,
62                          "returncode": self.returncode,
63                          "segment_count": self.segment_count,
64                          "status": "success",
65                      },
66                      f,
67                  )
68              return "exec-123"
69
70          def cancel_execution(self, execution):
71              raise AssertionError(
72                  "cancel must never fire on the happy path"
73              )  # pragma: no cover
74
75
76      class _NoopRunClient(CloudRunJobClient):
77          """Records the dispatch but does NO filesystem writes ? for tests that fake the
storage layer
78          directly (a gs:// out_uri has no local dir to makedirs)."""
```

```
79
80          def __init__(self):
81              self.calls: list[dict] = []
82              self.cancelled: list[str] = []
83
84          def run_job(self, job_name, argv, *, region, project=None):
85              self.calls.append(
86                  {"job": job_name, "argv": list(argv), "region": region, "project": project}
87              )
88              return "exec"
89
90          def cancel_execution(self, execution):
91              self.cancelled.append(execution)
92
93
94      class _NeverDoneClient(CloudRunJobClient):
95          """Simulates a still-running execution: dispatch succeeds (returning a fully-
qualified
96          execution name) but NO done-marker is ever written, so the bucket-poll must time
out."""
97
98          EXEC_NAME = "projects/p/locations/asia-south1/jobs/rally-worker/executions/exec-
slow"
99
100         def __init__(self, *, cancel_error: Exception | None = None):
101             self.calls: list[dict] = []
102             self.cancelled: list[str] = []
103             self._cancel_error = cancel_error
104
105         def run_job(self, job_name, argv, *, region, project=None):
106             self.calls.append(
107                 {"job": job_name, "argv": list(argv), "region": region, "project": project}
108             )
109             return self.EXEC_NAME
110
111         def cancel_execution(self, execution):
112             self.cancelled.append(execution)
113             if self._cancel_error is not None:
114                 raise self._cancel_error
115
116
117     class _CrashedNoMarkerClient(CloudRunJobClient):
118         """Simulates the bug's failure class: the worker execution TERMINATES (crash / OOM /
an old
119         image argparse-aborting on an unknown flag) WITHOUT ever writing a done-marker.
Dispatch returns
120         a resolvable execution name, no marker is ever written, and the Executions API
reports a terminal
121         state ? so run_infer must fail FAST via the status probe instead of spinning the
poll budget."""
122
123         EXEC_NAME = "projects/p/locations/asia-south1/jobs/rally-worker/executions/exec-
crash"
124
125         def __init__(self, *, state: str = "Failed"):
126             self.calls: list[dict] = []
127             self.cancelled: list[str] = []
128             self.status_polls = 0
129             self._state = state
130
131         def run_job(self, job_name, argv, *, region, project=None):
132             self.calls.append(
133                 {"job": job_name, "argv": list(argv), "region": region, "project": project}
134             )
135             return self.EXEC_NAME
```

```
136
137         def cancel_execution(self, execution):
138             self.cancelled.append(execution)
139
140         def execution_terminal_state(self, execution):
141             self.status_polls += 1
142             # Anti-hang guard: the fast path MUST abort on the first terminal observation.
If a
143             # regression kept polling despite a terminal status, trip here instead of
spinning.
144             assert self.status_polls <= 5, "status probed repeatedly ? fast path did not
abort the poll"
145             return self._state
146
147
148     # ------------------------------------------------------------------ factory (default-
OFF)
149
150
151     def _cfg(compute_target="", storage_backend="local"):
152         return AppConfig.model_validate(
153             {
154                 "deployment": {"compute_target": compute_target},
155                 "storage": {"backend": storage_backend},
156             }
157         )
158
159
160     def test_factory_is_none_for_local_targets():
161         """DEFAULT-OFF: no/local/cpu_mock compute target ? None (run the in-process
path)."""
162         assert get_compute_backend(_cfg("")) is None
163         assert get_compute_backend(_cfg("local_gpu")) is None
164         assert get_compute_backend(_cfg("cpu_mock")) is None
165
166
167     def test_factory_returns_cloud_run_for_cloud_target():
168         backend = get_compute_backend(_cfg("cloud_run", "gcs"), run_client=_FakeRunClient())
169         assert isinstance(backend, CloudRunCompute)
170
171
172     # ------------------------------------------------------- real client: missing-project
guard
173
174
175     def test_real_client_rejects_missing_project():
176         """The real GoogleCloudRunJobClient must fail loudly when GOOGLE_CLOUD_PROJECT is
unset ?
177         a None/empty project would build a bogus "projects/None/..." job path (job_path
requires a
178         real str). The guard runs BEFORE the google-cloud-run import, so it is exercisable
in CI
179         (the SDK is intentionally not a test dependency)."""
180         from backend.compute.cloud_run import GoogleCloudRunJobClient
181
182         client = GoogleCloudRunJobClient()
183         for missing in (None, ""):
184             with pytest.raises(RuntimeError, match="GOOGLE_CLOUD_PROJECT"):
185                 client.run_job(
186                     "rally-worker", ["infer"], region="asia-south1", project=missing
187                 )
188
189
190     def test_real_client_cancel_rejects_unresolvable_execution_name():
191         """cancel_execution must fail loudly on a non-resolvable execution name ? run_job
```

```
falls back
192         to the literal "execution" when the operation metadata carries no name, and
cancelling that
193         would be a bogus API call. The guard runs BEFORE the google-cloud-run import
(mirrors the
194         project guard), so it is exercisable in CI (the SDK is intentionally not a test
dependency)."""
195         from backend.compute.cloud_run import GoogleCloudRunJobClient
196
197         client = GoogleCloudRunJobClient()
198         for bogus in ("", "execution", "exec-123"):
199             with pytest.raises(RuntimeError, match="fully-qualified"):
200                 client.cancel_execution(bogus)
201
202
203     # ------------------------------------------------------------ CloudRunCompute dispatch
+ poll
204
205
206     def test_cloud_run_dispatch_polls_and_returns_result(tmp_path):
207         client = _FakeRunClient(video_id="abc123", segment_count=3)
208         backend = CloudRunCompute(
209             run_client=client,
210             job_name="rally-worker",
211             region="asia-south1",
212             project="proj",
213             storage_config={},
214             policy=_FAST,
215             token="tok",
216         )
217         out = str(tmp_path / "bucket")
218         result = backend.run_infer(
219             ComputeJobSpec(
220                 video_uri="gs://b/videos/clip.mp4", out_uri=out, segmenter="wasb_hybrid"
221             )
222         )
223
224         assert result.returncode == 0
225         assert result.video_id == "abc123"
226         assert result.outputs_uri == f"{out}/outputs/abc123/"
227         assert result.report.get("segment_count") == 3
228         # exactly one Cloud Run Job execution was triggered, in the right region/project
229         assert len(client.calls) == 1
230         assert (
231             client.calls[0]["job"] == "rally-worker"
232             and client.calls[0]["region"] == "asia-south1"
233         )
234
235
236     def test_dispatch_argv_forces_inline_and_carries_overrides(tmp_path):
237         """The dispatched container must run INLINE (never re-dispatch) ?
compute_target=local_gpu +
238         native detector appended AFTER the user overrides (so they win); --done-uri always
present."""
239         client = _FakeRunClient()
240         backend = CloudRunCompute(
241             run_client=client, job_name="j", region="r", policy=_FAST, token="tok"
242         )
243         backend.run_infer(
244             ComputeJobSpec(
245                 video_uri="gs://b/v.mp4",
246                 out_uri=str(tmp_path / "bk"),
247                 segmenter="wasb_hybrid",
248                 config_overrides=("sport=tennis",),
249             )
```

```
        )

        argv = client.calls[0]["argv"]
        assert argv[0] == "infer"
        assert "--done-uri" in argv and "--video-uri" in argv and "--out-uri" in argv
        assert argv[argv.index("--segmenter") + 1] == "wasb_hybrid"
        # A DEFAULT spec (skip_validation=False) must NOT carry --skip-validation: the
worker still
        # validates unless the dispatcher explicitly opts out. Guards the `if
spec.skip_validation:` gate
        # against a refactor that appends the flag unconditionally (which would silently
bypass the direct
        # `worker infer` cloud path's validation ? the very thing #skip-validation is scoped
away from).
        assert "--skip-validation" not in argv
        # the container-inline overrides come last (win) and the user override is carried
through
        joined = " ".join(argv)
        assert "deployment.compute_target=local_gpu" in joined
        assert "indexing.detector_impl=native" in joined
        assert "sport=tennis" in joined
        # user override precedes the inline-forcing overrides in --set order
        assert joined.index("sport=tennis") < joined.index(
            "deployment.compute_target=local_gpu"
        )


    def test_dispatch_argv_carries_skip_validation_when_set(tmp_path):
        """The builder must emit --skip-validation when the spec opts out ? the control-
plane sets
        skip_validation=True on every dispatch (it is the authoritative validation gate),
and the worker
        honours the flag to skip its redundant re-validation. Positive builder-level guard,
independent of
        the control-plane path (which hardcodes True)."""
        client = _FakeRunClient()
        backend = CloudRunCompute(
            run_client=client, job_name="j", region="r", policy=_FAST, token="tok"
        )
        backend.run_infer(
            ComputeJobSpec(
                video_uri="gs://b/v.mp4",
                out_uri=str(tmp_path / "bk"),
                skip_validation=True,
            )
        )
        assert "--skip-validation" in client.calls[0]["argv"]


    def test_unique_token_per_call_when_not_injected(tmp_path):
        """No injected token ? a fresh uuid per dispatch, so concurrent jobs don't collide
on the marker."""
        client = _FakeRunClient()
        backend = CloudRunCompute(run_client=client, job_name="j", region="r", policy=_FAST)
        backend.run_infer(
            ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bk"))
        )
        backend.run_infer(
            ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bk"))
        )
        done1 = client.calls[0]["argv"][client.calls[0]["argv"].index("--done-uri") + 1]
        done2 = client.calls[1]["argv"][client.calls[1]["argv"].index("--done-uri") + 1]
        assert done1 != done2


```

```
    def test_read_json_passes_dst_for_cloud_backends(monkeypatch):
        """BLOCKER guard: GCS/S3 fetch_to_local REQUIRES a dst (a None dst raises) ?
_read_json must
        pass one, else every real cloud marker/report read silently returns {} (local hid
this via its
        identity passthrough). Mimic a cloud backend that rejects dst=None."""
        from backend.compute import cloud_run as cr

        captured = {}

        def fake_fetch(uri, dst=None, *, config=None, overwrite=False):
            captured["dst"] = dst
            if dst is None:
                raise ValueError(
                    "cloud backend requires a dst path"
                )  # mirror GCSStorage/S3Storage
            with open(dst, "w", encoding="utf-8") as f:
                f.write('{"video_id": "v", "returncode": 0}')
            return dst

        monkeypatch.setattr(cr, "fetch", fake_fetch)
        backend = cr.CloudRunCompute(run_client=_FakeRunClient(), job_name="j", region="r")
        data = backend._read_json("gs://b/x.json")
        assert (
            captured["dst"] is not None
        )  # the fix: a real dst is always passed (no silent {})
        assert data == {"video_id": "v", "returncode": 0}
        assert not os.path.exists(os.path.dirname(captured["dst"]))


    def test_cloud_run_polls_until_marker_appears(monkeypatch):
        """The bucket-poll WAIT path (D7 scale-to-zero cold start): the marker is ABSENT for
the first
        polls then appears ? poll_until must LOOP, not resolve on poll 0. The synchronous
fake elsewhere
        never exercises this; here exists() is False twice then True."""
        from backend.compute import cloud_run as cr

        exists_calls = {"n": 0}

        class _FakeStore:
            def exists(self, ref):
                exists_calls["n"] += 1
                return exists_calls["n"] >= 3  # absent for polls 1-2, present on poll 3

        monkeypatch.setattr(cr, "get_storage", lambda backend, config=None: _FakeStore())

        def fake_fetch(uri, dst=None, *, config=None, overwrite=False):
            if dst is None:
                dst = os.path.join(tempfile.mkdtemp(prefix="cr-poll-"), "m.json")
            with open(dst, "w", encoding="utf-8") as f:
                f.write('{"video_id": "vlate", "returncode": 0, "segment_count": 1}')
            return dst

        monkeypatch.setattr(cr, "fetch", fake_fetch)
        backend = cr.CloudRunCompute(
            run_client=_NoopRunClient(),
            job_name="j",
            region="r",
            token="t",
            policy=ExecutionPolicy(
                poll_every_n_sec=0.0, op_timeout_sec=5.0, max_wait_sec=5.0
            ),
        )
        result = backend.run_infer(
```

```
            ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri="gs://b")
        )
        assert result.video_id == "vlate"
        assert (
            exists_calls["n"] >= 3
        )  # >=1 absent poll occurred before the marker appeared (loop ran)


    # ----------------------------------------- poll timeout: rescue-or-cancel (R1-21 GPU
cost leak)

    # max_wait_sec=0.0 ? poll_until gives up after its FIRST absent check (no real waiting).
    _TIMEOUT_NOW = ExecutionPolicy(poll_every_n_sec=0.0, op_timeout_sec=5.0,
max_wait_sec=0.0)


    def test_poll_timeout_cancels_execution_and_raises(tmp_path):
        """Poll deadline + no marker ? the still-running execution is cancelled (with the
EXACT name
        run_job returned) and PolicyTimeout surfaces carrying that name for the operator."""
        client = _NeverDoneClient()
        backend = CloudRunCompute(
            run_client=client,
            job_name="rally-worker",
            region="asia-south1",
            policy=_TIMEOUT_NOW,
            token="tok",
        )
        with pytest.raises(PolicyTimeout) as exc:
            backend.run_infer(
                ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bucket"))
            )
        assert client.cancelled == [_NeverDoneClient.EXEC_NAME]  # cancel got the right name
        msg = str(exc.value)
        assert _NeverDoneClient.EXEC_NAME in msg  # operator can find the execution in GCP
        assert "cancel requested" in msg
        assert "max_wait_sec" in msg  # the original timeout text is preserved, not masked


    def test_poll_timeout_cancel_failure_still_raises_policy_timeout(tmp_path):
        """Best-effort means BEST-EFFORT: a cancel API failure must never mask the original
timeout ?
        the caller (main.py's dispatch handler) still sees PolicyTimeout, with the failure
noted."""
        client = _NeverDoneClient(cancel_error=RuntimeError("cancel API unreachable"))
        backend = CloudRunCompute(
            run_client=client,
            job_name="rally-worker",
            region="asia-south1",
            policy=_TIMEOUT_NOW,
            token="tok",
        )
        with pytest.raises(PolicyTimeout) as exc:
            backend.run_infer(
                ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bucket"))
            )
        assert client.cancelled == [_NeverDoneClient.EXEC_NAME]  # the attempt WAS made
        msg = str(exc.value)
        assert _NeverDoneClient.EXEC_NAME in msg
        assert "cancel API unreachable" in msg  # the cancel failure is surfaced, not
swallowed
        assert "max_wait_sec" in msg  # ...alongside the original timeout text


    def test_poll_timeout_late_marker_rescues_result(monkeypatch):
```

```
        """A slow-but-healthy worker that finishes JUST past the poll deadline
(max_wait_sec=1800 can
        undercut the job's --task-timeout=3600) is rescued by the ONE final post-timeout
marker check:
        the completed GPU result is ingested, nothing is cancelled, no timeout surfaces."""
        from backend.compute import cloud_run as cr

        exists_calls = {"n": 0}

        class _FakeStore:
            def exists(self, ref):
                exists_calls["n"] += 1
                return exists_calls["n"] >= 2  # absent for the poll, present on the final
check

        monkeypatch.setattr(cr, "get_storage", lambda backend, config=None: _FakeStore())

        def fake_fetch(uri, dst=None, *, config=None, overwrite=False):
            if dst is None:
                dst = os.path.join(tempfile.mkdtemp(prefix="cr-late-"), "m.json")
            with open(dst, "w", encoding="utf-8") as f:
                f.write('{"video_id": "vlate", "returncode": 0, "segment_count": 1}')
            return dst

        monkeypatch.setattr(cr, "fetch", fake_fetch)
        client = _NoopRunClient()
        backend = cr.CloudRunCompute(
            run_client=client, job_name="j", region="r", token="t", policy=_TIMEOUT_NOW
        )
        result = backend.run_infer(
            ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri="gs://b")
        )
        assert result.returncode == 0
        assert result.video_id == "vlate"  # the late result is ingested, not wasted
        assert client.cancelled == []  # a finished execution is never cancelled
        assert exists_calls["n"] == 2  # exactly ONE extra post-timeout check


    # ---------------------------------- execution-status fast negative path (worker crashed
pre-marker)

    # poll_every_n_sec=0.0 ? the poll spins as fast as possible; max_wait_sec=2.0 BOUNDS any
regression
    # (a fast path that silently fell back to marker-only polling fails in ~2s here, never a
65-min hang)
    # while the working fast path aborts on poll 0 ? instantly.
    _FAILFAST = ExecutionPolicy(poll_every_n_sec=0.0, op_timeout_sec=5.0, max_wait_sec=2.0)


    def test_run_infer_fails_fast_when_execution_terminal_without_marker(tmp_path):
        """The core fix: a worker execution that TERMINATED (Failed) without ever writing a
marker is
        caught by the Executions-API probe ? RemoteExecutionFailed FAST (poll 0), not a
~65-min opaque
        hang. The diagnostic names the execution, the terminal state, and the log-inspection
command."""
        client = _CrashedNoMarkerClient(state="Failed")
        backend = CloudRunCompute(
            run_client=client,
            job_name="rally-worker",
            region="asia-south1",
            policy=_FAILFAST,
            token="tok",
        )
        with pytest.raises(RemoteExecutionFailed) as exc:
```

```
            backend.run_infer(
                ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bucket"))
            )
        msg = str(exc.value)
        assert _CrashedNoMarkerClient.EXEC_NAME in msg  # operator can find the execution
        assert "Failed" in msg  # the terminal state is surfaced
        assert "gcloud run jobs executions describe" in msg  # actionable log pointer
        assert client.status_polls == 1  # aborted on the FIRST terminal observation (fast,
not spun)
        assert client.cancelled == []  # a finished execution is never cancelled (nothing to
cancel)


    def test_dispatch_remote_maps_terminal_without_marker_to_rc5(tmp_path):
        """The dispatch handler maps RemoteExecutionFailed to a distinct, clearly-labelled
rc 5 ? NOT
        the rc 4 poll-timeout path (a still-maybe-running execution) and NOT a raw traceback
? so
        'worker crashed before the marker' is operationally distinguishable and surfaces
immediately."""
        import argparse

        from backend.worker import __main__ as wmain

        client = _CrashedNoMarkerClient(state="Failed")
        backend = CloudRunCompute(
            run_client=client,
            job_name="rally-worker",
            region="asia-south1",
            policy=_FAILFAST,
            token="tok",
        )
        args = argparse.Namespace(
            video_uri="gs://b/v.mp4",
            out_uri=str(tmp_path / "bucket"),
            segmenter="wasb_hybrid",
            set=None,
        )
        rc = wmain._dispatch_remote(backend, args)
        assert rc == 5  # the crashed-pre-marker fast-fail code...
        assert rc != 4  # ...distinct from the poll-budget timeout
        assert client.status_polls == 1  # reached the fast path (not the timeout branch)


    def test_succeeded_state_without_marker_also_fails_fast(tmp_path):
        """Generalization: ANY terminal state with no marker is a guaranteed-futile wait. A
'Succeeded'
        execution that nonetheless wrote no marker (the marker upload itself failed) also
fails fast,
        with the state surfaced ? the whole 'execution over, no marker' class collapses, not
just Failed."""
        client = _CrashedNoMarkerClient(state="Succeeded")
        backend = CloudRunCompute(
            run_client=client,
            job_name="rally-worker",
            region="asia-south1",
            policy=_FAILFAST,
            token="tok",
        )
        with pytest.raises(RemoteExecutionFailed) as exc:
            backend.run_infer(
                ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bucket"))
            )
        assert "Succeeded" in str(exc.value)

```

```
539
540     def test_present_marker_wins_over_terminal_status(tmp_path):
541         """The done-marker stays AUTHORITATIVE: a worker that wrote a marker ? even a
FAILURE marker
542         (rc 2) ? resolves through the normal path with its REAL returncode, never a
synthesized
543         RemoteExecutionFailed, even when the execution status also reads terminal. Status is
a negative
544         path only; it must never override a real marker (which is checked first)."""
545
546         class _MarkerAndFailedStatus(_FakeRunClient):
547             def execution_terminal_state(self, execution):  # pragma: no cover - never
reached
548                 return "Failed"  # the marker resolves poll 0, so this is never consulted
549
550         client = _MarkerAndFailedStatus(video_id="abc", returncode=2)
551         backend = CloudRunCompute(
552             run_client=client, job_name="j", region="r", policy=_FAST, token="tok"
553         )
554         result = backend.run_infer(
555             ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bucket"))
556         )
557         assert result.returncode == 2  # the worker's REAL rc, not RemoteExecutionFailed
558         assert result.video_id == "abc"
559
560
561     def test_status_probe_error_is_swallowed_and_falls_back_to_marker_polling(tmp_path):
562         """Best-effort probe: a raising Executions API must NEVER fail a healthy run. The
wait degrades
563         to today's marker-only polling ? PolicyTimeout (the existing timeout path), NOT
564         RemoteExecutionFailed ? a flaky status API cannot manufacture a failure."""
565
566         class _RaisingStatusClient(_NeverDoneClient):
567             def execution_terminal_state(self, execution):
568                 raise RuntimeError("executions API unreachable")
569
570         client = _RaisingStatusClient()
571         backend = CloudRunCompute(
572             run_client=client,
573             job_name="rally-worker",
574             region="asia-south1",
575             policy=_TIMEOUT_NOW,
576             token="tok",
577         )
578         with pytest.raises(PolicyTimeout):  # swallowed probe error ? the unchanged timeout
path
579             backend.run_infer(
580                 ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bucket"))
581             )
582         assert client.cancelled == [_NeverDoneClient.EXEC_NAME]  # still cancels on the real
timeout
583
584
585     def test_default_execution_terminal_state_is_none():
586         """The ABC default is None (additive / default-OFF): a client that doesn't override
status
587         polling degrades transparently to marker-only behaviour, so every pre-existing fake
is
588         unaffected and the seam never changes a healthy run's outcome."""
589         assert _NoopRunClient().execution_terminal_state("whatever") is None
590         assert (
591             _NeverDoneClient().execution_terminal_state(_NeverDoneClient.EXEC_NAME) is None
592         )
593
594
```

```
    def test_real_client_terminal_state_none_for_unresolvable_name():
        """The real client's status probe is best-effort: a non-resolvable execution name
returns None
        (can't probe ? keep marker-polling), NOT a raise ? exercisable in CI (the SDK is
absent)."""
        from backend.compute.cloud_run import GoogleCloudRunJobClient

        client = GoogleCloudRunJobClient()
        for bogus in ("", "execution", "exec-123"):
            assert client.execution_terminal_state(bogus) is None


    def test_terminal_status_with_marker_written_in_race_resolves_via_marker(monkeypatch):
        """Concurrency race: the execution reads terminal exactly as the worker finishes
writing its
        marker. _poll_check re-checks the marker ONCE after a terminal status ? if it landed
in between,
        the completed result is ingested (the marker wins), NOT a spurious
RemoteExecutionFailed."""
        from backend.compute import cloud_run as cr

        exists_calls = {"n": 0}

        class _FakeStore:
            def exists(self, ref):
                exists_calls["n"] += 1
                # absent on the pre-status read, present on the ONE post-status race re-
check
                return exists_calls["n"] >= 2

        monkeypatch.setattr(cr, "get_storage", lambda backend, config=None: _FakeStore())

        def fake_fetch(uri, dst=None, *, config=None, overwrite=False):
            if dst is None:
                dst = os.path.join(tempfile.mkdtemp(prefix="cr-race-"), "m.json")
            with open(dst, "w", encoding="utf-8") as f:
                f.write('{"video_id": "vrace", "returncode": 0, "segment_count": 1}')
            return dst

        monkeypatch.setattr(cr, "fetch", fake_fetch)
        client = _CrashedNoMarkerClient(state="Failed")  # status says terminal-Failed...
        backend = cr.CloudRunCompute(
            run_client=client, job_name="j", region="r", token="t", policy=_FAILFAST
        )
        result = backend.run_infer(
            ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri="gs://b")
        )
        assert result.video_id == "vrace"  # ...but the marker that raced in WINS
        assert exists_calls["n"] == 2  # exactly the pre-status read + the ONE race re-check
        assert client.status_polls == 1  # status was consulted once, then the marker
superseded it


    def test_classify_execution_state_maps_proto_plus_fields():
        """Regression guard for the proto-plus `.seconds` bug: google-cloud-run marshals a
Timestamp to a
        datetime (which has `.second`, NOT `.seconds`) when set and to None when unset.
Terminality MUST
        key on `completion_time is None`, and the int per-task counts classify it. The
original
        `getattr(completion_time, "seconds", 0)` guard was ALWAYS 0 ? the real fast-fail
path was inert."""
        from datetime import datetime, timezone

        from backend.compute.cloud_run import _classify_execution_state
```

```
649
650         class _Exe:  # mimics a proto-plus run_v2.Execution (datetime completion_time + int
counts)
651             def __init__(
652                 self, completion_time=None, cancelled_count=0, failed_count=0,
succeeded_count=0
653             ):
654                 self.completion_time = completion_time
655                 self.cancelled_count = cancelled_count
656                 self.failed_count = failed_count
657                 self.succeeded_count = succeeded_count
658
659         done = datetime(2026, 7, 7, 12, 0, tzinfo=timezone.utc)
660         # a datetime has `.second`, never `.seconds` ? the exact reason the old guard mis-
fired
661         assert hasattr(done, "second") and not hasattr(done, "seconds")
662         # still pending/running (no completion timestamp) ? None, even with a stray count
set
663         assert _classify_execution_state(_Exe(completion_time=None)) is None
664         assert _classify_execution_state(_Exe(completion_time=None, failed_count=1)) is None
665         # terminal classifications
666         assert _classify_execution_state(_Exe(completion_time=done, failed_count=1)) ==
"Failed"
667         assert (
668             _classify_execution_state(_Exe(completion_time=done, cancelled_count=1)) ==
"Cancelled"
669         )
670         assert (
671             _classify_execution_state(_Exe(completion_time=done, succeeded_count=1)) ==
"Succeeded"
672         )
673         # cancel takes precedence over a co-incident failed_count
674         assert (
675             _classify_execution_state(
676                 _Exe(completion_time=done, cancelled_count=1, failed_count=1)
677             )
678             == "Cancelled"
679         )
680         # terminal but no positive count ? conservative Failed
681         assert _classify_execution_state(_Exe(completion_time=done)) == "Failed"
682
683
684     def test_classify_execution_state_against_real_proto_plus():
685         """Belt-and-suspenders: exercise the classifier against a REAL google-cloud-run
run_v2.Execution
686         so a future proto-plus marshaling change is caught. Skipped where the SDK isn't
installed (it's
687         intentionally NOT a CI dependency); the fake-based test above pins the shape for
CI."""
688         run_v2 = pytest.importorskip("google.cloud.run_v2")
689         from datetime import datetime, timezone
690
691         from backend.compute.cloud_run import _classify_execution_state
692
693         ts = datetime(2026, 7, 7, 12, 0, tzinfo=timezone.utc)
694         running = run_v2.Execution()  # completion_time unset ? proto-plus marshals to None
695         assert _classify_execution_state(running) is None
696
697         for count_attr, expected in (
698             ("failed_count", "Failed"),
699             ("cancelled_count", "Cancelled"),
700             ("succeeded_count", "Succeeded"),
701         ):
702             exe = run_v2.Execution()
703             exe.completion_time = ts
```

```
            setattr(exe, count_attr, 1)
            assert _classify_execution_state(exe) == expected


    def test_terminal_execution_state_none_for_empty_execution():
        """An empty/unknown execution name (run_job could not resolve one) can't be probed ?
None
        (degrade to marker-only polling), and the client's status API is never even
called."""
        class _Boom(_NoopRunClient):
            def execution_terminal_state(self, execution):  # pragma: no cover - must never
be called
                raise AssertionError("must not probe an empty execution name")

        backend = CloudRunCompute(run_client=_Boom(), job_name="j", region="r")
        assert backend._terminal_execution_state("") is None


    # ------------------------------------------------- worker wiring: dispatch branch +
done-marker


    def test_cmd_infer_dispatches_when_cloud_target(tmp_path, monkeypatch):
        """cmd_infer with compute_target=cloud_run hands off to the ComputeBackend (no in-
process run)."""
        from backend.compute.base import ComputeResult
        from backend.worker import __main__ as wmain

        seen = {}

        class _FakeBackend:
            def run_infer(self, spec):
                seen["spec"] = spec
                return ComputeResult(
                    returncode=0, video_id="vidX", outputs_uri="gs://b/outputs/vidX/"
                )

        monkeypatch.setattr(wmain, "get_compute_backend", lambda config: _FakeBackend())
        # cloud-serving startup checks require the AI-handoff key (AI_HANDOFF_KEY_MISSING is
fatal).
        monkeypatch.setenv("GEMINI_API_KEY", "test-key")
        cfg = tmp_path / "c.json"
        cfg.write_text(
            '{"deployment": {"profile": "cloud-serving"}, "storage": {"backend": "gcs"}}'
        )
        rc = wmain.cmd_infer(
            wmain.build_parser().parse_args(
                [
                    "infer",
                    "--video-uri",
                    "gs://b/videos/clip.mp4",
                    "--out-uri",
                    "gs://b",
                    "--config",
                    str(cfg),
                    "--segmenter",
                    "wasb_hybrid",
                ]
            )
        )
        assert rc == 0
        assert seen["spec"].video_uri == "gs://b/videos/clip.mp4"
        assert seen["spec"].segmenter == "wasb_hybrid"

```

```
763
764     def test_worker_writes_done_marker_on_success(tmp_path, monkeypatch):
765         """The container side: cmd_infer with --done-uri writes a completion marker
(video_id + rc) so a
766         remote dispatcher's bucket-poll terminates. Driven by the deterministic stub + skip-
AI."""
767         from backend.worker import __main__ as wmain
768
769         class _OK:
770             passed = True
771             validator_name = "fake"
772             message = "ok"
773
774             def model_dump(self):
775                 return {"validator_name": "fake", "passed": True, "message": "ok"}
776
777         monkeypatch.setattr(
778             wmain.validator_registry,
779             "run_all_with_context",
780             lambda path, cfg: ([_OK()], {}),
781         )
782         monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
783         monkeypatch.setattr(
784             "backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
785             lambda p: 30.0,
786         )
787
788         cfg = tmp_path / "c.json"
789         cfg.write_text(
790             json.dumps(
791                 {
792                     "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
793                     "output": {"output_dir": str(tmp_path / "o")},
794                 }
795             )
796         )
797         video = tmp_path / "in.mp4"
798         video.write_bytes(b"FAKEVIDEO")
799         out = tmp_path / "bucket"
800         done = str(out / "_dispatch" / "tok.done")
801
802         rc = wmain.cmd_infer(
803             wmain.build_parser().parse_args(
804                 [
805                     "infer",
806                     "--video-uri",
807                     str(video),
808                     "--out-uri",
809                     str(out),
810                     "--config",
811                     str(cfg),
812                     "--scratch",
813                     str(tmp_path / "s"),
814                     "--segmenter",
815                     "wasb_hybrid",
816                     "--done-uri",
817                     done,
818                 ]
819             )
820         )
821         assert rc == 0
822         marker = json.loads(open(done, encoding="utf-8").read())
823         assert marker["returncode"] == 0 and marker["status"] == "success"
824         assert marker["video_id"] and marker["segment_count"] >= 1
825         # CP-rebuild P3 (#485): the report records the SOURCE the detection ran on, so a
```

```
        # restart-wiped control plane re-hydrating from outputs/<md5>/ can restore the video
        # row's truthful path (not just the segments).
        report_path = out / "outputs" / marker["video_id"] / "report.json"
        report = json.loads(report_path.read_text(encoding="utf-8"))
        assert report["video_uri"] == str(video)


    def test_worker_no_done_uri_is_unchanged(tmp_path, monkeypatch):
        """DEFAULT-OFF: without --done-uri the inline run writes NO marker (today's
behaviour)."""
        from backend.worker import __main__ as wmain

        class _OK:
            passed = True
            validator_name = "fake"
            message = "ok"

            def model_dump(self):
                return {"validator_name": "fake", "passed": True, "message": "ok"}

        monkeypatch.setattr(
            wmain.validator_registry,
            "run_all_with_context",
            lambda path, cfg: ([_OK()], {}),
        )
        monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
        monkeypatch.setattr(
            "backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
            lambda p: 30.0,
        )
        cfg = tmp_path / "c.json"
        cfg.write_text(
            json.dumps(
                {
                    "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
                    "output": {"output_dir": str(tmp_path / "o")},
                }
            )
        )
        video = tmp_path / "in.mp4"
        video.write_bytes(b"FAKEVIDEO")
        out = tmp_path / "bucket"
        rc = wmain.cmd_infer(
            wmain.build_parser().parse_args(
                [
                    "infer",
                    "--video-uri",
                    str(video),
                    "--out-uri",
                    str(out),
                    "--config",
                    str(cfg),
                    "--scratch",
                    str(tmp_path / "s"),
                    "--segmenter",
                    "wasb_hybrid",
                ]
            )
        )
        assert rc == 0
        assert not (out / "_dispatch").exists()  # no marker dir when --done-uri is omitted


    def test_worker_writes_failure_marker_on_validation_fail(tmp_path, monkeypatch):
        """The container side: a worker-side FAILURE (validation rc=2) with --done-uri
```

```
writes a marker
890         carrying the REAL rc, so a remote dispatcher's bucket-poll terminates FAST (not a
30-min hang)."""
891         from backend.worker import __main__ as wmain
892
893         class _BAD:
894             passed = False
895             validator_name = "blur"
896             message = "blurry"
897
898             def model_dump(self):
899                 return {"validator_name": "blur", "passed": False, "message": "blurry"}
900
901         monkeypatch.setattr(
902             wmain.validator_registry,
903             "run_all_with_context",
904             lambda path, cfg: ([_BAD()], {}),
905         )
906         monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
907         cfg = tmp_path / "c.json"
908         cfg.write_text(
909             json.dumps(
910                 {
911                     "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
912                     "output": {"output_dir": str(tmp_path / "o")},
913                 }
914             )
915         )
916         video = tmp_path / "in.mp4"
917         video.write_bytes(b"FAKEVIDEO")
918         out = tmp_path / "bucket"
919         done = str(out / "_dispatch" / "tok.done")
920
921         rc = wmain.cmd_infer(
922             wmain.build_parser().parse_args(
923                 [
924                     "infer",
925                     "--video-uri",
926                     str(video),
927                     "--out-uri",
928                     str(out),
929                     "--config",
930                     str(cfg),
931                     "--scratch",
932                     str(tmp_path / "s"),
933                     "--segmenter",
934                     "wasb_hybrid",
935                     "--done-uri",
936                     done,
937                 ]
938             )
939         )
940         assert rc == 2  # validation failure
941         marker = json.loads(open(done, encoding="utf-8").read())
942         assert marker["returncode"] == 2 and marker["status"] == "failed"
943         assert marker[
944             "video_id"
945         ]  # carries the id so the dispatcher's poll terminates (rc != 0 ? failed)
946
947
948     def test_worker_failure_writes_no_marker_without_done_uri(tmp_path, monkeypatch):
949         """DEFAULT-OFF: a worker failure with NO --done-uri writes no marker (the unchanged
local run)."""
950         from backend.worker import __main__ as wmain
951
```

```
        class _BAD:
            passed = False
            validator_name = "blur"
            message = "blurry"

            def model_dump(self):
                return {"validator_name": "blur", "passed": False, "message": "blurry"}

        monkeypatch.setattr(
            wmain.validator_registry,
            "run_all_with_context",
            lambda path, cfg: ([_BAD()], {}),
        )
        monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
        cfg = tmp_path / "c.json"
        cfg.write_text(
            json.dumps(
                {
                    "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
                    "output": {"output_dir": str(tmp_path / "o")},
                }
            )
        )
        video = tmp_path / "in.mp4"
        video.write_bytes(b"FAKEVIDEO")
        out = tmp_path / "bucket"
        rc = wmain.cmd_infer(
            wmain.build_parser().parse_args(
                [
                    "infer",
                    "--video-uri",
                    str(video),
                    "--out-uri",
                    str(out),
                    "--config",
                    str(cfg),
                    "--scratch",
                    str(tmp_path / "s"),
                    "--segmenter",
                    "wasb_hybrid",
                ]
            )
        )
        assert rc == 2
        assert not (out / "_dispatch").exists()


    # ---------------------------------------------------------- Dockerfile structural guard
(no build)


    def test_dockerfile_has_required_directives():
        """CI can't build a CUDA image, so guard the Dockerfile's load-bearing directives
structurally:
        the GPU base, ffmpeg + DejaVu fonts (the watermark), the WASB conda env, and the
worker entrypoint."""
        from pathlib import Path

        text = Path("deploy/cloudrun/Dockerfile").read_text(encoding="utf-8")
        assert "FROM nvidia/cuda" in text  # GPU base (CUDA)
        assert "ffmpeg" in text  # stitch/decode
        assert "fonts-dejavu" in text  # the watermark filter font
        # The #506 modernized WASB stack (L4-validated): torch 2.6 cu124 + PyNvVideoCodec,
and
        # the NVDEC/NVENC driver userspace libs PyNvVideoCodec dlopens (compute-only GPU
```

```
hosts
1013        # inject neither) ? a silent drop of any of these bricks
decode_backend=nvdec_resident.
1014        assert "torch==2.6.0" in text
1015        assert "download.pytorch.org/whl/cu124" in text
1016        assert "PyNvVideoCodec" in text
1017        assert "libnvcuvid.so.1" in text and "libnvidia-encode.so.1" in text
1018        assert "torch==1.11.0+cu113" not in text  # the old stack is gone, not co-installed
1019        assert "WASB-SBDT" in text  # the shuttle detector repo
1020        assert 'ENTRYPOINT ["python3", "-m", "backend.worker"]' in text
1021        assert "WASB_PYTHON=/opt/conda/envs/wasb/bin/python" in text
1022        # The cloud worker reads the gs:// input + writes the gs:// outputs/done-marker
through
1023        # backend.storage (storage-gcs); and when this image is reused as the deployed
control plane it
1024        # dispatches the L4 Job via google.cloud.run_v2 (cloud-run) and verifies Cf-Access
JWTs when
1025        # edge-auth is on (auth ? PyJWT). A bare `pip install .` boots but dies on the first
gs:// fetch
1026        # (ModuleNotFoundError: google.cloud), on dispatch (google-cloud-run missing), or on
the first
1027        # edge-authed request (jwt missing) ? real Cloud Run failures the mock-client tests
can't see.
1028        # Lock the extras in structurally.
1029        assert ".[storage-gcs,cloud-run,auth]" in text
1030
1031
1032    def test_dockerfile_uses_micromamba_not_anaconda_tos():
1033        """P3a: the WASB env is built with micromamba + conda-forge (BSD, no ToS), NOT
Miniconda +
1034        `conda tos accept` (the ?200-employee Anaconda Terms-of-Service trap,
PACKAGING_PLAN.md §3).
1035        Locked structurally so a revert to the ToS path fails CI; the env path (WASB_PYTHON)
is unchanged."""
1036        from pathlib import Path
1037
1038        text = Path("deploy/cloudrun/Dockerfile").read_text(encoding="utf-8")
1039        assert "micromamba" in text  # the conda-forge env builder
1040        assert "conda-forge" in text  # the BSD, ToS-free channel
1041        assert (
1042            "MAMBA_ROOT_PREFIX=/opt/conda" in text
1043        )  # keeps the env at /opt/conda/envs/wasb
1044        assert "Miniconda3" not in text  # the Anaconda installer is gone
1045        assert "conda tos accept" not in text  # no ToS acceptance needed (conda-forge)
1046
1047
1048    def test_remote_poll_budget_comes_from_config_unless_policy_injected():
1049        """CP-rebuild §4.3: deployment.remote_poll_max_wait_sec drives the dispatch poll
budget
1050        (default 3900s > the Job's --task-timeout 3600s ? the old 1800s default could
abandon a
1051        slow-but-healthy paid GPU run), and an explicitly injected policy always wins."""
1052        from backend.infrastructure.execution_policy import DEFAULT_POLICY
1053
1054        cfg = AppConfig()
1055        cfg.deployment.compute_target = "cloud_run"
1056
1057        be = get_compute_backend(cfg, run_client=_FakeRunClient())
1058        assert be is not None
1059        assert be._policy.max_wait_sec == 3900.0
1060        # Only the budget is derived ? the other resilience knobs stay at their defaults.
1061        assert be._policy.poll_every_n_sec == DEFAULT_POLICY.poll_every_n_sec
1062        assert be._policy.op_timeout_sec == DEFAULT_POLICY.op_timeout_sec
1063
1064        cfg.deployment.remote_poll_max_wait_sec = 42.0
```

```
1065        assert get_compute_backend(cfg, run_client=_FakeRunClient())._policy.max_wait_sec ==
42.0
1066
1067        # An injected policy is never overridden by config.
1068        be = get_compute_backend(cfg, run_client=_FakeRunClient(), policy=_FAST)
1069        assert be._policy.max_wait_sec == _FAST.max_wait_sec
1070
1071
1072    def test_run_infer_forwards_on_wait_into_the_bucket_poll(tmp_path, monkeypatch):
1073        """The #482 heartbeat plumbing: the on_wait callback handed to run_infer must be the
1074        exact on_tick the bucket-poll drives (cloud_run threads it through poll_until)."""
1075        from backend.compute import cloud_run as cr
1076
1077        seen = {}
1078
1079        def fake_poll(check, policy, *, label="op", logger=None, on_tick=None, **kw):
1080            seen["on_tick"] = on_tick
1081            return {"video_id": "vidCloud", "returncode": 0}
1082
1083        monkeypatch.setattr(cr, "poll_until", fake_poll)
1084        client = _FakeRunClient()  # still writes report.json so the post-marker read
succeeds
1085        backend = CloudRunCompute(
1086            run_client=client,
1087            job_name="rally-worker",
1088            region="asia-southeast1",
1089            storage_config={},
1090            policy=_FAST,
1091            token="tok",
1092        )
1093
1094        def heartbeat(_elapsed):
1095            pass
1096
1097        result = backend.run_infer(
1098            ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bk")),
1099            on_wait=heartbeat,
1100        )
1101        assert seen["on_tick"] is heartbeat
1102        assert result.video_id == "vidCloud"
1103
```

49. assistant (2026-07-08T05:17:02.444Z)

I have all the patterns I need. Let me check `backend/main.py` module-level side effects (to decide whether the worker can reuse the `_main.VideoCompiler` seam) and how the compile job is tested.

50. user (2026-07-08T05:17:03.958Z)

```
1       import argparse
2       import json
3       import logging
4       import os
5       import shutil
6       import sys
7       import tempfile
8       import threading
9       import uuid
10      from importlib.metadata import PackageNotFoundError
11      from importlib.metadata import version as _pkg_version
12      from urllib.parse import quote
13
14      import uvicorn
15      from dotenv import load_dotenv
```

```
16
17      # PACKAGING_PLAN.md P0a: prefer an app-home .env ONLY when RALLY_APP_HOME is set (the
packaged
18      # launcher pins it); otherwise env_file_to_load() returns None and load_dotenv(None)
keeps the
19      # exact historical frame-relative walk-up ? byte-identical to the pre-P0a bare
load_dotenv().
20      # stdlib-only import ? keeps this pre-pydantic line light.
21      from backend.utils.app_home import env_file_to_load, resolve_output_dir
22
23      load_dotenv(env_file_to_load())
24
25      # Centralized logging setup for the indexer (hot paths use logger, CLI output uses print
for direct user-facing messages).
26      logging.basicConfig(
27          level=logging.INFO,
28          format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
29          datefmt="%H:%M:%S",
30      )
31      logger = logging.getLogger(__name__)
32      from typing import Any, Dict, List, Optional
33
34      from fastapi import FastAPI, HTTPException, Request
35      from fastapi.middleware.cors import CORSMiddleware
36      from fastapi.responses import (
37          FileResponse,
38          HTMLResponse,
39          JSONResponse,
40          RedirectResponse,
41      )
42      from fastapi.staticfiles import StaticFiles
43      from pydantic import BaseModel
44      from starlette.middleware.trustedhost import TrustedHostMiddleware
45
46      from backend import (
47          billing,  # M8: per-job cost ledger (USD internal / INR display)
48          storage,  # M2: URI-aware input acquisition (local = identity passthrough)
49      )
50      from backend.api import (
51          auth as edge_auth,  # M9: Cloudflare Access edge-auth (default-OFF)
52      )
53      from backend.api import (
54          jobs as jobs_api,  # /api/jobs list route (router) + submit-dedup helper (#448)
55      )
56      from backend.config import (
57          AppConfig,
58          StartupCheckError,
59          enforce_startup_checks,
60          write_light_default_config,  # P2b: batteries-included first-run config seeding
61      )
62      from backend.config import (  # new typed config
63          load_config as load_app_config,
64      )
65      from backend.infrastructure.database import Database
66      from backend.pipeline.segmenters.base import segmenter_registry
67      from backend.pipeline.stitcher.compiler import VideoCompiler
68      from backend.pipeline.validators.base import registry
69      from backend.providers import provider_registry
70      from backend.reporting import emit_indexer_report
71      from backend.results import Failure  # standardized error/result contract
72
73      # Server/network bootstrap helpers (CORS/host-allowlist config + the `indexer --app`
launcher)
74      # live in backend.server_runtime (P24 god-module split). Imported HERE, above `app =
FastAPI(...)`,
```

```
75      # because the module-level add_middleware(...) below calls
_cors_origins_from_env/_allowed_hosts_from_env
76      # at import time; re-exported so `import backend.main as m; m._resolve_bind_host(...)`
keeps working.
77      from backend.server_runtime import (  # noqa: F401  (re-exports: names stay addressable
on backend.main)
78          _allowed_hosts_from_env,
79          _cors_origins_from_env,
80          _find_free_port,
81          _open_browser_when_ready,
82          _port_is_free,
83          _resolve_app_port,
84          _resolve_bind_host,
85      )
86      from backend.storage.capabilities import (
87          storage_capabilities,  # P3: secret-free "which media can this box use" report
88      )
89      from backend.utils.ffmpeg import (  # P2a: single ffmpeg/ffprobe resolver
90          ffmpeg_bin,
91          ffprobe_bin,
92      )
93      from backend.utils.freespace import require_free_bytes  # HTTP-507 disk guard (P2, D11)
94      from backend.utils.hashing import compute_video_id  # canonical file-identity hashing
95      from backend.utils.paths import (
96          output_base,
97          resolve_under_root,
98          safe_output_filename,
99          validate_input_path,
100     )
```

51. user (2026-07-08T05:17:05.556Z)

```
480:def test_create_compile_job_persists_kind_and_params(tmp_path):
481:    """#329: an async compile job rides the jobs table with kind='compile' + a JSON params
payload
484:    assert db.create_compile_job(
488:    assert job["kind"] == "compile"
498:def test_worker_dispatches_compile_kind_to_execute_compile(env):
499:    """The claimed-job dispatcher routes kind='compile' to _execute_compile (NOT
_execute_processing),
512:    mp.setattr(m, "_execute_compile", _fake_compile)
518:    db.create_compile_job("cj2", "vidX", "/v.mp4", [1], "r.mp4")
567:    db.create_compile_job("cmp_q", "vid1", "/v.mp4", [1, 2], "reel_q.mp4")
568:    db.create_compile_job("cmp_r", "vid1", "/v.mp4", [1, 2], "reel_r.mp4")
570:    db.create_compile_job("cmp_done", "vid1", "/v.mp4", [1, 2], "reel_done.mp4")
683:    db.create_compile_job("c1", "vid", str(vid), [1], "reel.mp4")  # active compile, owner
None
```

52. assistant (2026-07-08T05:17:33.537Z)

`backend.main` is importable without creating a DB (heavy but fine on the worker image). Let me check how `_execute_compile`'s stitch is currently tested (VideoCompiler seam) so my refactor keeps those green.

53. user (2026-07-08T05:17:36.420Z)

```
=== VideoCompiler / _execute_compile / _stitch usages in tests ===
tests/test_api_endpoints.py
tests/test_async_jobs.py
tests/test_compiler.py
tests/test_generate_highlights.py
tests/test_main_module_budget.py
tests/test_upload_download.py
tests/test_watermark.py
```

```
tests/test_watermark_i18n.py
tests/__pycache__/test_async_jobs.cpython-313-pytest-9.0.3.pyc
tests/__pycache__/test_compiler.cpython-313-pytest-9.0.3.pyc
=== compile docs dir ===
architecture.svg
README.md
runlogs
TESTING_STRATEGY.md
=== localize_watermark ===
"""Localized reel-watermark text + font selection.

The SPA passes the UI locale to ``POST /api/compile``; this maps it to a localized branding
string
and a script-appropriate bundled font, so a Kannada/Hindi/? user's reel carries the watermark in
their language. The brand token ("KhelSutra.guru") stays Latin; only the surrounding phrase
localizes.

Fonts (``backend/assets/fonts/``):
  - en/es/da/id (Latin)  ? the configured default (bundled RobotoSlab, Apache-2.0)
  - kn ? Noto Sans Kannada · hi ? Noto Sans Devanagari  (OFL-1.1, bundled ? the D7 OFL-for-media
    carve-out; FRIENDLY_AND_MULTILINGUAL §4 anticipated this).

Honest fallbacks: an unknown/missing locale (or a locale with no entry, incl. ``en``) returns
the
watermark UNCHANGED, so existing/default behavior is preserved. If a script font file is somehow
absent, ``font_path`` is left at the default (RobotoSlab) rather than pointing at a missing file
?
and the compiler already retries the encode without the watermark if drawtext can't render, so a
font gap can never nuke the reel. Kept config-package-agnostic (operates on a dict).
"""

from pathlib import Path
from typing import Any, Dict, Optional

## This module is backend/pipeline/stitcher/ ? parents[2] == backend/ (matches compiler

_FONTS_DIR = Path(__file__).resolve().parents[2] / "assets" / "fonts"

## Localized watermark phrase per locale. AI-drafted ? native-speaker review pending (sa
## the SPA catalogs). `en` is intentionally absent: it falls through to the configured defaul

WATERMARK_TEXT: Dict[str, str] = {
    "es": "Generado con KhelSutra.guru",
    "da": "Lavet med KhelSutra.guru",
    "id": "Dibuat dengan KhelSutra.guru",
    "kn": "KhelSutra.guru ???? ??????????",
    "hi": "KhelSutra.guru ?? ????? ???",
}

## Script-specific bundled fonts ? only where the Latin default (RobotoSlab) can't render
## Latin locales (es/da/id) localize the TEXT but keep the default font.

FONT_FILE_BY_LOCALE: Dict[str, str] = {
    "kn": "NotoSansKannada-Variable.ttf",
    "hi": "NotoSansDevanagari-Variable.ttf",
}
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

54. user (2026-07-08T05:17:42.793Z)

```
1       """VideoCompiler coverage (previously zero ? the user-facing highlight-reel output path,
2       where the f-string SyntaxError lived undetected). ffmpeg/ffprobe are mocked."""
3
4       import os
```

```
    import subprocess
    from unittest.mock import MagicMock, patch

    import pytest

    from backend.pipeline.stitcher.compiler import VideoCompiler


    def test_parse_time_formats():
        f = VideoCompiler._parse_time
        assert f(12.5) == 12.5
        assert f("12.5") == 12.5
        assert f("1:30") == 90.0
        assert f("1:00:00") == 3600.0
        assert f("garbage") == 0.0


    def test_merge_overlapping_collapses_duplicates_and_overlaps():
        m = VideoCompiler._merge_overlapping
        # Exact duplicates + near-duplicates (the "every rally twice" re-run bug) collapse
to one each.
        assert m([(28.2, 34.5), (28.5, 34.5), (40.5, 45.5), (40.5, 45.5)]) == [
            (28.2, 34.5),
            (40.5, 45.5),
        ]
        # Unsorted input is sorted; an overlapping range extends the previous.
        assert m([(100.0, 110.0), (0.0, 5.0), (104.0, 120.0)]) == [
            (0.0, 5.0),
            (100.0, 120.0),
        ]
        # Genuinely distinct (gapped) rallies are preserved, not merged.
        assert m([(0.0, 5.0), (6.0, 9.0)]) == [(0.0, 5.0), (6.0, 9.0)]
        # Degenerate rows (end <= start) are dropped; string timestamps are accepted.
        assert m([("0:10", "0:20"), (30.0, 30.0), (31.0, 29.0)]) == [(10.0, 20.0)]


    def test_compile_dedups_before_stitching(tmp_path):
        """A segment list with every rally twice must yield ONE clip per unique rally."""
        comp = VideoCompiler(str(tmp_path / "out"))
        raw = tmp_path / "raw.mp4"
        raw.write_bytes(b"x")
        trims = []

        def fake_run(cmd, *a, **k):
            if cmd[0] == "ffprobe":
                return MagicMock(stdout="100.0\n", returncode=0)
            out = cmd[-1]
            if isinstance(out, str) and out.endswith(".mp4"):
                open(out, "wb").write(b"clip")
                # count per-clip trims (the concat list file input is not an .mp4 output)
                if "-i" in cmd and cmd[cmd.index("-i") + 1] == str(raw):
                    trims.append(cmd)
            return MagicMock(returncode=0, stdout=b"", stderr=b"")

        dup_segments = [(0.0, 5.0), (0.2, 5.0), (10.0, 15.0), (10.0, 15.0), (20.0, 25.0)]
        with patch.object(subprocess, "run", side_effect=fake_run):
            comp.compile_highlights(str(raw), dup_segments, "reel.mp4")
        # 5 input segments collapse to 3 unique rallies -> 3 per-clip trims, not 5.
        assert len(trims) == 3, f"expected 3 deduped trims, got {len(trims)}"


    def test_compile_basenames_output_and_stitches(tmp_path):
        comp = VideoCompiler(str(tmp_path / "out"))
        raw = tmp_path / "raw.mp4"
        raw.write_bytes(b"x")
```

```
69
70          calls = []
71
72          def fake_run(cmd, *a, **k):
73              calls.append(cmd)
74              # Create whatever output file ffmpeg/ffprobe was told to write so existence
checks pass.
75              if cmd[0] == "ffprobe":
```

55. user (2026-07-08T05:17:47.405Z)

```
495         assert db.get_job("pj")["kind"] is None
496
497
498     def test_worker_dispatches_compile_kind_to_execute_compile(env):
499         """The claimed-job dispatcher routes kind='compile' to _execute_compile (NOT
_execute_processing),
500         records the result as 'done', and skips the process-only cost/flywheel hooks."""
501         _client, db, _tmp_path, mp = env
502         seen = {}
503
504         def _fake_compile(cfg, db, job, progress=None):
505             seen["id"] = job["id"]
506             return {
507                 "status": "success",
508                 "video_id": "vidX",
509                 "download_url": "/api/highlights/vidX/r.mp4",
510             }
511
512         mp.setattr(m, "_execute_compile", _fake_compile)
513         mp.setattr(
514             m,
515             "_execute_processing",
516             lambda *a, **k: pytest.fail("a compile job must not hit _execute_processing"),
517         )
518         db.create_compile_job("cj2", "vidX", "/v.mp4", [1], "r.mp4")
519         m._run_claimed_job(db.get_job("cj2"), m.global_config, m.db_instance)
520         assert seen["id"] == "cj2"
521         done = db.get_job("cj2")
522         assert done["status"] == "done"
523         assert done["result"]["download_url"] == "/api/highlights/vidX/r.mp4"
524
525
526     def test_create_job_persists_and_reconstructs_compute(tmp_path):
527         """v8: the compute-picker choice must round-trip create_job ? get_job ?
_job_to_request so the
528         worker honours it across the async submit?reconstruct boundary. NULL ? server
default (the
529         pre-picker default-OFF behaviour). Regression guard for the dropped-across-boundary
bug."""
530         db = Database(str(tmp_path / "jobs.db"))
531         db.create_job("j-cloud", "gs://b/clip.mp4", compute="cloud")
532         db.create_job("j-default", "/v.mp4")  # no compute ? NULL ? server default
533         assert db.get_job("j-cloud")["compute"] == "cloud"
534         assert db.get_job("j-default")["compute"] is None
535         assert m._job_to_request(db.get_job("j-cloud")).compute == "cloud"
536         assert m._job_to_request(db.get_job("j-default")).compute is None
537
538
539     # --- crash recovery ----------------------------------------------------------
```

56. assistant (2026-07-08T05:18:30.420Z)

I have everything I need. This is a large, coherent change. Let me track it with a task list, then implement ? starting with the config layer.

57. user (2026-07-08T05:18:32.152Z)

```
context OK
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

58. assistant (2026-07-08T05:18:56.314Z)

Shell cwd resets each call (I'll use absolute worktree paths `C:/kg-wt521/...` everywhere). Let me read the worktree copies of files I'll edit (to register them for editing) ? starting with the config `__init__` which I haven't seen yet.

59. user (2026-07-08T05:18:56.384Z)

```
1       """Configuration package.
2
3       Public API:
4           from backend.config import load_config, AppConfig, Config
5
6       This package is the canonical place for all configuration access.
7       """
8
9       import os
10
11      from .loader import (
12          LIGHT_DEFAULT_CONFIG,
13          load_config,
14          save_default_config,
15          write_light_default_config,
16      )
17      from .models import (  # re-export common ones
18          AppConfig,
19          Config,
20          DeploymentConfig,
21          IndexingConfig,
22          StitchingConfig,
23          StorageConfig,
24          ValidationConfig,
25      )
26      from .presets import PROFILE_PRESETS, probe_cuda_available, suggest_profile
27      from .startup import (
28          StartupCheck,
29          StartupCheckError,
30          enforce_startup_checks,
31          run_startup_checks,
32      )
33
34
35      def default_db_path(config_path: str = "config.json") -> str:
36          """Resolve the DB path from the typed config ? the single shared helper.
37
38          Replaces the copy-pasted ``_default_db_path()`` in ``run_eval.py`` and
39          ``run_regression.py``.
40          """
41          cfg = load_config(config_path)
42          return os.path.join(cfg.output.output_dir, cfg.output.db_name)
43
44
45      __all__ = [
46          "load_config",
47          "save_default_config",
48          "write_light_default_config",
49          "LIGHT_DEFAULT_CONFIG",
50          "default_db_path",
51          "AppConfig",
52          "Config",
```

```
53          "DeploymentConfig",
54          "IndexingConfig",
55          "StitchingConfig",
56          "StorageConfig",
57          "ValidationConfig",
58          "PROFILE_PRESETS",
59          "suggest_profile",
60          "probe_cuda_available",
61          "StartupCheck",
62          "StartupCheckError",
63          "enforce_startup_checks",
64          "run_startup_checks",
65      ]
66
```

60. assistant (2026-07-08T05:18:59.523Z)

Now let me implement. First, the new placement resolver module:

61. assistant (2026-07-08T05:19:13.203Z)

```
"""Phase ? machine placement resolution (#521).

A *phase placement* answers one question per pipeline phase ? **where does the heavy compute for
this phase run?** ? with a config knob (``deployment.phase_placement``) instead of code. The
three
placeable phases and their values:

    validation  ?  control_plane | cloud_run_l4
    segment     ?  control_plane | cloud_run_l4   (a.k.a. "detect")
    compile     ?  control_plane | cloud_run_l4   (a.k.a. "stitch")

``control_plane`` runs the phase IN-PROCESS on the (CPU-only) control-plane service;
``cloud_run_l4``
dispatches it to the GPU/NVENC Cloud Run worker Job (the same L4 detection already uses). This
module
is the SINGLE source of truth so ``backend.orchestration`` (segment + compile) and the worker
never
re-derive the mapping.

**Default = today's behaviour (pure / additive).** When a phase's knob is unset (``None``) the
resolver reproduces the pre-#521 placement:

    validation  ?  control_plane                                    (the authoritative gate,
#512/#513)
    segment     ?  cloud_run_l4 IFF deployment.compute_target == "cloud_run", else control_plane
    compile     ?  control_plane                                    (the in-process ffmpeg
stitch)

so a config with no ``phase_placement`` block behaves byte-identically to before this module
existed.
"""

from __future__ import annotations

from typing import Any

# Phase identities (kept as constants so callers don't stringly-type the phase names).
PHASE_VALIDATION = "validation"
PHASE_SEGMENT = "segment"
PHASE_COMPILE = "compile"
PHASES: tuple[str, ...] = (PHASE_VALIDATION, PHASE_SEGMENT, PHASE_COMPILE)
```

Placement targets.

```
PLACE_CONTROL_PLANE = "control_plane"
PLACE_CLOUD_L4 = "cloud_run_l4"
PLACEMENTS: tuple[str, ...] = (PLACE_CONTROL_PLANE, PLACE_CLOUD_L4)


def resolve_phase_placement(cfg: Any, phase: str) -> str:
    """Where does ``phase`` run ? ``"control_plane"`` or ``"cloud_run_l4"``?

    Precedence: an explicit ``deployment.phase_placement.<phase>`` wins; otherwise the legacy
default
    (mirrors the pre-#521 behaviour, see the module docstring). Never raises ? an absent
    ``deployment`` / ``phase_placement`` degrades to the legacy default.
    """
    deployment = getattr(cfg, "deployment", None)
    placement = getattr(deployment, "phase_placement", None)
    explicit = getattr(placement, phase, None) if placement is not None else None
    if explicit:
        return str(explicit)
    # Legacy defaults ? reproduce today's placement exactly.
    if phase == PHASE_SEGMENT:
        compute_target = (getattr(deployment, "compute_target", "") or "").strip()
        return PLACE_CLOUD_L4 if compute_target == "cloud_run" else PLACE_CONTROL_PLANE
    # validation + compile default to the control-plane (their pre-#521 home).
    return PLACE_CONTROL_PLANE
```

62. user (2026-07-08T05:19:13.235Z)

```
File created successfully at: C:\kg-wt521\backend\config\placement.py (file state is current in
your context ? no need to Read it back)
```

63. assistant (2026-07-08T05:19:18.274Z)

Now edit the config models. Let me register the worktree file with targeted reads, then edit.

64. user (2026-07-08T05:19:19.264Z)

```
354     class StitchingConfig(BaseModel):
355         begin_padding: float = 0.5
356         end_padding: float = 0.5
357         use_cache: bool = False
358         min_shot_count: int = 1
359         # Long-rally reel filter (seconds; 0 = OFF). Drop detected rally pairs whose
duration
360         # (end ? start) is below this from the highlight reel, BEFORE compiling. A cleaner
361         # reel-selection knob than `min_shot_count`: shot_count is unlabeled / "Unknown" on
the
362         # fully-local no-AI path, whereas duration is derived from the rally's own
start/end. The
363         # local detector over-fires and its false positives are mostly SHORT (motion blips,
364         # background-court fragments), so this keeps the reel to the eye-catching long
rallies.
365         # OFF by default ? the detector output is unchanged, only which rallies reach the
reel.
366         min_rally_duration: float = Field(default=0.0, ge=0.0)
367         watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)
368
```

65. user (2026-07-08T05:19:19.743Z)

```
454     class DeploymentConfig(BaseModel):
455         """Deployment profile ? one switch that selects a coherent bundle of compute +
storage
456         knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md``
```

```
§4.3).
457
458         **Pure / additive:** the empty default profile (``""``) applies NO preset, so the
pipeline
459         behaves exactly as before this section existed. A profile is opt-in (``config.json``
460         ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); auto-detect only ever
461         *suggests* one (D15 ? cloud spend is never entered silently). Nothing in the core
reads
462         these fields yet (M1): selecting a profile works purely by pre-setting the EXISTING
knobs
463         (``indexing.detector_impl`` / ``skip_ai_handoff``, ``storage.backend``) in the
loader.
464
465         Valid values mirror ``backend/config/presets.py`` (parity-pinned by a unit test)."""
466
467         # "" = no profile (today's manual knobs). Others apply presets.PROFILE_PRESETS.
468         profile: Literal["", "truly-local", "cloud-serving", "cpu-mock"] = ""
469         # Where the core runs. Set by the profile preset; overridable in config.json. Inert
in M1
470         # (recorded for observability; consumed by the ComputeBackend seam from M2+).
471         compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""
472         # Cloud-serving throughput cap (#466): how many DETECT jobs the control-plane drain
may dispatch +
473         # bucket-poll CONCURRENTLY when compute_target=="cloud_run". Each is an I/O-bound
Cloud Run Job
474         # execution (the GPU work is on ephemeral L4s, WASB-only has no Gemini), so ? unlike
a truly-local
475         # single box ? they need not serialize. Bounds concurrent PAID L4 executions. Non-
cloud_run targets
476         # ignore this and stay serial (max_workers=1), preserving the truly-local GPU/Gemini
serialization.
477         max_concurrent_remote_jobs: int = Field(default=4, ge=1, le=64)
478         # Poll budget for one dispatched remote job (CONTROL_PLANE_DURABLE_REBUILD §4.3).
Must sit
479         # ABOVE the worker Job's own --task-timeout (3600s) so the worker's timeout stays
the
480         # authority and the dispatcher's one-shot post-timeout marker rescue still applies ?
the old
481         # 1800s ExecutionPolicy default undercut real 1080p60 jobs (~22 min detect +
queue/cold-start,
482         # observed on the 2026-07-05 owner CUJ) and could abandon a slow-but-healthy paid
GPU run.
483         remote_poll_max_wait_sec: float = Field(default=3900.0, gt=0)
```

66. assistant (2026-07-08T05:19:23.358Z)

Now the StitchingConfig encoder field:

67. user (2026-07-08T05:19:29.794Z)

The file C:\kg-wt521\backend\config\models.py has been updated successfully. (file state is current in your context ? no need to Read it back)

68. assistant (2026-07-08T05:19:33.470Z)

Now add the `PhasePlacementConfig` model and wire it into `DeploymentConfig` with the encoder knob and the footgun guard:

69. user (2026-07-08T05:19:40.671Z)

The file C:\kg-wt521\backend\config\models.py has been updated successfully. (file state is current in your context ? no need to Read it back)

70. user (2026-07-08T05:19:53.301Z)

The file C:\kg-wt521\backend\config\models.py has been updated successfully. (file state is current in your context ? no need to Read it back)

71. assistant (2026-07-08T05:19:56.153Z)

Now export the resolver and new model from the config package:

72. user (2026-07-08T05:19:59.936Z)

The file C:\kg-wt521\backend\config__init__.py has been updated successfully. (file state is current in your context ? no need to Read it back)

73. user (2026-07-08T05:20:04.846Z)

The file C:\kg-wt521\backend\config__init__.py has been updated successfully. (file state is current in your context ? no need to Read it back)

74. assistant (2026-07-08T05:20:08.941Z)

Now set compile?cloud_run_l4 in the cloud-serving preset:

75. user (2026-07-08T05:20:09.549Z)

```
51          "cloud-serving": {
52              "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
53              # Linux L4 (cuda); Gemini handoff ON until the owned model clears the #172 floor
(D11).
54              # wasb GPU-feed tuning: the parity defaults (batch 8, no prefetch) left the paid
L4
55              # <1% utilized on the first real serving run (2026-07-03) ? the pipeline was
56              # CPU-feed-bound. Serving opts into the measured fast values; local/WSL
unchanged.
57              "indexing": {
58                  "detector_impl": "native",
59                  "skip_ai_handoff": False,
60                  # GPU-resident NVDEC decode (#506/#507): validated on a real L4 (golden gate
PASS,
61                  # F1 0.574 ? baseline 0.582 @ SERVED) AND on the live Cloud Run L4 (the
served
62                  # trajectory reproduced the VM run ? 26479 ? 26481 visible on GX010128),
then rolled
63                  # out + canaried (DEPLOY_REPORT_cloud-serving_gpu-resident-
nvdec_2026-07-07). This makes
64                  # nvdec the code-level serving default so it survives an image rebuild; the
worker's
65                  # WASB_DECODE_BACKEND env was the reversible rollout lever. cuda-only
(validator-enforced).
66                  "wasb": {
67                      "batch_size": 64,
68                      "prefetch_batches": 4,
69                      "decode_backend": "nvdec_resident",
70                  },
71              },
72              "storage": {"backend": "gcs"},
73          },
```

76. user (2026-07-08T05:20:20.983Z)

The file C:\kg-wt521\backend\config\presets.py has been updated successfully. (file state is current in your context ? no need to Read it back)

77. assistant (2026-07-08T05:20:30.706Z)

Now the compute layer. Let me add `CompileJobSpec` and `run_compile` to the base:

78. user (2026-07-08T05:20:31.395Z)

```
52
53
54      @dataclass(frozen=True)
55      class ComputeResult:
56          """The terminal state of a dispatched job. ``returncode`` mirrors the worker's exit
code
57          (0 ok / 2 validation-or-config / 3 segmenter-failure), plus **4 = remote dispatch
timeout or an
58          empty/unreadable completion marker** (the cloud path only). ``video_id`` is known
once the job's
59          done-marker is read; ``outputs_uri`` is where ``intervals.csv`` / ``report.json``
landed."""
60
61          returncode: int
62          outputs_uri: str = ""
63          video_id: str = ""
64          report: dict = field(default_factory=dict)
65          # The backend's execution identity for PROVENANCE (e.g. the Cloud Run execution
name) ? surfaced
66          # on the job result so "the cloud path actually ran" is independently verifiable
(gcloud run jobs
67          # executions describe <execution>). Empty for the in-process path / backends without
one.
68          execution: str = ""
69
70
71      class ComputeBackend(ABC):
72          """Backend-agnostic compute. ``run_infer`` dispatches ONE inference job and returns
only once
73          its outputs are durably available (bucket-poll for cloud), so a caller treats local
and cloud
74          execution identically."""
75
76          @abstractmethod
77          def run_infer(
78              self,
79              spec: ComputeJobSpec,
80              *,
81              on_wait: "Callable[[float], None] | None" = None,
82          ) -> ComputeResult:
83              """Run ``spec`` to completion and return its ``ComputeResult``.
84
85              ``on_wait(elapsed_sec)`` (optional) is invoked periodically while the backend is
86              still WAITING on the remote work ? the caller's live-progress heartbeat (#482),
so a
87              long GPU run doesn't look frozen. Implementations without a wait loop may ignore
it."""
88
```

79. user (2026-07-08T05:20:40.427Z)

```
The file C:\kg-wt521\backend\compute\base.py has been updated successfully. (file state is
current in your context ? no need to Read it back)
```

80. user (2026-07-08T05:20:48.301Z)

```
The file C:\kg-wt521\backend\compute\base.py has been updated successfully. (file state is
current in your context ? no need to Read it back)
```

81. assistant (2026-07-08T05:20:50.883Z)

Now export `CompileJobSpec` from the compute package:

82. user (2026-07-08T05:20:54.593Z)

<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>

83. user (2026-07-08T05:20:57.796Z)

```
16      from backend.compute.base import (
17          ComputeBackend,
18          ComputeJobSpec,
19          ComputeResult,
20          RemoteExecutionFailed,
21      )
22      from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy
23
24      __all__ = [
25          "ComputeBackend",
26          "ComputeJobSpec",
27          "ComputeResult",
28          "RemoteExecutionFailed",
29          "get_compute_backend",
30      ]
```

84. user (2026-07-08T05:21:04.505Z)

The file C:\kg-wt521\backend\compute__init__.py has been updated successfully. (file state is current in your context ? no need to Read it back)

85. assistant (2026-07-08T05:21:09.444Z)

Now the `CloudRunCompute` ? extract a shared marker-await helper and add `run_compile`. Let me register and read the worktree file's relevant regions:

86. user (2026-07-08T05:21:10.281Z)

```
244         def run_infer(
245             self,
246             spec: ComputeJobSpec,
247             *,
248             on_wait: "Callable[[float], None] | None" = None,
249         ) -> ComputeResult:
250             out_root = spec.out_uri.rstrip("/")
251             token = self._token or uuid.uuid4().hex
252             done_uri = f"{out_root}/_dispatch/{token}.done"
253             argv: List[str] = [
254                 "infer",
255                 "--video-uri",
256                 spec.video_uri,
257                 "--out-uri",
258                 spec.out_uri,
259                 "--done-uri",
260                 done_uri,
261             ]
262             if spec.segmenter:
263                 argv += ["--segmenter", spec.segmenter]
264             if spec.skip_validation:
265                 # The control-plane already validated + only dispatches on PASS ? tell the
worker not to
266                 # re-decode and re-validate (redundant, and its config may resolve different
thresholds).
267                 argv.append("--skip-validation")
268             for kv in (*spec.config_overrides, *self._container_overrides):
269                 argv += ["--set", kv]
270
271             execution = self._client.run_job(
```

```
272                 self._job_name, argv, region=self._region, project=self._project
273             )
274             LOG.info(
275                 "cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
276                 self._job_name,
277                 execution,
278                 done_uri,
279             )
280
281             try:
282                 marker = poll_until(
283                     lambda: self._poll_check(done_uri, str(execution or "")),
284                     self._policy,
285                     label="cloud-run-infer",
286                     logger=LOG,
287                     # Live heartbeat (#482): each still-waiting tick reaches the caller so
288                     # job.progress moves during the whole GPU run instead of freezing.
289                     on_tick=on_wait,
290                 )
291             except PolicyTimeout as timeout_exc:
292                 # RemoteExecutionFailed (the fast negative path in _poll_check) is NOT a
PolicyTimeout,
293                 # so it propagates past here uncaught ? the dispatch handler maps it to a
distinct,
294                 # clearly-labelled failure instead of the ~65-min opaque timeout this branch
handles.
295                 marker = self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
296             video_id = str(marker.get("video_id", ""))
297             # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
298             # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
299             rc = int(marker.get("returncode", 1))
300             if not video_id:
301                 LOG.warning(
302                     "cloud-run: job=%s completion marker present but empty/unreadable (no "
303                     "video_id) ? treating as failure",
304                     self._job_name,
305                 )
306                 rc = rc or 1
307             outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
308             report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
309             LOG.info(
310                 "cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id, rc
311             )
312             return ComputeResult(
313                 returncode=rc,
314                 outputs_uri=outputs_uri,
315                 video_id=video_id,
316                 report=report,
317                 execution=str(execution or ""),
318             )
```

87. user (2026-07-08T05:21:18.458Z)

The file C:\kg-wt521\backend\compute\cloud_run.py has been updated successfully. (file state is current in your context ? no need to Read it back)

88. assistant (2026-07-08T05:21:20.895Z)

Now add `_await_marker` and `run_compile` after `run_infer` (before `_poll_check`):

89. user (2026-07-08T05:21:36.491Z)

The file C:\kg-wt521\backend\compute\cloud_run.py has been updated successfully. (file state is current in your context ? no need to Read it back)

90. assistant (2026-07-08T05:21:39.404Z)

Now import `CompileJobSpec` into cloud_run.py:

91. user (2026-07-08T05:21:41.185Z)

The file C:\kg-wt521\backend\compute\cloud_run.py has been updated successfully. (file state is current in your context ? no need to Read it back)

92. assistant (2026-07-08T05:21:47.282Z)

Now the VideoCompiler NVENC encoder support. Let me register the worktree file and edit the constructor, add encoder selection, and use it in the trim:

93. user (2026-07-08T05:21:48.213Z)

```
27      class VideoCompiler:
28          def __init__(
29              self,
30              output_dir: str,
31              end_padding: float = 1.0,
32              begin_padding: float = 0.5,
33              watermark: Optional[Any] = None,
34          ):
35              self.output_dir = output_dir
36              self.end_padding = end_padding
37              self.begin_padding = begin_padding
38              # Optional branding overlay. Accepts a WatermarkConfig model or a plain dict;
39              # normalized to a dict (or None) so this module stays config-package-agnostic.
40              self.watermark = self._normalize_watermark(watermark)
41              os.makedirs(output_dir, exist_ok=True)
42
```

94. user (2026-07-08T05:21:56.522Z)

The file C:\kg-wt521\backend\pipeline\stitcher\compiler.py has been updated successfully. (file state is current in your context ? no need to Read it back)

95. assistant (2026-07-08T05:22:03.969Z)

Now add the encoder-selection logic. First a module-level probe cache, then the selection method:

96. user (2026-07-08T05:22:13.977Z)

The file C:\kg-wt521\backend\pipeline\stitcher\compiler.py has been updated successfully. (file state is current in your context ? no need to Read it back)

97. assistant (2026-07-08T05:22:16.596Z)

Now add the `_select_encoder` method (before `_trim_one_segment`) and thread the resolved codec args through the trim:

98. user (2026-07-08T05:22:27.278Z)

The file C:\kg-wt521\backend\pipeline\stitcher\compiler.py has been updated successfully. (file state is current in your context ? no need to Read it back)

99. assistant (2026-07-08T05:22:29.726Z)

Now use the codec args in the `base_cmd`:

100. user (2026-07-08T05:22:34.809Z)

The file C:\kg-wt521\backend\pipeline\stitcher\compiler.py has been updated successfully. (file state is current in your context ? no need to Read it back)

101. assistant (2026-07-08T05:22:36.849Z)

Now resolve the encoder once in `compile_highlights` and pass it to each trim:

102. user (2026-07-08T05:22:43.519Z)

The file C:\kg-wt521\backend\pipeline\stitcher\compiler.py has been updated successfully. (file state is current in your context ? no need to Read it back)

103. assistant (2026-07-08T05:22:49.769Z)

Now the core: `backend/orchestration.py`. Let me register the worktree file and add the placement import:

104. user (2026-07-08T05:22:50.322Z)

```
21      import csv
22      import json
23      import logging
24      import math
25      import os
26      import re
27      import tempfile
28      import threading
29      from typing import TYPE_CHECKING, Any, Dict, List, Optional, Tuple
30      from urllib.parse import quote
31
32      from backend import storage
33      from backend.config import AppConfig, StitchingConfig
34      from backend.pipeline.segmenters.base import segmenter_registry
35      from backend.pipeline.stitcher.watermark_i18n import localize_watermark
36      from backend.pipeline.validators.base import ValidationResult, registry
37      from backend.results import Failure
38      from backend.utils.paths import output_base, safe_output_filename
```

105. user (2026-07-08T05:22:54.362Z)

The file C:\kg-wt521\backend\orchestration.py has been updated successfully. (file state is current in your context ? no need to Read it back)

106. assistant (2026-07-08T05:22:58.513Z)

Now add the `_will_detect_on_cloud` predicate after `_cloud_available`:

107. user (2026-07-08T05:23:05.745Z)

The file C:\kg-wt521\backend\orchestration.py has been updated successfully. (file state is current in your context ? no need to Read it back)

108. assistant (2026-07-08T05:23:10.786Z)

Now update `_maybe_dispatch_remote`'s default target branch to consult segment placement:

109. user (2026-07-08T05:23:16.434Z)

The file C:\kg-wt521\backend\orchestration.py has been updated successfully. (file state is

current in your context ? no need to Read it back)

110. assistant (2026-07-08T05:23:19.566Z)

Now the `skip_validation` derivation in the `ComputeJobSpec`:

111. user (2026-07-08T05:23:24.732Z)

The file C:\kg-wt521\backend\orchestration.py has been updated successfully. (file state is current in your context ? no need to Read it back)

112. assistant (2026-07-08T05:23:28.496Z)

Now wire the validation-delegation into `_execute_processing`. Let me read the current validation block region:

113. user (2026-07-08T05:23:29.215Z)

```
795             storage.cleanup_acquired_local_input(
796                 req.video_path, local_video, getattr(config, "storage", None)
797             )
798             raise
799         try:
800             existing_video = db.get_video(video_id)
801         except Exception:
802             storage.cleanup_acquired_local_input(
803                 req.video_path, local_video, getattr(config, "storage", None)
804             )
805             raise
806         was_reingest = existing_video is not None
807
808         # typed AppConfig: attribute access for the default segmenter (with safe fallback)
809         default_seg = getattr(
810             getattr(config, "indexing", None), "default_segmenter", "motion"
811         )
812         seg_name = req.segmenter or default_seg
813         segmenter_actual = None
814         segmentation_ran = False
815         val_results: List[Any] = []
816         val_context: Dict[str, Any] = {}
817         # Compute-path PROVENANCE (the validation method): which backend actually ran the
DETECT ?
818         # "in_process" once the local segmenter is reached, set on the response as
"cloud_run" by
819         # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
fail).
820         compute_actual: Optional[str] = None
821         response: Optional[Dict[str, Any]] = None
822         try:
823             if (
824                 existing_video
825                 and existing_video.get("status") == "processed"
826                 and not req.force
827             ):
828                 cached_segments = db.query_play_segments(video_id, {})
829                 if cached_segments:
830                     logger.info(
831                         "[API] Reusing cached segmentation for video_id=%s; force=true
bypasses cache.",
832                         video_id,
833                     )
834                     response = _cached_process_payload(
835                         video_id,
836                         existing_video.get("validation_results", []),
```

```
                        cached_segments,
                        getattr(req, "compute", None),
                    )
                    return response

            # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for
the report) ?
            #    UNLESS this exact content already has a cached verdict under the current
validator config.
            notify("validating")
            # VALIDATION-RESULT CACHE: video_id is the MD5 of the file bytes, so an
identical-content
            # reupload keys the same row. When that content was ALREADY validated under the
SAME validator
            # configuration, reuse the stored verdict instead of re-decoding the clip
through the whole
            # validator suite ? minutes on a heavy 4K clip on the CPU-only control-plane,
which the
            # 2026-07-07 CUJ re-paid on every retry after a segmentation failure (validation
passed but
            # detection failed ? no cached segments, so the whole-pipeline cache above
misses and we land
            # here). The fingerprint folds in the validation.* config + sport + validator
set + a schema
            # version, so ANY config change misses and re-validates (never serve a stale
PASS/FAIL).
            # ``force=true`` bypasses the reuse and re-writes a fresh verdict below (mirrors
how it bypasses
            # the whole-pipeline cache). Composes with #513 (the worker skips its own re-
validation on a
            # cloud dispatch), so a post-failure reupload reaches segmentation in ~0
validation time.
            val_fingerprint = registry.config_fingerprint(config)
            # The whole read ? fingerprint-match ? replay is guarded: ANY cache fault
(unreadable row,
            # corrupt or schema-incompatible stored results) degrades to a real validation
rather than
            # failing the job. The cache is a pure speedup ? never a new failure mode.
            reused_validation = False
            if not req.force:
                try:
                    cached = db.get_validation_cache(video_id)
                    if cached is not None and cached.get("fingerprint") == val_fingerprint:
                        # Replay the stored ValidationResult list; val_context stays {} (no
fresh
                        # ffprobe_meta ? the report's media block degrades gracefully to
size-only, the
                        # deliberate cost of skipping the decode). Segmentation reads
neither of these.
                        val_results = [
                            ValidationResult(**r) for r in cached.get("results", [])
                        ]
                        reused_validation = True
                        logger.info(
                            "[API] Reusing cached validation verdict (passed=%s) for
video_id=%s; "
                            "validator config unchanged ? skipping the validator suite. "
                            "force=true re-validates.",
                            cached.get("passed"),
                            video_id,
                        )
                except Exception:  # noqa: BLE001 ? any cache read/replay fault degrades to
a real validation
                    reused_validation = False
                    logger.warning(
```

```
882                         "[API] validation-cache reuse failed for video_id=%s ? re-
validating.",
883                         video_id,
884                         exc_info=True,
885                     )
886             if not reused_validation:
887                 logger.info(f"Running validations for {req.video_path}...")
888                 val_results, val_context = registry.run_all_with_context(local_video,
config)
889             failures = [r for r in val_results if not r.passed]
890             if not reused_validation:
891                 # Persist the verdict keyed by (content, config) so the NEXT identical
reupload skips the
892                 # decode. Best-effort (the writer swallows + logs its own faults); a miss
just re-validates.
893                 db.set_validation_cache(
894                     video_id,
```

114. user (2026-07-08T05:23:33.726Z)

```
886             if not reused_validation:
887                 logger.info(f"Running validations for {req.video_path}...")
888                 val_results, val_context = registry.run_all_with_context(local_video,
config)
889             failures = [r for r in val_results if not r.passed]
890             if not reused_validation:
891                 # Persist the verdict keyed by (content, config) so the NEXT identical
reupload skips the
892                 # decode. Best-effort (the writer swallows + logs its own faults); a miss
just re-validates.
893                 db.set_validation_cache(
894                     video_id,
895                     val_fingerprint,
896                     not failures,
897                     [r.model_dump() for r in val_results],
898                 )
899
900             if failures:
901                 # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
902                 # status; it no longer destroys the video's prior good segments.
903                 db.add_video(
904                     video_id,
905                     req.video_path,
906                     "failed",
907                     [r.model_dump() for r in val_results],
908                 )
909                 response = {
910                     "video_id": video_id,
911                     "status": "failed",
912                     "message": _validation_failure_message(failures),
913                     "results": [r.model_dump() for r in val_results],
914                 }
915                 return response
916
917             # 2. Run segmenter & indexer
918             logger.info("[API] Video passed checks. Segmenting and indexing...")
919             db.add_video(
920                 video_id, req.video_path, "processed", [r.model_dump() for r in val_results]
921             )
922
923             segmenter_cls = segmenter_registry.get(seg_name)
924             if not segmenter_cls:
925                 # Submit-time validation already 400s unknown names; this is the defensive
```

115. assistant (2026-07-08T05:24:00.876Z)

I'll gate the validator suite on a `validate_on_control_plane` flag (delegating to the worker when validation is placed on the L4). Let me make targeted edits:

116. user (2026-07-08T05:24:10.406Z)

The file C:\kg-wt521\backend\orchestration.py has been updated successfully. (file state is current in your context ? no need to Read it back)

117. user (2026-07-08T05:24:16.669Z)

The file C:\kg-wt521\backend\orchestration.py has been updated successfully. (file state is current in your context ? no need to Read it back)

118. assistant (2026-07-08T05:24:21.219Z)

Now the compile dispatch. Let me read the current `_execute_compile` region in the worktree (shifted by earlier edits):

119. user (2026-07-08T05:24:21.833Z)

```
1155        return f"{root.rstrip('/')}/highlights/{video_id}/{name}"
1156
1157
1158    def _storage_settings(cfg) -> Optional[Dict[str, Any]]:
1159        """The active storage settings dict (per-backend config for the storage layer ? S3
endpoint/region,
1160        GCS project/client/signed_url_ttl), or None. Threaded into storage calls so a non-
default backend
1161        isn't silently constructed as a fresh default."""
1162        sc = getattr(cfg, "storage", None)
1163        return sc.model_dump() if sc is not None else None
1164
1165
1166    def signed_reel_url(cfg, video_id: str, name: str) -> Optional[str]:
1167        """A short-lived signed URL for a reel mirrored to the output bucket, so
/api/highlights can
1168        307-redirect off the app's response path (bypassing Cloud Run's ~32 MiB cap + the
ephemeral local
1169        /tmp; #463). None ? no bucket copy ? serve the local file. Uses the active storage
config for BOTH
1170        the existence check and the signing (extracted here to keep backend/main.py lean ?
P23/P24)."""
1171        reel_uri = reel_object_uri(cfg, video_id, name)
1172        if not reel_uri:
1173            return None
1174        settings = _storage_settings(cfg)
1175        if storage.exists(reel_uri, config=settings):
1176            return storage.signed_url(reel_uri, config=settings, method="GET")
1177        return None
1178
1179
1180    def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:
1181        """Worker handler for a ``kind='compile'`` job (#329): re-resolve the segments,
stitch the reel,
1182        and return the {status, filepath, download_url} result the SPA polls for. Returns a
1183        ``status='failed'`` result for an EXPECTED problem (missing video / no segments / a
stitch error)
1184        so the SPA always gets a clean verdict instead of a worker-crash error."""
1185        import backend.main as _main  # call-time seam: VideoCompiler stays patchable on
backend.main
1186
1187        notify = progress or (lambda stage: None)
1188        params = json.loads(job.get("params") or "{}")
1189        video_id = job.get("video_id") or ""
```

```
        segment_ids = params.get("segment_ids") or []
        locale = params.get("locale")

        notify("resolving")
        try:
            video, kept, out_name = _resolve_compile(
                db, cfg, video_id, segment_ids, params.get("output_filename", "")
            )
        except _CompileError as e:
            return {"status": "failed", "message": e.detail, "video_id": video_id}

        stitching = getattr(cfg, "stitching", None) or StitchingConfig()
        output_dir = (
            getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
        )
        # Localize the branding watermark to the UI locale the SPA rendered in (recorded on
the job);
        # unknown/absent locale keeps the configured default unchanged.
        localized_watermark = localize_watermark(stitching.watermark, locale)
        compiler = _main.VideoCompiler(
            output_dir,
            begin_padding=stitching.begin_padding,
            end_padding=stitching.end_padding,
            watermark=localized_watermark,
        )
        time_pairs = [(s["start_time"], s["end_time"]) for s in kept]

        notify("stitching")
        local_src = ""
        try:
            # The stored `filepath` is the SOURCE ref (gs://, s3://, gdrive://, or a local
path). Resolve to
            # a LOCAL file the stitcher can read ? identity for a local path, fetch-to-
scratch for a URI (the
            # same seam the process path uses); without it, compiling a bucket-ingested
video can't open gs://.
            local_src = storage.acquire_local_input(
                video["filepath"], getattr(cfg, "storage", None)
            )
            # The ffmpeg stitch is CPU-heavy ? serialize it so a raised drain worker count
(for remote
            # dispatch) doesn't run multiple compiles at once on the box (see
_LOCAL_COMPUTE_LANE).
            with _LOCAL_COMPUTE_LANE:
                out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
        except Exception as e:  # noqa: BLE001 ? a stitch/ffmpeg failure is a clean job
FAILURE, not a crash
            logger.error(
                "[compile] stitching failed for video_id=%s: %s", video_id, e, exc_info=True
            )
            return {
                "status": "failed",
                "message": f"Highlight stitching failed: {e}",
                "video_id": video_id,
            }
        finally:
            if local_src:
                storage.cleanup_acquired_local_input(
                    video["filepath"], local_src, getattr(cfg, "storage", None)
                )

        # #463: mirror the reel to the output bucket so /api/highlights can 307 to a signed
URL ? Cloud Run
        # caps a buffered response at ~32 MiB (a real reel is bigger) and the local /tmp
copy dies when the
```

```
1246        # instance recycles. A mirror-upload failure must NOT fail an otherwise-good stitch
(the endpoint
1247        # still falls back to the local file on the same instance).
1248        reel_uri = reel_object_uri(cfg, video_id, out_name)
1249        if reel_uri:
1250            try:
1251                notify("uploading reel")
1252                # Thread the active storage settings so the configured backend (S3
endpoint/region, GCS
1253                # project/client) is used, not a fresh default.
1254                storage.put(out_path, reel_uri, config=_storage_settings(cfg))
1255            except Exception as e:  # noqa: BLE001 ? best-effort mirror; local stitch
already succeeded
1256                logger.warning("[compile] could not mirror reel to %s: %s", reel_uri, e)
1257
1258        # `filepath` is server-local (not browser-reachable); also return the download URL
so the SPA needn't
1259        # hand-build it (P6 / PER_RALLY_SELECTION.md §4.3).
1260        download_url = f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
1261        notify("finished")
1262        return {
1263            "status": "success",
1264            "filepath": out_path,
```

120. user (2026-07-08T05:24:24.779Z)

```
1262        return {
1263            "status": "success",
1264            "filepath": out_path,
1265            "download_url": download_url,
1266            "video_id": video_id,
1267        }
1268
```

121. assistant (2026-07-08T05:25:23.516Z)

Now I'll refactor `_execute_compile` ? extract the shared `_stitch_reel` core, add the placement branch, and the `_dispatch_compile_remote` handler. Let me replace the whole function:

122. user (2026-07-08T05:26:25.553Z)

The file C:\kg-wt521\backend\orchestration.py has been updated successfully. (file state is current in your context ? no need to Read it back)

123. assistant (2026-07-08T05:26:38.297Z)

Let me check the `backend.main` re-export block to add the new orchestration symbols for addressability:

124. user (2026-07-08T05:26:39.177Z)

```
226     )
227
228     # The processing/compile ORCHESTRATION (hash?validate?segment?report pipeline, Cloud Run
229     # dispatch+ingest, compile resolution + worker handler) now lives in
backend.orchestration
230     # (P23 god-module split; re-cut of #390). Re-exported here so the names stay addressable
on
231     # backend.main ? `import backend.main as m; m._execute_processing(...)`, the job
worker's
232     # call-time `_main._execute_compile(...)` seam, and monkeypatch.setattr(backend.main,
...)
233     # all keep working unchanged.
234     from backend.orchestration import (  # noqa: F401  (re-exports: names stay addressable
```

```
on backend.main)
235         _cloud_available,
236         _CompileError,
237         _execute_compile,
238         _execute_processing,
239         _finalize_reingest,
240         _ingest_remote_segments,
241         _maybe_dispatch_remote,
242         _remote_input_size_bytes,
243         _report_config_for_input,
244         _resolve_compile,
245         read_path_rehydrate,
246         signed_reel_url,
247     )
248
249
250     # Legacy thin wrapper for any remaining direct calls during transition.
251     # New code should import load_app_config / AppConfig from backend.config directly.
252     def load_config(config_path: str = "config.json") -> Dict[str, Any]:
253         cfg = load_app_config(config_path)
254         return cfg.model_dump(mode="json")
255
256
257     # --- API Models ---
```

125. user (2026-07-08T05:26:45.055Z)

```
The file C:\kg-wt521\backend\main.py has been updated successfully. (file state is current in
your context ? no need to Read it back)
```

126. assistant (2026-07-08T05:27:35.071Z)

Now the worker `compile` subcommand. Let me register the worktree worker file and add `cmd_compile` + helpers after `cmd_infer`:

127. user (2026-07-08T05:27:36.037Z)

```
415         if getattr(args, "done_uri", None):
416             _write_done_marker(
417                 args.done_uri, video_id, 0, len(segments), status, storage_cfg, scratch
418             )
419         return 0
420
421
422     #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a
423     #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
collision).
424     _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")
425
426     #: Leading ISO-date prefix on a canonical label name (``labels/<YYYY-MM-
DD>_<name>.rallies.csv``,
```

128. user (2026-07-08T05:27:59.483Z)

```
The file C:\kg-wt521\backend\worker\__main__.py has been updated successfully. (file state is
current in your context ? no need to Read it back)
```

129. assistant (2026-07-08T05:28:03.203Z)

Now add the `compile` subparser. Let me read the parser region and add it after the infer subparser:

130. user (2026-07-08T05:28:04.261Z)

```
852             metavar="k=v",
853             help="config override, e.g. --set indexing.skip_ai_handoff=true",
854         )
855         pi.add_argument(
856             "--skip-validation",
857             action="store_true",
858             help="skip the worker's re-validation: the dispatcher (control-plane) already
ran the "
859             "validators with its resolved config and only dispatches on PASS, so re-
validating here is "
860             "redundant, decodes the video again, and can contradict the gate. Default-OFF:
the direct "
861             "`worker infer` CLI still validates.",
862         )
863         pi.set_defaults(func=cmd_infer)
864
865         pt = sub.add_parser(
866             "train",
867             help="Gen-0 train-from-bucket (labels [+videos] -> trajectories -> harness)",
868         )
869         pt.add_argument(
```

131. user (2026-07-08T05:28:13.542Z)

```
The file C:\kg-wt521\backend\worker\__main__.py has been updated successfully. (file state is
current in your context ? no need to Read it back)
```

132. assistant (2026-07-08T05:28:17.273Z)

```
Core implementation done. Let me run a quick import + lint smoke, then the existing target tests
to catch regressions:
```

133. user (2026-07-08T05:28:22.701Z)

```
placement defaults: control_plane control_plane control_plane
imports OK
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

134. user (2026-07-08T05:28:30.795Z)

```
cloud compile: cloud_run_l4
cloud segment: cloud_run_l4
cloud validation: control_plane
GUARD OK: 1 validation error for AppConfig
deployment
  Value error, d
validation+cloud-segment OK
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

135. assistant (2026-07-08T05:28:33.876Z)

```
The resolver and guard work. Now let me run the existing target tests to check for regressions:
```

136. user (2026-07-08T05:29:43.698Z)

```
E
E         Omitting 4 identical items, use -vv to show
E         Left contains 2 more items:
E         {'cloud_stitch_encoder': 'h264_nvenc',
E          'phase_placement': {'compile': None, 'segment': None, 'validation': None}}
E         Use -v to get more diff

tests\test_config_deployment.py:395: AssertionError
============================== warnings summary ==============================
```

```
tests/test_api_cloud_dispatch.py::test_full_api_process_to_query_via_cloud
tests/test_api_cloud_dispatch.py::test_full_api_process_to_query_via_cloud
tests/test_api_cloud_dispatch.py::test_local_override_survives_api_boundary
tests/test_api_cloud_dispatch.py::test_local_override_survives_api_boundary
tests/test_api_cloud_dispatch.py::test_cloud_override_survives_api_boundary_on_local_server
tests/test_api_cloud_dispatch.py::test_cloud_override_survives_api_boundary_on_local_server
tests/test_api_cloud_dispatch.py::test_full_api_process_job_result_carries_cloud_provenance
tests/test_api_cloud_dispatch.py::test_full_api_process_job_result_carries_cloud_provenance
  C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
    warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
========================== short test summary info ==========================
FAILED tests/test_config_deployment.py::test_default_config_dump_only_adds_deployment_section
FAILED tests/test_config_deployment.py::test_save_default_config_includes_deployment
2 failed, 195 passed, 8 warnings in 64.34s (0:01:04)
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

137. assistant (2026-07-08T05:29:48.024Z)

Only 2 failures ? both are config-shape snapshot tests that need the new additive fields. Let me read and update them:

138. user (2026-07-08T05:29:49.424Z)

```
63:def test_default_config_dump_only_adds_deployment_section():
392:def test_save_default_config_includes_deployment(tmp_path):
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

139. user (2026-07-08T05:29:52.089Z)

```
63      def test_default_config_dump_only_adds_deployment_section():
64          """Adding the section must not perturb any other default ? the dump equals a fresh
65          AppConfig()'s dump, and the new section is present with empty values."""
66          dumped = load_config(NONEXISTENT).model_dump(mode="json")
67          assert dumped == AppConfig().model_dump(mode="json")
68          assert dumped["deployment"] == {
69              "profile": "",
70              "compute_target": "",
71              "max_concurrent_remote_jobs": 4,
72              "remote_poll_max_wait_sec": 3900.0,
73          }
74
75
76      # ---------------------------------------------------------------------------------------
77      # Parity: the model Literal must match the presets table (no drift).
78      # ---------------------------------------------------------------------------------------
79      def test_profile_literal_matches_presets():
80          import typing
81
82          literal_profiles = set(
83              typing.get_args(DeploymentConfig.model_fields["profile"].annotation)
84          )
85          assert literal_profiles == {"", *PROFILES}
86          assert set(PROFILE_PRESETS) == set(PROFILES)
87          assert set(COMPUTE_TARGET_FOR_PROFILE) == set(PROFILES)
88
89
90      def test_compute_target_literal_matches_presets():
91          import typing
```

```
92
93          literal_targets = set(
94              typing.get_args(DeploymentConfig.model_fields["compute_target"].annotation)
95          )
96          assert literal_targets == {"", *COMPUTE_TARGETS}
97          assert set(COMPUTE_TARGET_FOR_PROFILE.values()) == set(COMPUTE_TARGETS)
98
99
100     def test_each_preset_only_touches_existing_knobs():
101         """A preset must only set knobs that exist on the typed models ? otherwise it would
be a
102         silent no-op. Validates the merged result against AppConfig for every profile."""
```

140. user (2026-07-08T05:30:01.916Z)

```
The file C:\kg-wt521\tests\test_config_deployment.py has been updated successfully. (file state
is current in your context ? no need to Read it back)
```

141. user (2026-07-08T05:30:04.968Z)

```
395     def test_save_default_config_includes_deployment(tmp_path):
396         p = save_default_config(str(tmp_path / "out.json"))
397         saved = json.loads(p.read_text())
398         assert saved["deployment"] == {
399             "profile": "",
400             "compute_target": "",
401             "max_concurrent_remote_jobs": 4,
402             "remote_poll_max_wait_sec": 3900.0,
403         }
404         cfg = load_config(str(p))
405         assert cfg.deployment.profile == ""
406
407
408     def test_repo_config_json_has_no_profile():
409         """The shipped config.json must not silently opt into a profile (default path stays
today's
410         behaviour for the real deployment)."""
411         repo_root = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
412         cfg_path = os.path.join(repo_root, "config.json")
413         if not os.path.exists(cfg_path):
414             pytest.skip("no repo config.json")
415         cfg = load_config(cfg_path)
416         assert cfg.deployment.profile == ""
417
418
419     def test_remote_poll_max_wait_default_and_validation():
420         """CP-rebuild §4.3: the dispatch poll budget must outlast the worker Job's
421         --task-timeout (3600s) by default, and a non-positive budget is a config error."""
422         from pydantic import ValidationError
423
424         from backend.config.models import DeploymentConfig
```

142. user (2026-07-08T05:30:10.312Z)

```
The file C:\kg-wt521\tests\test_config_deployment.py has been updated successfully. (file state
is current in your context ? no need to Read it back)
```

143. assistant (2026-07-08T05:30:16.989Z)

```
Config-shape tests fixed. Now let me write the new tests. First, the phase-placement resolver +
guard tests:
```

144. assistant (2026-07-08T05:30:38.981Z)

```
"""Config-driven phase ? machine placement (#521).

Proves the ``deployment.phase_placement`` knob + ``resolve_phase_placement`` resolver: the
legacy
defaults reproduce today's placement exactly (validation + compile on the control-plane, segment
on
the L4 iff compute_target==cloud_run), an explicit knob overrides, the cloud-serving preset
places
compile on the L4, and the "nobody validates" footgun is rejected at config load.
"""

import pytest

from backend.config import (
    PHASE_COMPILE,
    PHASE_SEGMENT,
    PHASE_VALIDATION,
    PLACE_CLOUD_L4,
    PLACE_CONTROL_PLANE,
    AppConfig,
    load_config,
    resolve_phase_placement,
)
```

---------- legacy defaults (byte-identical)

```
def test_default_placement_is_todays_behaviour():
    """No phase_placement block ? validation + compile on the control-plane, segment on the
    control-plane (compute_target unset). Byte-identical to pre-#521."""
    cfg = AppConfig()
    assert resolve_phase_placement(cfg, PHASE_VALIDATION) == PLACE_CONTROL_PLANE
    assert resolve_phase_placement(cfg, PHASE_SEGMENT) == PLACE_CONTROL_PLANE
    assert resolve_phase_placement(cfg, PHASE_COMPILE) == PLACE_CONTROL_PLANE


def test_segment_default_follows_compute_target():
    """The legacy segment default is cloud IFF compute_target==cloud_run ? so a cloud_run server
    keeps dispatching DETECT without any phase_placement config."""
    cloud = AppConfig.model_validate({"deployment": {"compute_target": "cloud_run"}})
    assert resolve_phase_placement(cloud, PHASE_SEGMENT) == PLACE_CLOUD_L4
    # validation + compile still default to the control-plane on a cloud_run server (unset
knobs).
    assert resolve_phase_placement(cloud, PHASE_VALIDATION) == PLACE_CONTROL_PLANE
    assert resolve_phase_placement(cloud, PHASE_COMPILE) == PLACE_CONTROL_PLANE

    local = AppConfig.model_validate({"deployment": {"compute_target": "local_gpu"}})
    assert resolve_phase_placement(local, PHASE_SEGMENT) == PLACE_CONTROL_PLANE
```

---------- explicit overrides

```
def test_explicit_knob_overrides_default():
    """Each phase's explicit knob wins over the legacy default."""
    cfg = AppConfig.model_validate(
        {
            "deployment": {
                "compute_target": "cloud_run",
                "phase_placement": {
                    "segment": "control_plane",  # force DETECT back on the control-plane
                    "compile": "cloud_run_l4",  # move stitch to the L4
```

```
                },
            }
        }
    )
    assert resolve_phase_placement(cfg, PHASE_SEGMENT) == PLACE_CONTROL_PLANE
    assert resolve_phase_placement(cfg, PHASE_COMPILE) == PLACE_CLOUD_L4


def test_resolver_never_raises_on_missing_deployment():
    """A config-ish object with no deployment/phase_placement degrades to the legacy default."""

    class _Bare:
        pass

    assert resolve_phase_placement(_Bare(), PHASE_COMPILE) == PLACE_CONTROL_PLANE
    assert resolve_phase_placement(_Bare(), PHASE_SEGMENT) == PLACE_CONTROL_PLANE
```

---------- the cloud-serving preset

```
def test_cloud_serving_preset_places_compile_on_l4(tmp_path):
    """The cloud-serving profile preset moves compile to the L4 (the #521 headline) while
keeping
    validation on the control-plane and segment on the L4 (via compute_target)."""
    cfg_path = tmp_path / "c.json"
    cfg_path.write_text('{"deployment": {"profile": "cloud-serving"}}')
    cfg = load_config(str(cfg_path))
    assert resolve_phase_placement(cfg, PHASE_COMPILE) == PLACE_CLOUD_L4
    assert resolve_phase_placement(cfg, PHASE_SEGMENT) == PLACE_CLOUD_L4
    assert resolve_phase_placement(cfg, PHASE_VALIDATION) == PLACE_CONTROL_PLANE
```

---------- the "nobody validates" guard

```
def test_validation_on_l4_requires_cloud_segment():
    """validation=cloud_run_l4 with segment on the control-plane would leave NOBODY validating
(the
    control-plane skips + no worker runs) ? rejected at config load."""
    with pytest.raises(Exception, match="phase_placement.validation"):
        AppConfig.model_validate(
            {
                "deployment": {
                    "compute_target": "",
                    "phase_placement": {
                        "validation": "cloud_run_l4",
                        "segment": "control_plane",
                    },
                }
            }
        )


def test_validation_on_l4_ok_with_cloud_segment():
    """validation=cloud_run_l4 is accepted when segment also runs on the L4 (explicit)."""
    cfg = AppConfig.model_validate(
        {
            "deployment": {
                "phase_placement": {
                    "validation": "cloud_run_l4",
                    "segment": "cloud_run_l4",
                }
```

```
            }
        }
    )
    assert resolve_phase_placement(cfg, PHASE_VALIDATION) == PLACE_CLOUD_L4


def test_validation_on_l4_ok_when_compute_target_is_cloud_run():
    """The guard honours the LEGACY segment default: with compute_target=cloud_run the segment
runs
    on the L4 by default, so validation=cloud_run_l4 needs no explicit segment knob."""
    cfg = AppConfig.model_validate(
        {
            "deployment": {
                "compute_target": "cloud_run",
                "phase_placement": {"validation": "cloud_run_l4"},
            }
        }
    )
    assert resolve_phase_placement(cfg, PHASE_VALIDATION) == PLACE_CLOUD_L4
    assert resolve_phase_placement(cfg, PHASE_SEGMENT) == PLACE_CLOUD_L4
```

145. user (2026-07-08T05:30:39.022Z)

```
File created successfully at: C:\kg-wt521\tests\test_phase_placement.py (file state is current
in your context ? no need to Read it back)
```

146. assistant (2026-07-08T05:31:07.339Z)

Now the compile-dispatch test mirroring `test_api_cloud_dispatch.py`:

147. assistant (2026-07-08T05:31:44.836Z)

```
"""Compile/stitch ? L4 worker dispatch (#521).

Proves the control-plane can OFFLOAD the reel compile/stitch to the Cloud Run L4 worker by
config
(``deployment.phase_placement.compile=cloud_run_l4``), mirroring the DETECT dispatch ? all GPU-
free +
cloud-free via a FAKE ``CloudRunJobClient`` that simulates the compile worker (reads the staged
compile-spec + writes the reel + done-marker into a local "bucket"). Cardinal invariant: DEFAULT
(compile on the control-plane) still stitches in-process, byte-identically.
"""

import json
import os

import pytest

import backend.compute as compute_pkg
import backend.orchestration as orch
from backend.compute.cloud_run import CloudRunJobClient
from backend.config import AppConfig
from backend.infrastructure.database import Database
from backend.infrastructure.execution_policy import ExecutionPolicy

_FAST = ExecutionPolicy(poll_every_n_sec=0.0, op_timeout_sec=5.0, max_wait_sec=5.0)


class _FakeCompileClient(CloudRunJobClient):
    """Simulates the Cloud Run compile worker: on dispatch, READ the staged compile-spec
(proving the
    control-plane resolved + staged it), then write the reel done-marker into the local bucket
exactly
    where the real worker would, so the dispatcher's bucket-poll resolves."""
```

```
    def __init__(self, *, returncode=0, video_id=None):
        self.calls: list[list[str]] = []
        self.spec: dict | None = None
        self.returncode = returncode
        self._video_id = video_id

    def run_job(self, job_name, argv, *, region, project=None):
        self.calls.append(list(argv))
        spec_uri = argv[argv.index("--spec-uri") + 1]
        done_uri = argv[argv.index("--done-uri") + 1]
        with open(spec_uri, encoding="utf-8") as f:
            self.spec = json.load(f)
        vid = self._video_id or self.spec["video_id"]
        out_name = self.spec["output_name"]
        status = "success" if self.returncode == 0 else "failed"
        marker = {
            "video_id": vid,
            "returncode": self.returncode,
            "status": status,
            "download_url": f"/api/highlights/{vid}/{out_name}",
            "reel_uri": f"gs://bucket/highlights/{vid}/{out_name}",
            "message": "" if self.returncode == 0 else "stitch blew up",
        }
        os.makedirs(os.path.dirname(done_uri), exist_ok=True)
        with open(done_uri, "w", encoding="utf-8") as f:
            json.dump(marker, f)
        return "exec-compile-1"

    def cancel_execution(self, execution):  # pragma: no cover - happy path never cancels
        raise AssertionError("cancel must never fire on the happy path")


def _inject_fake(monkeypatch, fake):
    real = compute_pkg.get_compute_backend
    monkeypatch.setattr(
        compute_pkg,
        "get_compute_backend",
        lambda config, **kw: real(config, run_client=fake, policy=_FAST, token="tok", **kw),
    )


@pytest.fixture
def compile_env(tmp_path, monkeypatch):
    """A control-plane DB with a video + 3 play segments, wired for cloud with compile placed on
the
    L4 and a LOCAL bucket (so the fake worker's spec/marker are plain file reads/writes)."""
    db = Database(str(tmp_path / "t.db"))
    bucket = tmp_path / "bucket"
    bucket.mkdir()
    video_id = "vidCompile"
    src = tmp_path / "source.mp4"
    src.write_bytes(b"SRC")
    db.add_video(video_id, str(src), "processed", [])
    sid1 = db.add_play_segment(video_id, 2.0, 6.0, metadata={"shot_count": 5})
    sid2 = db.add_play_segment(video_id, 10.0, 14.0, metadata={"shot_count": 3})
    sid3 = db.add_play_segment(video_id, 20.0, 22.0, metadata={"shot_count": 4})
    cfg = AppConfig.model_validate(
        {
            "deployment": {
                "compute_target": "cloud_run",
                "phase_placement": {"compile": "cloud_run_l4"},
            },
            "storage": {"backend": "local", "output_root": str(bucket)},
        }
    )
```

```
    return db, cfg, video_id, [sid1, sid2, sid3], bucket, monkeypatch


def _job(video_id, segment_ids, name="reel.mp4", locale=None):
    return {
        "id": "cj1",
        "video_id": video_id,
        "kind": "compile",
        "params": json.dumps(
            {"segment_ids": segment_ids, "output_filename": name, "locale": locale}
        ),
    }


def test_compile_dispatches_to_l4_and_returns_reel(compile_env):
    db, cfg, video_id, seg_ids, _bucket, monkeypatch = compile_env
    fake = _FakeCompileClient()
    _inject_fake(monkeypatch, fake)

    out = orch._execute_compile(cfg, db, _job(video_id, seg_ids, "myreel.mp4"))

    assert out["status"] == "success"
    assert out["video_id"] == video_id
    assert out["download_url"] == f"/api/highlights/{video_id}/myreel.mp4"
    assert out["compute"]["path"] == "cloud_run"
    assert out["compute"]["execution"] == "exec-compile-1"
    assert len(fake.calls) == 1  # exactly one Cloud Run Job execution


def test_compile_dispatch_stages_resolved_spec_with_nvenc(compile_env):
    """The staged compile-spec carries the RESOLVED time-pairs (padding-agnostic start/end from
the DB)
    and the L4 encoder (deployment.cloud_stitch_encoder = h264_nvenc by default) ? so the worker
    hardware-encodes and needs no control-plane DB."""
    db, cfg, video_id, seg_ids, _bucket, monkeypatch = compile_env
    fake = _FakeCompileClient()
    _inject_fake(monkeypatch, fake)

    orch._execute_compile(cfg, db, _job(video_id, seg_ids))

    assert fake.spec is not None
    assert fake.spec["video_id"] == video_id
    assert fake.spec["video_uri"].endswith("source.mp4")
    # 3 resolved (start, end) pairs from the DB rows
    assert fake.spec["time_pairs"] == [[2.0, 6.0], [10.0, 14.0], [20.0, 22.0]]
    # the L4 stitch uses NVENC (forwarded from deployment.cloud_stitch_encoder)
    assert fake.spec["stitching"]["encoder"] == "h264_nvenc"
    # the argv is a `compile` subcommand carrying the spec + done marker
    argv = fake.calls[0]
    assert argv[0] == "compile"
    assert "--spec-uri" in argv and "--done-uri" in argv
    joined = " ".join(argv)
    assert "storage.output_root=" in joined  # bucket forwarded so the worker mirrors the reel


def test_compile_dispatch_forwards_stitch_config_overrides(compile_env):
    """The worker Job has no DEPLOYMENT_PROFILE, so the dispatch forwards the bucket coordinates
    (storage.output_root/backend) ? else the worker can't read the spec or mirror the reel."""
    db, cfg, video_id, seg_ids, bucket, monkeypatch = compile_env
    fake = _FakeCompileClient()
    _inject_fake(monkeypatch, fake)
    orch._execute_compile(cfg, db, _job(video_id, seg_ids))
    joined = " ".join(fake.calls[0])
    assert f"storage.output_root={bucket}" in joined
    assert "storage.backend=local" in joined
```

```
def test_compile_cloud_failure_returns_failed(compile_env):
    """A worker stitch failure (rc != 0) surfaces as a failed job with the worker's message ?
the SPA
    gets a clean verdict, not a crash."""
    db, cfg, video_id, seg_ids, _bucket, monkeypatch = compile_env
    fake = _FakeCompileClient(returncode=3)
    _inject_fake(monkeypatch, fake)

    out = orch._execute_compile(cfg, db, _job(video_id, seg_ids))
    assert out["status"] == "failed"
    assert "stitch blew up" in out["message"]
    assert out["video_id"] == video_id


def test_compile_default_stitches_in_process_no_dispatch(compile_env, monkeypatch):
    """DEFAULT (compile on the control-plane): _execute_compile stitches in-process via
_stitch_reel
    and NEVER dispatches to Cloud Run. The stitch core is stubbed so no real ffmpeg runs."""
    db, cfg, video_id, seg_ids, _bucket, _mp = compile_env
    # flip compile back to the control-plane (the default) ? everything else stays cloud-wired
    cfg.deployment.phase_placement.compile = "control_plane"
    fake = _FakeCompileClient()
    _inject_fake(monkeypatch, fake)
    seen = {}

    def _fake_stitch(c, stitching, vid, source, pairs, name, locale, notify):
        seen["pairs"] = pairs
        seen["source"] = source
        return {"status": "success", "video_id": vid, "download_url":
f"/api/highlights/{vid}/{name}"}

    monkeypatch.setattr(orch, "_stitch_reel", _fake_stitch)

    out = orch._execute_compile(cfg, db, _job(video_id, seg_ids, "local.mp4"))
    assert out["status"] == "success"
    assert fake.calls == []  # NEVER dispatched ? the control-plane stitched in-process
    assert seen["pairs"] == [(2.0, 6.0), (10.0, 14.0), (20.0, 22.0)]
    assert seen["source"].endswith("source.mp4")


def test_compile_missing_video_fails_before_dispatch(compile_env):
    """A compile for an unknown video_id fails at resolve (404) before any dispatch."""
    db, cfg, _video_id, _seg_ids, _bucket, monkeypatch = compile_env
    fake = _FakeCompileClient()
    _inject_fake(monkeypatch, fake)
    out = orch._execute_compile(cfg, db, _job("nope", [1, 2]))
    assert out["status"] == "failed"
    assert fake.calls == []  # never dispatched ? resolve failed first
```

148. user (2026-07-08T05:31:44.880Z)

```
File created successfully at: C:\kg-wt521\tests\test_compile_dispatch.py (file state is current
in your context ? no need to Read it back)
```

149. assistant (2026-07-08T05:31:50.238Z)

Let me run the new placement and compile-dispatch tests:

150. user (2026-07-08T05:31:57.622Z)

```
            )

        # Build the compile-spec the worker consumes. time-pairs are JSON lists (frozen-safe on
the wire);
```

```
        # the stitching config is the control-plane's RESOLVED one, with the encoder swapped to
the L4's
        # (deployment.cloud_stitch_encoder, NVENC by default) so the worker hardware-encodes the
reel.
        stitch_dump = stitching.model_dump()
        stitch_dump["encoder"] = getattr(
            getattr(cfg, "deployment", None), "cloud_stitch_encoder", "h264_nvenc"
        )
        spec_payload = {
            "video_id": video_id,
            "video_uri": video["filepath"],
            "output_name": out_name,
            "locale": locale,
            "time_pairs": [
                [float(s["start_time"]), float(s["end_time"])] for s in kept
            ],
            "stitching": stitch_dump,
        }
>       token = uuid.uuid4().hex
                ^^^^
E       NameError: name 'uuid' is not defined. Did you forget to import 'uuid'?

backend\orchestration.py:1339: NameError
========================== short test summary info ==========================
FAILED tests/test_compile_dispatch.py::test_compile_dispatches_to_l4_and_returns_reel
FAILED tests/test_compile_dispatch.py::test_compile_dispatch_stages_resolved_spec_with_nvenc
FAILED tests/test_compile_dispatch.py::test_compile_dispatch_forwards_stitch_config_overrides
FAILED tests/test_compile_dispatch.py::test_compile_cloud_failure_returns_failed
4 failed, 10 passed in 3.28s
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

151. assistant (2026-07-08T05:32:00.933Z)

`uuid` isn't imported in orchestration.py. Let me add it:

152. user (2026-07-08T05:32:02.298Z)

The file C:\kg-wt521\backend\orchestration.py has been updated successfully. (file state is current in your context ? no need to Read it back)

153. user (2026-07-08T05:32:10.764Z)

```
..............                                                           [100%]
14 passed in 2.88s
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

154. assistant (2026-07-08T05:32:20.686Z)

All 14 pass. Now add `run_compile` tests to `test_compute_backend.py`. Let me register the worktree file's tail:

155. user (2026-07-08T05:32:21.656Z)

```
1094        def heartbeat(_elapsed):
1095            pass
1096
1097        result = backend.run_infer(
1098            ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bk")),
1099            on_wait=heartbeat,
1100        )
1101        assert seen["on_tick"] is heartbeat
1102        assert result.video_id == "vidCloud"
1103
```

156. user (2026-07-08T05:32:44.926Z)

```
The file C:\kg-wt521\tests\test_compute_backend.py has been updated successfully. (file state is
current in your context ? no need to Read it back)
```

157. assistant (2026-07-08T05:32:48.282Z)

Let me remove the unused import in that last test:

158. user (2026-07-08T05:32:49.760Z)

```
The file C:\kg-wt521\tests\test_compute_backend.py has been updated successfully. (file state is
current in your context ? no need to Read it back)
```

159. assistant (2026-07-08T05:32:55.406Z)

Now the worker `cmd_compile` + parser tests. Let me find the end of `test_worker.py`:

160. user (2026-07-08T05:32:57.145Z)

```
945 tests/test_worker.py
    )  # graceful: _resolve_train_detector caught NotImplementedError, returned None


def test_probe_video_fps_width_returns_none_on_unreadable(tmp_path):
    """The strict ffprobe probe must return None (caller skips) for a non-video file, never
guess."""
    bad = tmp_path / "not_a_video.mp4"
    bad.write_bytes(b"NOTAVIDEO")
    assert wmain._probe_video_fps_width(str(bad)) is None
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

161. user (2026-07-08T05:33:00.234Z)

```
938         )  # graceful: _resolve_train_detector caught NotImplementedError, returned None
939
940
941     def test_probe_video_fps_width_returns_none_on_unreadable(tmp_path):
942         """The strict ffprobe probe must return None (caller skips) for a non-video file,
never guess."""
943         bad = tmp_path / "not_a_video.mp4"
944         bad.write_bytes(b"NOTAVIDEO")
945         assert wmain._probe_video_fps_width(str(bad)) is None
```

162. user (2026-07-08T05:33:27.700Z)

```
The file C:\kg-wt521\tests\test_worker.py has been updated successfully. (file state is current
in your context ? no need to Read it back)
```

163. assistant (2026-07-08T05:33:31.875Z)

test_worker.py needs `import os`. Let me add it:

164. user (2026-07-08T05:33:32.842Z)

```
The file C:\kg-wt521\tests\test_worker.py has been updated successfully. (file state is current
in your context ? no need to Read it back)
```

165. assistant (2026-07-08T05:33:35.717Z)

Now add an encoder-selection test to `test_compiler.py`:

166. user (2026-07-08T05:33:37.489Z)

```
def test_compile_raises_when_source_missing(tmp_path):
    comp = VideoCompiler(str(tmp_path / "out"))
    with pytest.raises(FileNotFoundError):
        comp.compile_highlights(str(tmp_path / "nope.mp4"), [(0.0, 5.0)], "reel.mp4")
---lines---
160 tests/test_compiler.py
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

167. user (2026-07-08T05:33:42.130Z)

```
155
156
157     def test_compile_raises_when_source_missing(tmp_path):
158         comp = VideoCompiler(str(tmp_path / "out"))
159         with pytest.raises(FileNotFoundError):
160             comp.compile_highlights(str(tmp_path / "nope.mp4"), [(0.0, 5.0)], "reel.mp4")
```

168. user (2026-07-08T05:34:00.347Z)

```
The file C:\kg-wt521\tests\test_compiler.py has been updated successfully. (file state is
current in your context ? no need to Read it back)
```

169. assistant (2026-07-08T05:34:03.208Z)

Let me run all the new/modified test files:

170. user (2026-07-08T05:34:15.021Z)

```
.................................................................... [ 53%]
.....................................................                [100%]
135 passed in 8.14s
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

171. assistant (2026-07-08T05:34:47.831Z)

All 135 pass. Now let me add tests for the validation-placement wiring (the `skip_validation` flip + CP delegation) to `test_api_cloud_dispatch.py`:

172. user (2026-07-08T05:34:48.712Z)

```
1097    def test_wait_progress_message_is_honest_and_minute_granular():
1098        """#482: the CP knows ELAPSED time, not frames ? the heartbeat says so plainly,
shows
1099        movement from the first tick ("starting up" during queue/cold-start), then counts
minutes."""
1100        from backend.orchestration import _wait_progress_message
1101
1102        early = _wait_progress_message("wasb_hybrid", 12.0)
1103        assert "wasb_hybrid" in early and "starting up" in early
1104        later = _wait_progress_message("wasb_hybrid", 125.0)
1105        assert later.endswith("2 min elapsed")
1106
```

173. user (2026-07-08T05:35:03.239Z)

```
The file C:\kg-wt521\tests\test_api_cloud_dispatch.py has been updated successfully. (file state
is current in your context ? no need to Read it back)
```

174. user (2026-07-08T05:35:41.682Z)

```
tests/test_api_cloud_dispatch.py::test_full_api_process_to_query_via_cloud
tests/test_api_cloud_dispatch.py::test_local_override_survives_api_boundary
tests/test_api_cloud_dispatch.py::test_local_override_survives_api_boundary
```

```
tests/test_api_cloud_dispatch.py::test_cloud_override_survives_api_boundary_on_local_server
tests/test_api_cloud_dispatch.py::test_cloud_override_survives_api_boundary_on_local_server
tests/test_api_cloud_dispatch.py::test_full_api_process_job_result_carries_cloud_provenance
tests/test_api_cloud_dispatch.py::test_full_api_process_job_result_carries_cloud_provenance
  C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
    warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
40 passed, 8 warnings in 33.32s
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

175. assistant (2026-07-08T05:35:49.665Z)

All 40 pass. Now let me run lint + type checks (matching CI). Let me check the exact CI commands and run them:

176. user (2026-07-08T05:35:51.173Z)

```
16:  # Lint gate (ruff.toml at repo root): pyflakes/pycodestyle + bugbear.
17:  # Also the repo-wide syntax guard ? ruff reports invalid-syntax on every
29:      - name: Install ruff
30:        # Pinned so an upstream ruff release can't redden every PR overnight; bump
deliberately.
31:        run: python -m pip install ruff==0.15.16
33:        run: ruff check . --output-format github
35:  # PII / attribution leak guard (tools/leak_scan.py): fails on a committed 32-hex MD5
49:        run: python -m tools.leak_scan
51:  # Internal-link guard (tools/check_md_links.py): fails on a broken internal markdown link
64:        run: python -m tools.check_md_links
66:  # Type gate (mypy.ini at repo root): lenient green baseline with an explicit
68:  # up. Typed core deps are installed so mypy sees their real signatures
79:      - name: Install mypy + typed core deps
80:        # mypy pinned alongside ruff so a type-checker release can't redden CI overnight.
81:        run: python -m pip install mypy==2.1.0 pydantic fastapi uvicorn python-dotenv google-
genai numpy
83:        run: mypy backend training
136:        # (pytest, pytest-cov, httpx for the fastapi TestClient, ruff, mypy).
146:        run: python run_all_tests.py --cov=backend --cov=training --cov-report=term --cov-
fail-under=70
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

177. user (2026-07-08T05:35:55.377Z)

```
=== RUFF ===
I001 [*] Import block is un-sorted or un-formatted
  --> tests\test_phase_placement.py:9:1
   |
 7 |   """
 8 |
 9 | / import pytest
10 | |
11 | | from backend.config import (
12 | |     PHASE_COMPILE,
13 | |     PHASE_SEGMENT,
14 | |     PHASE_VALIDATION,
15 | |     PLACE_CLOUD_L4,
16 | |     PLACE_CONTROL_PLANE,
17 | |     AppConfig,
18 | |     load_config,
19 | |     resolve_phase_placement,
```

```
20 | | )
   | |_^
   |
help: Organize imports

Found 1 error.
[*] 1 fixable with the `--fix` option.
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

178. assistant (2026-07-08T05:35:58.315Z)

One import-sort nit. Let me auto-fix and re-check:

179. user (2026-07-08T05:36:00.348Z)

```
Found 1 error (1 fixed, 0 remaining).
=== RE-CHECK ===
All checks passed!
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
[This command modified 1 file you've previously read: ..\..\..\..\..\kg-
wt521\tests\test_phase_placement.py. Call Read before editing.]
```

180. assistant (2026-07-08T05:36:03.325Z)

Ruff clean. Now mypy on the changed modules:

181. user (2026-07-08T05:37:09.208Z)

```
backend\annotations\base.py:88: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\sports\__init__.py:9: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs  [annotation-unchecked]
backend\pipeline\stitcher\compiler.py:57: error: Value of type variable
"SupportsRichComparisonT" of "min" cannot be "float | None"  [type-var]
backend\pipeline\validators\base.py:123: note: By default the bodies of untyped functions are
not checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\providers\__init__.py:9: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
Found 1 error in 1 file (checked 150 source files)
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

182. assistant (2026-07-08T05:37:17.871Z)

mypy flags the probe's `min(30, ffmpeg_timeout_sec())` (the timeout can be `None`). Let me fix it:

183. user (2026-07-08T05:37:24.270Z)

```
The file C:\kg-wt521\backend\pipeline\stitcher\compiler.py has been updated successfully. (file
state is current in your context ? no need to Read it back)
```

184. user (2026-07-08T05:37:35.411Z)

```
=== exit summary ===
backend\providers\__init__.py:9: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
Success: no issues found in 150 source files
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

185. assistant (2026-07-08T05:37:43.579Z)

mypy clean. Now let me run the FULL test suite with coverage (the pre-LGTM gate), running in the background isn't needed ? let me run it directly:

186. user (2026-07-08T05:41:08.359Z)

```
backend\storage\gdrive_api.py                        105      6    94%
backend\storage\local.py                              49      3    94%
backend\storage\ref.py                                49      0   100%
backend\storage\s3.py                                 47      5    89%
backend\tools\__init__.py                              0      0   100%
backend\tools\export.py                               75      2    97%
backend\tools\rally_slicer.py                        105     47    55%
backend\utils\__init__.py                              0      0   100%
backend\utils\annotation_proxy.py                     64     11    83%
backend\utils\app_home.py                             40      0   100%
backend\utils\ffmpeg.py                               47      3    94%
backend\utils\freespace.py                            32      2    94%
backend\utils\geometry.py                             29      0   100%
backend\utils\hashing.py                               9      0   100%
backend\utils\logging_setup.py                        11      0   100%
backend\utils\paths.py                                44      3    93%
backend\utils\redact.py                               16      0   100%
backend\utils\run_telemetry.py                       126     12    90%
backend\utils\single_instance.py                      43     11    74%
backend\utils\video.py                                34      3    91%
backend\utils\workspace.py                            49      0   100%
backend\worker\__init__.py                             0      0   100%
backend\worker\__main__.py                           371     32    91%
training\__init__.py                                   0      0   100%
training\gen0\__init__.py                              0      0   100%
training\gen0\harness.py                             130      9    93%
----------------------------------------------------------------------
TOTAL                                              12663   1810    86%
Required test coverage of 70% reached. Total coverage: 85.71%
========================== short test summary info ==========================
FAILED tests/test_main_module_budget.py::test_main_py_line_budget - Assertion...
1 failed, 1874 passed, 8 skipped, 25 warnings in 199.57s (0:03:19)
======================================================================
 RESULT: FAIL - pytest exit 1
======================================================================
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

187. assistant (2026-07-08T05:41:17.428Z)

Coverage 85.7% (?70). One failure: the `main.py` line-budget guard (I added 2 re-export lines).
Let me check it:

188. user (2026-07-08T05:41:19.086Z)

```
"""Anti-bloat ratchet for the ``backend.main`` god-module split (P23; re-cut of #390).

``backend/main.py`` grew 1448 -> 2360 lines in under a week because every endpoint/fix PR
landed in it. The P23 extraction moved the processing/compile orchestration to
``backend/orchestration.py``; this ratchet keeps the split from silently regressing:

- **Line budget:** ``backend/main.py`` must stay at/below ``MAIN_LINE_BUDGET``. When a
  later split lands (CLI -> P23 step 2, routes -> P24), TIGHTEN the budget to the new
  size + small slack ? never loosen it without an owner-visible PR discussion.
- **Route ceiling:** no NEW ``@app.<method>`` routes in ``main.py`` ? new endpoints go in
  ``backend/api/`` (see ``backend/api/uploads.py`` for the pattern).
- **Re-export contract:** every orchestration name stays addressable on ``backend.main``
  (``import backend.main as m; m._execute_processing`` and
  ``monkeypatch.setattr(backend.main, ...)`` are load-bearing seams ? see
  ``backend/api/job_worker.py``'s module docstring).

If this test fails your PR: extract, don't inline. Umbrella tracker (archived):
``docs/archives/past_projects/AUDIT_REPORT_khelsutra-guru_2026-07-02-COMPLETED-2026-07-02.md``
(Remediation tracker P23/P24).
```

```
"""

import ast
from pathlib import Path

REPO = Path(__file__).resolve().parents[1]
MAIN = REPO / "backend" / "main.py"

## main.py was 1623 lines after the P24 server_runtime extraction (2026-07-02; 1703 after
## P23 orchestration extraction). Slack covers small in-place fixes; systematic growth mu
## a module instead (orchestration -> backend/orchestration.py; server/launcher -> back
## new routes -> backend/api/). TIGHTEN this budget whenever a further split lands.
## 1650 -> 1662 (#448): the /api/jobs list ROUTE was extracted to backend/api/jobs.py (ro
## respected); the only in-place growth was the idempotent-submit dedup hook in proces
## 1662 -> 1653 (#481 review): the dedup became an ATOMIC single-op claim (backend.ap
## + db.claim_or_create_process_job), which REPLACED the inline create_job block ? re
MAIN_LINE_BUDGET = 1653

## The 18 routes present at the P23 extraction. New endpoints belong in backend/api/.
MAIN_ROUTE_BUDGET = 18

ORCHESTRATION_REEXPORTS = [
    "_cloud_available",
    "_CompileError",
    "_execute_compile",
    "_execute_processing",
    "_finalize_reingest",
    "_ingest_remote_segments",
    "_maybe_dispatch_remote",
    "_remote_input_size_bytes",
    "_report_config_for_input",
    "_resolve_compile",
]


def test_main_py_line_budget():
    n = sum(1 for _ in MAIN.open(encoding="utf-8"))
    assert n <= MAIN_LINE_BUDGET, (
        f"backend/main.py is {n} lines (budget {MAIN_LINE_BUDGET}). Don't grow the "
        f"god-module back ? extract to backend/orchestration.py / backend/api/ / a new "
        f"module instead (P23/P24 in docs/archives/past_projects/"
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

189. assistant (2026-07-08T05:41:32.967Z)

```
The two re-exports I added aren't load-bearing (nothing calls `m._stitch_reel`; the worker uses
`orchestration._stitch_reel` directly), and they push `main.py` over its anti-bloat budget. Let
me revert them:
```

190. user (2026-07-08T05:41:37.300Z)

```
The file C:\kg-wt521\backend\main.py has been updated successfully. (file state is current in
your context ? no need to Read it back)
```

191. user (2026-07-08T05:41:51.985Z)

```
...                                                                      [100%]
3 passed in 2.67s
```

```
=== leak_scan ===
[leak-scan] clean ? no MD5 / personal-path / private-name leaks in the code surface.
=== md_links ===
[check-md-links] clean ? no broken internal links in live docs.
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

192. assistant (2026-07-08T05:41:56.130Z)

All gates green. Now the docs. Let me read the COMPUTE_DECOUPLED_SERVING README to place the new doc:

193. user (2026-07-08T05:41:57.794Z)

Compute-Decoupled Rally Serving ? one pure core, profile-selected compute + storag

> **Status:** ? **M0 APPROVED** (owner, 2026-06-21) ? **IN PROGRESS (build M1+)** · **Owner:** Avi · **Created:** 2026-06-21 · **Last updated:** 2026-06-21
> **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/past_projects/` when DONE)
> **Tracking anchors:** §7 progress tracker is the source of truth; pointer in memory `current-state` + `docs/NEXT_STEPS.md` once work starts.
> **Relation to existing docs:** the productionization successor to the archived `DECOUPLED_COMPUTE` (M3/M4); realizes the `ComputeBackend`/`StorageBackend` registries from [PLATFORM_ARCHITECTURE.md](../PLATFORM_ARCHITECTURE.md) §3; the **truly-local** profile is the L0 tier of [VIDEO_LOCALITY_MODEL.md](../VIDEO_LOCALITY_MODEL.md) and the `LocalDesktop` backend of [DESKTOP_APP_PLAN.md](../DESKTOP_APP_PLAN.md).
> **Naming:** this was the "Cloud Run + GPU serving subproject" (ADR-012). Renamed **compute-decoupled serving** because **truly-local is a co-equal, first-class profile**, not an afterthought.
> **Honesty note:** claims tagged `[verified]` (checked against code by the design workflow `wf_56274ff0`), `[design]` (proposed), `[researched]`.

> **? Northstar (D14):** a **mobile, app-like** flow ? point at a **Google Drive** video ? **fully cloud-native** run ? the stitched, **ready-to-share** reel lands **back in Drive, next to the source**. Two design promises around it: a user can **switch local?cloud anytime** (D13, a headline feature), and execution is **never a surprise** ? auto-detect *suggests*, the user *confirms* (D15).

0. TL;DR

The rally engine is a **pure function of `(input bytes + config) ? outputs`** in three deterministic stages ? **DETECT ? WINDOW ? STITCH** ? that **no deployment profile may branch on**. A **`ComputeBackend` (deployment profile)** decides *where* the core runs (a local process vs a Cloud Run GPU job); a **`StorageBackend`** decides *where bytes come from* (local FS / USB / mounted Google Drive / bucket). **One startup config** picks both. We ship **truly-local** (local GPU + in-process stitch, zero cloud) and **cloud-serving** (upload `<2GB` / gdrive / bucket ? a Cloud Run GPU job runs detect **and** stitch, so stitch's compute is metered), plus **cpu-mock** for CI. The single most important caveat: detection is **byte-identical only on the same GPU** (cross-GPU is sub-pixel-different, per the 2026-06-20 deploy report) ? so the **parity guarantee is enforced at the WINDOW + STITCH layer**, not detector-CSV bytes.

1. Problem & goal

- **Why now:** DECOUPLED_COMPUTE proved compute *portability* (M0?M2 + M3.5 on a GCP L4) and is archived; its deferred **M3 (serving endpoint + per-video billing) + M4 (scale-to-zero + cost guard)** become this project. The owner needs **both** a hosted service *and* a truly-local mode, with the **core (detect+stitch) unaffected** by which one runs.
- **The two user-facing outcomes** (owner, 2026-06-21):
  - **(a) Hosted service** ? operate on small **local uploads (`<2GB`)** *and* load from **gdrive / a bucket**, spinning up **Cloud Run for both rally segmentation AND stitching** (stitching is compute-intensive ? run it where its cost is accounted for).
  - **(b) Truly-local** ? on a local machine with videos on **local drives/USB + a local GPU**, run inference **and** stitching **completely locally**.

- **"Good" looks like:** the same `video ? rallies ? reel` core gives identical windows + stitched ranges whether it runs on a laptop or Cloud Run; switching is a **config/startup choice**, never a code branch in the core.

2. Decisions locked

#	Decision	Source	Implication
D1	The core is a **pure 3-stage function** (DETECT/WINDOW/STITCH); **no profile branch inside it**.	design `[verified]`	All profile difference lives in config + the two registries.
D2	**Two registries:** `ComputeBackend` (where) selected by `deployment.profile` + `compute_target`; `StorageBackend` (what-bytes) by `input_backend`/`storage.backend`.	ADR-014	Realizes PLATFORM_ARCHITECTURE's `local|hosted|byo_worker`.
D3	**Profiles shipped:** `truly-local`, `cloud-serving`, `cpu-mock` (CI); `byo_worker` reserved/deferred.	ADR-014	Enum/seam reserved so BYO isn't precluded.
D4	In **cloud-serving, STITCH runs on Cloud Run** (co-located with detect) so CPU/wall + egress are metered into the per-job cost ledger.	owner (a)	Stitch-on-laptop would hide real cost.
D5	**Parity enforced at WINDOW+STITCH** (byte-exact for a fixed trajectory); **detect** is byte-exact only same-GPU.	deploy report 2026-06-20	Cross-GPU asserted at the rally/window level, never CSV bytes.
D6	**Default-OFF, smallest-safe-first, one PR per milestone**; any core-caller change (e.g. configurable output base) ships behind a flag + a Launch Report.	[[launch-report-convention]]	Zero-regression discipline.
D7	**Cloud Run cold-start: SCALE-TO-ZERO** ? accept the ~1?3 min cold-start (it's an async job: upload?poll?reel, amortized over the multi-minute job). **No warm pool** (that ~$500/mo idle would break per-video billing).	owner 2026-06-21	M6/M7. Revisit a warm lane only on a UX complaint.
D8	**Pricebook = a versioned DB table**; cost computed in **USD** (matches GCP billing = one source of truth), **displayed in INR** at a configurable FX rate (Bangalore market); re-versioned when GCP prices change.	owner 2026-06-21	M8.
D9	**Edge auth = Cloudflare Access** (reuse the site's existing CF). Lock the Cloud Run **ingress to the CF proxy** + **verify the `Cf-Access` JWT signature** (so a direct hit to the Cloud Run URL can't spoof the identity header).	owner 2026-06-21	M9. One identity system.
D10	**Free-tier "data-for-reels" trade:** free reels in exchange for **training rights** (uploaded footage + derived trajectories/labels); **paid tier = no-train** (privacy is the paid feature). **Carve-outs:** train ONLY on attested **ADULT** footage (minor footage ? reel yes, training pool **NO** ? DPDP/GDPR need verifiable parental consent); train on **DERIVED** data + **auto-delete raw**; **ToS counsel-reviewed**; live training-on-uploads **gated to M9** (public signup). Truly-local needs no DPA.	owner 2026-06-21	M9 + D12; ties [[paper-coauthor-aim]] / PERSONALIZATION flywheel.
D11	**Quality floor: Gemini handoff ON** for cloud-serving until the owned model clears the #172 ship-gate (e.g. ?0.70 multi-court) ? then flip via a **Launch Report**; the owned model runs in **shadow/eval** meanwhile. (Gemini = serving/inference ONLY, never a training source ? CLAUDE.md guardrail.)	owner 2026-06-21	M7 + D12.
D12	**Data-flywheel harness is IN SCOPE (owner add, Q5):** capture the (consented, adult) uploaded video + the Gemini reel/rally output ? **reliably store** in the per-video workspace/bucket ? **run shadow evals** (owned-vs-Gemini, dual-eval-set) ? **promote good ones to the golden TRAINING set** (human-reviewed labels) so the owned model climbs toward the D11 floor (then Gemini flips off).	owner 2026-06-21	new **M11**; ties D10 (consent) + `data_pipeline/` + the eval/experiment harness.
D13	**Profile is user-switchable ANYTIME ? a headline FEATURE.** The same user runs **local today, cloud tomorrow, back to local** ? it's just a config/startup choice; the pure core gives **equivalent** output either way. **Advertise it** ("your machine *or* our cloud ? your call, switch anytime").	owner 2026-06-21	Honest caveat: equivalent at the **rally/window** level, not bit-identical across GPUs (D5); a single job runs in one profile, switching is *between* jobs.
D14	**NORTHSTAR ? operate toward this:** a **mobile, app-like** flow ? point at a **Google Drive** video ? **fully cloud-native** run ? the stitched, **ready-to-share** reel lands **back in Drive, next to the source**.	owner 2026-06-21	Implies a **`gdrive-api` (OAuth) StorageBackend** (read source + write reel back) ? distinct from the local **mount** backend; **resolves §8 #7**, new **M12**. The mobile UI is the khelsutra track; the engine must support it (async jobs + status + Drive round-trip / signed URL). Net-new scope (OAuth + Drive write);

not necessarily v1, but the design must not preclude it (the `StorageBackend` ABC accommodates it). |
D15	**Auto-detect SUGGESTS, the user CONFIRMS ? execution is NEVER a surprise.** A GPU owner may still choose cloud; **cloud (= spend) is never entered silently**; the active profile is always explicit + surfaced.	owner 2026-06-21	Amends §4.3 (auto-detect = a proposed default, not an auto-decision).
D16	**M6/M7 implementation SHAPE + the cheap §8 picks (owner 2026-06-21):** (a) **Cloud Run JOBS** (not Services-with-GPU) for the detect+stitch unit, **bucket-poll for completion** (reuses `JobWaiting`/`claim_due_job` verbatim, fits async upload?poll?reel + scale-to-zero D7); (b) **stitch ENCODE = libx264** (protects the D5 byte-determinism/parity guarantee + the L4 parity test) ? **NVDEC for DECODE only** (a speed lever that never touches output bytes); **NVENC encode is DEFERRED behind a config knob**, revisit only if M7 cost data justifies relaxing parity (§8 Q8); (c) **input cap = hard-reject `<2GB`** for v1 (a config cap so raising to 4GB later is one line; matches the shipped M2 chunked-upload threshold ? §8 Q9).	owner 2026-06-21	M6/M7. Resolves §8 Q2 (co-located, = D4), Q8, Q9, Q10 (dir name confirmed).
D17	**Real cloud verification BUDGET = $100 / ?10k cap (owner 2026-06-21):** the owner authorized actual GCP spend for due-diligence + production-grade testing/verification of M5?M7 (real L4 run, the M5 runlog, the M7 DEPLOY_REPORT), **provided the cap is respected** ? confirm exact cmd + hourly cost before each spend, prefer the smallest viable run + cheapest-adequate GPU, and **DELETE every resource after** ([[gcp-vm-always-delete]]). Removes the M6/M7 "spend" gate within the envelope; the calibrated pricebook (M8) + counsel ToS (M9) + OAuth app (M12) stay owner/counsel-side.	owner 2026-06-21	Unblocks the real M5/M7 runs (not just the code) within the cap.

3. Foundation ? evolution / ADR lineage ? *trace the idea end-to-end*

This project is the latest step in a long decision chain; each ADR contributed a piece and a **subtlety that carries forward**:

ADR / doc	Contributed	Subtlety carried into this project
ADR-005	Permissive licensing (MIT/Apache-2.0/BSD only)	The engine + any served model + the **container image** (ffmpeg, fonts, weights) stay commercial-clean.
ADR-008	Owned-model topology; **train==serve featurizer single-sourcing**	The "core unaffected across profiles" is the same parity discipline, extended from train/serve to **local/cloud**.
ADR-009	KhelSutra Guru; **local-first delivery**	The **truly-local profile** is the direct continuation ? privacy/offline self-host stays first-class.
ADR-010	DECOUPLED: DO?GCP pivot	The cloud-compute origin (GPU + co-located bucket).
ADR-011	DECOUPLED scope = M0?M2+M3.5; **deferred M3 (serving) + M4 (billing/cost-guard)**; overrode the *"rent nothing / not a service"* posture	**This project realizes M3/M4.** Carries §06's owner-gated gates: **DPA/consent for non-owner uploads, collector `md5` keying, WASB-C3**.
ADR-012	GPU-native; TPU deferred; **Cloud Run+GPU scale-to-zero = serving model**; **NVDEC** named as the decode lever	The serving substrate + the 7-step seed scope this expands.
ADR-013	Drop DO; **GCP sole compute stack**; $200 DO credits ? non-compute only	The provider for cloud-serving; DO Spaces remains only a possible S3 *storage* target.
? ADR-014 (new)	**Compute-decoupled serving:** one pure core, profile-selected compute+storage; stitch-where-metered	This doc. **Refines** ADR-011/012, does **not** supersede them.
PLATFORM_ARCHITECTURE.md	The `ComputeBackend`/`StorageBackend` registries	The abstraction this realizes in code.
VIDEO_LOCALITY_MODEL.md / DESKTOP_APP_PLAN.md	L0 fully-local tier / `LocalDesktop` backend	The truly-local profile = these, productionized.

Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer

194. assistant (2026-07-08T05:42:31.781Z)

This is important context ? **D16(b)** explicitly deferred NVENC encode "behind a config knob, revisit only if M7 cost data justifies relaxing parity." The 2026-07-08 CUJ is exactly that justification, and my design keeps libx264 the default with NVENC opt-in via config ? so it aligns. Let me write the phase-placement doc capturing this:

195. assistant (2026-07-08T05:43:26.970Z)

Phase ? machine placement + compile-on-L4 (#521)

> **Status:** ? **SHIPPED (code)** · **Owner:** Avi · **Created:** 2026-07-08 · **Issue:**
[#521](https://github.com/Khelsutra/badminton-highlight-indexer/issues/521)
> **Relation:** extends [COMPUTE_DECOUPLED_SERVING](./README.md) (the `ComputeBackend` dispatch
seam) and **refines D16(b)** (NVENC-encode-deferred-behind-a-config-knob) now that the
2026-07-08 live CUJ supplies the cost data D16(b) said to revisit on.
> **Evidence:** `runlogs/DEPLOY_REPORT_cloud-serving_validation-latency_2026-07-07.md` §9
(2026-07-08 live CUJ, rev `rally-control-plane-00019`).

---

0. TL;DR

The compile/stitch phase used to run **in-process on the CPU-only 2-vCPU control-plane**, where
the
2026-07-08 live CUJ measured **~12m55s** for an 11-clip / ~80 s reel from a 1.83 GiB 4K source
(~45?90 s per ~7 s clip ? the per-clip branding watermark forces a full 4K re-encode with **no
NVENC**). Validation (183 s) runs there too; both serialize on `_LOCAL_COMPUTE_LANE`, so one run
pins
the control-plane for ~16 min ? the API can't scale past a handful of concurrent users.

#521 makes **where each pipeline phase runs a config edit, not a code change**, and moves the
stitch
to the **L4 worker (NVENC + 8 vCPU)** the way detection already dispatches:

1. **`deployment.phase_placement`** ? a per-phase knob (`validation` / `segment` / `compile`)
with
   values `control_plane | cloud_run_l4`. Unset ? **today's placement, byte-identical**.
2. **compile ? L4** ? a `kind='compile'` Cloud Run dispatch mirroring the detect path: the
   control-plane resolves the reel's segments, stages a compile-spec to the bucket, dispatches
   `python -m backend.worker compile` on the L4, bucket-polls a done-marker, and returns the
same
   `{status, filepath, download_url}` shape the SPA already expects.
3. **NVENC on the L4** ? `deployment.cloud_stitch_encoder` (default `h264_nvenc`) is forwarded
onto
   the worker's `stitching.encoder`; libx264 stays the everywhere-safe fallback (auto-probed).

The cloud-serving preset opts compile onto the L4 (`phase_placement.compile=cloud_run_l4`).

---

1. The placement model

`resolve_phase_placement(cfg, phase)` (in `backend/config/placement.py`) is the **single source
of
truth** ? orchestration and the worker never re-derive the mapping. Precedence: an explicit
`deployment.phase_placement.<phase>` wins; otherwise the **legacy default** reproduces pre-#521
behaviour exactly:

phase	knob unset (legacy default)	meaning
`validation`	`control_plane`	the authoritative gate (#512/#513) stays on the control-plane
`segment` (detect)	`cloud_run_l4` **iff** `compute_target==cloud_run`, else `control_plane`
today's DETECT dispatch
`compile` (stitch)	`control_plane`	the in-process ffmpeg stitch

So a config with **no** `phase_placement` block behaves exactly as before this existed. To
relocate a
phase, set its knob:

```jsonc
// Move the stitch to the L4 (what the cloud-serving preset does):
```

```
{ "deployment": { "compute_target": "cloud_run",
                  "phase_placement": { "compile": "cloud_run_l4" } } }

// Force detection back onto the control-plane on a cloud box (e.g. a debugging run):
{ "deployment": { "compute_target": "cloud_run",
                  "phase_placement": { "segment": "control_plane" } } }

// Move validation off the control-plane too (must pair with a cloud segment ? see §4):
{ "deployment": { "compute_target": "cloud_run",
                  "phase_placement": { "validation": "cloud_run_l4" } } }
```

`control_plane` runs the phase in-process on the (CPU-only) control-plane service;
`cloud_run_l4`
dispatches it to the GPU/NVENC Cloud Run worker Job.

---

2. compile ? L4 dispatch (the flow)

`orchestration._execute_compile` consults `resolve_phase_placement(cfg, "compile")`. When it
resolves
to `cloud_run_l4` **and** cloud dispatch is available (`_cloud_available`), it calls
`_dispatch_compile_remote`; otherwise it stitches in-process via the shared `_stitch_reel` core
(unchanged).

`_dispatch_compile_remote` (mirrors `_maybe_dispatch_remote`):

1. **Resolve** the reel's segments HERE ? the control-plane owns the DB (`_resolve_compile` ?
   `(video, kept, out_name)`), applying the `stitching.min_shot_count` floor as before.
2. **Stage a compile-spec** to `<output_root>/_compile/<token>/spec.json` ? the resolved
   `time_pairs`, the control-plane's resolved `stitching` config (watermark + padding, with
`encoder`
   swapped to `cloud_stitch_encoder`), the source `video_uri`, `video_id`, `output_name`,
`locale`.
   The worker therefore needs **no control-plane DB** ? everything rides the spec.
3. **Dispatch** a `compile` Cloud Run Job execution (`CompileJobSpec` ?
`CloudRunCompute.run_compile`)
   running `python -m backend.worker compile --spec-uri ? --done-uri ? --set
storage.output_root=?`,
   forwarding the bucket coordinates (the worker Job has no `DEPLOYMENT_PROFILE`).
4. **Bucket-poll** the done-marker at `<output_root>/_compile/<token>.done` ? the SAME
   `_await_marker` machinery `run_infer` uses (crash-fast on a terminal-without-marker
execution,
   rescue-or-cancel on a poll timeout). Distinct `_compile/` prefix so a compile and a detection
for
   one bucket never collide on a marker.
5. The worker (`cmd_compile`) reads the spec, stitches the reel via the **shared `_stitch_reel`
   core** (no forked pipeline), mirror-uploads it to
`<output_root>/highlights/<video_id>/<name>`,
   and writes the marker (rc + `download_url` + `reel_uri`).
6. The control-plane returns `{status, filepath, download_url, video_id}` ? the reel is already
in the
   bucket, so `/api/highlights` serves it via the existing signed-URL 307. On a worker rc?0 ?
   `status='failed'` with the worker's message (clean SPA verdict, never a crash).

Fallback: if compile is placed on the L4 but the box isn't actually cloud-wired (no output
bucket
/ Cloud Run env), `_dispatch_compile_remote` logs and stitches in-process ? a slow reel beats no
reel.

---

3. NVENC on the L4 ? and the D5/D16(b) parity note

The dominant term in the CUJ stitch was the **per-clip 4K re-encode** the branding watermark forces
(~67 s/clip on 2 vCPU with libx264). The L4 has a hardware H.264/HEVC encoder, so #521 adds an encoder
knob:

- `StitchingConfig.encoder` ? `libx264` (default, CPU, works everywhere incl. CI) | `h264_nvenc` |
  `hevc_nvenc`. `VideoCompiler` **probes** the requested encoder once per run (a 1-frame lavfi test
  encode) and **falls back to libx264** if this ffmpeg build can't run it, so a misconfig / non-GPU
  box never breaks. All clips share the one chosen encoder ? the concat stays `-c copy`.
- `deployment.cloud_stitch_encoder` (default `h264_nvenc`) ? what the compile-dispatch forwards onto
  the worker's `stitching.encoder` for the L4 run. Set it to `libx264` to run the L4 stitch on CPU.
- The control-plane's **own** local stitch always uses `stitching.encoder` (libx264 by default) ? the
  CP has no GPU.

Parity note (honest): [README.md](./README.md) **D5/D16(b)** kept stitch **encode = libx264** to
protect a byte-exact stitch parity guarantee, and explicitly **deferred NVENC encode behind a config
knob, "revisit only if M7 cost data justifies relaxing parity."** The 2026-07-08 CUJ (~13 min stitch,
a hard scaling wall past ~5 concurrent users) **is** that cost data. #521 lands exactly the deferred
shape ? NVENC **behind a config knob**, libx264 the default, config-reversible per phase. Enabling
NVENC relaxes byte-identity **at the reel ENCODE layer only** (a cosmetic, shareable artifact); the
WINDOW/rally-level parity (the analytical output) is untouched ? detection and windowing are
unchanged. If bit-exact reels are ever required, set `cloud_stitch_encoder=libx264`.

> Follow-up (tracked, not in #521's first cut): a **pre-rendered / overlay-filter watermark** so the
> stitch isn't a full re-encode at all (the deeper win beyond swapping the encoder).

---

4. Validation placement + the "nobody validates" guard

`phase_placement.validation` moves the CPU-bound validator suite off the 2-vCPU control-plane too,
via the existing `--skip-validation` lever (#513):

- **`control_plane`** (default): the control-plane runs the validators and is the authoritative gate;
  the worker **skips** its redundant re-validation (`skip_validation=True`) ? today's #512/#513
  behaviour, byte-identical.
- **`cloud_run_l4`**: when detection **actually** dispatches to the L4, the control-plane **delegates**
  ? it does NOT run the validator suite (`_execute_processing` skips it), and the dispatch carries
  no `--skip-validation`, so the **worker** validates and rejects a bad clip via a non-zero rc
  (surfaced as a failed job).

Footgun guard: a clip can only be validated on the worker if the worker also **detects** it there.
Two protections:

```
1. **Config-load guard** (`DeploymentConfig` model-validator): `validation=cloud_run_l4`
requires the
   effective segment placement to also be `cloud_run_l4` (explicit, or via
`compute_target=cloud_run`)
   ? else config load fails with an actionable error.
2. **Runtime guard** (`_will_detect_on_cloud`): if a per-request `compute='local'` forces
detection
   in-process (no worker), the control-plane **still validates** ? never leave a clip
unvalidated
   because nobody ran the gate.

---
```

5. Files & tests

```
| Area | Code | Tests |
|---|---|---|
| Placement resolver + config | `backend/config/placement.py`, `backend/config/models.py`
(`PhasePlacementConfig`, `DeploymentConfig.phase_placement` / `cloud_stitch_encoder`, guard) |
`tests/test_phase_placement.py`, `tests/test_config_deployment.py` |
| Preset | `backend/config/presets.py` (`cloud-serving` ?
`phase_placement.compile=cloud_run_l4`) | `tests/test_phase_placement.py` |
| compile dispatch | `backend/orchestration.py` (`_execute_compile`, `_dispatch_compile_remote`,
`_stitch_reel`, `_will_detect_on_cloud`) | `tests/test_compile_dispatch.py` |
| compute backend | `backend/compute/base.py` (`CompileJobSpec`, `run_compile`),
`backend/compute/cloud_run.py` (`run_compile`, `_await_marker`) |
`tests/test_compute_backend.py` |
| worker | `backend/worker/__main__.py` (`compile` subcommand, `cmd_compile`) |
`tests/test_worker.py` |
| NVENC encoder | `backend/pipeline/stitcher/compiler.py` (`encoder`, `_select_encoder`,
`_probe_encoder_available`) | `tests/test_compiler.py` |
| validation delegation | `backend/orchestration.py` (`_maybe_dispatch_remote` skip flag,
`_execute_processing`) | `tests/test_api_cloud_dispatch.py` |

All GPU-free + cloud-free: a fake `CloudRunJobClient` simulates the worker (reads the staged
spec,
writes the reel + done-marker into a local "bucket"), mirroring
`tests/test_api_cloud_dispatch.py`.
```

196. user (2026-07-08T05:43:27.007Z)

```
File created successfully at: C:\kg-wt521\docs\COMPUTE_DECOUPLED_SERVING\PHASE_PLACEMENT.md
(file state is current in your context ? no need to Read it back)
```

197. assistant (2026-07-08T05:43:29.786Z)

```
Now add a pointer to the new doc from the README, right after the honesty note:
```

198. user (2026-07-08T05:43:34.375Z)

```
<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>
```

199. user (2026-07-08T05:43:38.900Z)

```
9
10      ---
11
12      > **? Northstar (D14):** a **mobile, app-like** flow ? point at a **Google Drive** video
? **fully cloud-native** run ? the stitched, **ready-to-share** reel lands **back in Drive, next
to the source**. Two design promises around it: a user can **switch local?cloud anytime** (D13,
a headline feature), and execution is **never a surprise** ? auto-detect *suggests*, the user
*confirms* (D15).
```

200. user (2026-07-08T05:43:41.708Z)

```
8       > **Honesty note:** claims tagged `[verified]` (checked against code by the design
workflow `wf_56274ff0`), `[design]` (proposed), `[researched]`.
9
10      ---
```

201. user (2026-07-08T05:43:46.740Z)

```
The file C:\kg-wt521\docs\COMPUTE_DECOUPLED_SERVING\README.md has been updated successfully.
(file state is current in your context ? no need to Read it back)
```

202. assistant (2026-07-08T05:43:50.396Z)

Now re-run the doc gates and the full set of touched test files to confirm everything's green:

203. user (2026-07-08T05:43:53.429Z)

```
=== md_links ===
[check-md-links] clean ? no broken internal links in live docs.
=== leak_scan ===
[leak-scan] clean ? no MD5 / personal-path / private-name leaks in the code surface.
=== ruff ===
All checks passed!
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

204. user (2026-07-08T05:44:44.227Z)

```
tests/test_api_cloud_dispatch.py::test_full_api_process_job_result_carries_cloud_provenance
  C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
    warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
225 passed, 8 warnings in 44.19s
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

205. assistant (2026-07-08T05:44:57.688Z)

All 225 pass. Let me run the complete suite once more with coverage to confirm the whole thing is green after the budget fix (honoring the pre-LGTM full-suite gate):

206. user (2026-07-08T05:48:36.692Z)

```
training\gen0\harness.py                            130      9    93%
----------------------------------------------------------------------
TOTAL                                             12663   1810    86%
Required test coverage of 70% reached. Total coverage: 85.71%
1875 passed, 8 skipped, 25 warnings in 213.65s (0:03:33)
======================================================================
 RESULT: PASS - all tests green
======================================================================
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

207. assistant (2026-07-08T05:48:49.355Z)

Full suite green (1875 passed, 85.71% coverage). Let me re-confirm mypy and check the worktree's diff before committing (branching-safety: verify only my commits, stage explicit paths):

208. user (2026-07-08T05:48:56.168Z)

```
=== mypy ===
```

```
backend\providers\__init__.py:9: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
Success: no issues found in 150 source files
=== branch ===
feat/521-compile-on-l4
=== ahead of origin/master (should be empty) ===
=== changed files ===
 M backend/compute/__init__.py
 M backend/compute/base.py
 M backend/compute/cloud_run.py
 M backend/config/__init__.py
 M backend/config/models.py
 M backend/config/presets.py
 M backend/orchestration.py
 M backend/pipeline/stitcher/compiler.py
 M backend/worker/__main__.py
 M docs/COMPUTE_DECOUPLED_SERVING/README.md
 M tests/test_api_cloud_dispatch.py
 M tests/test_compiler.py
 M tests/test_compute_backend.py
 M tests/test_config_deployment.py
 M tests/test_worker.py
?? backend/config/placement.py
?? docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md
?? tests/test_compile_dispatch.py
?? tests/test_phase_placement.py
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

209. assistant (2026-07-08T05:49:07.247Z)

mypy clean, branch clean (no sibling commits), `main.py` correctly nets to zero. Let me stage explicit paths (never `-A`) and commit:

210. user (2026-07-08T05:49:10.960Z)

```
warning: in the working copy of 'backend/config/placement.py', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md', LF will be
replaced by CRLF the next time Git touches it
warning: in the working copy of 'tests/test_compile_dispatch.py', LF will be replaced by CRLF
the next time Git touches it
warning: in the working copy of 'tests/test_phase_placement.py', LF will be replaced by CRLF the
next time Git touches it
=== staged ===
M  backend/compute/__init__.py
M  backend/compute/base.py
M  backend/compute/cloud_run.py
M  backend/config/__init__.py
M  backend/config/models.py
A  backend/config/placement.py
M  backend/config/presets.py
M  backend/orchestration.py
M  backend/pipeline/stitcher/compiler.py
M  backend/worker/__main__.py
A  docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md
M  docs/COMPUTE_DECOUPLED_SERVING/README.md
M  tests/test_api_cloud_dispatch.py
A  tests/test_compile_dispatch.py
M  tests/test_compiler.py
M  tests/test_compute_backend.py
M  tests/test_config_deployment.py
A  tests/test_phase_placement.py
M  tests/test_worker.py
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

211. assistant (2026-07-08T05:49:16.143Z)

All 19 paths staged (the CRLF warnings are harmless line-ending normalization). Now commit:

212. user (2026-07-08T05:49:30.634Z)

```
[feat/521-compile-on-l4 8866ffe] perf(serving): run compile/stitch on the L4 worker; config-
driven phase?machine placement (#521)
 19 files changed, 1710 insertions(+), 55 deletions(-)
 create mode 100644 backend/config/placement.py
 create mode 100644 docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md
 create mode 100644 tests/test_compile_dispatch.py
 create mode 100644 tests/test_phase_placement.py
=== verify ===
8866ffe perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine
placement (#521)
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

213. assistant (2026-07-08T05:49:34.632Z)

Clean single commit. Now push and open the PR against master:

214. user (2026-07-08T05:49:38.643Z)

```
remote:
remote: Create a pull request for 'feat/521-compile-on-l4' on GitHub by visiting:
remote:      https://github.com/Khelsutra/badminton-highlight-indexer/pull/new/feat/521-compile-
on-l4
remote:
branch 'feat/521-compile-on-l4' set up to track 'origin/feat/521-compile-on-l4'.
To https://github.com/Khelsutra/badminton-highlight-indexer.git
 * [new branch]      feat/521-compile-on-l4 -> feat/521-compile-on-l4
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

215. assistant (2026-07-08T05:49:45.626Z)

Let me check the PR template, then open the PR:

216. user (2026-07-08T05:49:46.730Z)

```
<!-- Describe the change and link the issue it closes. -->
```

Summary

Checks

- [ ] `python run_all_tests.py` green; `ruff check .` + `mypy` clean; coverage not lowered.
- [ ] Internal markdown links valid (`python -m tools.check_md_links`).

Launch / "bit-flip" PRs only

A PR that flips a default / "launch bit" so a tracked project reaches terminal state MUST also run
the `docs/DOC_STATUS.md` §2 archival due-diligence on that project's docs, **in this same PR**:
- [ ] Verified each doc's claims against the code (tagged load-bearing facts `[verified]`).
- [ ] Flipped the doc Status ? DELIVERED/DONE/REJECTED + a completion note (what shipped, PR, paths).
- [ ] Updated all inbound references (`docs/README.md`, `CLAUDE.md`, `docs/NEXT_STEPS.md`, cross-doc links).
- [ ] Kept the ADR forward/back lineage.
- [ ] `git mv` the terminal docs to `docs/archives/?` and recorded the move in `DOC_STATUS.md` §3.
- [ ] Fixed any internal-link breakage from the move (`link-check` CI job green).

(Not a launch/bit-flip PR? Delete this section.)
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer

217. user (2026-07-08T05:50:11.329Z)

https://github.com/Khelsutra/badminton-highlight-indexer/pull/525
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer

218. assistant (2026-07-08T05:50:26.128Z)

PR #525 created. Let me verify it targets master and CI is running:

219. user (2026-07-08T05:50:31.007Z)

{"base":"master","head":"feat/521-compile-
on-l4","number":525,"state":"OPEN","title":"perf(serving): run compile/stitch on the L4 worker;
config-driven phase?machine placement (#521)","url":"https://github.com/Khelsutra/badminton-
highlight-indexer/pull/525"}
=== checks (may be pending) ===
Golden fixtures cross-OS (ubuntu-latest)	pending	0
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28920721803/job/85797160128
Ruff lint	pass	11s	https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28920721803/job/85797160091
Golden fixtures cross-OS (macos-latest)	pending	0	https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28920721803/job/85797160157
Leak scan (PII / attribution guard)	pass	8s	https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28920721803/job/85797160101
Markdown link check (live docs)	pass	6s	https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28920721803/job/85797160086
Built wheel smoke (install + launch)	pending	0	https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28920721803/job/85797160154
Dependency license scan (no copyleft in runtime deps)	pending	0
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28920721803/job/85797160080
Full test suite (offline, mocked)	pending	0	https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28920721803/job/85797160106
Golden fixtures cross-OS (windows-latest)	pending	0
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28920721803/job/85797160184
Mypy (lenient baseline)	pending	0	https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28920721803/job/85797160107
Syntax sweep + import smoke (no GPU stack)	pending	0
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28920721803/job/85797160088
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer

220. assistant (2026-07-08T05:50:41.795Z)

PR #525 is open against master; early gates (ruff, leak-scan, md-links) already pass. Let me
watch CI to completion:

221. user (2026-07-08T05:53:19.523Z)

Full test suite (offline, mocked)	pass	2m50s	https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28920721803/job/85797160106
Golden fixtures cross-OS (macos-latest)	pass	25s	https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28920721803/job/85797160157
Golden fixtures cross-OS (ubuntu-latest)	pass	27s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28920721803/job/85797160128
Golden fixtures cross-OS (windows-latest)	pass	53s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28920721803/job/85797160184

```
Leak scan (PII / attribution guard)     pass    8s      https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28920721803/job/85797160101
Markdown link check (live docs) pass    6s      https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28920721803/job/85797160086
Mypy (lenient baseline) pass    34s     https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28920721803/job/85797160107
Ruff lint       pass    11s     https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28920721803/job/85797160091
Syntax sweep + import smoke (no GPU stack)      pass    20s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28920721803/job/85797160088
Built wheel smoke (install + launch)    pass    1m4s    https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28920721803/job/85797160154
Dependency license scan (no copyleft in runtime deps)   pass    34s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28920721803/job/85797160080
Full test suite (offline, mocked)       pass    2m50s   https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28920721803/job/85797160106
Golden fixtures cross-OS (macos-latest) pass    25s     https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28920721803/job/85797160157
Golden fixtures cross-OS (ubuntu-latest)        pass    27s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28920721803/job/85797160128
Golden fixtures cross-OS (windows-latest)       pass    53s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28920721803/job/85797160184
Leak scan (PII / attribution guard)     pass    8s      https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28920721803/job/85797160101
Markdown link check (live docs) pass    6s      https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28920721803/job/85797160086
Mypy (lenient baseline) pass    34s     https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28920721803/job/85797160107
Ruff lint       pass    11s     https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28920721803/job/85797160091
Syntax sweep + import smoke (no GPU stack)      pass    20s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28920721803/job/85797160088
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

222. assistant (2026-07-08T05:53:27.609Z)

All 11 CI checks green. Let me file the deferred Part 3 (cheaper watermark) as a tracked follow-up and record a brief session-handoff memory:

223. user (2026-07-08T05:53:42.799Z)

Noted (position 1, task_id: task_217edf9c). A chip is showing for the user ? they can start it in a fresh worktree with one click, or dismiss it. If this suggestion becomes stale or superseded, call dismiss_task with this task_id. Currently pending: task_217edf9c "Cheaper reel watermark (avoid full re-encode)". Continue your current work.

224. assistant (2026-07-08T05:53:58.391Z)

```
---
name: pr-525-compile-on-l4
description: PR #525 (#521) shipped config-driven phase?machine placement + compile-on-L4 +
NVENC; Part 3 (cheaper watermark) deferred
metadata:
  type: project
---

**PR [#525](https://github.com/Khelsutra/badminton-highlight-indexer/pull/525)** (closes #521) ?
opened 2026-07-08, all 11 CI checks green, awaiting owner review/merge.
```

Drove the compile/stitch off the CPU-only 2-vCPU control-plane (2026-07-08 live CUJ: ~12m55s for an 11-clip 4K reel ? §9 of the validation-latency runlog) to the L4 worker.

What shipped:
- `deployment.phase_placement` (`validation`/`segment`/`compile` ? `control_plane|cloud_run_l4`), resolved by `backend/config/placement.py::resolve_phase_placement`. Unset ? legacy placement, byte-identical. Config-load guard blocks validation-on-L4 without a cloud segment.
- `kind='compile'` Cloud Run dispatch (`orchestration._dispatch_compile_remote`) mirroring `_maybe_dispatch_remote`; worker `compile` subcommand (`backend/worker/__main__.py::cmd_compile`) stitches via the shared `orchestration._stitch_reel` core (no forked pipeline, no worker DB ? a staged bucket spec carries resolved time-pairs + stitching config).
- NVENC: `StitchingConfig.encoder` + `deployment.cloud_stitch_encoder` (default `h264_nvenc`); `VideoCompiler._select_encoder` probes + falls back to libx264. Lands the D16(b)-deferred "NVENC behind a config knob".
- cloud-serving preset sets `phase_placement.compile=cloud_run_l4` (validation stays on control_plane by default even in cloud-serving).

Deferred (tracked): Part 3 = cheaper watermark so the stitch isn't a full re-encode (pre-rendered overlay, or watermark-at-concat so per-clip trims stream-copy). Flagged in `docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md` §3.

Not verified (owner-side): real L4 NVENC stitch wall ? deploy was out of scope; no `gcloud run deploy` made. Worktree at `C:/kg-wt521` (branch `feat/521-compile-on-l4`).

225. user (2026-07-08T05:53:58.427Z)

File created successfully at: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\memory\pr-525-compile-on-l4.md (file state is current in your context ? no need to Read it back)

226. user (2026-07-08T05:54:03.832Z)

pr-525-compile-on-l4.md

227. assistant (2026-07-08T05:54:07.961Z)

Memory index

- [PR #525 ? compile-on-L4 (#521)](pr-525-compile-on-l4.md) ? config-driven phase?machine placement + L4 NVENC stitch; Part 3 (cheaper watermark) deferred.

228. user (2026-07-08T05:54:07.997Z)

File created successfully at: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\memory\MEMORY.md (file state is current in your context ? no need to Read it back)

229. assistant (2026-07-08T05:54:28.996Z)

Done. All three parts of #521 are implemented, tested, and shipped as **[PR #525](https://github.com/Khelsutra/badminton-highlight-indexer/pull/525)** (against `master`, all **11 CI checks green**).

What landed

1. Compile/stitch dispatched to the L4 worker. A `kind='compile'` Cloud Run dispatch mirroring `_maybe_dispatch_remote`: the control-plane resolves the reel's segments, stages a compile-spec (resolved time-pairs + stitching config + source URI) to the bucket, dispatches `python -m backend.worker compile` on the L4, bucket-polls a done-marker (same crash-fast/timeout-rescue machinery as detect), and returns the same `{status, filepath, download_url}` shape the SPA expects. The worker stitches via the **shared `_stitch_reel` core**

(no forked pipeline, no worker DB). Control-plane stitch stays available as a config option.

2. Config-driven phase?machine placement. `deployment.phase_placement` (`validation`/`segment`/`compile` ? `control_plane | cloud_run_l4`), resolved by [`backend/config/placement.py`](backend/config/placement.py). **Unset ? today's placement, byte-identical.** A config-load guard rejects the "nobody validates" footgun. The cloud-serving preset sets `compile ? cloud_run_l4`. Flipping the knob moves validation/segment/compile between machines with no code change.

3. NVENC on the L4. `StitchingConfig.encoder` + `deployment.cloud_stitch_encoder` (default `h264_nvenc`) forwarded to the worker; `VideoCompiler` probes it once and **falls back to libx264** on CI/non-GPU boxes. This lands the D16(b)-deferred "NVENC-encode behind a config knob" now that the CUJ supplies the justifying cost data ? libx264 stays the parity-safe default; window/rally-level parity is untouched.

Verification (pre-LGTM gate, run locally)

- **Full suite: 1875 passed, 8 skipped**; `ruff check .` + `mypy backend training` clean; `tools.leak_scan` + `tools.check_md_links` clean; **coverage 85.7%** (floor 70). All GPU-free/cloud-free via a fake `CloudRunJobClient`.
- New tests: `test_phase_placement.py`, `test_compile_dispatch.py`; additions to `test_compute_backend.py`, `test_worker.py`, `test_compiler.py`, `test_api_cloud_dispatch.py`.

Honest limits

- **Deploy was out of scope** ? no `gcloud run deploy`/traffic change was made, so the *real L4 NVENC stitch wall* isn't measured here (that needs an owner-driven deploy + CUJ). The dispatch plumbing is proven end-to-end with mocks.
- **Part 3 deferred** as the task allowed: the NVENC change hardware-accelerates the *encode*, but each clip is still fully re-encoded because the watermark filter forces it. The deeper win (pre-rendered overlay / watermark-at-concat so trims stream-copy) is filed as a tracked follow-up chip and noted in `docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md` §3.

Work was done in an isolated worktree (`C:/kg-wt521`, branch `feat/521-compile-on-l4`) ? the shared checkout and sibling branches were never touched.

230. user (2026-07-08T05:54:31.372Z)

please open a PR with the changes and ensure thorough testing and use the rally-worker setup to test on real videos if possible

231. assistant (2026-07-08T05:55:30.719Z)

The PR is already open (#525) ? I'll confirm it, then focus on the new asks: **thorough testing** (including an adversarial review of my own diff) and **real-video testing via the rally-worker setup**. Let me start with parallel recon: PR state, available real videos in GCS, local ffmpeg/NVENC, the deploy tooling, and the currently-deployed worker image.

232. user (2026-07-08T05:55:34.790Z)


```
=== PR 525 ===
{"checks":[{"concl":"SUCCESS","name":"Built wheel smoke (install +
launch)"},{"concl":"SUCCESS","name":"Dependency license scan (no copyleft in runtime
deps)"},{"concl":"SUCCESS","name":"Full test suite (offline,
mocked)"},{"concl":"SUCCESS","name":"Golden fixtures cross-OS (macos-
latest)"},{"concl":"SUCCESS","name":"Golden fixtures cross-OS (ubuntu-
latest)"},{"concl":"SUCCESS","name":"Golden fixtures cross-OS (windows-
latest)"},{"concl":"SUCCESS","name":"Leak scan (PII / attribution
guard)"},{"concl":"SUCCESS","name":"Markdown link check (live
docs)"},{"concl":"SUCCESS","name":"Mypy (lenient baseline)"},{"concl":"SUCCESS","name":"Ruff
lint"},{"concl":"SUCCESS","name":"Syntax sweep + import smoke (no GPU
stack)"}],"mergeStateStatus":"CLEAN","mergeable":"MERGEABLE","state":"OPEN"}
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```


233. user (2026-07-08T05:55:41.168Z)

```
=== gcloud auth/project ===
[accessibility]
screen_reader = False
[billing]
quota_project = gen-lang-client-0306026826
[core]
account = avi.dullu@gmail.com
disable_usage_reporting = False
project = gen-lang-client-0306026826

Your active configuration is: [default]
=== corpus videos (small ones) ===
gs://khelsutra-rally-corpus/videos/GX010128.MP4
gs://khelsutra-rally-corpus/videos/GX010142_1080p.mp4
gs://khelsutra-rally-corpus/videos/GX010143_1080p.mp4
gs://khelsutra-rally-corpus/videos/GX010147_1080p.mp4
gs://khelsutra-rally-corpus/videos/GX030094_proxy.mp4
gs://khelsutra-rally-corpus/videos/mahadevpura-1.MP4
gs://khelsutra-rally-corpus/videos/mahadevpura_1.mp4
gs://khelsutra-rally-corpus/videos/mahadevpura_smoke_180s.mp4
gs://khelsutra-rally-corpus/videos/SHANAN/
gs://khelsutra-rally-corpus/videos/golden/
gs://khelsutra-rally-corpus/videos/golden_1080p/
```

234. user (2026-07-08T05:55:42.925Z)

```
=== ffmpeg version ===
ffmpeg version 8.1.1-full_build-www.gyan.dev Copyright (c) 2000-2026 the FFmpeg developers
built with gcc 15.2.0 (Rev13, Built by MSYS2 project)
=== nvenc encoders present? ===
 V....D av1_nvenc            NVIDIA NVENC av1 encoder (codec av1)
 V....D libx264              libx264 H.264 / AVC / MPEG-4 AVC / MPEG-4 part 10 (codec h264)
 V....D libx264rgb           libx264 H.264 / AVC / MPEG-4 AVC / MPEG-4 part 10 RGB (codec h264)
 V....D h264_amf             AMD AMF H.264 Encoder (codec h264)
 V....D h264_d3d12va         D3D12VA h264 encoder (codec h264)
 V....D h264_mf              H264 via MediaFoundation (codec h264)
```

235. user (2026-07-08T05:55:43.686Z)

```
=== deploy tooling ===
deploy/cloudrun:
cloudbuild.yaml
Dockerfile
gen_service_yaml.py
post_deploy_smoke.ps1
README.md

deploy/gcp:
buckets.env
create_gpu_vm.sh
nightly_regression
ops-agent-startup.sh
provision.sh
README.md
teardown.sh
=== build/deploy scripts ===
ls: cannot access 'deploy/cloudrun/*.sh': No such file or directory
deploy/gcp/create_gpu_vm.sh
deploy/gcp/ops-agent-startup.sh
deploy/gcp/provision.sh
deploy/gcp/teardown.sh
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

236. assistant (2026-07-08T05:55:56.142Z)

PR #525 is confirmed **OPEN, MERGEABLE, all 11 checks green**. Local ffmpeg has NVENC-family encoders but **no `h264_nvenc`** (this is an AMD box) ? so I can test the libx264 fallback path against a real probe. Let me check the deployed worker image, video sizes, and the build flow:

237. user (2026-07-08T05:56:04.183Z)

```
=== rally-worker Job image ===
asia-southeast1-docker.pkg.dev/gen-lang-client-0306026826/rally/rally-worker:latest
=== candidate video sizes ===
 767766440  2026-06-22T13:42:54Z  gs://khelsutra-rally-corpus/videos/GX010142_1080p.mp4
TOTAL: 1 objects, 767766440 bytes (732.20MiB)
  90194321  2026-06-20T14:20:32Z  gs://khelsutra-rally-corpus/videos/mahadevpura_smoke_180s.mp4
TOTAL: 1 objects, 90194321 bytes (86.02MiB)
1429944539  2026-06-20T14:27:35Z  gs://khelsutra-rally-corpus/videos/GX030094_proxy.mp4
TOTAL: 1 objects, 1429944539 bytes (1.33GiB)
```

238. user (2026-07-08T05:56:04.819Z)

```
=== cloudbuild.yaml ===
# Cloud Build config for the Cloud Run GPU worker image (COMPUTE_DECOUPLED_SE
#
# `gcloud builds submit --tag ? -f ?` does NOT work ? `gcloud builds submit` has no `-f`
# non-root Dockerfile needs a build CONFIG like this one. Build with:
#
#   gcloud builds submit --config=deploy/cloudrun/cloudbuild.yaml --project <PROJECT
#
# Override the region/repo/image with --substitutions=_REGION=?,_REPO=?,_IMAGE=?
# PROJECT_ID is provided by Cloud Build automatically.
substitutions:
  _REGION: asia-southeast1   # L4 GPU is region-limited (asia-southeast1 / europe-west1 /
europe-west4 / us-central1 / us-east4)
  _REPO: rally
  _IMAGE: rally-worker
steps:
  - name: 'gcr.io/cloud-builders/docker'
    args:
      - build
      - -t
      - '${_REGION}-docker.pkg.dev/${PROJECT_ID}/${_REPO}/${_IMAGE}:latest'
      - -f
      - deploy/cloudrun/Dockerfile
      - .
images:
  - '${_REGION}-docker.pkg.dev/${PROJECT_ID}/${_REPO}/${_IMAGE}:latest'
options:
  machineType: E2_HIGHCPU_8   # the CUDA + micromamba image is heavy; a bigger builder keeps it
~8?10 min
timeout: 3600s
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

239. assistant (2026-07-08T05:56:40.721Z)

PR is fully green and mergeable. Now the important findings for the L4 test: the deployed `rally-worker` runs `:latest`, built from an **old commit without my `compile` subcommand** ? so a real L4 compile test requires **building a new image**, and `:latest` is the *shared live tag* (control-plane + worker both use it). That's a traffic-affecting deploy I must not do unilaterally.

Let me do the strongest tests I *can* do safely first: a **real-video end-to-end worker compile** (real ffmpeg, real clips) and a **real-GCS dispatch round-trip**. Let me pull a real

clip:

240. user (2026-07-08T05:56:55.910Z)

```
pulling real 180s clip...
.....................................

Average throughput: 21.7MiB/s
=== source probe ===
codec_name=h264
width=1920
height=1080
r_frame_rate=30000/1001
codec_name=aac
r_frame_rate=0/0
codec_name=unknown
r_frame_rate=0/0
duration=180.046167
size=90194321
```

241. assistant (2026-07-08T05:57:13.529Z)

Real 1080p/180s H.264 clip. Now let me run the **worker `compile` subcommand end-to-end on the real video** ? with `encoder=h264_nvenc` in the spec (as the L4 dispatch forwards), which will exercise the real probe ? libx264 fallback (no NVIDIA GPU here) plus a real watermark burn:

242. user (2026-07-08T05:57:20.482Z)

```
video_id=05e0e577029daafcf425ca70c0c4f0a3
=== spec written, video_id in reel path ===
```

243. user (2026-07-08T05:57:42.128Z)

```
=== RUN worker compile on the REAL 180s clip ===
11:27:34 [INFO] backend.pipeline.stitcher.compiler: Source video duration: 180.05s
11:27:34 [INFO] backend.pipeline.stitcher.compiler: Trimming 4 segments (begin_padding=0.5s,
end_padding=0.5s)...
11:27:34 [INFO] backend.pipeline.stitcher.compiler: [watermark] applying branding overlay to
each clip.
11:27:35 [WARNING] backend.pipeline.stitcher.compiler: [compile] encoder 'h264_nvenc' is not
runnable in this ffmpeg build ? falling back to libx264 (CPU). On the L4 worker this should be
available.
11:27:35 [INFO] backend.pipeline.stitcher.compiler: Trimming Segment #1/4 [5.00s - 12.00s] ->
[4.500s - 12.500s] (duration: 8.00s)
11:27:36 [INFO] backend.pipeline.stitcher.compiler: Trimming Segment #2/4 [40.00s - 48.00s] ->
[39.500s - 48.500s] (duration: 9.00s)
11:27:38 [INFO] backend.pipeline.stitcher.compiler: Trimming Segment #3/4 [90.00s - 98.00s] ->
[89.500s - 98.500s] (duration: 9.00s)
11:27:39 [INFO] backend.pipeline.stitcher.compiler: Trimming Segment #4/4 [150.00s - 158.00s] ->
[149.500s - 158.500s] (duration: 9.00s)
11:27:41 [INFO] backend.pipeline.stitcher.compiler: Successfully trimmed 4/4 clips. Preparing to
stitch...
11:27:41 [INFO] backend.pipeline.stitcher.compiler: Stitching clips together...
11:27:41 [INFO] backend.pipeline.stitcher.compiler: Dynamic highlight reel saved:
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-khelsutra-guru-badminton-
highlight-indexer/75d3d121-f240-4293-afa5-b436a379ad31/scratchpad/compiletest/wscratch\smoke-
reel.mp4 (21.8 MB)
11:27:41 [INFO] backend.worker: [worker] wrote compile marker ->
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-khelsutra-guru-badminton-
highlight-
indexer/75d3d121-f240-4293-afa5-b436a379ad31/scratchpad/compiletest/bucket/_compile/tok.done
[worker] compile: video_id=05e0e577029daafcf425ca70c0c4f0a3 clips=4 encoder=h264_nvenc
out=smoke-reel.mp4
[worker] compile: stitching
```

```
[worker] compile: finished

real    0m10.161s
user    0m0.123s
sys     0m0.123s
RC=0
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

244. assistant (2026-07-08T05:57:49.224Z)

The worker compile ran end-to-end on the real clip in ~10s: probed `h264_nvenc` ? **fell back to libx264** (fallback works on real ffmpeg), trimmed 4/4 real clips with watermark, stitched a 21.8 MB reel, wrote the marker, RC=0. Let me verify the mirrored reel + marker + that the reel is a valid playable mp4:

245. user (2026-07-08T05:57:54.868Z)

```
=== done-marker ===
{"video_id": "05e0e577029daafcf425ca70c0c4f0a3", "returncode": 0, "status": "success",
"download_url": "/api/highlights/05e0e577029daafcf425ca70c0c4f0a3/smoke-reel.mp4", "reel_uri":
"", "message": ""}
=== mirrored reel in bucket (highlights/<video_id>/) ===
ls: cannot access 'C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-khelsutra-
guru-badminton-highlight-indexer/75d3d121-f240-4293-afa5-
b436a379ad31/scratchpad/compiletest/bucket/highlights/05e0e577029daafcf425ca70c0c4f0a3/': No
such file or directory
=== reel is a valid mp4? (ffprobe the mirrored copy) ===
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-khelsutra-guru-badminton-
highlight-indexer/75d3d121-f240-4293-afa5-
b436a379ad31/scratchpad/compiletest/bucket/highlights/05e0e577029daafcf425ca70c0c4f0a3/smoke-
reel.mp4: No such file or directory
=== expected reel duration ~ sum of padded clips (8+9+9+9 = 35s) ===
```

246. assistant (2026-07-08T05:58:32.987Z)

The reel was produced (21.8 MB) but the mirror was correctly skipped ? my local-path `output_root` has no `://`, so `reel_object_uri` returns `""` (that's the intended "truly-local ? serve the local file" path). On the real L4 the `output_root` is `gs://?`, so the mirror fires. Let me verify the produced reel is valid, then run the **real-GCS round-trip** (fetch source from GCS + upload reel to GCS + marker to GCS) to exercise the exact storage plumbing the L4 uses:

247. user (2026-07-08T05:58:49.018Z)

```
=== the produced reel (local output_dir) is a valid mp4 ===
codec_name=h264
width=1920
height=1080
codec_name=aac
codec_name=unknown
duration=35.089000
size=22863973
=== gcs lib present locally? ===
google-cloud-storage OK
=== write access to serving bucket scratch prefix? ===

...................
probe 1783490318
```

248. assistant (2026-07-08T05:59:00.525Z)

Reel is valid: 1920×1080 H.264+AAC, **35.09s** duration = exactly the 4 padded clips (8+9+9+9). GCS write access confirmed. Now the **full real-GCS round-trip** ? worker compile fetching a gs:// source + gs:// spec and uploading the reel + marker to gs:// (the exact plumbing the L4

uses):

249. user (2026-07-08T05:59:13.827Z)

```
=== stage spec to GCS ===
..............
staged: gs://khelsutra-rally-serving/_pr525test/_compile/tok/spec.json (locale=hi ? Hindi
watermark + Devanagari font)
```

250. user (2026-07-08T05:59:52.223Z)

```
=== RUN worker compile with REAL GCS I/O (fetch gs source + spec, upload reel + marker to gs)
===
C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
11:29:38 [INFO] backend.pipeline.stitcher.compiler: Source video duration: 180.05s
11:29:38 [INFO] backend.pipeline.stitcher.compiler: Trimming 3 segments (begin_padding=0.5s,
end_padding=0.5s)...
11:29:38 [INFO] backend.pipeline.stitcher.compiler: [watermark] applying branding overlay to
each clip.
11:29:38 [WARNING] backend.pipeline.stitcher.compiler: [compile] encoder 'h264_nvenc' is not
runnable in this ffmpeg build ? falling back to libx264 (CPU). On the L4 worker this should be
available.
11:29:38 [INFO] backend.pipeline.stitcher.compiler: Trimming Segment #1/3 [5.00s - 12.00s] ->
[4.500s - 12.500s] (duration: 8.00s)
11:29:39 [INFO] backend.pipeline.stitcher.compiler: Trimming Segment #2/3 [40.00s - 48.00s] ->
[39.500s - 48.500s] (duration: 9.00s)
11:29:41 [INFO] backend.pipeline.stitcher.compiler: Trimming Segment #3/3 [90.00s - 98.00s] ->
[89.500s - 98.500s] (duration: 9.00s)
11:29:43 [INFO] backend.pipeline.stitcher.compiler: Successfully trimmed 3/3 clips. Preparing to
stitch...
11:29:43 [INFO] backend.pipeline.stitcher.compiler: Stitching clips together...
11:29:43 [INFO] backend.pipeline.stitcher.compiler: Dynamic highlight reel saved:
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-khelsutra-guru-badminton-
highlight-
indexer/75d3d121-f240-4293-afa5-b436a379ad31/scratchpad/compiletest/gcs_wscratch\smoke-reel.mp4
(16.2 MB)
C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
11:29:51 [INFO] backend.worker: [worker] wrote compile marker -> gs://khelsutra-rally-
serving/_pr525test/_compile/tok.done
[worker] compile: video_id=05e0e577029daafcf425ca70c0c4f0a3 clips=3 encoder=h264_nvenc
out=smoke-reel.mp4
[worker] compile: stitching
[worker] compile: uploading reel
[worker] compile: finished
worker-rc=0
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

251. assistant (2026-07-08T06:00:00.729Z)

Real-GCS round-trip succeeded (fetch gs source + spec ? stitch ? **"uploading reel"** ? marker to gs://, RC=0). Let me verify the reel + marker landed in GCS and the reel is valid, then confirm the marker now carries a populated `reel_uri`:

252. user (2026-07-08T06:00:41.719Z)

```
=== reel mirrored to GCS? ===
  16940481  2026-07-08T05:59:48Z  gs://khelsutra-rally-
serving/_pr525test/highlights/05e0e577029daafcf425ca70c0c4f0a3/smoke-reel.mp4
TOTAL: 1 objects, 16940481 bytes (16.16MiB)
=== marker in GCS (reel_uri now populated) ===
{"video_id": "05e0e577029daafcf425ca70c0c4f0a3", "returncode": 0, "status": "success",
"download_url": "/api/highlights/05e0e577029daafcf425ca70c0c4f0a3/smoke-reel.mp4", "reel_uri":
"gs://khelsutra-rally-serving/_pr525test/highlights/05e0e577029daafcf425ca70c0c4f0a3/smoke-
reel.mp4", "message": ""}
=== download the GCS reel + ffprobe (real, playable) ===
Average throughput: 35.6MiB/s
codec_name=h264
width=1920
height=1080
codec_name=aac
codec_name=unknown
duration=26.080000
size=16940481
```

253. assistant (2026-07-08T06:01:30.508Z)

Fully verified against real GCS: reel mirrored (16.16 MB), marker carries a populated `reel_uri` + `download_url` (exactly what the CP reads back), and the downloaded reel is valid (1920×1080, 26.08s = 3 padded clips). The **entire worker path runs correctly against real GCS**; only NVENC + the CP-initiated L4 dispatch remain.

Since ultracode is on, let me launch an adversarial multi-agent review of my diff (to catch bugs before the owner) while I run the final integration test. First, capture the diff for the reviewers:

254. user (2026-07-08T06:01:33.798Z)

```
79 C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-khelsutra-guru-badminton-
highlight-indexer/75d3d121-f240-4293-afa5-b436a379ad31/scratchpad/review/pr525.diff
saved diff for reviewers
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

255. assistant (2026-07-08T06:01:47.397Z)

Only 79 lines ? that's wrong for a ~1700-line change. Let me diagnose:

256. user (2026-07-08T06:01:49.275Z)

```
=== refs ===
dac6736cb23a681135b707d768014820b239a981
8866ffe2757e8fb87b84dcdb4c4158fa1f42f98f
=== stat ===
 ..._cloud-serving_validation-latency_2026-07-07.md | 48 +++--------------------
 1 file changed, 6 insertions(+), 42 deletions(-)
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

257. assistant (2026-07-08T06:02:09.523Z)

`origin/master` **moved** (699d85b ? dac6736) ? a sibling session merged the §9 runlog update while I worked (exactly the "master moves under you" case the safety rules warn about). My branch doesn't touch that file, so the PR stays clean/mergeable. Let me verify branch

cleanliness and regenerate the diff against the true merge-base:

258. user (2026-07-08T06:02:14.378Z)

```
merge-base=699d85bc0abd6e35d3196a4566dcec23f649394e
=== my commits since merge-base (must be ONLY mine) ===
8866ffe perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine
placement (#521)
=== does new master's §9 change conflict with my branch? (files I touch vs master's change) ===
backend/compute/__init__.py
backend/compute/base.py
backend/compute/cloud_run.py
backend/config/__init__.py
backend/config/models.py
backend/config/placement.py
backend/config/presets.py
backend/orchestration.py
backend/pipeline/stitcher/compiler.py
backend/worker/__main__.py
docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md
docs/COMPUTE_DECOUPLED_SERVING/README.md
docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_validation-
latency_2026-07-07.md
tests/test_api_cloud_dispatch.py
tests/test_compile_dispatch.py
tests/test_compiler.py
tests/test_compute_backend.py
tests/test_config_deployment.py
tests/test_phase_placement.py
tests/test_worker.py
=== regenerate diff from merge-base ===
2166 C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-khelsutra-guru-badminton-
highlight-indexer/75d3d121-f240-4293-afa5-b436a379ad31/scratchpad/review/pr525.diff
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

259. assistant (2026-07-08T06:02:36.533Z)

The name-only list shows my backend files in `merge-base..origin/master` ? that suggests **PR #525 may have already been merged** (as a squash). Let me confirm the master state and PR status before anything else:

260. user (2026-07-08T06:02:40.481Z)

```
=== PR 525 state ===
{"mergeCommit":"dac6736cb23a681135b707d768014820b239a981","mergedAt":"2026-07-
08T05:59:33Z","state":"MERGED"}
=== recent origin/master commits ===
dac6736 perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine
placement (#521) (#525)
bf97700 docs(runlogs): record live SPA CUJ closing the 2 owner-gated validation-latency cells
(§9) (#522)
699d85b feat(validation): cache validation verdict by video_id to skip re-validation on reupload
(#518)
71c88ef docs(runlogs): validation-latency closure report for #512 + #513 (cloud-serving) (#519)
115b83d docs(runlogs): record live smoke of the crash-before-marker fast-fail (#515) (#517)
ef41473 fix(cloud-run): fail fast when a worker execution terminates without a done-marker
(#515)
=== dac6736 summary ===
dac6736 perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine
placement (#521) (#525)
 backend/compute/__init__.py                      |   2 +
 backend/compute/base.py                          |  34 +++
 backend/compute/cloud_run.py                     | 100 ++++++-
 backend/config/__init__.py                       |  16 ++
```

```
 backend/config/models.py                           |  52 ++++
 backend/config/placement.py                        |  59 ++++
 backend/config/presets.py                          |  10 +-
 backend/orchestration.py                           | 322 ++++++++++++++++++++---
 backend/pipeline/stitcher/compiler.py              |  94 ++++++-
 backend/worker/__main__.py                         | 139 ++++++++-
 docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md  | 169 ++++++++++
 docs/COMPUTE_DECOUPLED_SERVING/README.md           |   1 +
 tests/test_api_cloud_dispatch.py                   |  70 +++++
 tests/test_compile_dispatch.py                     | 204 +++++++++++++
 tests/test_compiler.py                             |  64 +++++
 tests/test_compute_backend.py                      | 111 ++++++++
 tests/test_config_deployment.py                    |   5 +
 tests/test_phase_placement.py                      | 138 ++++++++++
 tests/test_worker.py                               | 175 ++++++++++++
 19 files changed, 1710 insertions(+), 55 deletions(-)
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

261. assistant (2026-07-08T06:03:16.007Z)

PR #525 is already MERGED (squash `dac6736`, at 05:59 UTC ? merged while I was running the real-video tests). So master now carries all 19 files / 1710 insertions of #521, and my real-video tests above validated the *merged* code. The §9 runlog (#522) is also on master now, so my doc reference resolves.

Since it's merged, any issue a review surfaces should go in a follow-up PR. Let me run a rigorous adversarial multi-agent review of the merged diff (ultracode) to make "thorough testing" real ? findings verified by independent skeptics before I trust them:

262. user (2026-07-08T06:03:19.869Z)

```
Your configuration specifies to merge with the ref 'refs/heads/feat/521-compile-on-l4'
from the remote, but no such ref was fetched.
--- worktree still on my branch (has the exact merged code) ---
8866ffe perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine
placement (#521)
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

263. assistant (2026-07-08T06:03:25.491Z)

Branch was auto-deleted post-merge (normal for squash) ? my worktree at `8866ffe` holds the exact merged code. Now the adversarial review workflow:

264. user (2026-07-08T06:04:07.681Z)

```
Workflow launched in background. Task ID: ws7lmnehf
Summary: Adversarially review the merged #521 compile-on-L4 diff; verify each finding before
trusting it
Transcript dir: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-
badminton-highlight-
indexer\75d3d121-f240-4293-afa5-b436a379ad31\subagents\workflows\wf_5f13fc79-bad
Script file: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-
highlight-indexer\75d3d121-f240-4293-afa5-b436a379ad31\workflows\scripts\pr525-adversarial-
review-wf_5f13fc79-bad.js
(Edit this file with Write/Edit and re-invoke Workflow with {scriptPath:
"C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-
indexer\75d3d121-f240-4293-afa5-b436a379ad31\workflows\scripts\pr525-adversarial-review-
wf_5f13fc79-bad.js"} to iterate without resending the script.)
Run ID: wf_5f13fc79-bad
To resume after editing the script: Workflow({scriptPath: "C:\Users\avidu\.claude\projects\C--
Users-avidu-Projects-khelsutra-guru-badminton-highlight-
indexer\75d3d121-f240-4293-afa5-b436a379ad31\workflows\scripts\pr525-adversarial-review-
wf_5f13fc79-bad.js", resumeFromRunId: "wf_5f13fc79-bad"}) ? completed agents return cached
results (cached results may themselves be empty ? inspect journal.jsonl before assuming there is
```

something to recover).

You will be notified when it completes. Use /workflows to watch live progress.

265. assistant (2026-07-08T06:04:34.490Z)

The review runs in the background. Meanwhile, let me update memory (PR merged + tests done) and clean up the GCS test artifacts (leave-no-trace):

266. user (2026-07-08T06:04:42.657Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\memory\pr-525-compile-on-l4.md has been updated successfully. (file state is current in your context ? no need to Read it back)

267. user (2026-07-08T06:05:04.845Z)

```
=== clean up GCS test artifacts (leave-no-trace) ===
Removing gs://khelsutra-rally-serving/_pr525test/_compile/tok/spec.json#1783490354082363...
Removing gs://khelsutra-rally-serving/_pr525test/probe.txt#1783490326257775...
..............................
=== verify gone ===
ERROR: (gcloud.storage.ls) One or more URLs matched no objects.
```

268. assistant (2026-07-08T06:05:16.843Z)

GCS artifacts cleaned. Now let me gather the precise deployed-infra facts so I can give the user an accurate L4-test plan (does the control-plane also share `:latest`? what GPU/mounts does the Job need?):

269. user (2026-07-08T06:05:27.443Z)

```
=== control-plane SERVICE image (does it also use :latest?) ===
asia-southeast1-docker.pkg.dev/gen-lang-client-0306026826/rally/rally-worker:latest
=== rally-worker JOB: gpu / mounts / timeout / SA ===
spec:
  template:
    spec:
      template:
        spec:
          containers:
          - resources:
              limits:
                cpu: '8'
                memory: 32Gi
                nvidia.com/gpu: '1'
            volumeMounts:
            - mountPath: /gcs
              name: serving
          nodeSelector:
            run.googleapis.com/accelerator: nvidia-l4
          serviceAccountName: 492532266377-compute@developer.gserviceaccount.com
          timeoutSeconds: '3600'
          volumes:
          - csi:
              driver: gcsfuse.run.googleapis.com
              volumeAttributes:
                bucketName: khelsutra-rally-serving
            name: serving
```

270. assistant (2026-07-08T06:06:37.952Z)

The key infra fact for the L4 test: **both the live control-plane and the worker Job run `rally-worker:latest`** (8 vCPU / L4 / gcsfuse mount of `khelsutra-rally-serving`). The deployed image

predates my `compile` subcommand, so a real L4 exec needs a fresh image build ? which touches the shared live tag. That's a spend + live-infra decision (D17 requires confirming spend first), so I'll tee it up for you rather than deploy unilaterally.

Here's where things stand while the adversarial review finishes:

? PR ? done and MERGED. #525 squash-merged to master as `dac6736` (all 11 CI checks green). "Open a PR" is satisfied; no duplicate needed.

? Real-video testing (on the merged code, no deploy required): I ran the actual `python -m backend.worker compile` handler end-to-end on a real 1080p/180s corpus clip:
- **Local:** requested `h264_nvenc` ? the probe correctly found it unrunnable on this AMD box ? **fell back to libx264**, trimmed 4 real clips with the branding watermark, produced a valid 35.09s reel. The NVENC-fallback path works on real ffmpeg.
- **Real GCS round-trip:** fetched a `gs://` source + `gs://` spec, stitched, **uploaded the reel to `gs://khelsutra-rally-serving/?/highlights/<id>/` and wrote the `gs://` done-marker** (with `reel_uri` + `download_url` populated ? exactly what the control-plane reads back). Downloaded reel verified valid. This exercises the *exact* worker code path the L4 runs ? only NVENC hardware encode and the CP-initiated Cloud Run dispatch differ (both proven with mocks). Test artifacts cleaned up from GCS.

? Running now: an adversarial multi-agent review of the merged diff (5 review dimensions ? each finding refuted-or-confirmed by 2 independent skeptics) to catch anything the mocked tests hide. I'll report verified findings when it lands, and file a follow-up PR if anything real surfaces.

? Pending your call ? the real L4 execution. It's possible but needs an image build (~8?10 min, ~$0.5?1 total incl. one L4 exec, well under the D17 cap) and touches the shared `:latest`. I'll lay out the exact options (rebuild `:latest` vs. an isolated throwaway `rally-worker-pr525` Job) with the review results so you can decide with full information.

271. user (2026-07-08T06:13:32.137Z)

<system-reminder>
The user started your suggested background task task_217edf9c ("Cheaper reel watermark (avoid full re-encode)") in a separate local session. It is running independently. You will be notified here when it ends.
</system-reminder>

dont run the real test. I will make a release today and fold all these in

272. assistant (2026-07-08T06:13:50.121Z)

Understood ? I won't touch the L4 / any deploy. Since you're releasing today, let me make sure the adversarial review lands so any real finding can go into that release. Let me check its status:

273. user (2026-07-08T06:13:53.700Z)

```
=== dir ===
total 2533
drwxr-xr-x 1 avidu 197609      0 Jul  8 11:42 .
drwxr-xr-x 1 avidu 197609      0 Jul  8 11:34 ..
-rw-r--r-- 2 avidu 197609 392347 Jul  8 11:41 agent-a0baf7ebe53c21630.jsonl
-rw-r--r-- 1 avidu 197609     48 Jul  8 11:34 agent-a0baf7ebe53c21630.meta.json
-rw-r--r-- 2 avidu 197609 109226 Jul  8 11:43 agent-a26929d4c969fb9c5.jsonl
-rw-r--r-- 2 avidu 197609 437700 Jul  8 11:43 agent-a52a7d189c652e8cf.jsonl
-rw-r--r-- 1 avidu 197609     48 Jul  8 11:34 agent-a52a7d189c652e8cf.meta.json
-rw-r--r-- 2 avidu 197609 415462 Jul  8 11:43 agent-a588aba9828101736.jsonl
-rw-r--r-- 1 avidu 197609     48 Jul  8 11:34 agent-a588aba9828101736.meta.json
-rw-r--r-- 2 avidu 197609 123009 Jul  8 11:43 agent-a71a55a475829e001.jsonl
-rw-r--r-- 2 avidu 197609 143998 Jul  8 11:43 agent-a77adcbc098c7cca7.jsonl
-rw-r--r-- 2 avidu 197609 372457 Jul  8 11:42 agent-a7e807a82054c4b5a.jsonl
-rw-r--r-- 1 avidu 197609     48 Jul  8 11:34 agent-a7e807a82054c4b5a.meta.json
```

```
-rw-r--r-- 2 avidu 197609  94686 Jul  8 11:43 agent-a9ecb0b2f9a51ffea.jsonl
-rw-r--r-- 2 avidu 197609 468507 Jul  8 11:43 agent-af2e3f7269e288069.jsonl
-rw-r--r-- 1 avidu 197609     48 Jul  8 11:34 agent-af2e3f7269e288069.meta.json
-rw-r--r-- 2 avidu 197609   5495 Jul  8 11:42 journal.jsonl
=== journal tail (if present) ===
{"type":"started","key":"v2:e0ded2cd4e19dcc6fbbf04e22dc3900edf2d93a82cb14a97db37b6b966d5a367","a
gentId":"a7e807a82054c4b5a"}
{"type":"started","key":"v2:b51d73523cb7de4cba49008d6e0dc00b811137c64c4aa7ebf0cdf707b3d9e5a9","a
gentId":"a0baf7ebe53c21630"}
{"type":"started","key":"v2:f1a3dd4c46f003f0df7ee70898d1fc42eb8c5ea73cac6636837c9160023b0039","a
gentId":"a588aba9828101736"}
{"type":"started","key":"v2:0f8b659245135fe7e2ec10f1b5c8f40946026825bbc1040cf1b527e99a55c791","a
gentId":"af2e3f7269e288069"}
{"type":"started","key":"v2:7abb01a326ea80c73d898d3ccf7b6af41ab29be9762b23572bd7c03b865f5f0f","a
gentId":"a52a7d189c652e8cf"}
{"type":"result","key":"v2:b51d73523cb7de4cba49008d6e0dc00b811137c64c4aa7ebf0cdf707b3d9e5a9","ag
entId":"a0baf7ebe53c21630","result":{"findings":[]}}
{"type":"result","key":"v2:e0ded2cd4e19dcc6fbbf04e22dc3900edf2d93a82cb14a97db37b6b966d5a367","ag
entId":"a7e807a82054c4b5a","result":{"findings":[{"file":"backend/orchestration.py","line":689,"
severity":"medium","category":"robustness","summary":"When validation is delegated to the L4
worker (phase_placement.validation=cloud_run_l4), a worker validation REJECTION surfaces to the
user as an opaque \"cloud job failed (returncode 2).\" with no reason, silently dropping the
validation feedback the in-process path provides.","failure_scenario":"Config:
deployment.phase_placement.validation=cloud_run_l4 with segment=cloud_run_l4 (or
compute_target=cloud_run). A user uploads a too-blurry / not-badminton clip. _execute_processing
sets validate_on_control_plane=False (orch:865-868), so the control-plane skips its validators
and dispatches with skip_validation=worker_skips_validation=False (orch:666-674). The worker's
blur/relevance validator rejects the clip: cmd_infer hits `if failures: return _fail(2)`
(worker/__main__.py:333-338), which writes a done-marker via _write_done_marker containing ONLY
{video_id, returncode:2, segment_count:0, status:\"failed\"} ? NO message/validator-reason field
(worker/__main__.py:228-238). run_infer returns ComputeResult(returncode=2);
_maybe_dispatch_remote line 688-689 then returns _failed(f\"cloud job failed (returncode
{result.returncode}).\") ? it never reads result.report.get('message') (unlike the compile path
at orch:1393). The user sees \"cloud job failed (returncode 2).\" instead of the specific 'video
too blurry / not badminton' message the in-process control-plane path returns. This failure mode
is newly reachable in #521 (pre-#521 skip_validation was hardcoded True so the worker never
validated) and is untested ? the new tests
(test_validation_delegated_to_worker_when_placed_on_l4 / _not_delegated_...) only cover the PASS
path and never assert the worker-rejection message.","suggested_fix":"Carry the validation-
failure reason through the delegated path: (1) include a `message` (joined failing-validator
messages) in cmd_infer's failure done-marker (extend _write_done_marker like the compile
marker), and (2) in _maybe_dispatch_remote line 688-689 surface it, e.g. `msg =
str(result.report.get('message') or f\"cloud job failed (returncode {result.returncode}).\")` ?
mirroring _dispatch_compile_remote at orch:1391-1396. Add a test that a worker validation
rejection returns a reason, not just an rc."},{"file":"backend/worker/__main__.py","line":488,"s
everity":"low","category":"robustness","summary":"cmd_compile writes its done-marker ONLY after
_stitch_reel returns; any failure BEFORE that (spec fetch/parse, time_pairs/stitching coercion,
VideoCompiler init) raises out of an unwrapped main(), so the worker exits with a traceback and
NO marker ? unlike cmd_infer's _fail() which always writes a marker for every worker-side
failure.","failure_scenario":"The bucket-staged spec is truncated/corrupt (transient partial
read) or time_pairs contains a malformed pair. `_read_compile_spec` (worker:426-432) raises
JSONDecodeError/ValueError, or the coercion `[(float(pair[0]), float(pair[1])) for pair ...]`
(worker:493-495) / `StitchingConfig(**...)` (worker:496) raises IndexError/TypeError.
cmd_compile has no surrounding try/except and main() at line 973 calls args.func(args)
unwrapped, so the process exits non-zero with a raw traceback and never writes the --done-uri
marker. The dispatcher's bucket-poll then depends ENTIRELY on the #515 crash-fast Executions-API
probe to convert this into a clean failed job; if that probe is unavailable/delayed, the poll
hangs up to deployment.remote_poll_max_wait_sec (~65 min). cmd_infer avoids this asymmetry ? its
_fail() writes a real-rc marker for every pre-segmentation failure so the dispatcher terminates
fast regardless of the Executions API.","suggested_fix":"Give cmd_compile the same discipline as
cmd_infer: wrap spec-read + coercion in try/except and, on failure, write a failed done-marker
(rc + message) via _write_compile_marker before returning a non-zero rc, so a corrupt/partial
spec is a clean fast failed-job rather than relying solely on the crash-fast probe."}]}}
{"type":"started","key":"v2:f6f1728ecec9b7e3222de5ecf00b5f86a4e7cd1bfce22e89aa41434a766b8cba","a
gentId":"a9ecb0b2f9a51ffea"}
```

{"type":"started","key":"v2:943a9bc5cadee38ce2782798aeb82f817f8d1e098c5c89e63eb92592e8250e2c","agentId":"a26929d4c969fb9c5"}
{"type":"started","key":"v2:0699a9041ce9cddb51ac5784b5b77fd16b2f7170ed8df027f185cbc5c74587ae","agentId":"a71a55a475829e001"}
{"type":"started","key":"v2:acd6fa2cf2ea8decb1a545987f6f305435dd138dbae56d856b9447ed55af7798","agentId":"a77adcbc098c7cca7"}

274. assistant (2026-07-08T06:14:30.927Z)

Got it ? **no real L4 test, no deploy.** I'll leave `rally-worker` / `:latest` untouched; you fold everything into today's release.

The adversarial review is on its verification pass and has already surfaced **two real robustness gaps** worth folding into that release (both small, both in the newly-added #521 code):

1. **[medium] Opaque validator-rejection message when validation is delegated to the L4** (`orchestration.py` ~689). With `phase_placement.validation=cloud_run_l4`, a worker rejection surfaces as `"cloud job failed (returncode 2)."` instead of the specific "too blurry / not badminton" reason ? because the `infer` done-marker carries no `message` and `_maybe_dispatch_remote` doesn't read `result.report.get('message')` (the compile path already does). This is a UX regression for the delegation feature (I'd flagged the risk earlier; the review confirms it's worth fixing).

2. **[low] `cmd_compile` has no marker-on-early-failure** (`worker/__main__.py`). A corrupt/partial staged spec raises out of unwrapped `main()` ? no done-marker ? the dispatcher falls back solely to the #515 crash-fast probe instead of terminating cleanly. `cmd_infer`'s `_fail()` always writes a marker; `cmd_compile` should match.

Let me wait for the verification pass to finish (it may confirm/refute these and add findings from the other dimensions), then I'll fix the **confirmed** set and put up a small follow-up PR you can merge into today's release. I'll report the full verified list shortly.

275. user (2026-07-08T06:20:03.628Z)

<task-notification>
<task-id>ws7lmnehf</task-id>
<tool-use-id>toolu_01TjmDrzJMeY9Cv4QhEyfcS3</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\75d3d121-f240-4293-afa5-b436a379ad31\tasks\ws7lmnehf.output</output-file>
<status>completed</status>
<summary>Dynamic workflow "Adversarially review the merged #521 compile-on-L4 diff; verify each finding before trusting it" completed</summary>
<result>{"confirmed":[{"file":"backend/orchestration.py","line":1263,"severity":"high","category":"correctness","summary":"On the L4 worker path, a failed (or skipped) bucket mirror-upload is swallowed and _stitch_reel still returns status='success', so a compile job reports success while the reel is never delivered ? the worker's local file is discarded on Job exit and the control-plane has no local copy to fall back to, so the SPA's download 404s (silent data loss reported as success).","failure_scenario":"phase_placement.compile=cloud_run_l4. Worker cmd_compile -> _stitch_reel stitches the reel into scratch, then storage.put(out_path, reel_uri) raises (transient GCS 5xx / IAM drift / quota) OR reel_uri is empty because the forwarded storage.output_root has no '://' scheme. The except at line 1263 logs a warning and control falls through to `return {'status':'success', ..., 'reel_uri': reel_uri or ''}`. cmd_compile then writes a done-marker with returncode=0/status=success/download_url set. run_compile reads rc=0 and _dispatch_compile_remote returns status='success' with download_url=/api/highlights/<vid>/<name>. The user clicks it: download_highlight() calls signed_reel_url -> storage.exists(reel_uri)=False (mirror failed) -> None, then falls back to os.path.join(output_dir, name) on the CONTROL-PLANE, where the file does not exist (it was stitched on the worker) -> HTTP 404 'No compiled file named ...'. The reel is lost but the job/SPA both show success.","suggested_fix":"In the worker/no-local-fallback case the bucket mirror is the ONLY delivery mechanism (the code comment at lines 1254-1255 acknowledges this), so a mirror-upload failure ? or an empty reel_uri when a bucket output is required ? must fail the stitch there: return {'status':'failed', 'message': 'reel mirror-upload failed / no bucket destination'} instead of success, so cmd_compile writes rc!=0 and the dispatcher surfaces

a clean failed job (letting the user retry). Keep the best-effort swallow only for the true control-plane in-process path where the CP-local file is a valid fallback. Also add an integration test exercising the real _stitch_reel on the worker with a failing storage.put (current tests in test_compile_dispatch.py / test_worker.py stub _stitch_reel, so this failure mode is uncovered).","dimension":"correctness","verdicts":[{"is_real":true,"confidence":"high","reasoning":"CONFIRMED in the actual code. On the L4 worker compile path, a failed bucket mirror-upload is swallowed and the job still reports success, producing a download 404 (silent data loss reported as success). Verified chain:\n\n1) backend/orchestration.py:1287-1309 ? after stitching to a scratch out_path, `_stitch_reel` does `reel_uri = reel_object_uri(...)`; `if reel_uri:` it runs `storage.put(out_path, reel_uri, ...)` in a try/except that on ANY exception only logs `logger.warning(\"[compile] could not mirror reel to %s...\")` (1294-1295) and falls through to `return {\"status\":\"success\", ..., \"reel_uri\": reel_uri or \"\"}` (1301-1309). The comment at 1282-1286 explicitly acknowledges that on the L4 worker \"the mirror is how the reel reaches the bucket at all (the worker's local file is discarded when the Job exits)\" ? yet the swallow-and-succeed is applied unconditionally to both the CP caller (where a local fallback exists) and the worker caller (where it does not).\n\n2) backend/worker/__main__.py:506-524 ? cmd_compile calls the shared _stitch_reel, then `status = result.get(\"status\")` = 'success', `rc = 0 if status==\"success\" else 3` = 0, and _write_compile_marker records status='success' + download_url.\n\n3) backend/orchestration.py:1435-1468 ? _dispatch_compile_remote only checks `if result.returncode != 0:`; on rc=0 it returns status='success' with the download_url. It never re-verifies the reel exists in the bucket (no storage.exists re-check).\n\n4) backend/main.py:1046-1083 + orchestration.py:1180-1191 ? download_highlight calls signed_reel_url ? storage.exists(reel_uri) is False (mirror failed) ? returns None ? falls back to os.path.join(output_dir, name) on the CONTROL-PLANE, where the worker-stitched file never existed ? HTTP 404 \"No compiled file named ...\".\n\nReachability: the reel put and the marker put are separate operations on very different-sized objects (a large 4K reel vs a tiny JSON marker). A transient/partial failure where the large reel upload fails (timeout/size/quota/mid-transfer blip) while the small marker succeeds is realistic and reachable ? that is precisely the false-success case. A total storage outage would instead also fail the marker put and correctly time the dispatcher out (rc4/rc5), so it does not save the partial case. The Cloud Run Job scratch is ephemeral and on a separate instance from the control-plane, so there is genuinely no CP fallback copy.\n\nThe cited line (1263) is off in the current checkout ? the mirror except is at 1294-1295 and the success return at 1301-1309 ? but the described logic is present verbatim and behaves exactly as claimed; the line drift is only a snapshot difference and does not refute the defect. This violates the stated invariant that a stitch/delivery failure must be a clean failed-job."},{"is_real":true,"confidence":"high","reasoning":"CONFIRMED ? I could not refute this. The full chain is verified in the merged code.\n\nMechanism (backend/orchestration.py, `_stitch_reel`): after a successful stitch, the reel mirror-upload at lines 1287?1295 is unconditionally best-effort ? `storage.put(out_path, reel_uri, ...)` is wrapped in `except Exception` that only logs `logger.warning(\"[compile] could not mirror reel to %s...\")`. Control then falls through to `return {\"status\": \"success\", ..., \"reel_uri\": reel_uri or \"\"}` (1301?1309). This helper is SHARED by the control-plane handler and the L4 worker with NO distinction, even though its own comment (1282?1286) states the worker's local file is discarded on Job exit and \"the mirror is how the reel reaches the bucket at all.\" So a swallowed mirror failure that is harmless on the CP (local fallback exists) becomes silent data loss on the worker.\n\nEnd-to-end confirmation:\n- Worker (backend/worker/__main__.py cmd_compile, 506?525): status/rc are taken directly from the `_stitch_reel` result; on a swallowed mirror failure it gets status='success', rc=0, and `_write_compile_marker` (446?461) writes a marker with rc=0/status='success'/download_url/reel_uri. No re-verification that the reel actually landed.\n- Dispatcher (backend/compute/cloud_run.py `_poll_check` 405?407): a PRESENT marker is authoritative (\"success signal ? authoritative even if status later flips\"), so the crash-fast RemoteExecutionFailed path is NOT taken (it requires an ABSENT marker). run_compile returns returncode=0; `_dispatch_compile_remote` (orchestration.py 1435?1468) returns status='success' with download_url=/api/highlights/<vid>/<name>.\n- Download (backend/main.py download_highlight 1060?1081): signed_reel_url ? reel_object_uri ? storage.exists(reel_uri)=False (upload failed) ? None ? falls back to os.path.join(output_dir, name) on the CONTROL-PLANE, where the file never existed (it was stitched on the worker) ? HTTP 404 \"No compiled file named '<name>'.\" Job/SPA both showed success; reel is lost.\n\nRejected refutations: (a) the reel put and the tiny marker put are independent storage.put calls ? a transient 5xx / partial-upload / quota / IAM hit on the large reel object while the sub-1KB marker succeeds seconds later is a realistic, non-contrived failure mode (the large object has far more failure surface). (b) The crash-fast path cannot help ? it only fires on an absent marker; here the marker is present and says success. (c) No verify-reel-exists step exists anywhere in the compile/worker/dispatch/download path. Both sub-cases in the claim manifest: an empty reel_uri (misprovisioned output_root without '://') skips the mirror entirely

```
and st
... (truncated 9852 chars, full result in C:\Users\avidu\AppData\Local\Temp\claude\C--Users-
avidu-Projects-khelsutra-guru-badminton-highlight-
indexer\75d3d121-f240-4293-afa5-b436a379ad31\tasks\ws7lmnehf.output)</result>
<diagnostics>Per-agent results: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-
khelsutra-guru-badminton-highlight-
indexer\75d3d121-f240-4293-afa5-b436a379ad31\subagents\workflows\wf_5f13fc79-bad/journal.jsonl ?
one {"type":"result",...} line per completed agent with its full return value.
If the result above is empty or unexpected, Read this file BEFORE diagnosing ? do not assume
agents returned non-empty results.
To re-run with edited post-processing: Workflow({scriptPath: 'C:\Users\avidu\.claude\projects\C
--Users-avidu-Projects-khelsutra-guru-badminton-highlight-
indexer\75d3d121-f240-4293-afa5-b436a379ad31\workflows\scripts\pr525-adversarial-review-
wf_5f13fc79-bad.js', resumeFromRunId: 'wf_5f13fc79-bad', args:
"{\"diffPath\":\"C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-khelsutra-
guru-badminton-highlight-
indexer/75d3d121-f240-4293-afa5-b436a379ad31/scratchpad/review/pr525.diff\",\"root\":\"C:/kg-
wt521\"}"}) ? agents whose (prompt, opts) are unchanged replay from cache.</diagnostics>
<usage><agent_count>19</agent_count><agents_done>19</agents_done><agents_error>0</agents_error><
agents_skipped>0</agents_skipped><agents_empty_result>1</agents_empty_result><subagent_tokens>18
23930</subagent_tokens><tool_uses>305</tool_uses><duration_ms>955885</duration_ms></usage>
</task-notification>
```

276. assistant (2026-07-08T06:20:17.422Z)

The review completed (19 agents, ~16 min). It found a **HIGH-severity bug I missed**. Let me read the full verified result:

277. user (2026-07-08T06:20:26.871Z)

```
Exit code 1
Traceback (most recent call last):
  File "<string>", line 4, in <module>
    c=d['confirmed']
      ~^^^^^^^^^^^^^
KeyError: 'confirmed'
```

278. user (2026-07-08T06:20:34.431Z)

```
TOP KEYS: ['summary', 'agentCount', 'logs', 'result', 'workflowProgress', 'totalTokens',
'totalToolCalls']
--- head of file ---
{
  "summary": "Adversarially review the merged #521 compile-on-L4 diff; verify each finding
before trusting it",
  "agentCount": 19,
  "logs": [
    "review complete: 7 raw findings, 2 confirmed, 5 refuted"
  ],
  "result": {
    "confirmed": [
      {
        "file": "backend/orchestration.py",
        "line": 1263,
        "severity": "high",
        "category": "correctness",
        "summary"
```

279. user (2026-07-08T06:20:43.803Z)

```
CONFIRMED: 2 | refuted: 5

=== HIGH | correctness | backend/orchestration.py : 1263
SUMMARY: On the L4 worker path, a failed (or skipped) bucket mirror-upload is swallowed and
```

```
_stitch_reel still returns status='success', so a compile job reports success while the reel is
never delivered ? the worker's local file is discarded on Job exit and the control-plane has no
local copy to fall back to, so the SPA's download 404s (silent data loss reported as success).
FIX: In the worker/no-local-fallback case the bucket mirror is the ONLY delivery mechanism (the
code comment at lines 1254-1255 acknowledges this), so a mirror-upload failure ? or an empty
reel_uri when a bucket output is required ? must fail the stitch there: return
{'status':'failed', 'message': 'reel mirror-upload failed / no bucket destination'} instead of
success, so cmd_compile writes rc!=0 and the dispatcher surfaces a clean failed job (letting the
user retry). Keep the best-effort swallow only for the true control-plane in-process path where
the CP-local file is a valid fallback. Also add an integration test exercising the real
_stitch_reel on the worker with a failing storage.put (curren

=== HIGH | test-coverage | backend/orchestration.py : 1263
SUMMARY: No test exercises the L4 worker's real _stitch_reel mirror-upload; a swallowed mirror
failure there reports success (rc 0) but the reel is lost, so the SPA gets a 404 for a job it
was told succeeded.
FIX: On the worker path a mirror-upload failure must FAIL the stitch (return status=failed /
non-zero rc), because unlike the control-plane the worker's local reel does not survive the Job.
Distinguish the two callers (e.g. a `require_mirror`/`is_worker` flag into _stitch_reel), and
add a test that runs cmd_compile through the real _stitch_reel with storage.put patched to
raise, asserting rc != 0 and a failed marker (no false success).

--- REFUTED (dropped) ---
  - backend/orchestration.py : A cloud-dispatched compile with a non-URI storage.output_root
silently discards the reel yet reports success, because re
  - backend/pipeline/stitcher/compiler.py : The NVENC availability probe does not use the same
encoder args as the real per-clip encode, so it can report an encoder
  - backend/orchestration.py : When validation is delegated to the L4 worker
(phase_placement.validation=cloud_run_l4), a worker validation REJECTION s
  - backend/worker/__main__.py : cmd_compile writes its done-marker ONLY after _stitch_reel
returns; any failure BEFORE that (spec fetch/parse, time_pair
  - backend/worker/__main__.py : The dispatcher->worker contract (compile argv flags and the
staged spec JSON schema) is never round-tripped end-to-end;
```

280. assistant (2026-07-08T06:21:55.274Z)

The verification pass cut through it: **2 confirmed (both the same HIGH-severity bug), 5 refuted.** Notably, the two issues I mentioned in my last message (opaque validation message; `cmd_compile` no-early-marker) were **refuted** by the skeptics ? the validation one because delegation is opt-in/not preset-enabled, and the marker one because the #515 crash-fast probe already converts a worker crash into a clean `rc 5`. Good ? that's exactly why adversarial verification matters.

The **real** bug is one I missed:

? HIGH ? silent data loss on the L4 worker: a failed mirror-upload reports `success`.
`_stitch_reel`'s reel mirror-upload is best-effort (swallow+warn) ? correct for the *control-plane* path (the local file is a valid fallback), but **wrong for the worker**, whose local file is discarded when the Job exits. So if `storage.put` fails (transient GCS 5xx / quota) *or* `reel_uri` is empty (misprovisioned `output_root`), the worker still returns `status=success` ? success marker ? the SPA gets a `download_url` that **404s**. The shared-core refactor introduced this: the mirror was best-effort when only the CP called it.

This is worth fixing before your release. Let me put it in a follow-up PR off the current master. First, a fresh isolated worktree off `origin/master`:

281. user (2026-07-08T06:22:09.949Z)

```
origin/master=dac6736
branch 'fix/521-worker-mirror-required' set up to track 'origin/master'.
Updating files:  67% (372/552)
Updating files:  68% (376/552)
Updating files:  69% (381/552)
Updating files:  70% (387/552)
Updating files:  71% (392/552)
```

```
Updating files:  72% (398/552)
Updating files:  73% (403/552)
Updating files:  74% (409/552)
Updating files:  75% (414/552)
Updating files:  76% (420/552)
Updating files:  77% (426/552)
Updating files:  78% (431/552)
Updating files:  79% (437/552)
Updating files:  80% (442/552)
Updating files:  81% (448/552)
Updating files:  82% (453/552)
Updating files:  83% (459/552)
Updating files:  84% (464/552)
Updating files:  85% (470/552)
Updating files:  86% (475/552)
Updating files:  87% (481/552)
Updating files:  88% (486/552)
Updating files:  89% (492/552)
Updating files:  90% (497/552)
Updating files:  91% (503/552)
Updating files:  92% (508/552)
Updating files:  93% (514/552)
Updating files:  94% (519/552)
Updating files:  95% (525/552)
Updating files:  96% (530/552)
Updating files:  97% (536/552)
Updating files:  98% (541/552)
Updating files:  99% (547/552)
Updating files: 100% (552/552)
Updating files: 100% (552/552), done.
HEAD is now at dac6736 perf(serving): run compile/stitch on the L4 worker; config-driven
phase?machine placement (#521) (#525)
--- verify ---
fix/521-worker-mirror-required
dac6736 perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine
placement (#521) (#525)
--- only my commits (empty) ---
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

282. assistant (2026-07-08T06:22:17.183Z)

Clean worktree off `dac6736`. Now let me read the merged `_stitch_reel` in the new worktree to apply the exact fix:

283. user (2026-07-08T06:22:20.966Z)

```
1172:    reel_uri = reel_object_uri(cfg, video_id, name)
1190:def _stitch_reel(
1256:    reel_uri = reel_object_uri(cfg, video_id, out_name)
1264:            logger.warning("[compile] could not mirror reel to %s: %s", reel_uri, e)
1319:        return _stitch_reel(
1423:def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:
1458:    return _stitch_reel(
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

284. user (2026-07-08T06:22:24.438Z)

```
1190    def _stitch_reel(
1191        cfg,
1192        stitching,
1193        video_id: str,
1194        source_ref: str,
1195        time_pairs: List[Tuple[float, float]],
1196        out_name: str,
```

```
        locale: Optional[str],
        notify,
    ) -> Dict[str, Any]:
        """Stitch + mirror-upload core, shared by the control-plane compile handler AND the
L4 worker
        (#521). ``time_pairs`` are already RESOLVED (the caller filtered via
``_resolve_compile``), so this
        needs NO control-plane DB ? the worker reads them from the staged spec. Builds the
VideoCompiler
        with ``stitching.encoder`` (NVENC on the L4, libx264 on the CP), fetches the source,
runs the
        trim/watermark/concat, mirrors the reel to
``<output_root>/highlights/<video_id>/<name>``, and
        returns the ``{status, filepath, download_url, video_id, reel_uri}`` the SPA polls
for. Returns a
        ``status='failed'`` result on a stitch error (never raises for an expected ffmpeg
failure)."""
        import backend.main as _main  # call-time seam: VideoCompiler stays patchable on
backend.main

        output_dir = (
            getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
        )
        # Localize the branding watermark to the UI locale the SPA rendered in (recorded on
the job);
        # unknown/absent locale keeps the configured default unchanged.
        localized_watermark = localize_watermark(stitching.watermark, locale)
        compiler = _main.VideoCompiler(
            output_dir,
            begin_padding=stitching.begin_padding,
            end_padding=stitching.end_padding,
            watermark=localized_watermark,
            encoder=getattr(stitching, "encoder", "libx264"),
        )

        notify("stitching")
        local_src = ""
        try:
            # The stored `filepath` is the SOURCE ref (gs://, s3://, gdrive://, or a local
path). Resolve to
            # a LOCAL file the stitcher can read ? identity for a local path, fetch-to-
scratch for a URI (the
            # same seam the process path uses); without it, compiling a bucket-ingested
video can't open gs://.
            local_src = storage.acquire_local_input(
                source_ref, getattr(cfg, "storage", None)
            )
            # The ffmpeg stitch is CPU-heavy ? serialize it so a raised drain worker count
(for remote
            # dispatch) doesn't run multiple compiles at once on the box (see
_LOCAL_COMPUTE_LANE).
            with _LOCAL_COMPUTE_LANE:
                out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
        except Exception as e:  # noqa: BLE001 ? a stitch/ffmpeg failure is a clean job
FAILURE, not a crash
            logger.error(
                "[compile] stitching failed for video_id=%s: %s", video_id, e, exc_info=True
            )
            return {
                "status": "failed",
                "message": f"Highlight stitching failed: {e}",
                "video_id": video_id,
            }
        finally:
            if local_src:
```

```
1247                storage.cleanup_acquired_local_input(
1248                    source_ref, local_src, getattr(cfg, "storage", None)
1249                )
1250
1251        # #463: mirror the reel to the output bucket so /api/highlights can 307 to a signed
URL ? Cloud Run
1252        # caps a buffered response at ~32 MiB (a real reel is bigger) and the local /tmp
copy dies when the
1253        # instance recycles. A mirror-upload failure must NOT fail an otherwise-good stitch
(the endpoint
1254        # still falls back to the local file on the same instance). On the L4 worker the
mirror is how the
1255        # reel reaches the bucket at all (the worker's local file is discarded when the Job
exits).
1256        reel_uri = reel_object_uri(cfg, video_id, out_name)
1257        if reel_uri:
1258            try:
1259                notify("uploading reel")
1260                # Thread the active storage settings so the configured backend (S3
endpoint/region, GCS
1261                # project/client) is used, not a fresh default.
1262                storage.put(out_path, reel_uri, config=_storage_settings(cfg))
1263            except Exception as e:  # noqa: BLE001 ? best-effort mirror; local stitch
already succeeded
1264                logger.warning("[compile] could not mirror reel to %s: %s", reel_uri, e)
1265
1266        # `filepath` is server-local (not browser-reachable); also return the download URL
so the SPA needn't
1267        # hand-build it (P6 / PER_RALLY_SELECTION.md §4.3).
1268        download_url = f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
1269        notify("finished")
```

285. assistant (2026-07-08T06:22:29.322Z)

Now apply the fix ? add a `require_mirror` flag so the worker (no local fallback) fails on a lost mirror instead of falsely succeeding:

286. user (2026-07-08T06:22:40.384Z)

The file C:\kg-wt525fix\backend\orchestration.py has been updated successfully. (file state is current in your context ? no need to Read it back)

287. user (2026-07-08T06:22:56.524Z)

The file C:\kg-wt525fix\backend\orchestration.py has been updated successfully. (file state is current in your context ? no need to Read it back)

288. assistant (2026-07-08T06:22:59.384Z)

Now make the worker's `cmd_compile` pass `require_mirror=True`:

289. user (2026-07-08T06:23:02.723Z)

```
468:    the reel via the SAME ``orchestration._stitch_reel`` core the control-plane uses (so
there is no
503:    result = orchestration._stitch_reel(
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

290. user (2026-07-08T06:23:08.421Z)

```
503         result = orchestration._stitch_reel(
504             config,
505             stitching,
506             video_id,
```

```
507             source_ref,
508             time_pairs,
509             out_name,
510             locale,
511             lambda stage: print(f"[worker] compile: {stage}"),
512         )
513         status = str(result.get("status") or "failed")
514         rc = 0 if status == "success" else 3
515         if status != "success":
516             print(f"[worker] compile FAILED: {result.get('message')}")
517
```

291. user (2026-07-08T06:23:15.472Z)

```
The file C:\kg-wt525fix\backend\worker\__main__.py has been updated successfully. (file state is
current in your context ? no need to Read it back)
```

292. assistant (2026-07-08T06:23:53.279Z)

Now add tests that exercise the **real** `_stitch_reel` (the finding noted the current tests stub it). Let me append `require_mirror` tests to `test_compile_dispatch.py`:

293. user (2026-07-08T06:23:55.787Z)

```
def test_compile_missing_video_fails_before_dispatch(compile_env):
    """A compile for an unknown video_id fails at resolve (404) before any dispatch."""
    db, cfg, _video_id, _seg_ids, _bucket, monkeypatch = compile_env
    fake = _FakeCompileClient()
    _inject_fake(monkeypatch, fake)
    out = orch._execute_compile(cfg, db, _job("nope", [1, 2]))
    assert out["status"] == "failed"
    assert fake.calls == []  # never dispatched ? resolve failed first
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

294. user (2026-07-08T06:23:58.769Z)

```
197     def test_compile_missing_video_fails_before_dispatch(compile_env):
198         """A compile for an unknown video_id fails at resolve (404) before any dispatch."""
199         db, cfg, _video_id, _seg_ids, _bucket, monkeypatch = compile_env
200         fake = _FakeCompileClient()
201         _inject_fake(monkeypatch, fake)
202         out = orch._execute_compile(cfg, db, _job("nope", [1, 2]))
203         assert out["status"] == "failed"
204         assert fake.calls == []  # never dispatched ? resolve failed first
```

295. user (2026-07-08T06:24:19.927Z)

```
The file C:\kg-wt525fix\tests\test_compile_dispatch.py has been updated successfully. (file
state is current in your context ? no need to Read it back)
```

296. assistant (2026-07-08T06:24:24.456Z)

Now the worker end-to-end test ? `cmd_compile` through the **real** `_stitch_reel` with a failing upload. Let me add it after the existing compile tests:

297. user (2026-07-08T06:24:27.146Z)

```
1089:def test_cmd_compile_no_done_uri_writes_no_marker(tmp_path, monkeypatch):
    )
    cfg = tmp_path / "c.json"
    cfg.write_text('{"storage": {"backend": "local"}}')
    rc = wmain.cmd_compile(
        wmain.build_parser().parse_args(
            [
```

```
                "compile",
                "--spec-uri",
                spec_uri,
                "--out-uri",
                str(bucket),
                "--scratch",
                str(tmp_path / "s"),
                "--config",
                str(cfg),
            ]
        )
    )
    assert rc == 0
    assert not (bucket / "_compile" / "tok.done").exists()
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

298. user (2026-07-08T06:24:30.854Z)

```
1108                    "--spec-uri",
1109                    spec_uri,
1110                    "--out-uri",
1111                    str(bucket),
```

299. user (2026-07-08T06:24:33.882Z)

```
1115                    str(cfg),
1116                ]
1117            )
1118        )
1119        assert rc == 0
1120        assert not (bucket / "_compile" / "tok.done").exists()
1121
```

300. user (2026-07-08T06:24:49.509Z)

```
The file C:\kg-wt525fix\tests\test_worker.py has been updated successfully. (file state is
current in your context ? no need to Read it back)
```

301. assistant (2026-07-08T06:25:05.371Z)

```
The output_root must be a `gs://` URI (so the mirror is *attempted* and then fails) ? with a
local path it'd hit the no-bucket branch instead. Let me fix that:
```

302. user (2026-07-08T06:25:08.738Z)

```
The file C:\kg-wt525fix\tests\test_worker.py has been updated successfully. (file state is
current in your context ? no need to Read it back)
```

303. assistant (2026-07-08T06:25:20.097Z)

```
Let me run the fix's tests plus the full compile/worker suites:
```

304. user (2026-07-08T06:25:58.309Z)

```
            config,
            stitching,
            video_id,
            source_ref,
            time_pairs,
            out_name,
            locale,
            lambda stage: print(f"[worker] compile: {stage}"),
            # On the L4 worker the bucket mirror is the ONLY reel delivery ? the worker's local
file is
```

```
            # discarded when the Job exits. A lost mirror MUST fail the stitch (not a
404-on-"success").
            require_mirror=True,
        )
E       TypeError: test_cmd_compile_stitches_and_writes_marker.<locals>._fake_stitch() got an
unexpected keyword argument 'require_mirror'

backend\worker\__main__.py:503: TypeError
---------------------------- Captured stdout call ----------------------------
[worker] compile: video_id=vidReel clips=2 encoder=h264_nvenc out=reel.mp4
========================== short test summary info ==========================
FAILED tests/test_worker.py::test_cmd_compile_stitches_and_writes_marker - Ty...
1 failed, 46 passed in 21.51s
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

305. assistant (2026-07-08T06:26:09.529Z)

Two stub `_fake_stitch` signatures need the new kwarg. Let me update them to accept it:

306. user (2026-07-08T06:26:12.284Z)

```
The file C:\kg-wt525fix\tests\test_worker.py has been updated successfully. (file state is
current in your context ? no need to Read it back)
```

307. user (2026-07-08T06:26:17.808Z)

```
1041:    assert seen["encoder"] == "h264_nvenc"
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

308. user (2026-07-08T06:26:21.152Z)

```
1040        # the shared stitch core got the resolved pairs + the NVENC encoder from the spec
1041        assert seen["encoder"] == "h264_nvenc"
1042        assert seen["pairs"] == [(2.0, 6.0), (10.0, 14.0)]
1043        assert seen["source"] == str(source)
1044        marker = json.loads(open(done, encoding="utf-8").read())
1045        assert marker["returncode"] == 0 and marker["status"] == "success"
```

309. user (2026-07-08T06:26:27.607Z)

```
The file C:\kg-wt525fix\tests\test_worker.py has been updated successfully. (file state is
current in your context ? no need to Read it back)
```

310. user (2026-07-08T06:26:32.107Z)

```
The file C:\kg-wt525fix\tests\test_compile_dispatch.py has been updated successfully. (file
state is current in your context ? no need to Read it back)
```

311. user (2026-07-08T06:26:43.079Z)

```
.............................................                            [100%]
47 passed in 5.17s
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

312. assistant (2026-07-08T06:26:48.418Z)

Fix tests pass. Now the full verification gate (ruff, mypy, leak-scan, link-check, then full suite):

313. user (2026-07-08T06:28:06.929Z)

```
=== ruff ===
All checks passed!
```

```
=== mypy ===
backend\providers\__init__.py:9: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
Success: no issues found in 150 source files
=== leak_scan ===
[leak-scan] clean ? no MD5 / personal-path / private-name leaks in the code surface.
=== md_links ===
[check-md-links] clean ? no broken internal links in live docs.
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

314. assistant (2026-07-08T06:28:15.763Z)

Let me add a brief note to the phase-placement doc about the worker's required-mirror invariant, then run the full suite:

315. user (2026-07-08T06:28:17.727Z)

```
91:   core** (no forked pipeline), mirror-uploads it to
`<output_root>/highlights/<video_id>/<name>`,
92:   and writes the marker (rc + `download_url` + `reel_uri`).
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

316. user (2026-07-08T06:28:21.350Z)

```
89         one bucket never collide on a marker.
90      5. The worker (`cmd_compile`) reads the spec, stitches the reel via the **shared
`_stitch_reel`
91         core** (no forked pipeline), mirror-uploads it to
`<output_root>/highlights/<video_id>/<name>`,
92         and writes the marker (rc + `download_url` + `reel_uri`).
93      6. The control-plane returns `{status, filepath, download_url, video_id}` ? the reel is
already in the
94         bucket, so `/api/highlights` serves it via the existing signed-URL 307. On a worker
rc?0 ?
```

317. user (2026-07-08T06:28:29.001Z)

```
The file C:\kg-wt525fix\docs\COMPUTE_DECOUPLED_SERVING\PHASE_PLACEMENT.md has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

318. user (2026-07-08T06:37:47.645Z)

```
[check-md-links] clean ? no broken internal links in live docs.
=== FULL SUITE ===
TOTAL                                                12669   1805    86%
Required test coverage of 70% reached. Total coverage: 85.75%
1879 passed, 8 skipped, 25 warnings in 547.42s (0:09:07)
======================================================================
 RESULT: PASS - all tests green
======================================================================
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

319. assistant (2026-07-08T06:38:12.635Z)

Full suite green (**1879 passed**, coverage 85.75%).

I also notice a sibling session (the watermark follow-up task you started) has surfaced a **separate, important bug** in MEMORY.md as PR #527: my NVENC probe uses a **64×64** test frame, which is *below NVENC's minimum resolution* ? so on a real L4 the probe could falsely report `h264_nvenc` unavailable and silently fall back to libx264, **defeating #521's NVENC win entirely**. That's a real bug, being fixed independently in #527 (different file ? `compiler.py::_probe_encoder_available` ? no conflict with my `_stitch_reel` fix). Good catch by that session; I'll leave it to #527 and keep this PR focused on the mirror-data-loss fix.

Let me verify branch cleanliness and commit (fetching first, since master is moving under concurrent sessions):

320. user (2026-07-08T06:38:18.244Z)

```
origin/master=dac6736  myHEAD-base=dac6736
=== my branch ===
fix/521-worker-mirror-required
=== does master now overlap my files? (fetch drift check) ===
=== my changed files ===
 M backend/orchestration.py
 M backend/worker/__main__.py
 M docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md
 M tests/test_compile_dispatch.py
 M tests/test_worker.py
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

321. assistant (2026-07-08T06:38:22.109Z)

Clean ? `origin/master` still at `dac6736`, no file overlap. Let me stage explicit paths and commit:

322. user (2026-07-08T06:38:36.438Z)

```
[fix/521-worker-mirror-required 7a31a9b] fix(compile): fail the L4-worker stitch when the reel
mirror-upload fails (#521 follow-up)
 5 files changed, 210 insertions(+), 9 deletions(-)
=== verify: only my commit ahead ===
7a31a9b fix(compile): fail the L4-worker stitch when the reel mirror-upload fails (#521 follow-
up)
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

323. user (2026-07-08T06:38:58.748Z)

```
remote:
branch 'fix/521-worker-mirror-required' set up to track 'origin/fix/521-worker-mirror-required'.
To https://github.com/Khelsutra/badminton-highlight-indexer.git
 * [new branch]      fix/521-worker-mirror-required -> fix/521-worker-mirror-required
https://github.com/Khelsutra/badminton-highlight-indexer/pull/528
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

324. assistant (2026-07-08T06:39:02.524Z)

Fix PR **#528** opened. Let me watch its CI to green:

325. user (2026-07-08T06:42:39.793Z)

```
Mypy (lenient baseline) pass    36s     https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28922900489/job/85803837201
Ruff lint       pass    8s      https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28922900489/job/85803837194
Syntax sweep + import smoke (no GPU stack)      pass    18s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28922900489/job/85803837207
Built wheel smoke (install + launch)    pass    1m11s   https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28922900489/job/85803837217
Dependency license scan (no copyleft in runtime deps)   pass    33s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28922900489/job/85803837214
Full test suite (offline, mocked)       pass    3m14s   https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28922900489/job/85803837265
Golden fixtures cross-OS (macos-latest) pass    22s     https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28922900489/job/85803837235
Golden fixtures cross-OS (ubuntu-latest)        pass    21s
```

```
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28922900489/job/85803837234
Golden fixtures cross-OS (windows-latest)       pass    1m0s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28922900489/job/85803837211
Leak scan (PII / attribution guard)     pass    8s      https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28922900489/job/85803837218
Markdown link check (live docs) pass    6s      https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28922900489/job/85803837191
Mypy (lenient baseline) pass    36s     https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28922900489/job/85803837201
Ruff lint       pass    8s      https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28922900489/job/85803837194
Syntax sweep + import smoke (no GPU stack)      pass    18s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28922900489/job/85803837207
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

326. assistant (2026-07-08T06:42:49.625Z)

All 11 CI checks green on **#528**. Let me record it in memory (the sibling session added #527; I'll append #528):

327. user (2026-07-08T06:42:50.134Z)

```
1       # Memory index
2
3       - [PR #525 ? compile-on-L4 (#521)](pr-525-compile-on-l4.md) ? config-driven
phase?machine placement + L4 NVENC stitch; Part 3 (cheaper watermark) deferred.
4       - [PR #527 ? NVENC probe 64x64?256x256](pr-527-nvenc-probe-resolution.md) ? sub-minimum
probe silently defeated #521's NVENC-on-L4 win; open, awaits owner live-GPU confirm.
5
```

328. user (2026-07-08T06:42:57.066Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\memory\MEMORY.md has been updated successfully. (file state is current in your context ? no need to Read it back)

329. assistant (2026-07-08T06:43:09.280Z)

```
---
name: pr-528-worker-mirror-required
description: PR #528 (#521 follow-up) ? L4-worker compile fails on a lost reel mirror-upload
instead of a silent 404-on-success
metadata:
  type: project
---
```

PR [#528](https://github.com/Khelsutra/badminton-highlight-indexer/pull/528) ? open 2026-07-08, all 11 CI checks green, for today's release.

Fixes a HIGH-severity silent-data-loss bug an **adversarial multi-agent review** of the merged #521 ([[pr-525-compile-on-l4]]) surfaced + verified: `orchestration._stitch_reel`'s reel mirror-upload was best-effort (swallow+warn) ? correct for the control-plane (CP-local reel is a fallback), but WRONG on the L4 worker (its local reel is discarded when the Job exits). A failed `storage.put` or an empty `reel_uri` (non-bucket `output_root`) let the worker return `status=success` ? success marker ? a `download_url` that 404s.

Fix: `_stitch_reel(..., require_mirror=False)`; `worker.cmd_compile` passes `require_mirror=True` ? a lost mirror / no-bucket-destination returns `status=failed` (rc 3) ? clean retryable failed job. CP path unchanged (default False).

Review verdict (why trust it): 7 raw findings ? 2 confirmed (this bug, ×2 dimensions), 5

refuted by independent skeptics ? incl. the opaque-delegated-validation-message (opt-in, not preset-enabled) and cmd_compile-no-early-marker (#515 crash-fast probe covers it). So those were deliberately NOT changed.

Sibling concurrent fix: [[pr-527-nvenc-probe-resolution]] fixes the NVENC probe's 64×64 sub-minimum test frame (would silently defeat NVENC on the real L4) ? different file, no conflict. Both fold into today's release alongside #525.

330. user (2026-07-08T06:43:09.323Z)

File created successfully at: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\memory\pr-528-worker-mirror-required.md (file state is current in your context ? no need to Read it back)

331. assistant (2026-07-08T06:43:38.325Z)

Done. Here's the complete picture for your release today.

What I did (this session)

Real-video testing ? done, no deploy (as you instructed). I did *not* touch `rally-worker`, `:latest`, or run any GPU job. Instead I exercised the actual worker `compile` path on a real 1080p/180s corpus clip:
- **Real ffmpeg:** requested `h264_nvenc` ? probe correctly found it unrunnable on this box ? fell back to libx264, trimmed real clips + watermark ? valid reel.
- **Real GCS round-trip:** fetched a `gs://` source + `gs://` spec, stitched, uploaded the reel + wrote the done-marker to `gs://khelsutra-rally-serving/?` (marker carried `reel_uri` + `download_url`). Verified the reel, then cleaned up the GCS scratch. This is the exact worker code path the L4 runs ? only NVENC hardware encode and the CP-initiated dispatch differ (both covered by tests).

Adversarial review caught a real bug I'd missed. A 19-agent verify-before-trust review (5 dimensions ? each finding refuted-or-confirmed by 2 skeptics) returned **2 confirmed / 5 refuted**:
- ? **HIGH (fixed):** on the L4 worker a *failed reel mirror-upload* returned `status=success` ? the SPA gets a `download_url` that **404s**. The worker has no local fallback (its file dies with the Job), so a lost mirror is a lost reel ? but was reported as success. Fixed in **PR #528** (`require_mirror=True` on the worker path ? clean retryable failure; control-plane path unchanged).
- The two issues I'd flagged earlier (opaque delegated-validation message; `cmd_compile` no-early-marker) were **refuted** on verification ? opt-in feature / already covered by the #515 crash-fast probe ? so I deliberately left them.

PRs for today's release

PR	State	What
[#525](https://github.com/Khelsutra/badminton-highlight-indexer/pull/525)	**merged** (`dac6736`)	#521 ? config-driven placement + compile-on-L4 + NVENC
[#528](https://github.com/Khelsutra/badminton-highlight-indexer/pull/528)	**open, CI green**	the mirror-data-loss fix above
[#527](https://github.com/Khelsutra/badminton-highlight-indexer/pull/527)	open (sibling session)	NVENC probe 64×64?256×256 ? the 64×64 frame is **below NVENC's minimum resolution**, which would silently defeat NVENC on the real L4. Independent file, no conflict with #528.

Worth knowing before you cut the release: #527 is not cosmetic ? without it, #521's headline NVENC win could silently no-op on the L4 (probe fails on the tiny frame ? libx264 fallback). I'd fold **#528 and #527** in alongside #525. Both are green/small.

All checks on #528: full suite **1879 passed**, ruff + mypy + leak-scan + link-check clean, coverage 85.8%. Isolated worktrees (`C:/kg-wt521`, `C:/kg-wt525fix`) ? I never touched the shared checkout or sibling branches.

332. user (2026-07-08T06:43:42.605Z)

what should be the merge order between 527 and 528?

333. assistant (2026-07-08T06:44:22.578Z)

Let me verify the actual file overlap and conflict potential empirically rather than assume:

334. user (2026-07-08T06:44:27.465Z)

```
=== #527 ===
{"base":"master","files":["backend/pipeline/stitcher/compiler.py","tests/test_compiler.py"],"hea
d":"fix/nvenc-probe-resolution","mergeable":"MERGEABLE","state":"OPEN"}
=== #528 ===
{"base":"master","files":["backend/orchestration.py","backend/worker/__main__.py","docs/COMPUTE_
DECOUPLED_SERVING/PHASE_PLACEMENT.md","tests/test_compile_dispatch.py","tests/test_worker.py"],"
head":"fix/521-worker-mirror-required","mergeable":"MERGEABLE","state":"OPEN"}
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

335. assistant (2026-07-08T06:44:54.966Z)

Both PRs are **MERGEABLE** and touch **completely disjoint files** (#527: `compiler.py` + `test_compiler.py`; #528: `orchestration.py`, `worker/__main__.py`, `PHASE_PLACEMENT.md`, `test_compile_dispatch.py`, `test_worker.py`). Let me prove no conflict empirically with a trial three-way merge:

336. user (2026-07-08T06:45:02.474Z)

```
 * branch            fix/nvenc-probe-resolution -> FETCH_HEAD
 * branch            fix/521-worker-mirror-required -> FETCH_HEAD
=== trial merge both branch trees onto master (exit 0 + no conflict markers = clean in ANY
order) ===
merge-tree exit=0
0
=== overlapping files between the two PRs (should be NONE) ===
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

337. assistant (2026-07-08T06:45:17.111Z)

Empirically confirmed: **merge order doesn't matter.**

- `git merge-tree --write-tree` of the two branches ? **exit 0, zero conflict markers**.
- **Zero overlapping files** ? #527 touches only `compiler.py` + `test_compiler.py`; #528 touches `orchestration.py`, `worker/__main__.py`, the doc, and two *different* test files.
- **No semantic dependency:** #527 changes the NVENC *probe* resolution (which encoder gets selected); #528 changes *mirror-failure handling*. Neither reads the other's code ? and #528's tests stub `VideoCompiler`, so the probe is irrelevant to them.

So: **merge them in whatever order is convenient, no rebase required.** Git merges per-file, and disjoint file sets can't textually conflict.

Two practical notes:
1. **After the first one merges**, GitHub recomputes the second's status ? it'll stay `MERGEABLE` and green (disjoint). Just glance at it before clicking; no rebase. A merge queue, if enabled, handles this automatically.
2. **Both should land in the release.** They're independent but complementary on the cloud-serving compile path: #528 stops a silent 404-on-"success", #527 stops NVENC from silently no-op'ing to libx264 on the L4. Shipping #521 without #527 means the headline NVENC speedup may not actually engage.

If you want an arbitrary tiebreaker, take **#528 first** (higher-severity data-loss fix) then **#527** ? but it's genuinely a coin flip.

338. user (2026-07-08T06:46:53.978Z)

merged

339. assistant (2026-07-08T06:47:32.466Z)

Let me verify master is coherent post-merge and run the combined result through the suite ? since each PR's CI ran against a base *without* the other, a run against the actual merged commit closes that gap (the disjoint analysis predicts green, but a release deserves the real check).

340. user (2026-07-08T06:47:38.924Z)

```
=== master log ===
65ee4c6 fix(serving): probe NVENC at 256x256, not 64x64, so it isn't falsely rejected
(#521/#525) (#527)
008a5d4 fix(compile): fail the L4-worker stitch when the reel mirror-upload fails (#521 follow-
up) (#528)
dac6736 perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine
placement (#521) (#525)
bf97700 docs(runlogs): record live SPA CUJ closing the 2 owner-gated validation-latency cells
(§9) (#522)
699d85b feat(validation): cache validation verdict by video_id to skip re-validation on reupload
(#518)
=== reset my (merged) worktree to the released master ===
HEAD is now at 65ee4c6 fix(serving): probe NVENC at 256x256, not 64x64, so it isn't falsely
rejected (#521/#525) (#527)
=== both fixes present in master? ===
#528 require_mirror: backend/orchestration.py:6 backend/worker/__main__.py:1
#527 probe resolution (should NOT be 64x64):
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

341. assistant (2026-07-08T06:47:51.689Z)

Master has all three (#525 ? #528 ? #527). `require_mirror` is present (6× in orchestration, 1× in worker). Let me confirm #527's probe change and start the full suite against the exact released commit:

342. user (2026-07-08T06:47:52.648Z)

```
=== #527's probe in merged master ===
35:#: 64x64 and 128x128 but accepts ~145px and up; 256x256 keeps a safe margin. Do NOT shrink
below
36:#: 256x256 ? guarded by tests/test_compiler.py::test_probe_encoder_uses_safe_resolution.
37:_PROBE_SOURCE_SIZE = "256x256"
40:def _probe_encoder_available(encoder: str) -> bool:
42:    lavfi test encode to null at ``_PROBE_SOURCE_SIZE`` (a resolution ? NVENC's minimum, so
the
60:                "lavfi",
62:                f"color=c=black:s={_PROBE_SOURCE_SIZE}:d=0.1",
276:                box = f":box=1:boxcolor=black@{min(opacity, 0.5):.2f}:boxborderw=10"
279:                f":fontcolor=white@{opacity:.2f}:fontsize=h/{divisor}:{dt_xy}{box}"
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

343. user (2026-07-08T06:47:52.694Z)

Command running in background with ID: bihc5yblz. Output is being written to: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\75d3d121-f240-4293-afa5-b436a379ad31\tasks\bihc5yblz.output. You will be notified when it completes. To check interim output, use Read on that file path.

344. assistant (2026-07-08T06:48:16.591Z)

#527's fix is sound: `_PROBE_SOURCE_SIZE = "256x256"` with a guard test and a clear comment (NVENC rejects 64/128px, accepts ~145px+). The full suite is running against the exact released commit `65ee4c6` in the background.

Let me update the memory index to reflect the merges (re-reading first, since the sibling

session may have touched it):

345. user (2026-07-08T06:48:17.464Z)

```
1       # Memory index
2
3       - [PR #525 ? compile-on-L4 (#521)](pr-525-compile-on-l4.md) ? config-driven
phase?machine placement + L4 NVENC stitch; Part 3 (cheaper watermark) deferred.
4       - [PR #527 ? NVENC probe 64x64?256x256](pr-527-nvenc-probe-resolution.md) ? sub-minimum
probe silently defeated #521's NVENC-on-L4 win; MERGED to master (65ee4c6); STILL awaits owner
live-GPU confirm; monitoring follow-up #529.
5       - [PR #528 ? worker mirror-required](pr-528-worker-mirror-required.md) ? #521 follow-up:
L4-worker stitch now fails on a lost reel mirror-upload (was silent 404-on-success); found by
adversarial review, CI green.
6
```

346. user (2026-07-08T06:48:28.277Z)

```
The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-
highlight-indexer\memory\MEMORY.md has been updated successfully. (file state is current in your
context ? no need to Read it back)
```

347. user (2026-07-08T06:48:33.265Z)

```
The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-
highlight-indexer\memory\pr-528-worker-mirror-required.md has been updated successfully. (file
state is current in your context ? no need to Read it back)
```

348. assistant (2026-07-08T06:48:47.485Z)

Master is coherent ? `65ee4c6` carries all three (#525 ? #528 ? #527), both follow-up fixes are present, and #527's probe fix is sound (256×256 with a guard test). Memory updated.

I'm running the full suite against that exact released commit now (in the background) to close the one gap CI didn't cover ? each PR's CI ran against a base *without* the other, so this is the first run with all three combined. The disjoint-file analysis predicts green; I'll confirm when it finishes.

Release baseline summary:

commit	PR	what
`dac6736`	#525	#521 ? config-driven placement + compile-on-L4 + NVENC
`008a5d4`	#528	fail the L4 stitch on a lost reel mirror (was silent 404-on-success)
`65ee4c6`	#527	probe NVENC at 256×256 so it isn't falsely rejected on the L4

One thing to keep in the release notes: the real **L4 + NVENC execution is still unverified** ? the deployed `rally-worker:latest` predates the `compile` subcommand, so the compile-on-L4 path first runs for real only once you rebuild the worker image from this master. I validated the full worker path against real GCS locally (fetch ? stitch ? upload reel + marker), and both follow-up bugs were found/fixed pre-deploy, but the NVENC hardware encode itself will get its first real exercise on that rebuild. Worth a smoke check on the first cloud compile after deploy.

I'll report back the moment the suite lands.

349. user (2026-07-08T06:51:42.466Z)

```
<task-notification>
<task-id>bihc5yblz</task-id>
<tool-use-id>toolu_01WqYDTXQdCDnFt55Vzj5fbN</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-khelsutra-guru-
badminton-highlight-indexer\75d3d121-f240-4293-afa5-b436a379ad31\tasks\bihc5yblz.output</output-
file>
```

<status>completed</status>
<summary>Background command "Run full suite against merged master" completed (exit code 0)</summary>
</task-notification>

350. assistant (2026-07-08T06:51:46.975Z)

The suite finished with **exit 0**. Let me confirm the numbers:

351. user (2026-07-08T06:51:49.133Z)

```
SUITE EXIT=0
Required test coverage of 70% reached. Total coverage: 85.82%
1882 passed, 8 skipped, 25 warnings in 225.47s (0:03:45)
=============================================================================
 RESULT: PASS - all tests green
=============================================================================
```

352. assistant (2026-07-08T06:51:58.660Z)

Full suite **green against the exact released commit `65ee4c6`**: **1882 passed, 8 skipped, coverage 85.82%** ? the first run with #525 + #528 + #527 all combined (which no individual PR's CI had verified). The release baseline is confirmed coherent.

Let me clean up my two now-merged worktrees:

353. user (2026-07-08T06:52:02.914Z)

```
error: failed to delete '.git/worktrees/kg-wt525fix': Permission denied
removed C:/kg-wt525fix
error: failed to delete '.git/worktrees/kg-wt521': Permission denied
removed C:/kg-wt521
=== my worktrees now ===
C:/Users/avidu/Projects/khelsutra-guru/bhi-wt-cheap-watermark
dac6736 [perf/521-cheap-watermark]
```

354. assistant (2026-07-08T06:52:33.553Z)

Everything's wrapped up. Cleanup note: the worktree *directories* were removed, but Git couldn't delete their `.git/worktrees/*` admin entries (`Permission denied` ? the shared `.git` is locked by concurrent sessions, same contention we saw earlier). Those dangling entries are harmless and a `git worktree prune` will clear them once the other sessions release their locks. I left the sibling `bhi-wt-cheap-watermark` worktree (the watermark task you started) untouched.

Release baseline ? verified and coherent ?

Master `65ee4c6` carries all three, and the **full suite passes against that exact commit** (1882 passed, 8 skipped, coverage 85.82%) ? the first run exercising #525 + #528 + #527 together, which no individual PR's CI had done:

commit	PR	what
`dac6736`	#525	#521 ? config-driven phase placement + compile-on-L4 + NVENC
`008a5d4`	#528	fail the L4 stitch on a lost reel mirror (was a silent 404-on-"success")
`65ee4c6`	#527	probe NVENC at 256×256 so it isn't falsely rejected on the L4

For your release notes / first post-deploy check

- **Real L4 + NVENC is still unverified in prod** ? the deployed `rally-worker:latest` predated the `compile` subcommand, so the compile-on-L4 path runs for real only after you rebuild the worker image from this master (`gcloud builds submit --config=deploy/cloudrun/cloudbuild.yaml`). I validated the whole worker path against real GCS locally; the hardware NVENC encode gets its first real exercise on that rebuild.
- **Smoke check after deploy:** drive one cloud compile and confirm (a) the reel lands at

`gs://?/highlights/<video_id>/?` and (b) the worker log shows `encoding clips with h264_nvenc` (not the libx264 fallback) ? that confirms both #527 (probe passes) and the #521 speedup actually engaged.
- Still open and independent: the **cheaper-watermark** follow-up (your `perf/521-cheap-watermark` session) ? the deeper win where the stitch avoids a full re-encode entirely.

Net: #521 shipped with two review-caught bugs fixed before they could bite, and the release baseline is green.